



M U L 2 _ N E W

Implementation Guide

Architecture, data structures, algorithms and routine reference of
MUL2_NEW

Politecnico di Torino · MUL2 project
Version 1.0 (October 2026)

Contents

- 1 Overview, rules and build system**
 - 1.1 Purpose of this guide
 - 1.2 Rules the code follows
 - 1.3 Repository layout
 - 1.4 Toolchain
 - 1.5 CMake targets
 - 1.6 What happens when the program runs
- 2 Architecture**
 - 2.1 Layers
 - 2.2 Naming conventions
 - 2.3 Status and error handling
 - 2.4 Memory management
 - 2.5 OpenMP and determinism
- 3 Data model**
 - 3.1 The model
 - 3.2 Topology codes
 - 3.3 DOF layout
 - 3.4 Sparse system
 - 3.5 The cache
 - 3.6 Results
- 4 Input and output**
 - 4.1 Reading text files
 - 4.2 The readers
 - 4.3 Output
 - 4.4 Output grid
- 5 Preprocessing: the model cache**
 - 5.1 Step 0: mechanical check
 - 5.2 Step 1: DOF layout
 - 5.3 Step 2: element frames
 - 5.4 Step 3: reference quadrature rules
 - 5.5 Step 4: Gauss layout
 - 5.6 Step 5: structural and expansion geometry
 - 5.7 Step 6: materials
 - 5.8 MITC preparation
 - 5.9 What the cache costs
- 6 Element kernels**
 - 6.1 Building blocks
 - 6.2 Choosing the kernel
 - 6.3 The separable kernel in detail
 - 6.4 The reference (point-by-point) kernel
 - 6.5 Dense products
 - 6.6 MITC in the code
 - 6.7 Reduced and selective integration
- 7 Assembly**
 - 7.1 Step 1: DOF lists
 - 7.2 Step 2: pattern
 - 7.3 Step 3: chunks and parallel evaluation
 - 7.4 Step 4: scatter

	7.5	Reduction for the modal problem
	7.6	Complexity and memory
8		Constraints, loads and solvers
	8.1	Boundary data
	8.2	Analysis 101
	8.3	Analysis 103
	8.4	Results and memory in the modal analysis
9		Post-processing
	9.1	Evaluation context: <code>PLACE_TYPE</code>
	9.2	Locating a point
	9.3	Displacement, strain and stress
	9.4	Output grid
	9.5	Writers
10		Testing and validation
	10.1	Test programs
	10.2	The end-to-end suite
	10.3	Comparing the two kernels
	10.4	Benchmarks
	10.5	Debugging tips
11		Extending the program
	11.1	Adding a structural element topology
	11.2	Adding an expansion family
	11.3	Adding an analysis type
	11.4	Rules for a good contribution
12		The HLE implementation
	12.1	Modules
	12.2	Data structures
	12.3	Flow
	12.4	Tests
13		Multiphysics and the new analyses
	13.1	1. New and extended modules
	13.2	2. How a model flows through the analyses
	13.3	3. Conventions that are easy to get wrong
	13.4	4. Extending
	13.5	5. Tests
14		Curved beams and shells
	14.1	Where the general geometry enters
	14.2	<code>MUL2_GENERAL_GEOMETRY</code> (<code>SRC/GEOMETRY/mul2_general_geometry.for</code>)
	14.3	<code>MUL2_GENERAL_KERNEL</code> (<code>SRC/ELEMENTS/mul2_general_kernel.for</code>)
	14.4	Joining by coincidence (<code>MUL2_COINCIDENT_JOIN</code>)
	14.5	Tests
	14.6	Extending
A		File formats at a glance
B		Naming of the principal variables
C		Source reference
	C.1	Layer BASE
	C.2	Layer MODEL
	C.3	Layer IO
	C.4	Layer QUADRATURE
	C.5	Layer FEM

- C.6 Layer GEOMETRY
- C.7 Layer CUF
- C.8 Layer KINEMATICS
- C.9 Layer MATERIALS
- C.10 Layer GAUSS
- C.11 Layer ELEMENTS
- C.12 Layer ASSEMBLY
- C.13 Layer BOUNDARY
- C.14 Layer SOLVERS
- C.15 Layer ANALYSES
- C.16 Layer POST
- C.17 Layer APP

Overview, rules and build system

1.1 Purpose of this guide

The Theoretical Guide explains *what* is computed. This guide explains *where and how* each formula is coded in the Fortran sources of MUL2_NEW, so that a student can read a routine and know its mathematical meaning, its inputs and outputs, and the data structures it works on. It is organised as follows:

- Chapters 1 and 2: rules of the code, the build system, the layered architecture and the conventions (status handling, kinds, naming).
- Chapters 3 to 5: data model, input and preprocessing.
- Chapters 6 to 8: element kernels, assembly, constraints and solvers (with flowcharts).
- Chapters 9 and 10: post-processing and tests.
- Chapter 11: how to extend the program.
- Appendices: variable dictionary, file formats, and the **generated routine reference** (every module, derived type and procedure).

1.2 Rules the code follows

The program obeys the document `SPECIFICHE_CODICE_FORTRAN.md` of the repository. The points that matter when reading the code are:

Table 1.1. Coding rules and where they show.

Rule	Consequence in the sources
Fixed-form source, extension <code>.for</code> , 72 columns, upper case	continuation lines start with <code>&</code> in column 6; the <code>CHECK_FIXED_FORM</code> target fails the build if a line is longer
<code>REAL(R8)</code> for all reals, <code>INTEGER(I4)</code> for counts and ids, <code>INTEGER(I8)</code> for sparse indices	every literal carries its kind: <code>0.0_R8</code> , <code>1_I4</code> ; sparse arrays (<code>ROW_POINTER</code> , <code>COLUMN_INDEX</code>) are 64-bit
One implementation per formula	e.g. the strain operator exists only in <code>MUL2_LINEAR_KINEMATICS</code> ; the rotation of the stiffness only in <code>MUL2_MATERIAL_ROTATIONS</code>
No global mutable state	all data is passed as arguments; the only module variable is the debug flag of the logger
Errors and warnings through a status object	no <code>STOP</code> inside library routines; <code>STATUS_TYPE</code> travels upwards (Section 2.3)
Serial reference always available; OpenMP only where the result does not depend on the thread count	the assembly scatters serially; the parallel regions are listed in Section 2.5
Legacy input and output names are kept	<code>STATIC/POST_POINT.dat</code> , <code>DYNAMIC/FREQUENCIES.dat</code> , ... are written exactly as the baseline
The GUI never links the solver	<code>INTERFACE/</code> only writes input files, starts the executable and reads output files

1.3 Repository layout

```
MUL2_NEW/
  CMakeLists.txt  CMakePresets.json  CREATE_VS_SOLUTION.cmd
  SRC/            fixed-form sources by layer (BASE MODEL IO ... APP) and SRC/ARPACK
  TESTS/         unit, integration and end-to-end tests; CASES (golden inputs), BENCH
  INTERFACE/     web interface (HTML/JS + PowerShell server)
  DOC/           guides (this book, the User and Theoretical guides), notes
  REFERENCES/    baseline inputs and golden outputs used by the tests
  BUILD/         generated build trees (not versioned)
```

1.4 Toolchain

- **Compiler:** Intel oneAPI ifx (tested with 2025.0). OpenMP is enabled with /Qopenmp.
- **Libraries:** oneMKL (PARDISO, BLAS/LAPACK) with the ILP64 interface, sequential threading by default; ARPACK compiled from the sources in SRC/ARPACK with 64-bit INTEGER and LOGICAL (/4I8) so that it accepts the same ILP64 arrays.
- **Build tool:** CMake ≥ 3.25 is the single source of truth. A Visual Studio solution is always generated:

```
CREATE_VS_SOLUTION.cmd build      # configure + build Release
CREATE_VS_SOLUTION.cmd open      # also open BUILD\WINDOWS_IFX\MUL2_V3.sln
```

Equivalent manual commands:

```
cmake --preset windows-ifx
cmake --build BUILD\WINDOWS_IFX --config Release
ctest --test-dir BUILD\WINDOWS_IFX -C Release --output-on-failure
```

The preset windows-ifx uses the *Visual Studio 17 2022* generator with the ifx toolset and defines the configurations Debug (bounds, uninitialised-variable and traceback checks), Release and Release_Fast. The options of interest are:

Table 1.2. CMake options.

Option	Default	Meaning
MUL2_ENABLE_OPENMP	ON in the preset	compile with OpenMP
MUL2_ENABLE_MKL	ON in the preset	build the solvers (PARDISO, ARPACK, LAPACK); without it only the core library and a stub executable are built
MUL2_MKL_ILP64	ON	64-bit MKL integer interface
MUL2_MKL_THREADED	OFF	link the threaded MKL (faster factorisations on some machines, not bit-reproducible across thread counts)

1.5 CMake targets

Table 1.3. Targets defined in CMakeLists.txt.

Target	Type	Content
MUL2_CORE	static library	everything that does not need MKL: base, model, readers, geometry, caches, kernels, assembly, boundary, post-processing
MUL2_ARPACK	static library	ARPACK sources (SRC/ARPACK/*.f)
MUL2_SOLVERS	static library	PARDISO and modal solvers, analyses 101/103, driver
MUL2_V3	executable	mul2_main.for (or mul2_main_core.for when MKL is off)
MUL2_TESTS	executable	unit tests of the core
MUL2_PARDISO_TESTS	executable	tests of the PARDISO wrapper
MUL2_ANALYSIS_TESTS	executable	end-to-end tests against the golden cases
CHECK_FIXED_FORM	custom target	checks the 72-column rule

1.6 What happens when the program runs

```
MUL2_V3.exe [input_directory]
```

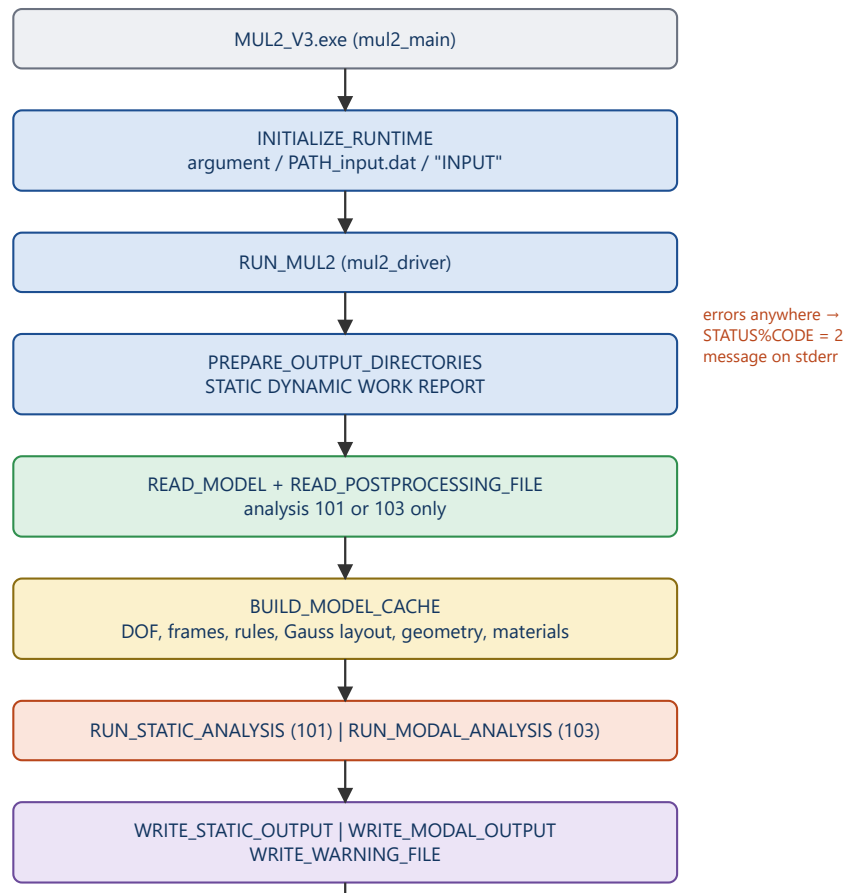


Figure 1.1. Overall flow of the program.

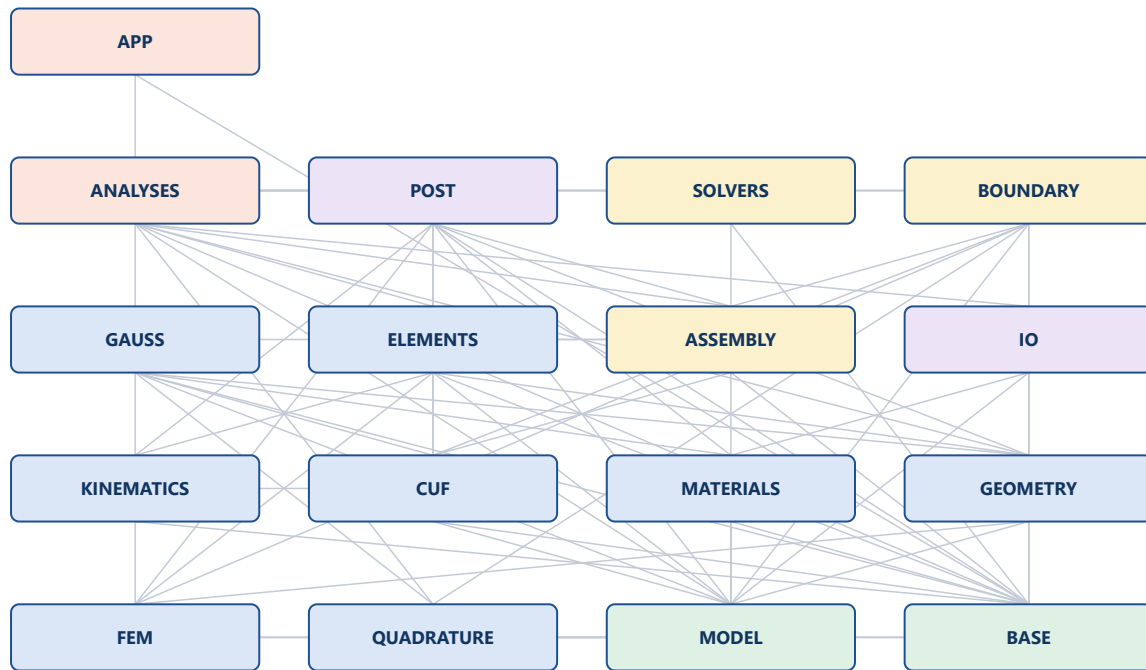
1. `mul2_main` decides the input directory: command-line argument, otherwise the first line of `PATH_input.dat`, otherwise `INPUT`; `--version` prints the version.
2. `RUN_MUL2` (module `MUL2_DRIVER`) creates `STATIC`, `DYNAMIC`, `WORK`, `REPORT`, reads the model, builds the cache, runs the analysis (101 or 103), writes the outputs and the warning report.
3. The exit code is 0 on success and 1 on any error; after printing `[ERROR] source: message`.

The environment variable `MUL2_DEBUG=1` enables debug logging; `MUL2_GENERAL_KERNEL=1` forces the point-by-point reference kernel (Chapter 6); `OMP_NUM_THREADS` sets the number of OpenMP threads.

Architecture

2.1 Layers

The source tree is divided into **layers**, one directory per layer. A module of a layer may use only modules of the same layer or of the layers below it. The diagram below is computed automatically from the `USE` statements: every line is a real dependency.



edges: a layer uses modules of the layers it points to (drawn from the `USE` statements; layers lower in the picture know nothing about upper ones)

Figure 2.1. Layers of the code and the dependencies actually present.

Table 2.1. Layers, their purpose and their modules.

Layer	Purpose	Modules
BASE	precision kinds, status, log, timers, strings, files, sorting	MUL2_KINDS, MUL2_STATUS, MUL2_LOG, MUL2_TIMER, MUL2_RUNTIME, MUL2_STRINGS, MUL2_FILES, MUL2_SORTING
MODEL	plain data types: nodes, elements, kinematics, expansion meshes, materials, laminations, boundary conditions, analysis and post requests, topologies	MUL2_NODES, MUL2_ELEMENTS, MUL2_KINEMATICS, MUL2_EXPANSION_MESHES, MUL2_LAMINATIONS, MUL2_BOUNDARY_CONDITIONS, MUL2_ANALYSIS_INPUT, MUL2_REFERENCE_SYSTEMS, MUL2_INCIDENCE, MUL2_TOPOLOGIES, MUL2_MODEL
IO	text readers; <code>READ_MODEL</code> builds a <code>MODEL_TYPE</code> from a directory	<code>READ_*</code> modules, MUL2_TEXT_IO
QUADRATURE, FEM	Gauss rules, shape functions	MUL2_QUADRATURE, MUL2_SHAPE_FUNCTIONS
GEOMETRY	element frames, Jacobians, local gradients	MUL2_ELEMENT_FRAMES, MUL2_JACOBIANS, MUL2_LOCAL_GRADIENTS
CUF, KINEMATICS	expansion bases, point factors, nucleus; strain operator	MUL2_CUF_BASES, MUL2_POINT_BASES, MUL2_FUNDAMENTAL_NUCLEUS, MUL2_LINEAR_KINEMATICS

Layer	Purpose	Modules
MATERIALS	stiffness matrices, rotations	MUL2_MATERIALS , MUL2_MATERIAL_ROTATIONS , MUL2_MATERIAL_RESOLUTION
GAUSS	reference rules, global Gauss layout, geometry and material caches	MUL2_GAUSS_POINTS , MUL2_GAUSS_GEOMETRY , MUL2_GAUSS_MATERIALS
ELEMENTS	element matrices (reference and separable kernels), MITC, dense products	MUL2_ELEMENT_MATRICES , MUL2_SEPARABLE_KERNEL , MUL2_MITC , MUL2_DENSE_PRODUCTS
ASSEMBLY	DOF layout, CSR pattern and assembly, sparse operations	MUL2_DOF_LAYOUT , MUL2_SPARSE_ASSEMBLY , MUL2_SPARSE_OPERATIONS
BOUNDARY	constraints and loads	MUL2_BOUNDARY_APPLICATION
SOLVERS	PARDISO and ARPACK/LAPACK wrappers	MUL2_PARDISO_FACTOR , MUL2_PARDISO_SOLVER , MUL2_MODAL_SOLVER
ANALYSES	model cache, parallel assembly, analyses 101 and 103, driver	MUL2_MODEL_CACHE , MUL2_MODEL_ASSEMBLY , MUL2_ANALYSIS_101 , MUL2_ANALYSIS_103 , MUL2_DRIVER
POST	point location and recovery, grids, output files	MUL2_RECOVERY , MUL2_POST_GRID , MUL2_POST_OUTPUT
APP	program entry	MUL2_V3

2.2 Naming conventions

- **Modules** are named `MUL2_<NAME>` ; the file has the same name in lower case.
- **Procedures** have verb-first names: `BUILD_...` creates or fills a structure, `EVALUATE_...` computes numbers at a point, `READ_...` / `WRITE_...` do I/O, `FIND_...` searches, `CLEAR_...` releases memory, `RUN_...` executes an analysis.
- **Derived types** end with `_TYPE` ; collections end with `_DB_TYPE` and contain an allocatable component `ITEM(:)` .
- **Dummy arguments** are declared with `INTENT` ; output arguments are `INTENT(OUT)` or `INTENT(INOUT)` ; the last argument of fallible routines is `STATUS` .
- **Indices**: `..._INDEX` is a position in an array (1-based) while `..._ID` is the number given in the input; the readers build sorted lookup tables so that `FIND_..._INDEX` is a binary search.
- **Frame vocabulary**: `GLOBAL_TO_LOCAL(:, :, e)` is the matrix R of element e (rows = local axes); `LOCAL_TO_GLOBAL` is its transpose.

2.3 Status and error handling

Every routine that can fail returns a `STATUS_TYPE` :

`STATUS_TYPE` (BASE/mul2_status.for)

```

TYPE, PUBLIC :: STATUS_TYPE
  INTEGER(I4) :: CODE = STATUS_OK           ! 0 OK, 1 warning, 2 error
  INTEGER(I4) :: WARNING_COUNT = 0_I4
  CHARACTER(LEN=256) :: MESSAGE = ' '
  CHARACTER(LEN=64) :: SOURCE = ' '        ! routine that raised it
END TYPE STATUS_TYPE

```

Rules:

1. A routine starts with `CALL CLEAR_STATUS(STATUS)` .
2. It reports with `SET_WARNING(STATUS, 'ROUTINE', 'text')` (the run continues) or `SET_ERROR(...)` (the routine returns). An error is never overwritten by a later message; warnings are counted.
3. A caller merges the status of a step into the status of the phase with `MERGE_STATUS(LOCAL, GLOBAL)` (the first error wins), tests it with `STATUS_IS_OK` and returns on error.
4. `RUN_MUL2` collects the warning texts in a `WARNING_LIST_TYPE` that is written to `REPORT/WARNING_file.dat` .

This convention replaces exceptions: reading any routine, the error paths are the `IF (.NOT. STATUS_IS_OK(...)) RETURN` lines.

2.4 Memory management

All large arrays are `ALLOCATABLE` and owned by the structure that contains them. Memory is released explicitly where it matters:

- `PARDISO_SET_SYSTEM` copies the upper triangle into the solver handle; the static analysis then frees the full matrix before the (memory-hungry) factorisation if no dump is requested.
- The modal analysis frees the full K , M and the reduction map once the reduced matrices are built.
- Element matrices exist only for the elements of the chunk being assembled and are released after the scatter.

2.5 OpenMP and determinism

Parallel regions (all marked with `!$OMP`):

Table 2.2. Parallel regions.

Where	What is parallel	Why it is thread-count independent
<code>ASSEMBLE_MODEL_SYSTEM</code>	the evaluation of the element matrices of a chunk	each element writes only its private matrices; the scatter into the CSR arrays is serial and in element order
<code>SPARSE_MULTIPLY</code>	rows of a matrix–vector product	rows are independent
<code>EVALUATE_GRID_STATES</code> , <code>EVALUATE_GRID_MODES</code>	cells of the output grid	each cell writes its own entries

Because the order of every floating-point addition is fixed, runs with different `OMP_NUM_THREADS` give **bit-identical** results (tested with 1 and 4 threads). Chunk size is a multiple of the thread count (so that no thread idles) and bounded by a memory budget of 10^9 bytes of element matrices.

Data model

The program is data-driven: the input is read into a set of plain databases (the *model*), derived data are computed once (the *cache*) and the analysis fills a *results* container. This chapter describes these structures. The complete component lists are generated in Appendix C; here the *meaning* and the *relations* are explained.

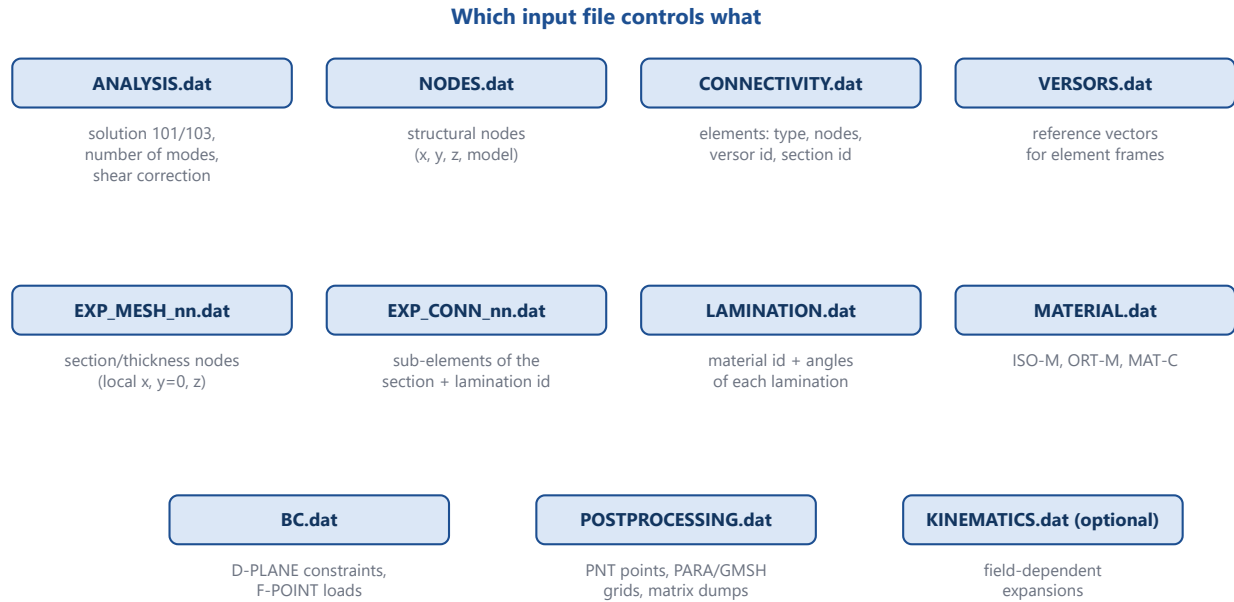


Figure 3.1. Which input file fills which database.

3.1 The model

`MODEL_TYPE` (module `MUL2_MODEL`) is the root of everything read from the input directory:

`MODEL_TYPE` (MODEL/mul2_model.for)

```
TYPE, PUBLIC :: MODEL_TYPE
CHARACTER(LEN=512) :: INPUT_PATH
TYPE(ANALYSIS_TYPE)      :: ANALYSIS      ! ANALYSIS.dat
TYPE(POST_DB_TYPE)       :: POST           ! POSTPROCESSING.dat
TYPE(KINEMATICS_DB_TYPE) :: KINEMATICS     ! KINEMATICS.dat or legacy NODES.dat
TYPE(NODE_DB_TYPE)       :: NODES         ! NODES.dat
TYPE(ELEMENT_DB_TYPE)    :: ELEMENTS      ! CONNECTIVITY.dat
TYPE(EXPANSION_DB_TYPE)  :: EXPANSIONS    ! EXP_MESH_nn.dat / EXP_CONN_nn.dat
TYPE(REFERENCE_VECTOR_DB_TYPE):: VECTORS  ! VERSORS.dat
TYPE(MATERIAL_DB_TYPE)   :: MATERIALS     ! MATERIAL.dat
TYPE(LAMINATION_DB_TYPE) :: LAMINATIONS   ! LAMINATION.dat
TYPE(BOUNDARY_DB_TYPE)  :: BOUNDARIES    ! BC.dat
END TYPE MODEL_TYPE
```

3.1.1 Nodes, elements and incidence

- `NODE_TYPE`: `ID` (number in the file), `COORDINATE(3)` (global), `KINEMATIC_ID` (which kinematic record the node uses). `NODE_DB_TYPE` also stores `ID_LIST` and `ID_ORDER`, a sorted copy of the ids built by `BUILD_NODE_INDEX`, so that `FIND_NODE_INDEX(NODES, id)` is a binary search.
- `ELEMENT_TYPE`: `ID`, `TOPOLOGY` (integer code, see `MUL2_TOPOLOGIES`), `NODE_ID(:)` (node ids in the shape-function order), `FRAME_ID` (versor id), `EXPANSION_ID` (index of the expansion mesh).
- `NODE_INCIDENCE_TYPE`: for node i the elements that contain it are `ELEMENT(FIRST(i):FIRST(i+1)-1)` (CSR-like, ascending). It is used to know which expansion mesh governs a node (the first incident element) and to detect

unused nodes.

3.1.2 Kinematics

A *kinematic* is the expansion specification of the nine fields:

Kinematics (MODEL/mul2_kinematics.for)

```
TYPE :: EXPANSION_SPEC_TYPE
  INTEGER(I4) :: FAMILY = EXPANSION_NONE    ! NONE, TE, LE, HLE, MISC
  INTEGER(I4) :: ORDER = 0                  ! Taylor order, HLE order
  INTEGER(I4) :: FIELD_REFERENCE = 0        ! index of user functions (M<n>)
END TYPE
TYPE :: KINEMATIC_TYPE
  INTEGER(I4) :: ID
  TYPE(EXPANSION_SPEC_TYPE) :: FIELD(N_FIELDS) ! U V W T P B SZZ SXZ SYZ
END TYPE
```

With the historical `NODES.dat` (`ID X Y Z TE|LE [order]`) the reader `READ_LEGACY_NODES_FILE` creates one kinematic for every distinct (family, order) pair, with the same expansion for the three displacement fields. With a `KINEMATICS.dat` file the table is read explicitly and `NODES.dat` carries the kinematic id.

3.1.3 Expansion meshes

`EXPANSION_MESH_TYPE` holds `NODE(:)` (id and local coordinate of every expansion node) and `ELEMENT(:)` (sub-elements with topology, lamination id and node ids). The mesh index equals the number `nn` of the files `EXP_MESH_nn.dat`, `EXP_CONN_nn.dat`. The coordinates of an expansion node are $(x_s, 0, z_s)$ for a beam section, $(0, 0, z_s)$ for a plate thickness and $(0, 0, 0)$ for a solid; the program determines which axes are active from the extent of the nodes (`FIND_EXPANSION_ACTIVE_AXES`).

3.1.4 Materials, laminations, boundary conditions

- `MATERIAL_TYPE`: ID, MODEL, DENSITY, STIFFNESS(6,6) in the material axes (Voigt order TT, LL, ZZ, TZ, LZ, TL).
- `LAMINATION_TYPE`: ID, KIND (constant), MATERIAL_ID, ANGLE_Y_DEG, ANGLE_Z_DEG.
- `BOUNDARY_CONDITION_TYPE`: KIND (`BC_DISPLACEMENT_PLANE` or `BC_FORCE_POINT`), PLANE(4), POINT(3), ACTIVE(3) (constrained components), VALUE(3) (prescribed displacements or load components).

3.1.5 Analysis and post-processing requests

`ANALYSIS_TYPE` holds `SOLUTION` (101/103), `MODE_COUNT` and `SHEAR(3)` (the correction codes `SHEAR_NONE`, `SHEAR_MITC`, ... for beams, plates, solids). `POST_DB_TYPE` holds the list of field requests (`FORMAT`, `FRAME`, `CELL_NODES`, `SPLIT(9)`), the list of point requests and the dump flags.

3.2 Topology codes

`MUL2_TOPOLOGIES` is the single registry of topologies. The integer code encodes the family in the hundreds digit: 101–103 beams, 201–205 surface elements, 301–306 solids, 401 the point element S1. Functions `TOPOLOGY_FROM_NAME`, `TOPOLOGY_NODE_COUNT` and `TOPOLOGY_NATURAL_DIMENSION` are used everywhere instead of hard-coded numbers.

3.3 DOF layout

`DOF_LAYOUT_TYPE` (module `MUL2_DOF_LAYOUT`, routine `BUILD_DOF_LAYOUT`):

Component	Meaning
<code>TERM_COUNT(node, field)</code>	number of terms of the field at the node (0 for inactive fields and unused nodes)
<code>FIRST(node, field)</code>	first global DOF of the block
<code>FIELD_FIRST(field), FIELD_LAST(field)</code>	range of the global DOFs of each field
<code>TOTAL_DOF</code>	total number of DOFs

TERM_COUNT comes from NODE_FIELD_TERM_COUNT: for Taylor, TAYLOR_BASIS_TERM_COUNT(order, dim) with dim the dimension of the expansion mesh (2 for beams, 1 for plates, 0 for solids); for Lagrange, the number of nodes of the mesh. GLOBAL_DOF(layout, node, field, term) returns FIRST(node,field) + term - 1. All other modules obtain global numbers only through this function.

3.4 Sparse system

SPARSE_SYSTEM_TYPE stores the CSR matrices:

SPARSE_SYSTEM_TYPE (ASSEMBLY/mul2_sparse_assembly.for)

```

TYPE :: SPARSE_SYSTEM_TYPE
  INTEGER(I8) :: ORDER, NONZERO_COUNT
  INTEGER(I8), ALLOCATABLE :: ROW_POINTER(:) ! size ORDER+1, 1-based
  INTEGER(I8), ALLOCATABLE :: COLUMN_INDEX(:) ! size NONZERO_COUNT, ascending per row
  REAL(R8), ALLOCATABLE :: STIFFNESS(:) ! K values
  REAL(R8), ALLOCATABLE :: MASS(:) ! M values (only when requested)
END TYPE

```

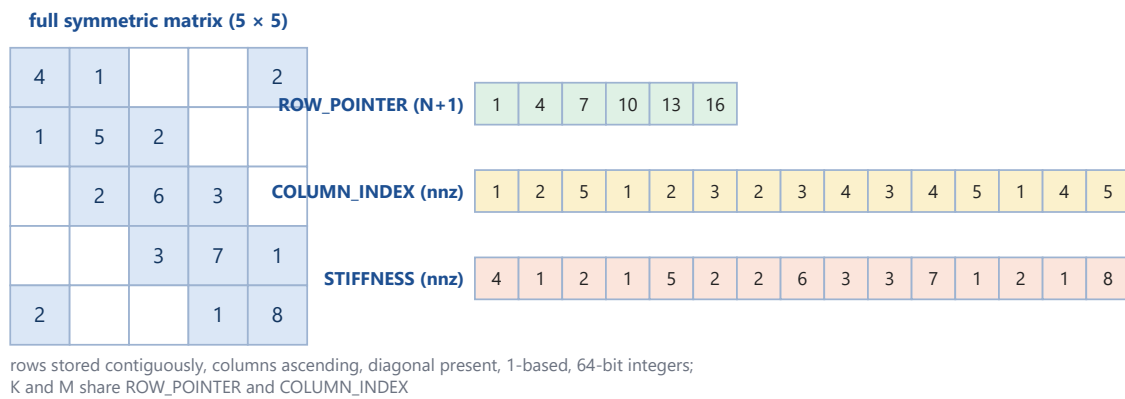


Figure 3.2. CSR storage.

3.5 The cache

MODEL_CACHE_TYPE collects everything that depends on the model but not on the analysis. It is built once by BUILD_MODEL_CACHE (Chapter 5):

Component	Type	Content
FRAMES	ELEMENT_FRAME_DB_TYPE	ORIGIN(3,e) , GLOBAL_TO_LOCAL(3,3,e) , LOCAL_TO_GLOBAL(3,3,e)
DOF_LAYOUT	DOF_LAYOUT_TYPE	numbering
RULES	REFERENCE_RULE_DB_TYPE	quadrature rules with shapes/derivatives at their points
GAUSS_LAYOUT	GAUSS_LAYOUT_TYPE	decomposition of the global Gauss-point number
STRUCTURAL_GEOMETRY	STRUCTURAL_GEOMETRY_CACHE_TYPE	Jacobian, inverse, determinant, local gradients of the structural points
EXPANSION_GEOMETRY	EXPANSION_GEOMETRY_CACHE_TYPE	same for the expansion points
GEOMETRY	GAUSS_GEOMETRY_TYPE	indices into the two caches, global coordinates, integration weights
MATERIAL_CACHE , MATERIAL_MAP		rotated stiffness per lamination and the material index of every Gauss point
EXPANSION_ORDER	INTEGER(:)	Gauss points per direction required by each expansion mesh

3.6 Results

ANALYSIS_RESULTS_TYPE holds SYSTEM (the sparse matrices), CONSTRAINTS, FORCE, SOLUTION (101) or MODE(:,k), EIGENVALUE, FREQUENCY, RESIDUAL (103) and the solver statistics SOLVER_BACKEND, SOLVER_ITERATIONS, SOLVER_OPERATIONS, SOLVER_SUBSPACE, ASSEMBLY_SECONDS, SOLVER_SECONDS.

Input and output

4.1 Reading text files

All input files share the same *legacy* layout: the first data record is the number of records; comments and blank lines may appear anywhere; text after the data is ignored. This is implemented once in `NEXT_DATA_LINE(UNIT, LINE, IOS)` (module `MUL2_TEXT_IO`), which returns the next line that is neither blank nor a comment. Every reader follows the same scheme:

Skeleton of a reader (`IO/read_connectivity.for`, abridged)

```
CALL CLEAR_STATUS(STATUS)
OPEN(NEWUNIT=UNIT, FILE=FILE_NAME, STATUS='OLD', ACTION='READ', IOSTAT=IOS)
IF (IOS .NE. 0) THEN
  CALL SET_ERROR(STATUS, 'READ_CONNECTIVITY_FILE', 'CANNOT OPEN: '//TRIM(FILE_NAME))
  RETURN
END IF
CALL NEXT_DATA_LINE(UNIT, LINE, IOS)      ! record count
READ(LINE,*,IOSTAT=IOS) COUNT             ! list-directed read of the line
...
DO I = 1, COUNT
  CALL NEXT_DATA_LINE(UNIT, LINE, IOS)    ! one record
  READ(LINE,*,IOSTAT=IOS) NAME, ID        ! header first: decides how many values follow
  ...
END DO
CALL NEXT_DATA_LINE(UNIT, LINE, IOS)      ! extra records after the declared count
IF (IOS .EQ. 0) CALL SET_WARNING(STATUS, ..., 'EXTRA RECORDS AFTER DECLARED COUNT')
```

List-directed reading accepts Fortran real formats (`1.0D-3`, `1e-3`), arbitrary blanks and tabs.

4.2 The readers

Table 4.1. Readers.

Reader (module)	File	Records and checks
<code>READ_ANALYSIS_FILE</code> (<code>MUL2_READ_ANALYSIS</code>)	<code>ANALYSIS.dat</code>	solution id, number of modes, three shear-treatment words (<code>NONE</code> , <code>REDI</code> , <code>SELI</code> , <code>MITC</code>); the obsolete solver record of the historical format is detected (second integer record) and skipped with a warning
<code>READ_POSTPROCESSING_FILE</code>	<code>POSTPROCESSING.dat</code>	<code>PARA / GMSH</code> (cell nodes 8/20/27, <code>LOC / GLB</code> , nine splits ≥ 1), <code>PNT</code> , <code>KMAT</code> <code>MMAT</code> <code>FRCE</code> <code>UNKN</code> <code>ENRG</code> ; fewer records than declared give a warning
<code>READ_LEGACY_NODES_FILE</code>	<code>NODES.dat</code> (historical)	<code>ID X Y Z TE LE [order]</code> ; creates the kinematics
<code>READ_KINEMATICS_FILE</code> , <code>READ_NODES_FILE</code>	<code>KINEMATICS.dat</code> , <code>NODES.dat</code> (V3)	<code>KINEMATIC id tok1...tok9</code> ; <code>ID X Y Z KINEMATIC_ID</code>
<code>READ_CONNECTIVITY_FILE</code>	<code>CONNECTIVITY.dat</code>	<code>TYPE id n1...nN [versor]</code> section; checks that all nodes exist and ids are unique
<code>READ_EXPANSION_SET</code> → <code>READ_EXPANSION_NODES</code> , <code>READ_EXPANSION_ELEMENTS</code>	<code>EXP_MESH_nn.dat</code> , <code>EXP_CONN_nn.dat</code>	one pair per expansion id used (1...99); duplicate ids and unknown nodes are errors
<code>READ_REFERENCE_VECTORS_FILE</code>	<code>VERSORS.dat</code>	<code>VECTOR id vx vy vz</code> ; zero vectors rejected
<code>READ_MATERIALS_FILE</code>	<code>MATERIAL.dat</code>	<code>ISO-M</code> , <code>ORT-M</code> , <code>MAT-C</code> build the stiffness (<code>BUILD_*_MATERIAL</code>); thermal/electrical records are accepted and ignored
<code>READ_LAMINATIONS_FILE</code>	<code>LAMINATION.dat</code>	<code>LAM2 id material angle_y angle_z</code> ; material must exist

Reader (module)	File	Records and checks
READ_BOUNDARY_CONDITIONS_FILE	BC.dat	D-PLANE id A B C D ux uy uz (N = free component), F-POINT id x y z Fx Fy Fz

READ_MODEL(INPUT_PATH, MODEL, STATUS) calls them in the order: analysis, kinematics/nodes, connectivity, expansions (the number of meshes is the largest EXPANSION_ID found in the connectivity), versors, materials, laminations, boundary conditions, post-processing, merging all statuses.

NOTE

The readers translate **syntax**; the **semantic** checks (for example that a lamination of an expansion element exists) are made by the caches. Duplicate ids are detected in $O(n \log n)$ with SORT_PERMUTATION and HAS_DUPLICATE_KEYS.

4.3 Output

All output routines live in MUL2_POST_OUTPUT; the file names and column layouts follow the baseline.

Table 4.2. Output files.

File	Written by	Content
STATIC/POST_POINT.dat	WRITE_POST_POINTS	one 46-column row per PNT request
STATIC/RESULTS_PARA_nn.vtk	WRITE_STATIC_VTK	legacy ASCII VTK with displacement, strain, stress, von Mises
STATIC/RESULTS_GMSH_nn.msh	WRITE_STATIC_GMSH	the same as GMSH views
DYNAMIC/FREQUENCIES.dat	WRITE_MODAL_OUTPUT	Frequency n: value [Hz]
DYNAMIC/RESULTS_DYN_PARA.vtk, RESULTS_DYN_GMSH.msh	WRITE_MODAL_VTK, WRITE_MODAL_GMSH	mode shapes
WORK/K_MAT.dat, M_MAT.dat, FORCES.dat, UNKNOWN.dat	WRITE_MATRIX_DUMPS	row column value lines (field-major numbering) / vector lines
WORK/ARPACK_SOLVER_INFO.dat	WRITE_SOLVER_INFO	eigenvalues, residuals, iterations
REPORT/WARNING_file.dat	WRITE_WARNING_FILE	warnings of the whole run

PREPARE_OUTPUT_DIRECTORIES creates the four output directories (using a shell command through EXECUTE_COMMAND_LINE, with the syntax of the current operating system, IS_WINDOWS).

Point rows. WRITE_POINT_ROW writes the 46 numbers: point id (1), evaluated position (3), requested position (3), lamination id, element id, sub-element id (3), displacement (3), strain local (6), strain global (6), stress local (6), stress global (6), three thermal/electric placeholders (3), six placeholders for electric fields (6). The placeholders are zeros, kept for compatibility. A point that is not found gives a row of NaN with zero ids.

4.4 Output grid

The grid (BUILD_POST_GRID) is built from the model and the request: for every element and every sub-element, cells with 8, 20 or 27 nodes are created in natural coordinates, subdivided according to SPLIT. VTK_CELL_ORDER and GMSH_CELL_ORDER give the node order required by each format (VTK quadratic hexahedron = type 25 with 20 nodes; the 27th node is dropped). The states at the grid nodes are computed in parallel (EVALUATE_GRID_STATES).

Preprocessing: the model cache

`BUILD_MODEL_CACHE(MODEL, CACHE, STATUS)` (module `MUL2_MODEL_CACHE`) turns the validated model into the arrays used by the kernels. It runs once per execution. Each step is a separate routine returning a status; the pipeline stops at the first error.

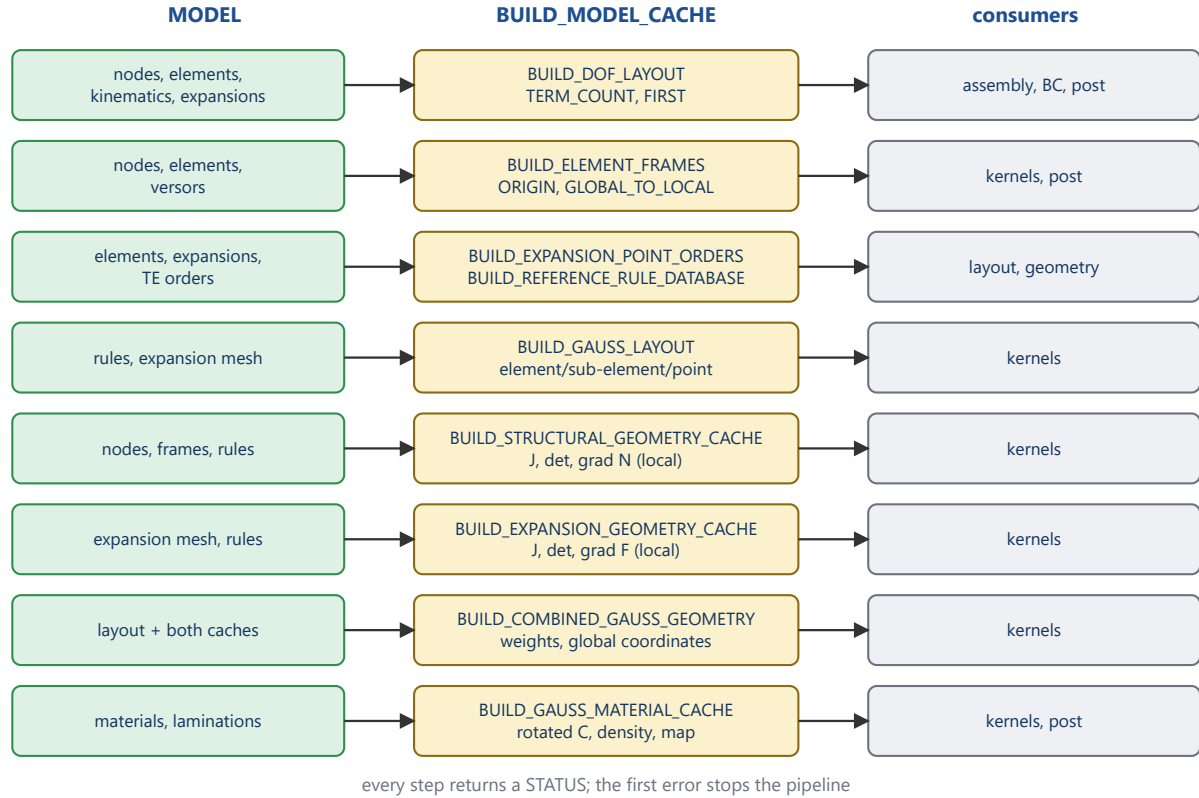


Figure 5.1. Pipeline of BUILD_MODEL_CACHE.

5.1 Step 0: mechanical check

`CHECK_MECHANICAL_MODEL` verifies that fields 4–9 (T, P, B, SZZ, SXZ, SYZ) are `NONE` in every kinematic (only displacements are supported by 101/103) and that the model has at least one element.

5.2 Step 1: DOF layout

`BUILD_DOF_LAYOUT` (Chapter 3). For every node it looks at the incident elements through `BUILD_NODE_INCIDENCE`; `NODE_FIELD_TERM_COUNT` returns the number of terms of each field: Taylor → `TAYLOR_BASIS_TERM_COUNT` of the dimension of the expansion mesh of the incident elements (the dimensions of the meshes of all incident elements must agree, error `EXPANSION HAS MIXED DIMENSIONS / NODE HAS INCOMPATIBLE EXPANSION DIMENSIONS` otherwise); Lagrange → the number of nodes of the expansion mesh, and **all elements incident to a Lagrange node must use the same expansion mesh** (error `LE NODE USES INCOMPATIBLE EXPANSIONS`). A Hierarchical or user-defined family gives an explicit "not implemented" error. A node with no incident element gets no DOF and a warning.

5.3 Step 2: element frames

`BUILD_ELEMENT_FRAMES` computes, for every element, `ORIGIN` (first node), `GLOBAL_TO_LOCAL` and `LOCAL_TO_GLOBAL` with `BUILD_BEAM_FRAME` (B2/B3/B4), `BUILD_SURFACE_FRAME` (Q4, Q9, Q16, T3, T6) or the identity (solids), using the formulas of Chapter 5 of the Theoretical Guide. The versor is looked up with `FIND_VECTOR_INDEX` from the `FRAME_ID` of the element.

5.4 Step 3: reference quadrature rules

`BUILD_EXPANSION_POINT_ORDERS` computes, for every expansion mesh m , the number of Gauss points per direction $n_g(m)$ needed by the Taylor orders of the nodes of the elements that use it: the largest `ORDER` + 1 over the three displacement fields of those nodes (0 when the default rule is enough). The result is `CACHE%EXPANSION_ORDER(m)`.

`BUILD_REFERENCE_RULE_DATABASE` creates one `REFERENCE_RULE_TYPE` for every distinct pair (topology, order): structural topologies with the default rule, expansion topologies with `EXPANSION_POINTS(topology, n_g)` points per direction when that exceeds `DEFAULT_POINTS_PER_DIRECTION`. A rule stores

Component	Shape	Content
<code>COORDINATE</code>	(points, 3)	natural coordinates of the points
<code>WEIGHT</code>	(points)	Gauss weights
<code>SHAPE</code>	(nodes, points)	shape function values $N_a(\xi_p)$
<code>DERIVATIVE</code>	(nodes, 3, points)	natural derivatives $\partial N_a / \partial \xi_d$

`FIND_REFERENCE_RULE(db, topology, order)` returns the index of the rule (order 0 = default). The rules for more than four points per direction come from `GAUSS_LEGENDRE_1D` (Newton iteration).

5.5 Step 4: Gauss layout

A **combined Gauss point** is a pair (structural point p , expansion point q of a sub-element e). `BUILD_GAUSS_LAYOUT` numbers all of them contiguously, element by element, with the order *sub-element*, *structural point*, *expansion point* (outermost to innermost):

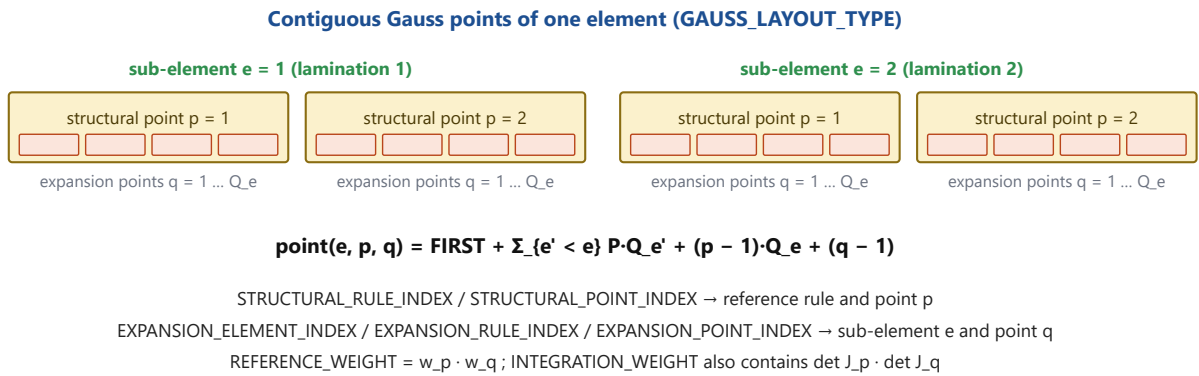


Figure 5.2. Layout of the combined Gauss points of one element.

`GAUSS_LAYOUT_TYPE` stores, for every global point number, the decomposition:

Table 5.2. Components of `GAUSS_LAYOUT_TYPE`.

Component	Size	Meaning
<code>COUNT</code>	scalar	total number of combined points
<code>ELEMENT_FIRST(e)</code> , <code>ELEMENT_LAST(e)</code>	elements	range of the points of element e

Component	Size	Meaning
ELEMENT_INDEX(g)	points	element of point g
EXPANSION_ELEMENT_INDEX(g)	points	sub-element e_x inside the mesh
STRUCTURAL_RULE_INDEX(g), STRUCTURAL_POINT_INDEX(g)	points	rule of the structural topology and point p in it
EXPANSION_RULE_INDEX(g), EXPANSION_POINT_INDEX(g)	points	rule of the sub-element topology and point q in it
LAMINATION_ID(g)	points	lamination of the sub-element
REFERENCE_WEIGHT(g)	points	$w_p w_q$

This is a *decomposition table*: no coordinates, no Jacobians are stored per combined point, only indices. The point of element e , sub-element j , structural point p , expansion point q has the number

$$g = \text{ELEMENT_FIRST}(e) + \sum_{j < j} P Q_j + (p - 1) Q_j + (q - 1).$$

5.6 Step 5: structural and expansion geometry

`BUILD_STRUCTURAL_GEOMETRY_CACHE` loops over the elements, builds the local coordinates of the nodes (`GLOBAL_TO_LOCAL × (X_node - ORIGIN)`), keeps only the active axes (`STRUCTURAL_ACTIVE_AXES`: y for beams, x, y for plates, x, y, z for solids) and, for every structural point, calls `EVALUATE_SQUARE_JACOBIAN` to obtain `JACOBIAN`, `INVERSE_JACOBIAN`, `DETERMINANT` and the physical derivatives of the shape functions. The latter are stored in `DERIVATIVE_LOCAL(3, entry)` with the active components filled and the others zero; `DERIVATIVE_OFFSET(point)` is the first entry of the point (nodes follow one after the other). The global coordinate of the structural point (`CENTER_GLOBAL`) is also stored.

`BUILD_EXPANSION_GEOMETRY_CACHE` does the same for every sub-element of every expansion mesh (`ACTIVE_AXIS(:, mesh)` from `FIND_EXPANSION_ACTIVE_AXES`, `COORDINATE_LOCAL` of the point, determinant, local gradients). The expansion cache does not depend on the structural element: **it is shared by all elements that use the mesh**. For a solid (`s1`, dimension 0) the determinant is 1 and the gradients are zero.

`BUILD_COMBINED_GAUSS_GEOMETRY` ties the two caches together: for every combined point it stores `STRUCTURAL_CACHE_INDEX` and `EXPANSION_CACHE_INDEX`, the global coordinate $X_p + R^T c_q$ and the **integration weight** $w_p w_q \det J_s \det J_e$.

5.7 Step 6: materials

`BUILD_GAUSS_MATERIAL_CACHE` resolves every distinct lamination id used by the layout with `RESOLVE_LAMINATION` (material rotation by `BUILD_MATERIAL_ROTATION`, stiffness rotation by `ROTATE_STIFFNESS`) into `STIFFNESS_LOCAL(6, 6, k)`, `DENSITY(k)`, `MATERIAL_FROM_LOCAL(3, 3, k)` and stores in `MATERIAL_MAP%CACHE_INDEX(g)` the entry of each point. The number of distinct entries is the number of laminations used, not the number of points.

5.8 MITC preparation

MITC data is element-local and is built on the fly, in the assembly, by `PREPARE_ELEMENT_MITC`: if the analysis requests MITC for the family of the element and the topology has a table (`MITC_IS_AVAILABLE`), `MITC_PREPARE_ELEMENT` fills a `MITC_DATA_TYPE` with the tied rows, the tying sets and the shape functions and local gradients at the tying points (computed with the *element* node coordinates, hence the Jacobian of the tying point itself).

5.9 What the cache costs

For a beam with n_e elements, P structural and Q expansion points per sub-element and E sub-elements, the layout has $n_e P \sum_e Q_e$ points (84 k for the 100 000-DOF Taylor-15 benchmark) but the geometry caches only $n_e P$ structural plus $\sum Q_e$ expansion entries; this is why the memory of the cache is negligible compared with that of the matrices.

Element kernels

This chapter follows a combined Gauss point from the cache to the element matrices. It first describes the small building blocks (shape functions, point factors, strain operator) and then the two complete kernels.

6.1 Building blocks

6.1.1 Shape functions and gradients

- `EVALUATE_SHAPE(topology, natural, N, DN_DNATURAL, STATUS)` returns the shape functions and their derivatives with respect to the natural coordinates; it dispatches on the topology to `SHAPE_B`, `SHAPE_Q4`, `SHAPE_Q9`, `SHAPE_Q16`, `SHAPE_T3`, `SHAPE_T6`, `SHAPE_H8`, `SHAPE_H27`. Tensor-product elements use `TENSOR_2D / TENSOR_3D` with the index tables `IX`, `IY`, `IZ` that define the node order, and `LAGRANGE_1D` for the 1D polynomials.
- `EVALUATE_SQUARE_JACOBIAN(coordinate, dn_dnatural, dimension, jacobian, determinant, inverse, dn_dphysical, status)` implements $J = x^T \partial N / \partial \xi$, $\det J$, J^{-1} and $\partial N / \partial x = J^{-T} \partial N / \partial \xi$ for dimension 1, 2 or 3 (`INVERT_SMALL_MATRIX`).
- `EVALUATE_SHAPE_AND_GRADIENT(topology, node_local, natural, shape, gradient, status)` (module `MUL2_LOCAL_GRADIENTS`) combines the two for one structural element at an arbitrary natural point; it is used for the MITC tying points and for post-processing.

6.1.2 Expansion functions

`EVALUATE_EXPANSION_FACTOR(spec, term, mesh, sub_element, dim, axes, coordinate_local, shape, gradient, F, dF, status)` (module `MUL2_POINT_BASES`) returns F_τ and its local gradient:

- *Taylor*: `EVALUATE_TAYLOR_TERM(order, dim, active_axis, coordinate, term, ...)` computes the monomial $x^a z^b$ and its derivatives (`INTEGER_POWER` for the powers). The term ordering is: increasing degree and, inside a degree, increasing power of the second active axis (z).
- *Lagrange*: the term τ is the node `MESH%NODE(tau)`; if it belongs to the active sub-element the value is the sub-element shape function of that node (`EXPANSION_SHAPE(local_node)`) and its gradient, otherwise it is **exactly zero** (this is what creates the structural zeros).

`EVALUATE_POINT_FACTORS(point, structural_node, term, spec, ..., structural_value, structural_gradient, factor_value, factor_gradient, status)` uses the Gauss layout to find the structural rule/point and the sub-element/expansion point of a global point number and returns N_i , ∇N_i , F_τ , ∇F_τ . `COMPOSE_PRODUCT_BASIS` builds $\phi = NF$ and $\nabla \phi = F \nabla N + N \nabla F$.

6.1.3 Strain operator

`BUILD_DISPLACEMENT_OPERATOR(gradient, operator, status)` returns the 6×3 matrix $B(g)$ of Chapter 4 of the Theoretical Guide (the strain operator of the fundamental nucleus), column by column through `BUILD_DISPLACEMENT_COLUMN`:

Strain column of each displacement component (KINEMATICS/mul2_linear_kinematics.for)

```
CASE(1) ! u:  exx = u,x ; gxz = u,z ; gxy = u,y
          COLUMN(1) = GRADIENT(1); COLUMN(4) = GRADIENT(3); COLUMN(6) = GRADIENT(2)
CASE(2) ! v:  eyy = v,y ; gyx = v,x ; gyz = v,z ; gxy = v,x
          COLUMN(2) = GRADIENT(2); COLUMN(5) = GRADIENT(3); COLUMN(6) = GRADIENT(1)
CASE(3) ! w:  ezz = w,z ; gxz = w,x ; gyz = w,y
          COLUMN(3) = GRADIENT(3); COLUMN(4) = GRADIENT(1); COLUMN(5) = GRADIENT(2)
```

6.1.4 Element DOF list

`BUILD_ELEMENT_DOF_LIST` creates, for one element, the local DOF arrays in the order **node**, **field**, **term**: `GLOBAL_DOF(I)` (field-major global number from `GLOBAL_DOF()`), `STRUCTURAL_NODE(I)` (local node $1 \dots N_n$), `FIELD(I)` (1, 2, 3), `TERM(I)` ($1 \dots T$), plus `NODE_INDEX` and `KINEMATIC_INDEX` per local node.

6.2 Choosing the kernel

`BUILD_LINEAR_ELEMENT_MATRICES` is the single entry point used by the assembly. After validation and DOF list it tries the separable kernel (unless `FORCE_GENERAL` is true, i.e. `MUL2_GENERAL_KERNEL=1`) and falls back to the reference kernel if `APPLICABLE` is false.

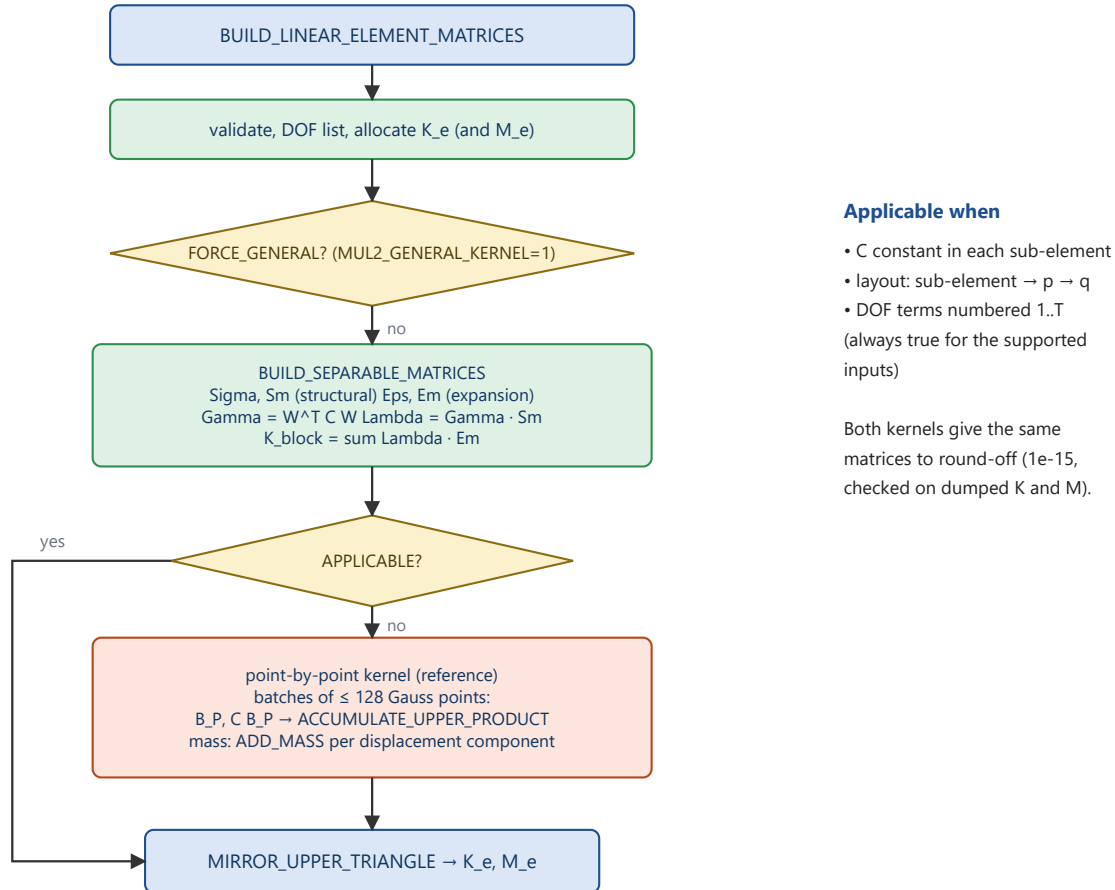


Figure 6.1. Decision flow of the element kernel.

6.3 The separable kernel in detail

Module `MUL2_SEPARABLE_KERNEL`, routine `BUILD_SEPARABLE_MATRICES`. It receives the element DOF arrays and the caches and produces `STIFFNESS` (upper triangle) and `MASS`. The correspondence between the formulas of Chapter 7 of the Theoretical Guide and the variables of the code is:

Table 6.1. Dictionary of `BUILD_SEPARABLE_MATRICES`.

Variable	Shape	Meaning
<code>NS, NP, NE</code>	scalars	structural nodes, structural points P , sub-elements
<code>PE(e)</code>	NE	global number of the first combined point of sub-element e
<code>NQ(e)</code>	NE	expansion points Q_e of sub-element e

Variable	Shape	Meaning
QO(e)	NE	offset of sub-element e in the concatenated list of expansion points; $NQT = \sum NQ$
MAT(e)	NE	material cache index of sub-element e (constant over its points; if not, the kernel returns <code>APPLICABLE=.FALSE.</code>)
SW(p)	NP	$w_p \det J_{s,p}$
SN(p,i), SG(:,p,i)	NP×NS, 3×NP×NS	N_i and ∇N_i (local frame)
STRUCTURAL_AXIS(d)	logical(3)	$d \in S$
CLASS(r)	int(6)	class of strain row r : 1 plain, 1 + s tied by MITC set s
SIGMA(d,p,i,c)	3×NP×NS×NCLASS	σ_d of the Theoretical Guide, Chapter 7 (tied for classes > 1)
SM(i,j,d,d',c,c')	NS×NS×3×3×NCLASS ²	$S_{dd'}^{cc'}(i,j) = \sum_p w_p \sigma_d^c \sigma_{d'}^{c'}$
SNN(i,j)	NS×NS	$\sum_p w_p N_i N_j$ for the mass
BASIS(m)%SPEC, %TERMS, %EPS(q,t,0:3)	per distinct specification m	F_τ (index 0) and ϵ_d (1...3) at all expansion points
WQ(q)	NQT	$w_q \det J_{e,q}$
PRODUCT(a,b)%EM(t,s,k,e)	per pair of specifications	$E_{dd'}^{(e)}(\tau,s)$ with $k = 3(d-1) + d'$, $k = 10$ for FF
W(6,d,f)	6×3×3	$w_{d,f} = B(e_d) Re_f$
GAMMA(f,h,d,d',c,c',e)		$\Gamma^{cc'}$ of Chapter 7 (coupling coefficient), split by row class
LAMBDA(i,j,f,h,k,e)	NS×NS×3×3×10×NE	$\sum_{cc'} \Gamma S$
OFFSET(i,f), TERMS(i,f), SPEC_OF(i,f)	NS×3	first local DOF (minus one), number of terms, index of the specification of the block (node i , field f)

The algorithm is the following (the numbers refer to the code sections):

- Layout recognition.** Walk the Gauss points of the element in the order sub-element $\rightarrow$ structural point $\rightarrow$ expansion point to fill `PE`, `NQ`, `QO`, `ERULE`, `MAT`; return `APPLICABLE=.FALSE.` if the number of points does not match, if some DOF terms are not numbered 1... T , or if the material index changes inside a sub-element.
- Structural factors.** For each structural point read `SW`, `SN`, `SG` from the structural cache (`DETERMINANT`, `DERIVATIVE_LOCAL`) and the rule (`WEIGHT`, `SHAPE`).
- Row classes and SIGMA.** Class 1: $\sigma_d = \partial_d N_i$ if $d \in S$ else N_i . For every MITC set s (class $s+1$): the same quantity evaluated at each tying point from `MITC_DATA%GRADIENT/SHAPE` and interpolated with `INTERPOLATION_WEIGHT`.
- Structural matrices.** Accumulate `SM` and `SNN` over p .
- Distinct specifications.** Group the blocks (node, field) by (family, order, field reference, number of terms): `BASIS(m)`. For each one evaluate all expansion factors at all expansion points with `EVALUATE_POINT_FACTORS` (node 1, point $(e, 1, q)$) and store `EPS(:, :, 0) = F` and `EPS(:, :, d) = F` if $d \in S$ else $\partial_d F$. The expansion weights `WQ` come from the rule and the expansion determinant.
- Coupling coefficients.** Compute `w` from the frame, `GAMMA` from the constitutive matrix `MATERIAL_CACHE%STIFFNESS_LOCAL(:, :, MAT(e))`, then `LAMBDA`.
- Block assembly.** For every block pair $(i, f) \leq (j, h)$: `PREPARE_PRODUCT` (once per pair of specifications) fills `EM` using `ACCUMULATE_PRODUCT`; the block of `STIFFNESS` is $\sum_e \sum_k \text{LAMBDA}(i, j, f, h, k, e) \cdot \text{EM}(:, :, k, e)$; for the mass, if $f = h$, $\sum_e \rho_e \text{SNN}(i, j) \text{EM}(:, :, 10, e)$.
- The caller mirrors the upper triangle (`MIRROR_UPPER_TRIANGLE`).

NOTE

The coupling of a Lagrange expansion with a Taylor expansion at different nodes is handled automatically: the pair of specifications selects the correct product `EM(a,b)`; the matrices of mixed pairs are not zero.

6.4 The reference (point-by-point) kernel

The reference kernel is the direct implementation of the element integral at the start of Chapter 7 of the Theoretical Guide. Its structure:

Reference kernel, structure (ELEMENTS/mul2_element_matrices.for, abridged)

```
DO WHILE (FIRST_POINT .LE. ELEMENT_LAST)      ! batches of up to BATCH points
DO POINT = FIRST_POINT, LAST_POINT
  STIFFNESS_LOCAL = WEIGHT * C(material of POINT) ! w C
  DO I = 1, N_DOF
    CALL EVALUATE_LOCAL_DOF(POINT, I, ..., BASIS_VALUE(Q,I), B_COLUMN, ...)
    B_TEST (6*Q+1:6*Q+6, I) = B_COLUMN          ! b_I
    B_WEIGHTED(6*Q+1:6*Q+6, I) = MATMUL(STIFFNESS_LOCAL, B_COLUMN) ! w C b_I
  END DO
END DO
CALL ACCUMULATE_UPPER_PRODUCT(B_TEST, B_WEIGHTED, 6*COUNT, STIFFNESS) ! K += B^T (w C B)
IF (MASS_REQUESTED) CALL ADD_MASS(...)
END DO
```

`EVALUATE_LOCAL_DOF` evaluates the factors at the point, composes the basis, builds the strain column with `BUILD_DISPLACEMENT_OPERATOR` and multiplies by the frame column of the field (`GLOBAL_TO_LOCAL(:, FIELD, element)`); if MITC is active the tied rows are replaced by `MITC_TIE_COLUMN`. Processing the points in *batches* (the points of a batch are stacked in the rows of `B_TEST` and `B_WEIGHTED`, up to 128 points and 2 million entries per array) turns the accumulation into one large matrix product instead of one 6×6 nucleus per pair of DOFs. `ADD_MASS` computes the mass block of each displacement component as a product of the basis values (`BASIS_VALUE(point, dof)`) weighted by $w\rho$.

6.5 Dense products

`MUL2_DENSE_PRODUCTS` contains the two loops used by both kernels, written without array temporaries (so that they are safe in threads with small stacks):

- `ACCUMULATE_UPPER_PRODUCT(A, B, ROWS, MATRIX, INDEX)`: $M_{IJ} += \sum_q A_{qi} B_{qj}$ for $I \leq J$ only; optional `INDEX` selects (ascending) the columns of `A`, `B` and their position in `MATRIX` (used for the mass blocks of one field).
- `ACCUMULATE_PRODUCT(A, B, ROWS, MATRIX)`: the same for all I, J (used for the expansion products).

Both process four columns of the right factor at a time, so that each streamed column of `A` is reused four times; the inner loop over the integration rows is contiguous and vectorised.

6.6 MITC in the code

`MUL2_MITC` contains the tying tables. `DEFINE_TYING(topology, data)` sets, for each topology, `ROW_SET(6)` (0 = untied, s = tied by set s) and the tying sets (`COUNT_1D`, `COORDINATE_1D`: the tensor product of one list per direction). `MITC_PREPARE_ELEMENT` evaluates shapes and local gradients at all tying points and stores them in `SHAPE(node, point, set)` and `GRADIENT(3, node, point, set)`. `INTERPOLATION_WEIGHT(set, natural, weights)` evaluates the tensor Lagrange weights M_I . The reference kernel uses `MITC_TIE_COLUMN` per DOF; the separable kernel builds `SIGMA` for the tied classes once per structural point.

6.7 Reduced and selective integration

`REDI` and `SELI` (`ANALYSIS.dat`, see Chapter 8 of the Theoretical Guide) are implemented without touching the formulas of the kernels, by **changing the integration set** and the constitutive matrix they read.

1. **Rules.** `BUILD_REFERENCE_RULE_DATABASE` adds, for the structural elements of a dimension that asks for `REDI` / `SELI`, a second rule of the same topology with the marker `ORDER = -1` (`BUILD_REDUCED_QUADRATURE`: one point per direction less, `REDUCED_POINTS_PER_DIRECTION`). Expansion elements never use it, so a thickness line B3 keeps its rule. `FIND_STRUCTURAL_RULE` returns the reduced rule when asked and the default one otherwise.
2. **Second set.** `BUILD_MODEL_CACHE` builds, when some dimension is reduced, a second `REDUCED_SET_TYPE` (Gauss layout, structural geometry, combined geometry and point-material map). The expansion geometry and the

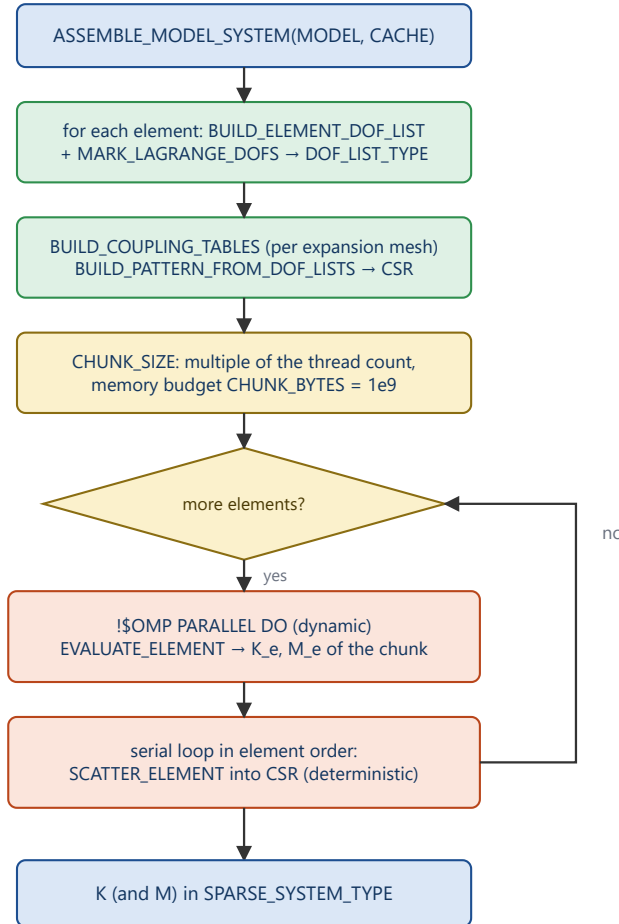
material cache are shared.

3. **Constitutive parts.** `STIFFNESS_PART(CACHE, INDEX, PART, DIM)` returns C , or its normal part `PART_NORMAL`, or its shear part `PART_SHEAR`; the shear part contains every entry whose row or column is a reduced strain (beam YZ and XY, plate XZ and YZ, solid XZ YZ XY), so the two parts add up to C .
4. **Assembly.** `EVALUATE_ELEMENT (MUL2_MODEL_ASSEMBLY)` calls `BUILD_LINEAR_ELEMENT_MATRICES` once for `REDI` (reduced set, full C , mass included) and twice for `SELI` (full set with `PART_NORMAL` and the mass; reduced set with `PART_SHEAR`, stiffness only; the two stiffness matrices are added). Both the separable and the reference kernel receive `PART` and `WITH_STIFFNESS`.
5. **Modal solver.** With `REDI` the mass is singular: `SOLVE_DENSE` falls back to $Mx = \mu Kx$ when LAPACK reports that M is not positive definite (ARPACK shift-invert accepts a semi-definite M).

Tests: `MUL2_ANALYSIS_TEST` compares beam and plate, static (displacements, stresses, VTK) and modal, with the baseline program (cases `*_redi`, `*_seli` in `TESTS/CASES`); `MUL2_REDUCED_TESTS` (`TESTS/REDUCED/reduced_tests.py`) checks the format of `ANALYSIS.dat`, the patch tests, the exact pure bending, the removal of locking and the modal analysis with a singular mass.

Assembly

`ASSEMBLE_MODEL_SYSTEM(MODEL, CACHE, SYSTEM, STATUS, WITH_MASS)` (module `MUL2_MODEL_ASSEMBLY`) builds the global stiffness (and optionally mass) matrix. It never forms a dense global matrix and never needs all element matrices in memory at the same time.



Why a serial scatter?

Floating-point sums are not associative. Scattering the chunk in element order fixes the order of every addition: the result is bit-identical for any thread count (tested with 1 and 4).

Figure 7.1. Assembly flow.

7.1 Step 1: DOF lists

For every element `BUILD_ELEMENT_DOF_LIST` fills the local DOF arrays; the global numbers are moved into a `DOF_LIST_TYPE` (`GLOBAL_DOF`), and `MARK_LAGRANGE_DOFS` adds two arrays that describe the *Lagrange* DOFs:

- `TABLE(k)` : index of the expansion mesh if the DOF belongs to a Lagrange expansion, 0 otherwise (Taylor DOFs are coupled with everything);
- `TERM(k)` : the expansion node (term) of the DOF.

`BUILD_COUPLING_TABLES` creates, for every expansion mesh, the logical matrix `COUPLED(t,s)` : true if the expansion nodes t and s belong to a common sub-element. It is $T \times T$ (81^2 for a 4×4 Q9 section) and is computed by scanning the sub-elements and locating each of their nodes in the node list.

7.2 Step 2: pattern

`BUILD_PATTERN_FROM_DOF_LISTS(total_dof, lists, system, status, with_mass, tables)` builds the CSR pattern without any dense intermediate:

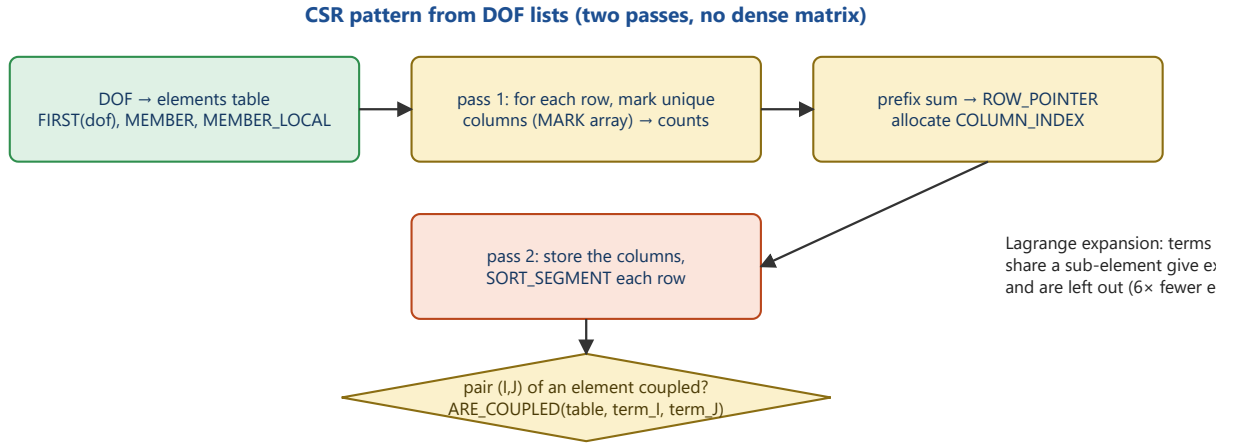


Figure 7.2. Two-pass construction of the pattern.

1. Build the *DOF* → *elements* table: `FIRST(dof)` (CSR pointer), `MEMBER(1)` (element) and `MEMBER_LOCAL(1)` (position of the DOF in the element list).
2. For every row (DOF) collect the unique columns: loop over the elements of the DOF and over the DOFs of each element, mark visited columns with the array `MARK(column) = row` (so no clearing is needed between rows), skip pairs for which `ARE_COUPLED` is false. **Pass 1** only counts, giving `ROW_POINTER` after a prefix sum; **pass 2** stores the columns and sorts each row with `SORT_SEGMENT` (insertion sort for short rows, heap sort otherwise).
3. Allocate `STIFFNESS` (and `MASS` if requested) with the same length.

`ARE_COUPLED(list, tables, k0, k1)` is `.FALSE.` only if both DOFs are Lagrange DOFs of the same table and `COUPLED(term_k0, term_k1)` is false. For Taylor expansions every pair is coupled.

7.3 Step 3: chunks and parallel evaluation

The element matrices of a big model do not fit in memory, so the elements are processed in **chunks**:

Chunked evaluation (`ANALYSES/mul2_model_assembly.for`, abridged)

```

CHUNK_SIZE = ...      ! multiple of the thread count, bounded by CHUNK_BYTES / (bytes per element)
DO FIRST = 1, N_ELEMENT, CHUNK_SIZE
  LAST = MIN(N_ELEMENT, FIRST + CHUNK_SIZE - 1)
  !$OMP PARALLEL DO DEFAULT(SHARED) PRIVATE(I) SCHEDULE(DYNAMIC,1)
  DO I = 1, LAST - FIRST + 1
    CALL EVALUATE_ELEMENT(MODEL, CACHE, FIRST+I-1, CHUNK(I), CHUNK_STATUS(I), MASS_REQUESTED, FORCE_GENERAL)
  END DO
!$OMP END PARALLEL DO
  DO I = 1, LAST - FIRST + 1
    ! serial, in element order
    CALL SCATTER_ELEMENT(SYSTEM, LISTS(FIRST+I-1)%GLOBAL_DOF, CHUNK(I)%STIFFNESS, CHUNK(I)%MASS, ...)
    CALL CLEAR_ELEMENT_MATRIX(CHUNK(I))
  END DO
END DO

```

`EVALUATE_ELEMENT` is thread-safe: it only reads the shared databases, prepares the MITC data of the element (`PREPARE_ELEMENT_MITC`) and calls `BUILD_LINEAR_ELEMENT_MATRICES`. The memory budget `CHUNK_BYTES` is 10^9 bytes; the bytes per element are $8n^2$ if only K is built and $16n^2$ with M (n = local DOFs). The number of elements per chunk is a multiple of the number of threads when memory allows, so that no thread idles inside a chunk.

7.4 Step 4: scatter

`SCATTER_ELEMENT(system, global_dof, stiffness, mass, status)` adds the dense element matrices to the CSR arrays. For every local pair (I, J) :

1. If the stiffness entry is zero and the mass entry is zero (or absent) the pair is **skipped**: it is a structural zero.
2. Otherwise `FIND_CSR_POSITION` finds $(\text{global_dof}(I), \text{global_dof}(J))$ by binary search in the sorted row and the values are added. If the position does not exist the routine raises `ENTRY IS MISSING FROM CSR PATTERN`, which guarantees that a non-zero entry never falls outside the pattern (a safeguard of the coupling tables).

The serial loop and the fixed element order make the sums reproducible for any number of threads.

7.5 Reduction for the modal problem

`BUILD_REDUCED_PATTERN(order, row_pointer, column_index, constrained, pattern)` creates the pattern of the sub-matrix of the free DOFs: `FREE_TO_GLOBAL(f)` maps free to global numbers, `ROW_POINTER / COLUMN_INDEX` describe the reduced matrix and `SOURCE_POSITION(k)` is the position in the full CSR of the k -th reduced entry. `REDUCE_VALUES(pattern, full_value, reduced_value)` copies values through the map; `EXPAND_VECTOR(pattern, reduced, full)` scatters a reduced vector back (constrained entries zero). `SPARSE_MULTIPLY` computes $y = Ax$ for the full CSR (row-parallel).

7.6 Complexity and memory

For a model with n_{DOF} unknowns and r non-zeros per row:

- the pattern needs $O(n_{DOF}r)$ operations and $16 n_{DOF}r$ bytes ($r \approx 200$ for the Lagrange benchmark, ≈ 2000 for Taylor-15);
- the element evaluation costs $\sum_e O(PN^2 + QT^2 + n^2)$ (separable kernel);
- the scatter costs $\sum_e n^2 \log r$ (binary search) and is the only serial step. In the 100 000-DOF Taylor-15 benchmark it takes about 7 s of the 12 s of the assembly, so it is now the dominant part and the natural target of future optimisation (for instance a parallel scatter by row ranges).

Constraints, loads and solvers

8.1 Boundary data

`BUILD_MECHANICAL_BOUNDARY_DATA(boundaries, nodes, elements, kinematics, expansions, frames, dof_layout, tolerance, constraints, force, status)` (module `MUL2_BOUNDARY_APPLICATION`) converts the records of `BC.dat` into

- `CONSTRAINT_SET_TYPE`: `ACTIVE(dof)` (logical), `VALUE(dof)` (prescribed value), `COUNT`;
- `FORCE(dof)`: the global load vector.

The geometric tolerance is 10^{-9} (relative, with a scale of at least one).

8.1.1 Planes

`ADD_PLANE_CONSTRAINT(plane, component_active, component_value, ...)` loops over all nodes. For a node it takes the *first incident element* (`FIRST_ELEMENT`, from the incidence table) to know the expansion mesh and the frame, computes the physical position of every expansion node (`EXPANDED_POINT`: `node + LOCAL_TO_GLOBAL × local coordinate`) and tests `POINT_ON_PLANE`. For each active displacement component:

- Lagrange: every term whose point is on the plane is constrained (`SET_DOF_CONSTRAINT(GLOBAL_DOF(...), value)`).
- Taylor: if all section points are on the plane, term 1 gets the value and the others zero; otherwise the error `PARTIAL PLANE CONSTRAINT REQUIRES MPC FOR TE`.
- Hierarchical / user-defined: explicit "not implemented" error.

`SET_DOF_CONSTRAINT` refuses two different values for the same DOF (`CONFLICTING PRESCRIBED VALUES`). If a plane selects no DOF a warning is issued.

8.1.2 Point loads

`ADD_POINT_FORCE(point, load, ...)` scans nodes and expansion nodes for the one whose physical position coincides with `point` (relative tolerance). Lagrange: `FORCE(GLOBAL_DOF(node, field, term)) += load(field)` where the term is the expansion node. Taylor: for every term τ , `FORCE(...) += load(field) × EVALUATE_TAYLOR_TERM(...)` evaluated at the coordinates of the expansion node. Failure: `POINT LOAD LOCATION WAS NOT FOUND`.

8.1.3 Static elimination

`APPLY_STATIC_CONSTRAINTS(system, force, constraints, status)` performs the two loops of the algorithm of Chapter 9 of the Theoretical Guide in place:

1. $F_r \leftarrow F_r - \sum_{c \in \text{constrained}} K_{rc} q_c$ for every row;
2. for every constrained row or column set the entry to 0; set the constrained diagonal to 1 and the right-hand side to q_r .

The CSR pattern is unchanged. A constrained row without a diagonal entry would be an error (`CONSTRAINED CSR ROW HAS NO DIAGONAL`); the pattern builder guarantees the diagonal.

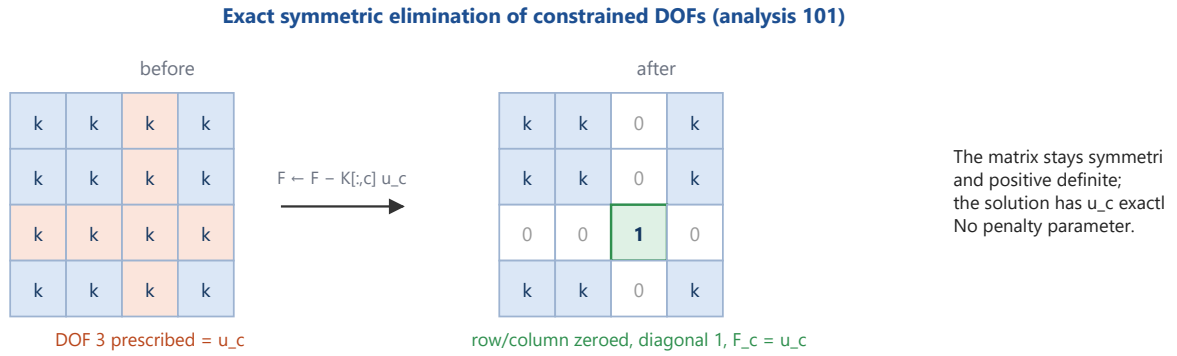


Figure 8.1. Elimination of constrained DOFs.

8.2 Analysis 101

`RUN_STATIC_ANALYSIS(model, cache, results, status):`

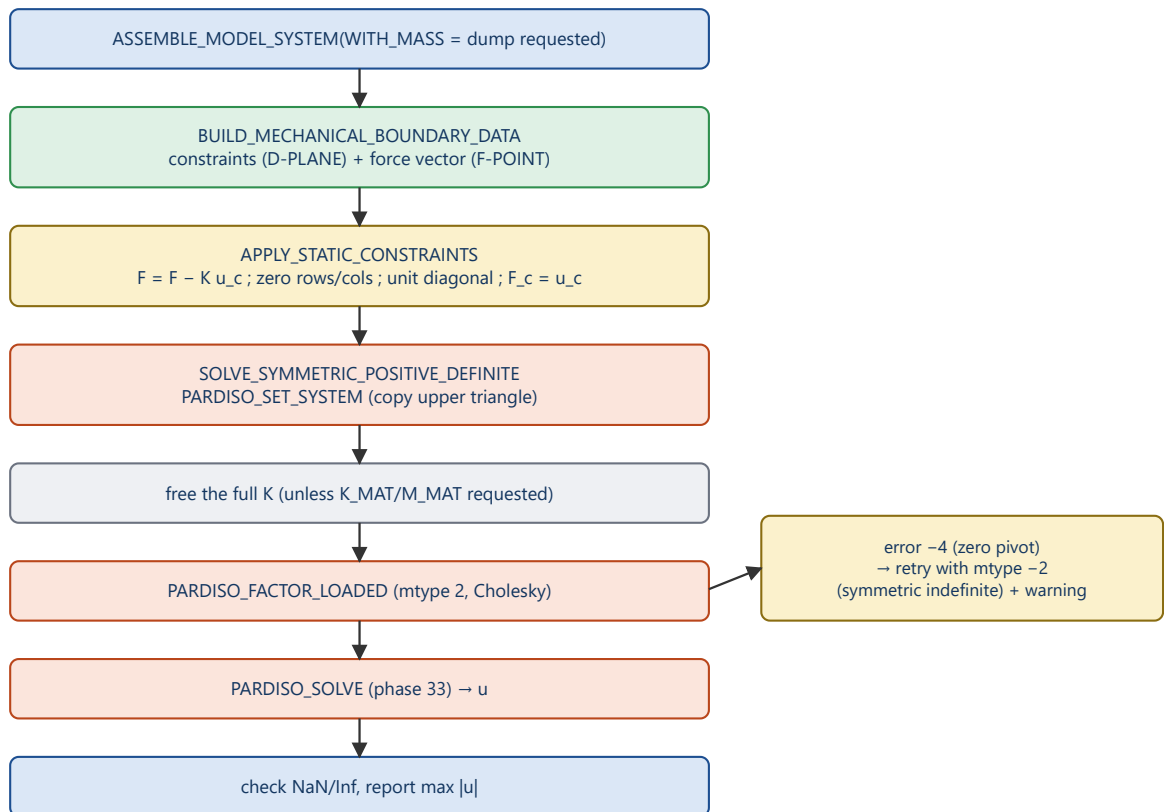


Figure 8.2. Static analysis flow.

The solve is done by `SOLVE_SYMMETRIC_POSITIVE_DEFINITE(system, rhs, solution, status, release_system):`

1. `PARDISO_SET_SYSTEM`: copies the upper triangle (`EXTRACT_UPPER_TRIANGLE`) into the handle `PARDISO_FACTOR_TYPE` (arrays `ROW_POINTER`, `COLUMN_INDEX`, `VALUE`);

2. if `release_system` is true (no K/M dump requested) the caller's `STIFFNESS`, `COLUMN_INDEX`, `ROW_POINTER` and `MASS` are deallocated, saving half of the matrix memory during the factorisation;
3. `PARDISO_FACTOR_LOADED(mtype, factor, status)`: PARDISO phase 12 (reordering, symbolic and numerical factorisation). On error code `-4` with `mtype = 2` the internal memory is freed (`PARDISO_FREE_INTERNAL`) and the factorisation is retried with `mtype = -2` and a warning;
4. `PARDISO_SOLVE`: phase 33, with `IPARM` at defaults;
5. `PARDISO_RELEASE`: phase `-1` and deallocation.

`PARDISO_HINT(error)` translates the numeric error codes into the messages shown to the user (`-2` not enough memory, `-3` reordering failed, `-4` zero pivot, `-5/-6` internal).

The routine then checks the solution for NaN and infinity and logs `MAX |U|`.

8.3 Analysis 103

`RUN_MODAL_ANALYSIS(model, cache, results, status)`:

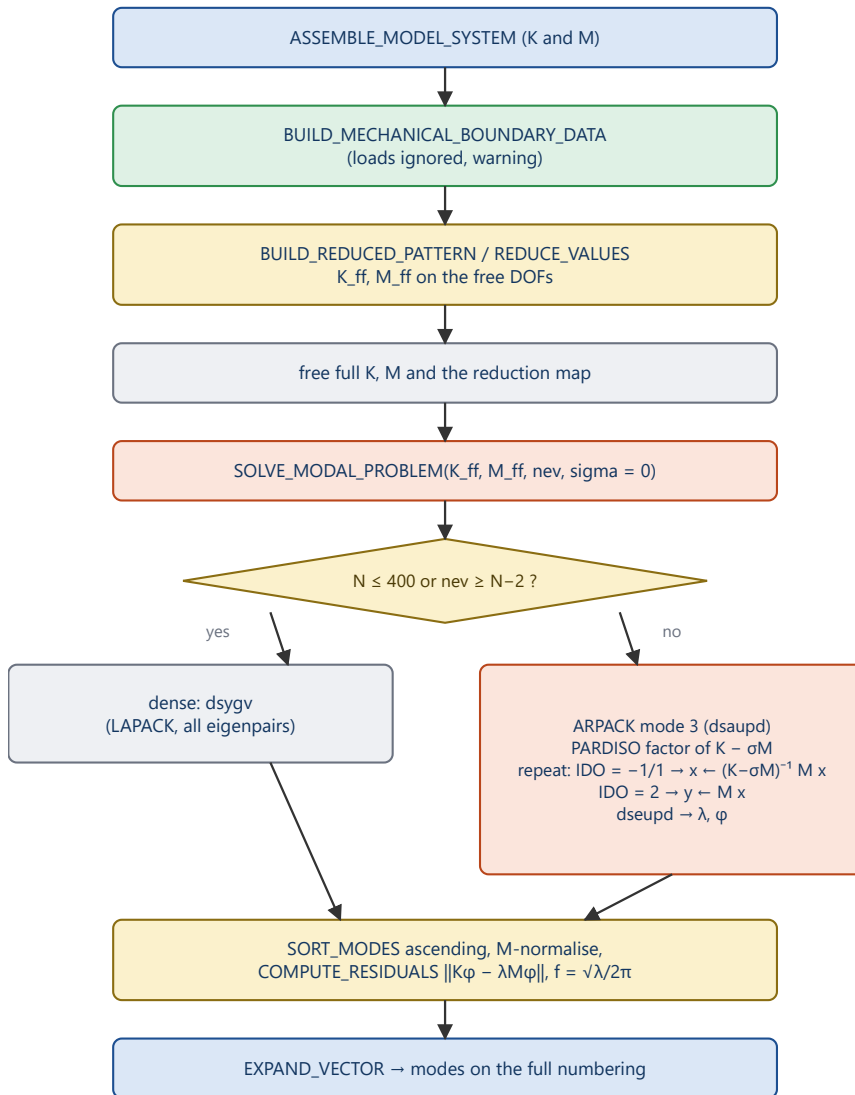


Figure 8.3. Modal analysis flow.

`SOLVE_MODAL_PROBLEM(order, row_pointer, column_index, stiffness, mass, nev, sigma, eigenvalue, vector, residual, info, status)` chooses between `SOLVE_DENSE` (`order ≤ DENSE_ORDER_LIMIT = 400` or `nev ≥ order-2`) and `SOLVE_ARPACK`. After either, `SORT_MODES` sorts ascending and `COMPUTE_RESIDUALS` evaluates $\|K\phi - \lambda M\phi\| / \|K\phi\|$ with `SPARSE_MULTIPLY`.

8.3.1 The ARPACK reverse-communication loop

`SOLVE_ARPACK` builds `SHIFTED = K -  $\sigma$ M` (same pattern), factorises it with `PARDISO_FACTORIZE` (`mtype 2` if $\sigma = 0$, else `-2`) and runs the `dsaupd` loop:

ARPACK mode 3 loop (SOLVERS/mul2_modal_solver.for, abridged)

```
IPARAM(1) = 1; IPARAM(3) = 500; IPARAM(7) = 3      ! exact shifts, max restarts, mode 3
IDO = 0
DO
  CALL DSAUPD(IDO, 'G', N, 'LM', NEV, TOL, RESID, NCV, V, N, IPARAM, IPNTR, WORKD, WORKL, LWORKL, INFO)
  SELECT CASE (IDO)
  CASE (-1, 1)    ! y <- (K - sigma M)^-1 M x
    CALL SPARSE_MULTIPLY(..., MASS, WORKD(IPNTR(1)), TEMPORARY)      ! M x
    CALL PARDISO_SOLVE(FACTOR, TEMPORARY, WORKD(IPNTR(2)), STATUS)    ! solve with the factors
  CASE (2)        ! y <- M x
    CALL SPARSE_MULTIPLY(..., MASS, WORKD(IPNTR(1)), WORKD(IPNTR(2)))
  CASE DEFAULT
    EXIT
  END SELECT
END DO
CALL DSEUPD(.TRUE., 'A', SELECT, D, V, N, SIGMA, 'G', ...)           ! eigenvalues, vectors
```

`IPNTR` points into `WORKD` for the input and the output vector; `INFO` tells the number of converged values; the eigenvalues of the original problem are recovered as $\lambda = \sigma + 1 / \mu$ by `dseupd`. `NCV = min(N, max(2 nevw + 1, nevw + 10))` Lanczos vectors are used. The ARPACK sources are compiled with 64-bit `INTEGER` and `LOGICAL` so that the `I8` arrays can be passed unchanged; the interface blocks in the module declare the external routines with `INTEGER(I8)` and `LOGICAL(I8)` arguments.

8.3.2 Dense fallback

`SOLVE_DENSE` expands the CSR matrices into dense `A(N,N)`, `B(N,N)` and calls LAPACK `DSYGV(1, 'V', 'U', ...)`; the first `nevw` eigenpairs are returned.

8.4 Results and memory in the modal analysis

After the reduced matrices are built, `RUN_MODAL_ANALYSIS` frees `PATTERN%SOURCE_POSITION` and, unless `KMAT/MMAT` were requested, the full system. The solution vectors are stored in `RESULTS%MODE(order, k)` after `EXPAND_VECTOR`; `FREQUENCY(k) = sqrt(max(0,  $\lambda$ ))/(2 $\pi$ )`.

Post-processing

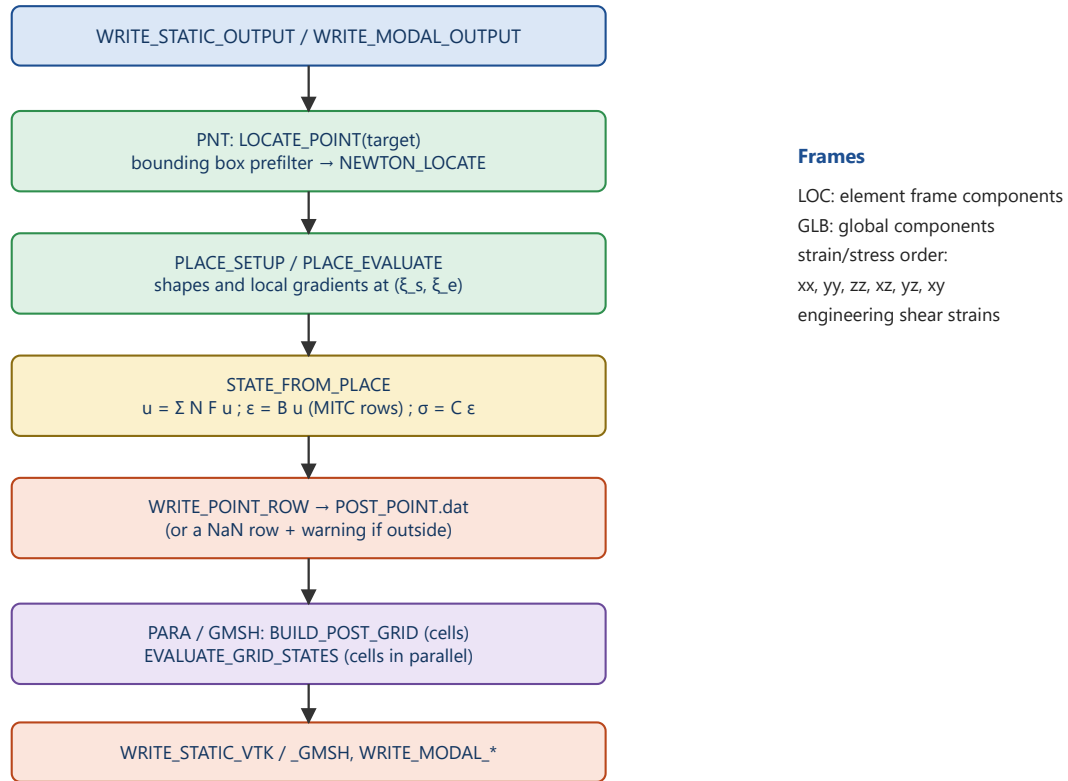


Figure 9.1. Flow of the post-processing.

9.1 Evaluation context: `PLACE_TYPE`

To evaluate fields at an arbitrary point the program needs, for one (element, sub-element) pair, the node coordinates, the frame and the shape functions. `PLACE_TYPE` (module `MUL2_RECOVERY`) holds them:

Component	Meaning
<code>ELEMENT</code> , <code>MESH</code> , <code>SUB_ELEMENT</code>	which element, expansion mesh, sub-element
<code>STRUCTURAL_DIMENSION</code> , <code>EXPANSION_DIMENSION</code> , <code>STRUCTURAL_TOPOLOGY</code> , <code>EXPANSION_TOPOLOGY</code>	dimensions and topologies
<code>STRUCTURAL_AXIS(3)</code> , <code>EXPANSION_AXIS(3)</code>	active local axes
<code>NODE_GLOBAL</code> , <code>NODE_LOCAL</code> , <code>EXPANSION_NODE</code>	coordinates of the structural nodes (global and local) and of the sub-element nodes
<code>SHAPE_STRUCTURAL</code> , <code>NATURAL_DERIVATIVE_S</code> , <code>GRADIENT_STRUCTURAL</code>	structural shapes, natural derivatives, local gradients at the current point
<code>SHAPE_EXPANSION</code> , <code>NATURAL_DERIVATIVE_E</code> , <code>GRADIENT_EXPANSION</code>	the same for the sub-element
<code>EXPANSION_POINT_LOCAL</code> , <code>POINT_GLOBAL</code> , <code>NATURAL_S</code>	local section point, physical point, structural natural coordinates
<code>MITC</code>	MITC data of the element when active

`PLACE_SETUP(model, cache, element, sub_element, place, status)` fills the constant part; `PLACE_EVALUATE(place, model, cache, natural_s, natural_e, with_gradient, status)` evaluates shapes (`EVALUATE_SHAPE_AND_GRADIENT`), physical position and, if requested, gradients at a natural point.

9.2 Locating a point

`LOCATE_POINT(model, cache, target, tolerance, element, sub_element, natural_s, natural_e, found, status)` implements the two-step search of Chapter 12 of the Theoretical Guide:

1. `POINT_NEAR_ELEMENT(model, element, target, margin)`: bounding-box test on the structural nodes enlarged by the section extent.
2. For each candidate (element, sub-element): `NEWTON_LOCATE`, with the Jacobian of the combined map (3×3 , `INVERT_3X3`), starting at the centre of the reference domain; converged when the correction is below the tolerance. `NATURAL_INSIDE(topology, natural, tolerance)` checks that the coordinates are inside the reference element.

For a beam the three unknowns are the axial natural coordinate (structural) and the two section coordinates (expansion); for a plate the two surface coordinates and the thickness coordinate; for a solid the three structural coordinates.

9.3 Displacement, strain and stress

`STATE_FROM_PLACE(model, cache, place, solution, state, status)` loops over the DOFs of the element: for each (node, field, term) it evaluates the expansion factor, composes the basis with the structural shape, adds the displacement contribution Nf_q and the strain column (`BUILD_DISPLACEMENT_OPERATOR` with the local gradient and the frame column of the field; MITC rows via `MITC_TIE_COLUMN`). Then:

Stress recovery (`POST/mul2_recovery.for`, concept)

```
STATE%DISPLACEMENT = sum_I phi_I * q_I           ! global components
STATE%STRAIN_LOCAL  = sum_I b_I  * q_I           ! element frame
STATE%STRESS_LOCAL  = MATMUL( C_LOCAL(lamination), STATE%STRAIN_LOCAL )
STATE%STRAIN_GLOBAL = TRANSFORM_INVERSE( R ) * STRAIN_LOCAL
STATE%STRESS_GLOBAL = (stress rotation) * STRESS_LOCAL
```

`DISPLACEMENT_FROM_PLACE` does only the displacement part for several vectors at once (the mode shapes).

9.4 Output grid

`BUILD_POST_GRID(model, nodes_per_cell, split, grid, status)` creates, for every element and sub-element, cells that sample the natural domain with $r \times s \times t$ divisions. `ELEMENT_SPLITS` and `SPLIT_NATURAL` map the nine integers of the request onto the structural and expansion natural coordinates of the element family (table of Chapter 12 of the Theoretical Guide); `CELL_LIMITS` gives the natural coordinates of the nodes of cell i of n along one direction; triangular parents use `COLLAPSE_TRIANGLE`. The grid stores `CELL_ELEMENT`, `CELL_SUB_ELEMENT`, `CELL_DIMENSION`, and for every grid node its natural coordinates `NATURAL_STRUCTURAL` and `NATURAL_EXPANSION`.

`EVALUATE_GRID_STATES` evaluates the states of all cells with `!$OMP PARALLEL DO` over cells (dynamic schedule, chunks of 16): each cell sets up its own `PLACE_TYPE` and writes its own entries of `STATE`.

9.5 Writers

- `WRITE_VTK_GEOMETRY`, `WRITE_VTK_VECTOR`, `WRITE_VTK_SCALAR`: legacy ASCII VTK (`UNSTRUCTURED_GRID`, cell types 12 and 25, `POINT_DATA` vectors and scalars).
- `WRITE_GMSH_GEOMETRY`, `WRITE_GMSH_DATA`: GMSH 2 ASCII (`$Nodes`, `$Elements`, `$NodeData`).
- `SELECT_FRAME` chooses local or global components of displacement, strain and stress for the request.
- `WRITE_CSR(file, system, value, status)` writes row column value lines; `WRITE_VECTOR` writes value lines.

The modal output always samples one quadratic cell per element (`EVALUATE_GRID_MODES`) and writes one displacement field per mode, with the frequency in the field title.

Testing and validation

10.1 Test programs

Table 10.1. Test executables and what they check.

Executable (source)	Level	Content
MUL2_TESTS (TESTS/mul2_tests.for)	unit and integration	base utilities, quadrature, shape functions (partition of unity, derivative consistency), Jacobians and frames, Taylor bases, materials and rotations, DOF layout, Gauss layout/geometry, element matrices (symmetry, rigid-body energy, patch), sparse assembly, boundary conditions
MUL2_PARDISO_TESTS (TESTS/mul2_pardiso_tests.for)	solver	factorisation and solution of small SPD systems, error paths
MUL2_ANALYSIS_TESTS (TESTS/mul2_analysis_tests.for, mul2_test_support.for)	end-to-end	runs the real executable on test inputs in isolated directories and compares point results, VTK fields and frequencies with references
CHECK_FIXED_FORM	static check	no source line longer than 72 columns

All are registered in CTest (`ctest --test-dir BUILD/WINDOWS_IFX -C Release`). The end-to-end test passes the executable, the repository root and a run directory as arguments.

10.2 The end-to-end suite

`mul2_test_support.for` provides the helpers: `RUN_SOLVER(exe, input, run_dir, threads, ok)` executes the program through `EXECUTE_COMMAND_LINE` (with `OMP_NUM_THREADS` when requested), `READ_POINT_ROWS`, `READ_FREQUENCIES`, `READ_VTK_BLOCK` parse the outputs, `MAX_RELATIVE_DIFFERENCE` compares arrays and `FILES_ARE_IDENTICAL` compares two files line by line.

The cases of `TESTS/mul2_analysis_tests.for`:

Table 10.2. End-to-end cases.

Case	Source of the reference	Checks
GOLDEN_BEAM_101, GOLDEN_PLATE_101, GOLDEN_BEAM_103	REFERENCES/GOLDEN (baseline outputs)	points, VTK displacement/stress/strain, frequencies and MAC of 20 modes
mixed_none, mixed_mitc	TESTS/CASES/<name>	B4 + Q9 + H27 in one model
beam_b3_te2_mitc, beam_te8_unit	TESTS/CASES	Taylor expansions; the TE8 case is the regression test of the quadrature order
plate_q4_mitc, plate_q9x4_mitc, solid_h8_none	TESTS/CASES	plates and solids
modal_plate_q9x4_mitc, modal_beam_b3_te2	TESTS/CASES	frequencies
patch_*	analytic (uniform strain, $\nu = 0$)	exact reproduction of a uniform strain
thread independence	the same run with 1 and 4 threads	outputs bit-identical

10.2.1 Tolerances

Tolerances are relative to the largest value of the field. The default is $2 \cdot 10^{-6}$ (limited by the penalty used in the baseline); point results use 10^{-4} because the baseline locates points only approximately; thin plates are ill-conditioned and use $4 \cdot 10^{-6}$; patch tests use 10^{-9} .

10.2.2 Creating a new reference case

1. Write the input directory `TESTS/CASES/<name>/INPUT`.
2. Run the *baseline* program on it (`MUL2_OPTIMIZED/INTERFACE/MUL2.exe`) in a scratch directory with `PATH_input.dat` containing `INPUT` and the output directories created.
3. Copy `STATIC/POST_POINT.dat`, `STATIC/RESULTS_PARA_01.vtk` (static) or `DYNAMIC/FREQUENCIES.dat`, `DYNAMIC/RESULTS_DYN_PARA.vtk` (modal) into `TESTS/CASES/<name>/REFERENCE`.
4. Add a `GENERATED_STATIC('<name>', tolerance)` or `GENERATED_MODAL(...)` line to the test program and rebuild.

Features that the baseline does not have (such as hierarchical expansions) are validated against internal equivalences (for example two bases spanning the same space must give the same displacements) instead.

10.3 Comparing the two kernels

Setting `MUL2_GENERAL_KERNEL=1` runs the reference kernel. To compare the two kernels, request `KMAT` and `MMAT` in `POSTPROCESSING.dat`, run once with and once without the variable, and compare `WORK/K_MAT.dat` and `WORK/M_MAT.dat` numerically. The comparison is made in the tests of the project at 10^{-15} (relative to the largest entry).

10.4 Benchmarks

`TESTS/BENCH` contains four inputs of about 100 000 DOF (`beam_te15_100k`, `beam_le_100k` and their modal versions) and the script `bench.ps1`, which runs an executable on a case in a clean run directory and reports exit code, wall time, CPU time and the peak working set:

```
powershell TESTS\BENCH\bench.ps1 -Exe BUILD\WINDOWS_IFX\Release\MUL2_V3.exe ^
-Case TESTS\BENCH\beam_te15_100k -RunDir C:\temp\run_new
```

Results are in the Theoretical Guide, Chapter 13.

10.5 Debugging tips

- Build the `Debug` configuration: array bounds, uninitialised variables and floating-point traps are checked and a traceback is printed on errors.
- `MUL2_DEBUG=1` enables debug lines from the logger.
- To isolate an element problem, run a model with one element and `KMAT` / `FRCE` / `UNKN` requests and inspect the dumps in `WORK`.
- Large Gauss point counts or odd memory use: check `PREPROCESSING: ... DOF, ... GAUSS POINTS` in the console and `NNZ` in the assembly line.
- A PARDISO error `-4` is almost always a modelling error (supports) before it is a solver problem.

Extending the program

This chapter lists, for the three most common extensions, **every place that must be touched**. It also shows how the layered design keeps the changes small.

11.1 Adding a structural element topology

Example: a new quadrilateral with 8 serendipity nodes.

1. **Registry** (MODEL/mul2_topologies.for): add a code constant, the name in `TOPOLOGY_FROM_NAME`, the node count in `TOPOLOGY_NODE_COUNT` and the dimension in `TOPOLOGY_NATURAL_DIMENSION`.
2. **Shape functions** (FEM/mul2_shape_functions.for): add a case in `EVALUATE_SHAPE` and a routine computing N_a and $\partial N_a / \partial \xi_a$ with the node order you want to document. Verify partition of unity and derivative consistency with the unit tests.
3. **Quadrature** (QUADRATURE/mul2_quadrature.for): add the default rule in `BUILD_DEFAULT_QUADRATURE`; if the topology is tensor-product also in `DEFAULT_POINTS_PER_DIRECTION` and `BUILD_ORDERED_QUADRATURE` so that Taylor expansions can use more points.
4. **Element frame** (GEOMETRY/mul2_element_frames.for): for surfaces, state which two corners define the plate normal (`BUILD_SURFACE_FRAME`); for beams, the end node (`BUILD_BEAM_FRAME`).
5. **MITC** (optional, ELEMENTS/mul2_mitc.for): add the tied rows and tying points in `DEFINE_TYING` and `MITC_IS_AVAILABLE`.
6. **Post-processing** (POST/mul2_post_grid.for, POST/mul2_recovery.for): the natural-coordinate handling of `NATURAL_INSIDE` and the cell mapping if the reference domain is new.
7. **Tests**: a patch test, a comparison of the new element with an equivalent existing one (for instance Q8 versus Q9 on a rectangular mesh) and the documentation tables.

No change is needed in the kernels, the assembly or the solvers: they only see shape functions, gradients and weights.

11.2 Adding an expansion family

Example: the hierarchical Legendre family, now implemented (Chapter 12); it shows the hooks that a new family must fill:

Table 11.1. Where an expansion family enters the code.

Place	Routine	Action for a new family
Parser	PARSE_EXPANSION_TOKEN (MODEL/mul2_kinematics.for)	recognise the token and set <code>FAMILY</code> , <code>ORDER</code> (done for HLE)
DOF count	NODE_FIELD_TERM_COUNT (ASSEMBLY/mul2_dof_layout.for)	return the number of terms of the field at the node (for HLE the number of terms of the mesh, <code>N_TERM</code>)
Basis	EVALUATE_EXPANSION_FACTOR (CUF/mul2_point_bases.for)	return F_τ and its local gradient at an expansion point
Quadrature order	BUILD_EXPANSION_POINT_ORDERS (ANALYSES/mul2_model_cache.for)	the Gauss points per direction needed for the products of two functions
Sparse pattern	MARK_LAGRANGE_DOFS, BUILD_COUPLING_TABLES (ANALYSES/mul2_model_assembly.for)	set a table with the term couplings if the functions have local support; otherwise the family is fully coupled
Constraints	ADD_PLANE_CONSTRAINT (BOUNDARY/mul2_boundary_application.for)	which terms are zero or equal to the prescribed value on a plane
Loads	ADD_POINT_FORCE	how a point load is distributed over the terms
Post-processing	STATE_FROM_PLACE (POST/mul2_recovery.for)	uses <code>EVALUATE_EXPANSION_FACTOR</code> ; normally nothing to change

Because the separable kernel only needs `EVALUATE_EXPANSION_FACTOR` (through `EVALUATE_POINT_FACTORS`) and the number of terms, a new family automatically gets the fast kernel. Hierarchical bases need two extra pieces: the *sign convention* of the shared entities (edge modes of odd order change sign if the edge is traversed in the opposite direction) and the *interface constraints* between sub-elements of different order.

11.3 Adding an analysis type

1. Create `ANALYSES/mul2_analysis_nnn.for` with `RUN_<NAME>_ANALYSIS(model, cache, results, status)` following the pattern of analysis 101: assemble with `ASSEMBLE_MODEL_SYSTEM`, build boundary data, solve, fill `ANALYSIS_RESULTS_TYPE`.
2. Register the source in the `MUL2_SOLVERS` list of `CMakeLists.txt`.
3. In `MUL2_DRIVER` accept the new `SOLUTION` value and add a `CASE` in the two `SELECT CASE` blocks (analysis and output).
4. Write the output routine in `MUL2_POST_OUTPUT` and add the fixed output file names.
5. Extend `ANALYSIS_TYPES` in `INTERFACE/app.js` so that the interface offers the new analysis.
6. Document the theory (new chapter), the user input and the tests.

11.4 Rules for a good contribution

- Keep every line within 72 columns (the build checks it).
- Never `STOP` in a library routine: set an error in the status and return.
- Do not add global variables.
- Every formula in one place; reuse the existing routine instead of writing a second one.
- Write the test first: a case with a known answer (analytic, patch, or reference program).
- Update the three guides and the generated reference (`python DOC/BOOKS/build_books.py`).

The HLE implementation

This chapter describes how the hierarchical Legendre expansion of Chapter 14 of the Theoretical Guide is implemented: the modules, the data, the flow and the tests.

12.1 Modules

Table 12.1. New and modified units.

Unit	Role
FEM/mul2_hle_shape.for (MUL2_HLE_SHAPE)	the functions: HLE_FUNCTION_COUNT, HLE_MODE_LIST (kind and indices of each local mode), HLE_SIDE_ENDS, HLE_PHI, EVALUATE_HLE_SHAPE (values and derivatives)
FEM/mul2_hle_map.for (MUL2_HLE_MAP)	the blending map: HLE_GEOMETRY_SETUP (arcs from vertices and mid nodes), HLE_MAP_EVALUATE (position and Jacobian)
MODEL/mul2_hle_modes.for (MUL2_HLE_MODES)	BUILD_HLE_MESH: numbering of the generalised terms, ownership, signs
MODEL/mul2_topologies.for	pseudo-topologies: HB = 1100+p, HQ = 2100+p; TOPOLOGY_FUNCTION_COUNT, TOPOLOGY_IS_HLE, TOPOLOGY_HLE_ORDER
MODEL/mul2_expansion_meshes.for	new fields of the mesh and element types (below)
IO/read_expansions.for	reading of HQ4 ... p [m1..m4], HB2 ... p
GAUSS/*, CUF/mul2_point_bases.for, GEOMETRY/mul2_local_gradients.for	number of functions larger than number of nodes; HLE case of the expansion factor; curved Jacobian
ASSEMBLY/mul2_dof_layout.for, ANALYSES/mul2_model_assembly.for	term count, consistency of node and mesh kind, coupling tables
BOUNDARY/mul2_boundary_application.for	plane constraints and point loads for HLE
POST/mul2_recovery.for	stress recovery on curved sub-elements

12.2 Data structures

The expansion element type gains `MODE_TERM(:)` (the global term of each local function) and `MODE_SIGN(:)` (± 1), plus `MID_NODE_ID(4)`. The expansion mesh gains `IS_HLE`, `N_TERM`, `TERM_NODE(2,T)` (vertex term: (*node*, 0); side term: the two end vertices; internal term: (0,0)), `TERM_OWNER` and `NODE_TERM`, the term of each vertex node. `EXPANSION_TERM_COUNT` returns the number of terms of a mesh: for Lagrange the nodes, for HLE `N_TERM`.

12.3 Flow

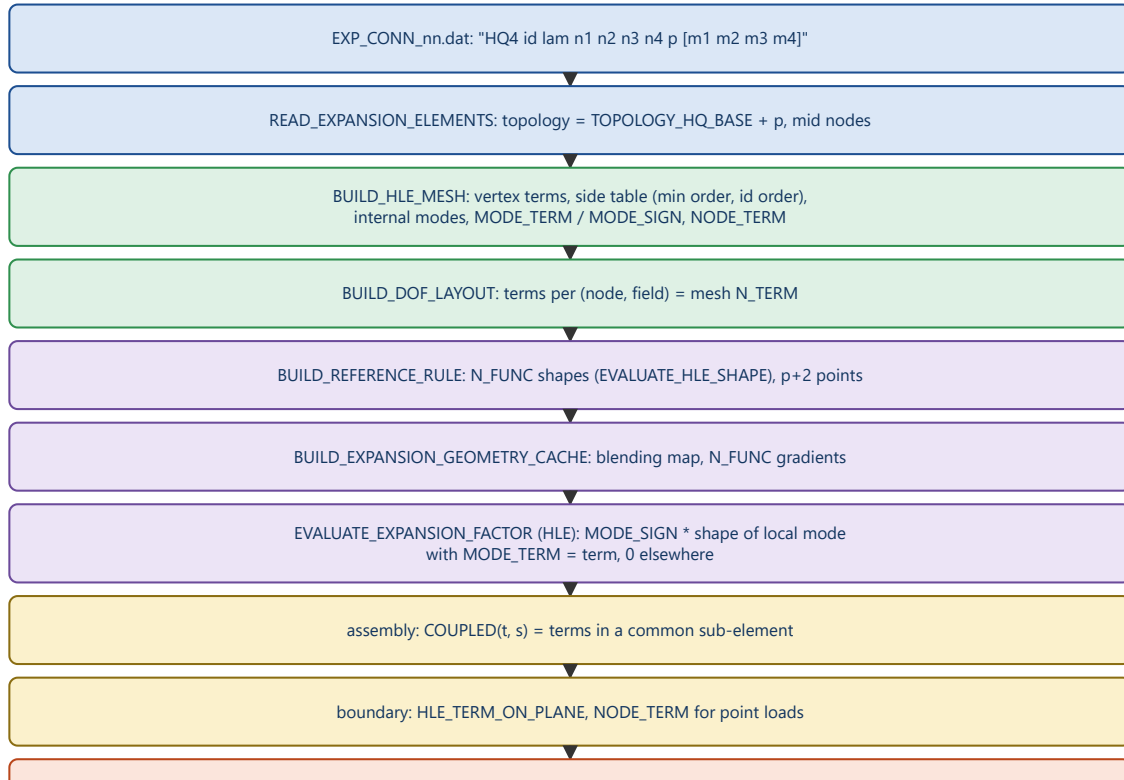


Figure 12.1. Construction and use of the HLE data.

- Reading.** `READ_EXPANSIONS` stores the order in the element topology code and the optional mid nodes; after all elements are read it calls `BUILD_HLE_MESH`.
- Numbering.** `BUILD_HLE_MESH` walks the sub-elements: vertex terms are numbered after the nodes; every side gets one term per order $k \leq \min$ of the orders of the elements that share it (the side is identified by its two vertex ids, lower first); internal terms belong to one element. For each element and each local function it stores the global term in `MODE_TERM`, or 0 if the function is omitted (side mode above the minimum order), and the sign $(-1)^k$ when the local side runs from the higher to the lower vertex id.
- Shape evaluation.** `EVALUATE_SHAPE` dispatches HB/HQ to `EVALUATE_HLE_SHAPE`, which returns all `HLE_FUNCTION_COUNT` functions in the local order of the paper. The quadrature uses $p + 2$ points per direction.
- Geometry cache.** For an element without mid nodes the Jacobian comes from the vertices; the gradient of every mode uses `MATMUL(DERIV, TRANSPOSE(INVERSE))`. With mid nodes the Jacobian of the map is passed through two pseudo nodes whose coordinates are $\partial Q / \partial r$ and $\partial Q / \partial s$ and the integration points use the mapped position.
- Kernels.** `EVALUATE_EXPANSION_FACTOR` finds the local mode whose `MODE_TERM` equals the requested term and multiplies by `MODE_SIGN`. The separable kernel and the reference kernel (`MUL2_GENERAL_KERNEL=1`) give the same matrices.
- Constraints and loads.** A plane constrains a vertex term if the vertex lies on it, a side term if both end vertices do, and all the terms of an element if all its vertices do; side and internal terms are prescribed 0. A point load goes to the vertex term of the node.

12.4 Tests

Table 12.2. HLE tests.

Test	File	Content
<code>MUL2_HLE_TESTS</code>	<code>TESTS/mul2_hle_tests.for</code>	function counts, Kronecker and partition of unity at the vertices, derivatives, C0 conformity on a shared side for several orders and numberings, annulus area, arcs on circles

Test	File	Content
MUL2_HLE_SOLVER_TESTS	TESTS/HLE/hle_tests.py	HQ4 p=1 vs Q4; HB vs B3/B4; reversed side; fast vs reference kernel; Ritz bounds; convergence vs TE4/TE6; tube vs Euler-Bernoulli; VTK radii
validation campaign	TESTS/VALIDATION	the HLE group of the validation report

NOTE

running the test executables needs the oneAPI folders `mk1\2025.0\bin` and `compiler\2025.0\bin` on `PATH`; otherwise Windows shows a hidden dialog for the missing DLL and the test hangs.

Multiphysics and the new analyses

This chapter lists what was added to the program in the multiphysics and analysis extension, where it lives and how the pieces fit. The routine-by-routine reference (Appendix) is generated from the sources and includes all the new modules.

13.1 1. New and extended modules

Module (file)	Role
MUL2_MATERIALS (MODEL)	adds PIEZO(3,6), PERMITTIVITY, EXPANSION(6), CONDUCTIVITY(3,3), PYRO(3), SPECIFIC_HEAT and the global DAMP_* coefficients
MUL2_READ_MATERIALS	records Z-EXP, Z-PRM, T-EXP, T-CON, T-SPC, PIROE, DAMP; records of a material may precede it (a pending list is attached at the end)
MUL2_MATERIAL_ROTATIONS, MUL2_MATERIAL_RESOLUTION	rotation of the tensors into the element frame (strain transform T , $A^T eT$, $A^T \varepsilon A$, $T(A^T)\alpha$, $A^T p$)
MUL2_GAUSS_MATERIALS	the cache of every lamination stores the rotated tensors; GENERALIZED_CONSTITUTIVE returns the 12×12 matrix M
MUL2_ELEMENT_MATRICES	the point-by-point kernel with $NR = 6 / 9 / 12$ rows (mechanics, +potential, +temperature) and four extra modes: COUPLING (block K_{uT} , $K_{\varphi T}$), GEOMETRIC (K_G of a state), the heat capacity in the mass, and BUILD_NONLINEAR_ELEMENT_MATRICES
MUL2_FIELDS, MUL2_READ_FIELDS	spatial functions of FIELDS.dat
MUL2_TIME_INPUT, MUL2_READ_TIME	TIME_RESP.dat (load amplitude $a(t)$), FREQ_RESP.dat, NL_INFO.dat
MUL2_BOUNDARY_APPLICATION	the constraints and loads of all the fields, the floating electrodes (APPLY_DOF_TIES, EXPAND_TIES) and the field multiplier
MUL2_SURFACE_LOADS	surface integration of heat flux, sun and convection
MUL2_MODEL_ASSEMBLY	parallel assembly of K , M and the coupling block; ASSEMBLE_GEOMETRIC_VALUES; ASSEMBLE_NONLINEAR_VALUES
MUL2_PARDISO_FACTOR	adds the non-symmetric type 11 with the full CSR
MUL2_MODAL_SOLVER	SOLVE_BUCKLING_DENSE; the condensation of the massless potential rows in the modal problem
MUL2_ANALYSIS_104/105/106/108	time response, buckling, frequency response, nonlinear statics
MUL2_POST_OUTPUT, MUL2_RECOVERY	temperature, heat flux and potential recovery; WRITE_TIME_STEP_OUTPUT, WRITE_FREQUENCY_OUTPUT

13.2 2. How a model flows through the analyses

```

READ_MODEL (FIELDS, TIME/FREQ/NL files by analysis id)
-> BUILD_MODEL_CACHE (DOF layout of 5 fields, frames, Gauss data, materials)
-> ASSEMBLE_MODEL_SYSTEM (K, M [, K_uT])
-> BUILD_MECHANICAL_BOUNDARY_DATA (constraints, F) -> APPLY_SURFACE_LOADS
-> analysis 101 | 103 | 104 | 105 | 106 | 108
-> output

```

DOF classes. The analyses 104 and 106 classify each degree of freedom by its field: class 1 (fields 1-3, second order in time), 2 (field 4, temperature, first order) and 3 (field 5, potential, algebraic). The effective matrix is one pass over the CSR pattern in which every row takes the scalar that belongs to its class.

The monolithic system. The pattern of K is built from the degree-of-freedom lists of the elements, so it already contains every coupling pair; the non-symmetric blocks are only *values*. `APPLY_STATIC_CONSTRAINTS` works entry by entry and therefore on non-symmetric matrices. Whether PARDISO uses the symmetric type 2 (with the automatic fallback to the indefinite type -2) or the non-symmetric type 11 depends only on the presence of the temperature.

Chunked parallel assembly. `ASSEMBLE_MODEL_SYSTEM` and `ASSEMBLE_NONLINEAR_VALUES` evaluate the elements of a chunk in parallel (OpenMP, one element per thread) into private arrays and scatter them **serially in element order**, so the result does not depend on the number of threads.

13.3 3. Conventions that are easy to get wrong

- The potential row of the element matrix is the **negative** of the permittivity block: the matrix is symmetric indefinite.
- The heat conductivity block is $+k$ (positive definite); a heat flux or power is a positive load into the body.
- A pyroelectric coefficient is stored at constant strain; the input (zero stress) is converted in the material cache.
- In the nonlinear element the column of a displacement degree of freedom is the MITC-tied linear strain **plus** $H^T d \otimes g$; the temperature columns only hold the gradient and carry the value separately for the coupling.
- The geometric matrix does not use MITC; it only needs the gradients and the local direction of every degree of freedom.
- A time-analysis output step writes rows with the time first; the nonlinear analysis uses the same file with the load factor as time.

13.4 4. Extending

To add a physics: a new field token in `KINEMATICS.dat` and `N_SOLVED_FIELDS`, its row block in `GENERALIZED_CONSTITUTIVE` and the column of its degree of freedom in `EVALUATE_LOCAL_DOF`; the coupling blocks go in the `COUPLING` branch; the recovery (`STATE_FROM_PLACE`) and the output follow. To add an analysis: an id in `MUL2_ANALYSIS_INPUT`, a module that fills `ANALYSIS_RESULTS_TYPE`, its input reader in `READ_MODEL` and a case in `MUL2_DRIVER`.

13.5 5. Tests

Suite	Content
<code>MUL2_PIEZO_TESTS</code>	slab statics, thickness resonance (short, open and floating electrode), beam/plate/solid equivalence, pyroelectric slab
<code>MUL2_THERMAL_TESTS</code>	expansion, clamped stress, conduction, heat power, flux, convection, sun (areas and power of beam, plate and solid), thermal bending, <code>FIELDS.dat</code>
<code>MUL2_DYNAMICS_TESTS</code>	Newmark period and amplitude, Rayleigh damping, ramp, conduction with capacity, thermoelastic damping, piezoelectric actuation, harmonic response, piezoelectric resonance, lumped sun and convection
<code>MUL2_BUCKLING_TESTS</code>	cantilever (beam and solid), clamped-clamped (mechanical and thermal)
<code>MUL2_NONLINEAR_TESTS</code>	linear limit, elastica, beam-column, solid, Green-measure thermal expansion

All the suites run in the Release and Debug configurations (`ctest -C Release` , `-C Debug`); the Debug build checks array bounds and uninitialised variables.

Curved beams and shells

This chapter documents the implementation of the general geometry (curved beams, shells). The theory is in Chapter 17 of the theoretical guide, the input in Chapter 13 of the user guide.

14.1 Where the general geometry enters

The ordinary path is untouched: an element is *general* only when it is named `CB*/S*` or its nodes are off the line/plane, and every other element keeps its frame, structural cache and kernels bit for bit. The general elements are intercepted at three places and use two new modules.

Place	What changes
<code>read_connectivity.for</code> (<code>ALIAS_NAME</code>)	<code>S4 S9 S16 CB2 CB3 CB4</code> are read as <code>Q4 Q9 Q16 B2 B3 B4</code> with <code>ELEMENT_TYPE%GENERAL = .TRUE.</code>
<code>read_directors.for</code> (new, <code>MUL2_READ_DIRECTORS</code>)	optional <code>DIRECTORS.dat</code> : <code>NODE_TYPE%DIRECTOR</code> , <code>HAS_DIRECTOR</code>
<code>mul2_model_cache.for</code> (<code>BUILD_MODEL_CACHE</code>)	after <code>BUILD_ELEMENT_FRAMES</code> , <code>BUILD_GENERAL_GEOMETRY</code> fills the new members of <code>ELEMENT_FRAME_DB_TYPE</code>
<code>mul2_gauss_geometry.for</code>	for a general element the flat structural Jacobian is not evaluated (identity, weight 1): the weight comes from the kernel
<code>mul2_model_assembly.for</code>	<code>EVALUATE_ELEMENT_BASE</code> and the coupled branch of <code>EVALUATE_ELEMENT</code> call <code>GENERAL_ELEMENT</code> ; <code>NONLINEAR_ELEMENT</code> and <code>GEOMETRIC_ELEMENT</code> raise an error
<code>mul2_boundary_application.for</code>	<code>PLACED_POINT</code> replaces <code>EXPANDED_POINT</code> at the four places that locate an expansion node (<code>D-PLANE</code> , <code>V-FLOAT</code> , <code>F-POINT</code> , <code>V-POINT</code>)
<code>mul2_recovery.for</code>	<code>PLACE_SETUP/PLACE_EVALUATE/STATE_FROM_PLACE/NEWTON_LOCATE</code> have a general branch
<code>mul2_surface_loads.for</code>	error when a model with <code>Q-*</code> has general elements

`ELEMENT_FRAME_DB_TYPE` (`mul2_reference_systems.for`) has four new members: `GENERAL_INDEX(element)` (0 = ordinary), `GENERAL_TRIAD(3,3,node,general)`, `GENERAL_REFERENCE(3,general)` and `GENERAL_KINK(node,general)` (shell nodes shared with another normal: 1 = edge, -1 = thickness axis opposite to the one of the first element at the node, 0 = smooth). The arrays are compact (the general elements only), allocated even when empty, so `IS_GENERAL_ELEMENT(FRAMES, E)` is always safe.

14.2 MUL2_GENERAL_GEOMETRY (SRC/GEOMETRY/mul2_general_geometry.for)

Procedure	Purpose
<code>BUILD_GENERAL_GEOMETRY</code>	finds the general elements (<code>IS_CURVED</code> : nodes against line/plane within 10^{-6}), computes raw tangents/normals at the nodes with <code>EVALUATE_SHAPE</code> at the natural coordinates of the node (<code>GENERAL_NODE_NATURAL</code>), groups the entries by node (<code>CSR: FIRST</code> , <code>ENTRY_G</code> , <code>ENTRY_J</code>), and averages them within <code>FEATURE_COSINE</code> (the exact director of <code>DIRECTORS.dat</code> replaces the average); fills the triad
<code>GENERAL_MAP</code>	position and Jacobian rows g_a at a point: $\sum_i N_i(X_i + c_k A_{ki})$, <code>AXIS</code> = active axes of the expansion mesh
<code>GENERAL_FRAME</code>	frame rows R at a point (same rules as the ordinary frames)
<code>GENERAL_OFFSET</code>	$\sum c_k A_{ki}$, used to place expansion nodes (loads, constraints)

14.3 MUL2_GENERAL_KERNEL (SRC/ELEMENTS/mul2_general_kernel.for)

`GENERAL_CONTEXT_TYPE` holds the geometry of one element (coordinates, triads, reference vector, active axes) and the tying families (`TIE_FAMILY_TYPE` : lists of points in ξ and η , the rows they replace, shape functions and derivatives at the tying points, precomputed once per element).

- `GENERAL_CONTEXT_SETUP`(CTX, element, ..., TYING): fills the context; with `TYING` chooses the tables of the topology (Q9: three families; Q4: two; B2-B4: one; Q16: none).
- `GENERAL_POINT_COLUMNS`(CTX, N, DN, NATURAL, C, DOFs, FV, FG, BCOL, VALUE, G, R, DET) : the strain columns of all the DOFs at a point. For each tying point the jacobian `GM` at the same c is formed once; the rows of the family are replaced by $\sum_m L_m \tilde{\gamma}(\xi_m)$ (`TIE_WEIGHTS`, `LAGRANGE`); then `BUILD_T` gives the 6×6 matrix $T(Q)$. The fields P and T get the Cartesian gradient.
- **Edges.** After the columns of a point are built, for every node with `KINK` $\neq 0$ the three first-order DOFs (same node, expansion function $\$,$ fields 1-3: recognised by `FG(axis)=1` and `FV=c`) are replaced by the combinations `BCOL*A` and `VALUE*A` with A = the skew matrix of the director ($q1 = \omega \times v$) or $-I$ for opposite axes. The transformation is therefore applied once, in the columns, and serves the stiffness, the mass, the coupling blocks and the recovery. It needs a Taylor expansion (TE) of order 1 or 2; an LE expansion has no first-order term.
- `BUILD_GENERAL_ELEMENT_MATRICES` : loops over the points in batches (`MAX_BATCH`), evaluates the expansion factors once per distinct (kinematic, field, term) with `EVALUATE_POINT_FACTORS` (so every expansion family, TE, LE, HLE and the node-dependent kinematics, goes through the existing code), calls `GENERAL_POINT_COLUMNS` and accumulates $B^T(WMB)$ with `ACCUMULATE_UPPER_PRODUCT` on the upper triangle, mass and heat capacity with the same helper. `COUPLING` returns only the thermoelastic and pyroelectric blocks (`EVALUATE_ELEMENT`).

The kernel keeps no state (OpenMP safe: the assembly evaluates the elements of a chunk in parallel exactly as for the ordinary ones, the result does not depend on the number of threads). The same two procedures serve the recovery: `GENERAL_STATE_PART` of `mul2_recovery.for` collects the DOFs of the element, the factors at the point and calls `GENERAL_POINT_COLUMNS`, then the ordinary material block gives stress, electric displacement and heat flux; `POINT_STATE_TYPE%FRAME` stores the frame of the point (used by the output in the local frame).

14.4 Joining by coincidence (MUL2_COINCIDENT_JOIN)

`SRC/BOUNDARY/mul2_coincident_join.for`, called by `BUILD_MODEL_CACHE` after the frames and the triads when `ANALYSIS%JOIN_COINCIDENT` is true. `JOIN_COINCIDENT_DOFS` builds the list of the LE and TE DOFs with their position (`PLACED_POINT` : node + offset of the expansion node, in the frame or in the triad of the element), a spatial hash (cells of twice the tolerance, counting sort), unites the pairs of the same field and class within the tolerance (union-find) and stores in `DOF_LAYOUT_TYPE%ALIAS` the map provisional number $\rightarrow$ final number; `GLOBAL_DOF` applies it and `TOTAL_DOF` is reduced. Every consumer uses `GLOBAL_DOF` and `TERM_COUNT`, so assembly, constraints, loads and recovery need no change. Cost: $O(N)$ with 27 cells per entry.

14.5 Tests

`TESTS/CURVED/curved_tests.py` (ctest `MUL2_CURVED_TESTS`, ~ 10 s, also run in Debug with `/check:a11`): equivalence of every general topology with the ordinary one, rigid modes, quarter ring (force, couple, stress, frequency), pinched cylinder (TE, LE, directors, locking), tubes, stringer joint. The unit checks of Chapter 10 are unchanged.

14.6 Extending

- A new shell topology: add its node-natural coordinates to `GENERAL_NODE_NATURAL` and a tying table to `GENERAL_CONTEXT_SETUP`.
- Curved elements in 105/108: `GENERAL_POINT_COLUMNS` has the linear part; the geometric/nonlinear terms need the gradient of φ in the Cartesian frame (available as `MATMUL(R, MATMUL(GINV, DPHI))`).
- Surface loads: the area element is $\|g_1 \times g_2\|$ at the face of the expansion mesh.

File formats at a glance

Table A.1. Input files (every file starts with the number of records; blank lines and comments are ignored).

File	Record	Fields
ANALYSIS.dat	1	solution (101 / 103) and a free comment
	2	number of modes (used by 103)
	3–5	treatment of beams, plates, solids: NONE , REDI , SELI , MITC (an old integer solver record before record 2 is skipped)
NODES.dat (legacy)	ID X Y Z TE n or ID X Y Z LE m	coordinates, expansion model (TE : order n ; LE : second number ignored)
NODES.dat (with KINEMATICS.dat)	ID X Y Z KID	kinematic id
KINEMATICS.dat	KINEMATIC id u v w t p b szz sxz syz	nine tokens: TEn , LE , HLEn , Mn , NONE
CONNECTIVITY.dat	TYPE id n1 ... nN [versor] section	topology name, element id, node ids, versor id (not needed for solids), expansion mesh number
VERSORS.dat	VERSOR id vx vy vz	reference vector
MATERIAL.dat	header nmat nrec	number of materials, number of records
	ISO-M id E nu rho	isotropic
	ORT-M id EL ET EZ nuLT nuLZ nuTZ GLT GLZ GTZ rho	orthotropic
	MAT-C id c11 ... c66 rho	36 entries of C (material axes) and density
LAMINATION.dat	LAM2 id material angle_y angle_z	degrees
EXP_MESH_nn.dat	id x y z	local coordinates of expansion nodes ($y = 0$ for beams)
EXP_CONN_nn.dat	TYPE id lamination n1 ... nN	sub-element: topology, id, lamination, node ids
BC.dat	D-PLANE id A B C D ux uy uz	plane $AX + BY + CZ + D = 0$; each component a number or N
	F-POINT id x y z Fx Fy Fz	point load at a section node
POSTPROCESSING.dat	PARA GMSH cells LOC GLB s1... s9	output grid
	PNT id x y z	point result
	KMAT , MMAT , FRCE , UNKN , ENRG	dump requests

Table A.2. Output files.

File	Layout
STATIC/POST_POINT.dat	header + one row per point, 46 columns
STATIC/RESULTS_PARA_nn.vtk	legacy VTK ASCII, point data Displacements , Epsilon_*, Sigma_*, von Mises
DYNAMIC/FREQUENCIES.dat	Frequency n: value
WORK/K_MAT.dat	row column value (1-based, field-major numbering)
REPORT/WARNING_file.dat	count + one warning per line

Naming of the principal variables

Table B.1. Variable names that recur in the sources.

Name	Meaning
NODE, NODE_INDEX, NODE_ID	node (position in the database / number in the input)
ELEMENT, ELEMENT_INDEX	structural element
SUB_ELEMENT, EXPANSION_ELEMENT_INDEX	element of the expansion mesh
MESH, MESH_INDEX	expansion mesh
POINT	combined Gauss point (global number)
STRUCTURAL_POINT_INDEX, EXPANSION_POINT_INDEX	point in the structural / expansion rule
TERM	expansion term index τ
FIELD	1 = U, 2 = V, 3 = W
LOCAL_DOF, I, J	DOF index inside an element
GLOBAL_DOF	global DOF number
N, DN	shape functions and derivatives
F, FACTOR_VALUE, FACTOR_GRADIENT	expansion function and gradient
B_COLUMN	6-vector strain column of a DOF
STIFFNESS_LOCAL	rotated 6×6 constitutive matrix
WEIGHT, INTEGRATION_WEIGHT	quadrature weight, with Jacobians
GLOBAL_TO_LOCAL	rotation R (rows = local axes)
ACTIVE_AXIS, AXIS	local axes spanned by an element or expansion mesh
SIGMA, EPS, SM, EM, GAMMA, LAMBDA	separable-kernel quantities of Chapter 6

Source reference

This appendix is generated from the sources by `DOC/BOOKS/tools/refgen.py`; it lists, layer by layer, every module with its purpose, the modules it uses, its public derived types and constants, and its procedures with the meaning of the dummy arguments. Procedure descriptions are the comments written above each routine in the code.

The code base has **83 modules** and **404 procedures** in 85 source files (ARPACK excluded).

C.1 Layer BASE

Precision kinds, status/error handling, logging, timers, string and file utilities, sorting and searching.

C.1.1 MUL2_FILES

File `SRC/BASE/mul2_files.for`.

File-system helpers (output directories).

Uses: `MUL2_KINDS`, `MUL2_STATUS`.

Table C.1. Procedures of `MUL2_FILES`.

Procedure	Kind	Visibility	Purpose
<code>IS_WINDOWS</code>	Function → logical	public	True when the program runs on Windows (selects shell syntax).
<code>ENSURE_DIRECTORY</code>	Subroutine	public	Create a relative directory if it does not exist yet.

`ENSURE_DIRECTORY`

Create a relative directory if it does not exist yet.

Argument	Declaration	Meaning
<code>NAME</code>	<code>CHARACTER(LEN=*), intent(IN)</code>	Name.
<code>STATUS</code>	<code>TYPE(STATUS_TYPE), intent(OUT)</code>	Status of the call (OK, warning or error with message and source).

C.1.2 MUL2_KINDS

File `SRC/BASE/mul2_kinds.for`.

Numeric kinds used by the whole code.

C.1.3 MUL2_LOG

File `SRC/BASE/mul2_log.for`.

Central console logger.

Table C.3. Procedures of `MUL2_LOG`.

Procedure	Kind	Visibility	Purpose
<code>LOG_INITIALIZE</code>	Subroutine	public	Select normal or debug logging.
<code>LOG_INFO</code>	Subroutine	public	Print an informational line on the console.
<code>LOG_WARNING</code>	Subroutine	public	Print a warning line.
<code>LOG_ERROR</code>	Subroutine	public	Print an error line on the error unit.
<code>LOG_DEBUG</code>	Subroutine	public	Print a line only in debug mode.
<code>IS_DEBUG_MODE</code>	Function → logical	public	Return the debug flag.

LOG_INITIALIZE

Select normal or debug logging.

Argument	Declaration	Meaning
DEBUG_ENABLED	LOGICAL, intent(IN)	True selects debug logging.

LOG_INFO

Print an informational line on the console.

Argument	Declaration	Meaning
MESSAGE	CHARACTER(LEN=*), intent(IN)	Message text.

LOG_WARNING

Print a warning line.

Argument	Declaration	Meaning
MESSAGE	CHARACTER(LEN=*), intent(IN)	Message text.

LOG_ERROR

Print an error line on the error unit.

Argument	Declaration	Meaning
MESSAGE	CHARACTER(LEN=*), intent(IN)	Message text.

LOG_DEBUG

Print a line only in debug mode.

Argument	Declaration	Meaning
MESSAGE	CHARACTER(LEN=*), intent(IN)	Message text.

C.1.4 MUL2_RUNTIME

File SRC/BASE/mul2_runtime.for.

Process-wide runtime initialization.

Uses: MUL2_LOG.

Table C.9. Procedures of MUL2_RUNTIME.

Procedure	Kind	Visibility	Purpose
INITIALIZE_RUNTIME	Subroutine	public	Start the global timer and print the banner.
FINALIZE_RUNTIME	Subroutine	public	Print the CPU and wall time of the run.

C.1.5 MUL2_SORTING

File SRC/BASE/mul2_sorting.for.

Heap sort of 64-bit keys with a permutation, and duplicate search.

Uses: MUL2_KINDS.

Table C.10. Procedures of MUL2_SORTING.

Procedure	Kind	Visibility	Purpose
SORT_PERMUTATION	Subroutine	public	Order(1:n) = permutation such that key(order(1:n)) is ascending.
SIFT_DOWN	Subroutine	private	Heap-sort helper.
BINARY_SEARCH	Function → integer(i4)	public	Position i of target in sorted_key(order(1:n)) as order(i), or 0.

Procedure	Kind	Visibility	Purpose
HAS_DUPLICATE_KEYS	Function → logical	public	True when a key array contains a repeated value.

SORT_PERMUTATION

Order(1:n) = permutation such that key(order(1:n)) is ascending.

Argument	Declaration	Meaning
KEY	INTEGER(I8), intent(IN)(:)	Array of keys to sort.
ORDER	INTEGER(I4), intent(OUT)(:)	Output: permutation.

BINARY_SEARCH

Position i of target in sorted_key(order(1:n)) as order(i), or 0.

Argument	Declaration	Meaning
SORTED_KEY	INTEGER(I8), intent(IN)(:)	Keys (in the order given by ORDER).
ORDER	INTEGER(I4), intent(IN)(:)	Order of the matrix, or polynomial/quadrature order (see routine).
TARGET	INTEGER(I8), intent(IN)	Key to find.

HAS_DUPLICATE_KEYS

True when a key array contains a repeated value.

Argument	Declaration	Meaning
KEY	INTEGER(I8), intent(IN)(:)	Integer keys.

C.1.6 MUL2_STATUS

File SRC/BASE/mul2_status.for.

Central status and error description.

Uses: MUL2_KINDS.

Table C.14. Public constants of MUL2_STATUS.

Name	Type	Value	Meaning
STATUS_OK	INTEGER(I4)	0_I4	Status code.
STATUS_WARNING	INTEGER(I4)	1_I4	Status code.
STATUS_ERROR	INTEGER(I4)	2_I4	Status code.

Table C.15. Derived type STATUS_TYPE — Result of any operation: code (0 OK, 1 warning, 2 error), warning counter, last message and its source routine.

Component	Type	Shape	Default	Meaning
CODE	INTEGER(I4)	''	STATUS_OK	Status code.
WARNING_COUNT	INTEGER(I4)	''	0_I4	Number of warnings.
MESSAGE	CHARACTER(LEN=256)	''	' '	Last message.
SOURCE	CHARACTER(LEN=64)	''	' '	Routine that reported it.

Table C.16. Procedures of MUL2_STATUS.

Procedure	Kind	Visibility	Purpose
CLEAR_STATUS	Subroutine	public	Reset a status to OK.
SET_WARNING	Subroutine	public	Record a warning (an existing error is not overwritten).
SET_ERROR	Subroutine	public	Record an error with its source and message.

Procedure	Kind	Visibility	Purpose
STATUS_IS_OK	Function → logical	public	True when the status is not an error (warnings are allowed).
MERGE_STATUS	Subroutine	public	Fold the status of one step into the status of a whole phase: the first error wins and warnings are counted, never lost.

CLEAR_STATUS

Reset a status to OK.

Argument	Declaration	Meaning
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

SET_WARNING

Record a warning (an existing error is not overwritten).

Argument	Declaration	Meaning
STATUS	TYPE(STATUS_TYPE), intent(INOUT)	Status of the call (OK, warning or error with message and source).
SOURCE	CHARACTER(LEN=*), intent(IN)	Name of the routine reporting the warning.
MESSAGE	CHARACTER(LEN=*), intent(IN)	Message text.

SET_ERROR

Record an error with its source and message.

Argument	Declaration	Meaning
STATUS	TYPE(STATUS_TYPE), intent(INOUT)	Status of the call (OK, warning or error with message and source).
SOURCE	CHARACTER(LEN=*), intent(IN)	Name of the routine reporting the error.
MESSAGE	CHARACTER(LEN=*), intent(IN)	Message text.

STATUS_IS_OK

True when the status is not an error (warnings are allowed).

Argument	Declaration	Meaning
STATUS	TYPE(STATUS_TYPE), intent(IN)	Status of the call (OK, warning or error with message and source).

MERGE_STATUS

Fold the status of one step into the status of a whole phase: the first error wins and warnings are counted, never lost.

Argument	Declaration	Meaning
LOCAL	TYPE(STATUS_TYPE), intent(IN)	Status of one step.
GLOBAL	TYPE(STATUS_TYPE), intent(INOUT)	Accumulated status of the phase.

C.1.7 MUL2_STRINGS

File SRC/BASE/mul2_strings.for .

Basic string utilities.

Table C.22. Procedures of MUL2_STRINGS .

Procedure	Kind	Visibility	Purpose
UPPERCASE	Subroutine	public	Convert a string to upper case in place.

UPPERCASE

Convert a string to upper case in place.

Argument	Declaration	Meaning
TEXT	CHARACTER(LEN=*), intent(INOUT)	Text.

C.1.8 MUL2_TIMER

File SRC/BASE/mul2_timer.for.

Wall-clock and cpu timer.

Uses: MUL2_KINDS.

Table C.24. Derived type `TIMER_TYPE` — CPU and wall-clock timer.

Component	Type	Shape	Default	Meaning
NAME	CHARACTER(LEN=64)	""	' '	Timer name.
CPU_BEGIN	REAL(R8)	""	0.0_R8	CPU time at start.
CPU_END	REAL(R8)	""	0.0_R8	CPU time at stop.
CLOCK_BEGIN	INTEGER(I8)	""	0_I8	Clock count at start.
CLOCK_END	INTEGER(I8)	""	0_I8	Clock count at stop.
CLOCK_RATE	INTEGER(I8)	""	0_I8	Clock ticks per second.
CLOCK_MAX	INTEGER(I8)	""	0_I8	Clock wrap value.
RUNNING	LOGICAL	""	.FALSE.	Timer running flag.

Table C.25. Procedures of `MUL2_TIMER`.

Procedure	Kind	Visibility	Purpose
TIMER_START	Subroutine	public	Start a named timer (CPU and wall clock).
TIMER_STOP	Subroutine	public	Stop a timer.
TIMER_CPU_SECONDS	Function → real(r8)	public	Elapsed CPU seconds.
TIMER_WALL_SECONDS	Function → real(r8)	public	Elapsed wall-clock seconds.

TIMER_START

Start a named timer (CPU and wall clock).

Argument	Declaration	Meaning
TIMER	TYPE(TIMER_TYPE), intent(INOUT)	Timer.
NAME	CHARACTER(LEN=*), intent(IN)	Name.

TIMER_STOP

Stop a timer.

Argument	Declaration	Meaning
TIMER	TYPE(TIMER_TYPE), intent(INOUT)	Timer.

TIMER_CPU_SECONDS

Elapsed CPU seconds.

Argument	Declaration	Meaning
TIMER	TYPE(TIMER_TYPE), intent(IN)	Timer.

TIMER_WALL_SECONDS

Elapsed wall-clock seconds.

Argument	Declaration	Meaning
TIMER	TYPE(TIMER_TYPE), intent(IN)	Timer.

C.2 Layer MODEL

Plain data types of the model: nodes, elements, kinematics, expansion meshes, materials, laminations, boundary conditions, analysis and post-processing requests.

C.2.1 MUL2_ANALYSIS_INPUT

File SRC/MODEL/mul2_analysis_input.for.

Analysis configuration and post-processing requests. shear(1:3) holds the shear-locking treatment of beam, plate and solid elements: none (full integration), redi (reduced integration of the structural element), seli (selective: the terms with a shear strain are reduced, the others full) or mitc (tied shear strains).

Uses: MUL2_KINDS.

Table C.30. Public constants of MUL2_ANALYSIS_INPUT.

Name	Type	Value	Meaning
SOLUTION_STATIC	INTEGER(I4)	101_I4	Analysis number.
SOLUTION_MODAL	INTEGER(I4)	103_I4	Analysis number.
SOLUTION_TIME	INTEGER(I4)	104_I4	Analysis number.
SOLUTION_BUCKLING	INTEGER(I4)	105_I4	Analysis number.
SOLUTION_FREQUENCY	INTEGER(I4)	106_I4	Analysis number.
SOLUTION_NONLINEAR	INTEGER(I4)	108_I4	Analysis number.
SHEAR_NONE	INTEGER(I4)	0_I4	Shear-correction code NONE.
SHEAR_REDUCED	INTEGER(I4)	1_I4	Shear-correction code REDUCED.
SHEAR_SELECTIVE	INTEGER(I4)	2_I4	Shear-correction code SELECTIVE.
SHEAR_MITC	INTEGER(I4)	3_I4	Shear-correction code MITC.
SHEAR_BEAM	INTEGER(I4)	1_I4	Shear-correction code BEAM.
SHEAR_PLATE	INTEGER(I4)	2_I4	Shear-correction code PLATE.
SHEAR_SOLID	INTEGER(I4)	3_I4	Shear-correction code SOLID.
POST_FORMAT_PARAVIEW	INTEGER(I4)	1_I4	Output format code.
POST_FORMAT_GMSH	INTEGER(I4)	2_I4	Output format code.
POST_FRAME_LOCAL	INTEGER(I4)	1_I4	Output frame code.
POST_FRAME_GLOBAL	INTEGER(I4)	2_I4	Output frame code.

Table C.31. Derived type ANALYSIS_TYPE — Content of ANALYSIS.dat.

Component	Type	Shape	Default	Meaning
SOLUTION	INTEGER(I4)	""	0_I4	Displacement coefficients in field-major DOF order.
MODE_COUNT	INTEGER(I4)	""	0_I4	Number of modes (requested / converged).
FIELD_RECORD	LOGICAL	""	.FALSE.	Optional fields record: the physics that are solved (the kinematics only define how each field is expanded). without the record (field_record false) every field expanded in kinematics.dat is solved, as in the historical format.
FIELD_MECH	LOGICAL	""	.TRUE.	
FIELD_THERMO	LOGICAL	""	.FALSE.	
FIELD_PIEZO	LOGICAL	""	.FALSE.	

Component	Type	Shape	Default	Meaning
JOIN_COINCIDENT	LOGICAL	``	.FALSE.	join coincident [tol]: dofs of different nodes are joined when they are the same field at the same point; tol is a fraction of the diagonal of the model.
JOIN_TOLERANCE	REAL(R8)	``	1.0E-6_R8	
SHEAR	INTEGER(I4)	(3)	SHEAR_NONE	Shear correction of beams, plates, solids.

Table C.32. Derived type POST_FIELD_REQUEST_TYPE — One para or gmsh record. split holds the nine subdivision counts: beam (section x, axis, section z), plate (x, y, thickness), solid (x, y, z). cell_nodes is 8 (linear) or 20/27 (quadratic).

Component	Type	Shape	Default	Meaning
FORMAT	INTEGER(I4)	``	POST_FORMAT_PARAVIEW	Output format (VTK or GMSH).
FRAME	INTEGER(I4)	``	POST_FRAME_GLOBAL	Output frame (local or global) or frame id.
CELL_NODES	INTEGER(I4)	``	20_I4	Nodes per output cell.
SPLIT	INTEGER(I4)	(9)	1_I4	Subdivision counts.

Table C.33. Derived type POST_POINT_REQUEST_TYPE — One PNT record.

Component	Type	Shape	Default	Meaning
ID	INTEGER(I4)	``	0_I4	Identifier.
COORDINATE	REAL(R8)	(3)	0.0_R8	Coordinates.

Table C.34. Derived type POST_DB_TYPE — Content of POSTPROCESSING.dat.

Component	Type	Shape	Default	Meaning
FIELD	TYPE(POST_FIELD_REQUEST_TYPE), ALLOCATABLE	(:)	``	Field of each local DOF (or request list).
POINT	TYPE(POST_POINT_REQUEST_TYPE), ALLOCATABLE	(:)	``	Point requests or point coordinate.
WRITE_STIFFNESS	LOGICAL	``	.FALSE.	Dump K.
WRITE_MASS	LOGICAL	``	.FALSE.	Dump M.
WRITE_FORCE	LOGICAL	``	.FALSE.	Dump F.
WRITE_UNKNOWN	LOGICAL	``	.FALSE.	Dump u.
WRITE_ENERGY	LOGICAL	``	.FALSE.	Energy request.

C.2.2 MUL2_BOUNDARY_CONDITIONS

File SRC/MODEL/mul2_boundary_conditions.for.

Mechanical boundary-condition input model.

Uses: MUL2_KINDS.

Table C.35. Public constants of MUL2_BOUNDARY_CONDITIONS.

Name	Type	Value	Meaning
BC_DISPLACEMENT_PLANE	INTEGER(I4)	1_I4	Boundary-condition kind.
BC_FORCE_POINT	INTEGER(I4)	2_I4	Boundary-condition kind.
BC_VALUE_POINT	INTEGER(I4)	3_I4	Prescribed value of a field (e.g. the electric potential) at a point; on a plane the record is a bc_displacement_plane.
BC_TIE_PLANE	INTEGER(I4)	4_I4	Floating electrode: the potential dofs on a plane share one value.
BC_SURFACE	INTEGER(I4)	5_I4	Load on the exposed surface (heat flux, sun, convection): see mul2_surface_loads. surface = 1 flux, 2 sun, 3 convection.

Table C.36. Derived type `BOUNDARY_CONDITION_TYPE` — One D-PLANE or F-POINT record.

Component	Type	Shape	Default	Meaning
KIND	INTEGER(I4)	``	0_I4	Record kind.
ID	INTEGER(I4)	``	0_I4	Identifier.
PLANE	REAL(R8)	(4)	0.0_R8	Plane A, B, C, D.
POINT	REAL(R8)	(3)	0.0_R8	Point requests or point coordinate.
ACTIVE	LOGICAL	(9)	.FALSE.	One entry per field (u v w t p b szz sxz syz).
VALUE	REAL(R8)	(9)	0.0_R8	Values.
FIELD_ID	INTEGER(I4)	``	0_I4	Spatial field (fields.dat) multiplying the value, 0 = none.
SURFACE	INTEGER(I4)	``	0_I4	
PARAM	REAL(R8)	(6)	0.0_R8	

Table C.37. Derived type `BOUNDARY_DB_TYPE` — Boundary-condition database.

Component	Type	Shape	Default	Meaning
ITEM	TYPE(BOUNDARY_CONDITION_TYPE), ALLOCATABLE	(:)	``	List of items.

C.2.3 MUL2_ELEMENTS

File `SRC/MODEL/mul2_elements.for`.

Finite-element connectivity definitions.

Uses: `MUL2_KINDS`.

Table C.38. Derived type `ELEMENT_TYPE` — One structural element.

Component	Type	Shape	Default	Meaning
ID	INTEGER(I8)	``	0_I8	Identifier.
TOPOLOGY	INTEGER(I4)	``	0_I4	Topology code.
NODE_ID	INTEGER(I8), ALLOCATABLE	(:)	``	Node ids of the element.
FRAME_ID	INTEGER(I4)	``	0_I4	
EXPANSION_ID	INTEGER(I4)	``	0_I4	Expansion mesh used.
GENERAL	LOGICAL	``	.FALSE.	True when the name of the element forces the general (curved) geometry: s4 s9 s16 (shells), cb2 cb3 cb4 (curved beams).

Table C.39. Derived type `ELEMENT_DB_TYPE` — Element database.

Component	Type	Shape	Default	Meaning
ITEM	TYPE(ELEMENT_TYPE), ALLOCATABLE	(:)	``	List of items.

C.2.4 MUL2_EXPANSION_MESHES

File `SRC/MODEL/mul2_expansion_meshes.for`.

Lagrange expansion meshes used on sections or thicknesses.

Uses: `MUL2_KINDS`.

Table C.40. Derived type `EXPANSION_NODE_TYPE` — Node of an expansion mesh (section or thickness).

Component	Type	Shape	Default	Meaning
ID	INTEGER(I8)	``	0_I8	Identifier.
COORDINATE	REAL(R8)	(3)	0.0_R8	Coordinates.

Table C.41. Derived type `EXPANSION_ELEMENT_TYPE` — Sub-element of an expansion mesh.

Component	Type	Shape	Default	Meaning
ID	INTEGER(I8)	``	0_I8	Identifier.
TOPOLOGY	INTEGER(I4)	``	0_I4	Topology code.
LAMINATION_ID	INTEGER(I4)	``	0_I4	Lamination id.
NODE_ID	INTEGER(I8), ALLOCATABLE	(:)	``	Node ids of the element.
MODE_TERM	INTEGER(I4), ALLOCATABLE	(:)	``	Hle only: global term and sign of every local function (term 0 = function omitted by the order of an adjacent sub-element).
MODE_SIGN	REAL(R8), ALLOCATABLE	(:)	``	
MID_NODE_ID	INTEGER(I8)	(4)	0_I8	Hq only: third point (a mesh node, 0 = none) of each side; a side with one is the circular arc through its vertices and that point.

Table C.42. Derived type `EXPANSION_MESH_TYPE` — One expansion mesh (`EXP_MESH_nn` / `EXP_CONN_nn`).

Component	Type	Shape	Default	Meaning
ID	INTEGER(I4)	``	0_I4	Identifier.
NODE	TYPE(EXPANSION_NODE_TYPE), ALLOCATABLE	(:)	``	
ELEMENT	TYPE(EXPANSION_ELEMENT_TYPE), ALLOCATABLE	(:)	``	Element index.
IS_HLE	LOGICAL	``	.FALSE.	Hle only: number of global terms and their origin. the first terms are the vertex modes; term_node(:,t) = (node,0) for a vertex, (node_a,node_b) for a side mode, (0,0) for an internal mode; term_owner(t) is the sub-element of an internal mode.
N_TERM	INTEGER(I4)	``	0_I4	
TERM_NODE	INTEGER(I4), ALLOCATABLE	(:,:)	``	
TERM_OWNER	INTEGER(I4), ALLOCATABLE	(:)	``	
NODE_TERM	INTEGER(I4), ALLOCATABLE	(:)	``	Term of the vertex mode of every node (0 = node used only to describe a curved side, it carries no degree of freedom).

Table C.43. Derived type `EXPANSION_DB_TYPE` — Expansion mesh database.

Component	Type	Shape	Default	Meaning
ITEM	TYPE(EXPANSION_MESH_TYPE), ALLOCATABLE	(:)	``	List of items.

Table C.44. Procedures of `MUL2_EXPANSION_MESHES`.

Procedure	Kind	Visibility	Purpose
HAS_EXPANSION_NODE_ID	Function → logical	public	True when the mesh has a node with this id.
FIND_EXPANSION_INDEX	Function → integer(i4)	public	Index of an expansion mesh by id.
EXPANSION_TERM_COUNT	Function → integer(i4)	public	Number of degrees of freedom per structural node and field of a lagrange or hle mesh.

HAS_EXPANSION_NODE_ID

True when the mesh has a node with this id.

Argument	Declaration	Meaning
MESH	TYPE(EXPANSION_MESH_TYPE), intent(IN)	Expansion mesh.
ID	INTEGER(I8), intent(IN)	Identifier.

FIND_EXPANSION_INDEX

Index of an expansion mesh by id.

Argument	Declaration	Meaning
DATABASE	TYPE(EXPANSION_DB_TYPE), intent(IN)	Database being filled.
ID	INTEGER(I4), intent(IN)	Identifier.

EXPANSION_TERM_COUNT

Number of degrees of freedom per structural node and field of a lagrange or hle mesh.

Argument	Declaration	Meaning
MESH	TYPE(EXPANSION_MESH_TYPE), intent(IN)	Expansion mesh.

C.2.5 MUL2_FIELDS

File SRC/MODEL/mul2_fields.for.

Spatial fields (fields.dat): $f(x,y,z)$ = sum of terms. a boundary condition may end with a field number: its value is then multiplied by f at the position of every node (or integration point of a surface load). terms (same grammar as the historical code): const c1 c1 x-exp c1 c2 c1 x**c2 (also y-exp, z-exp) cos-x c1 c2 c3 c1 cos(c2 x + c3) (also cos-y, cos-z) sin-x c1 c2 c3 c1 sin(c2 x + c3) (also sin-y, sin-z) atanx c1 c2 c3 c1 atan2(z-c2,x-c3) in degrees b-sin c1 c2 c3 c4 c5 c1 sin(c2 pi x/c3) sin(c4 pi y/c5)

Uses: MUL2_KINDS.

Table C.48. Public constants of MUL2_FIELDS.

Name	Type	Value	Meaning
TERM_CONSTANT	INTEGER(I4)	1_I4	
TERM_POWER_X	INTEGER(I4)	2_I4	
TERM_POWER_Y	INTEGER(I4)	3_I4	
TERM_POWER_Z	INTEGER(I4)	4_I4	
TERM_ARCTAN	INTEGER(I4)	5_I4	
TERM_COS_X	INTEGER(I4)	6_I4	
TERM_COS_Y	INTEGER(I4)	7_I4	
TERM_SIN_X	INTEGER(I4)	8_I4	
TERM_SIN_Y	INTEGER(I4)	9_I4	
TERM_BI_SINE	INTEGER(I4)	10_I4	
TERM_COS_Z	INTEGER(I4)	11_I4	
TERM_SIN_Z	INTEGER(I4)	12_I4	

Table C.49. Derived type FIELD_TYPE — component list

Component	Type	Shape	Default	Meaning
ID	INTEGER(I4)	``	0_I4	Identifier.
TERM	INTEGER(I4), ALLOCATABLE	(:)	``	Expansion term of each local DOF.
CONSTANT	REAL(R8), ALLOCATABLE	(:,:)	``	

Table C.50. Derived type `FIELD_DB_TYPE` — component list

Component	Type	Shape	Default	Meaning
ITEM	<code>TYPE(FIELD_TYPE), ALLOCATABLE</code>	<code>(:)</code>	<code>''</code>	List of items.

Table C.51. Procedures of `MUL2_FIELDS`.

Procedure	Kind	Visibility	Purpose
<code>FIND_FIELD_INDEX</code>	Function $\rightarrow$ integer(i4)	public	
<code>EVALUATE_FIELD</code>	Function $\rightarrow$ real(r8)	public	Value of field id at a point (1 for id = 0: no field).

`FIND_FIELD_INDEX`

Argument	Declaration	Meaning
DATABASE	<code>TYPE(FIELD_DB_TYPE), intent(IN)</code>	Database being filled.
ID	<code>INTEGER(I4), intent(IN)</code>	Identifier.

`EVALUATE_FIELD`

Value of field id at a point (1 for id = 0: no field).

Argument	Declaration	Meaning
DATABASE	<code>TYPE(FIELD_DB_TYPE), intent(IN)</code>	Database being filled.
ID	<code>INTEGER(I4), intent(IN)</code>	Identifier.
X	<code>REAL(R8), intent(IN)</code>	Vector or coordinate.
Y	<code>REAL(R8), intent(IN)</code>	Result vector.
Z	<code>REAL(R8), intent(IN)</code>	

C.2.6 MUL2_HLE_MODES

File `SRC/MODEL/mul2_hle_modes.for`.

Global terms of a hierarchical legendre (hle) expansion mesh. every sub-element carries its own polynomial order: the global terms (the degrees of freedom of one structural node and field) are first vertex modes, one per node that is a vertex of a sub-element (the nodal value); nodes that only describe a curved side carry no term (`mesh%node_term = 0`); next side modes: one group for every geometric side (pair of nodes), modes $k = 2..pmin$, $pmin$ being the lowest order of the sub-elements that share the side; last internal modes, owned by one sub-element. a side mode of a sub-element of higher order than its neighbour is omitted (constrained to zero): the trace on the shared side is then a polynomial of the lower degree on both sides, so the field stays c_0 . the side functions have the parity $(-1)^k$, **so a sub-element whose local side direction is opposite to the global one (from the lower to the higher node id) uses the sign $(-1)^k$.**

Uses: `MUL2_KINDS`, `MUL2_STATUS`, `MUL2_TOPOLOGIES`, `MUL2_EXPANSION_MESHES`, `MUL2_HLE_SHAPE`.

Table C.54. Procedures of `MUL2_HLE_MODES`.

Procedure	Kind	Visibility	Purpose
<code>BUILD_HLE_MESH</code>	Subroutine	public	

`BUILD_HLE_MESH`

Argument	Declaration	Meaning
MESH	<code>TYPE(EXPANSION_MESH_TYPE), intent(INOUT)</code>	Expansion mesh.
STATUS	<code>TYPE(STATUS_TYPE), intent(INOUT)</code>	Status of the call (OK, warning or error with message and source).

C.2.7 MUL2_INCIDENCE

File SRC/MODEL/mul2_incidence.for.

Node-to-element incidence (csr): the elements touching each node.

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_NODES, MUL2_ELEMENTS.

Table C.56. Derived type `NODE_INCIDENCE_TYPE` — Elements of node i: element(first(i):first(i+1)-1), ascending.

Component	Type	Shape	Default	Meaning
FIRST	INTEGER(I4), ALLOCATABLE	(:)	``	First global DOF of (node, field) or first element of a node.
ELEMENT	INTEGER(I4), ALLOCATABLE	(:)	``	Element index.

Table C.57. Procedures of `MUL2_INCIDENCE`.

Procedure	Kind	Visibility	Purpose
BUILD_NODE_INCIDENCE	Subroutine	public	Node -> incident elements table (CSR).

`BUILD_NODE_INCIDENCE`

Node -> incident elements table (CSR).

Argument	Declaration	Meaning
NODES	TYPE(NODE_DB_TYPE), intent(IN)	Structural node database.
ELEMENTS	TYPE(ELEMENT_DB_TYPE), intent(IN)	Structural element database.
INCIDENCE	TYPE(NODE_INCIDENCE_TYPE), intent(INOUT)	Node -> element incidence table.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.2.8 MUL2_KINEMATICS

File SRC/MODEL/mul2_kinematics.for.

Field-dependent kinematics definitions.

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_STRINGS.

Table C.59. Public constants of `MUL2_KINEMATICS`.

Name	Type	Value	Meaning
N_FIELDS	INTEGER(I4)	9_I4	Number of fields of the unified formulation.
N_SOLVED_FIELDS	INTEGER(I4)	5_I4	Fields solved by the 101/103 analyses: u, v, w, t, p.
FIELD_U	INTEGER(I4)	1_I4	Field index of U.
FIELD_V	INTEGER(I4)	2_I4	Field index of V.
FIELD_W	INTEGER(I4)	3_I4	Field index of W.
FIELD_T	INTEGER(I4)	4_I4	Field index of T.
FIELD_P	INTEGER(I4)	5_I4	Field index of P.
FIELD_B	INTEGER(I4)	6_I4	Field index of B.
FIELD_SZZ	INTEGER(I4)	7_I4	Field index of SZZ.
FIELD_SXZ	INTEGER(I4)	8_I4	Field index of SXZ.
FIELD_SYZ	INTEGER(I4)	9_I4	Field index of SYZ.
EXPANSION_NONE	INTEGER(I4)	0_I4	Expansion family NONE.
EXPANSION_TE	INTEGER(I4)	1_I4	Expansion family TE.
EXPANSION_LE	INTEGER(I4)	2_I4	Expansion family LE.
EXPANSION_HLE	INTEGER(I4)	3_I4	Expansion family HLE.

Name	Type	Value	Meaning
EXPANSION_MISC	INTEGER(I4)	4_I4	Expansion family MISC.

Table C.60. Derived type `EXPANSION_SPEC_TYPE` — Expansion of one field: family (NONE, TE, LE, HLE, M), order and field reference.

Component	Type	Shape	Default	Meaning
FAMILY	INTEGER(I4)	``	EXPANSION_NONE	Expansion family code.
ORDER	INTEGER(I4)	``	0_I4	Order of the matrix / expansion order / quadrature order (see type).
FIELD_REFERENCE	INTEGER(I4)	``	0_I4	Index of a user-defined function set.

Table C.61. Derived type `KINEMATIC_TYPE` — Expansion specification of the nine fields.

Component	Type	Shape	Default	Meaning
ID	INTEGER(I4)	``	0_I4	Identifier.
FIELD	TYPE(EXPANSION_SPEC_TYPE)	(N_FIELDS)	``	Field of each local DOF (or request list).

Table C.62. Derived type `KINEMATICS_DB_TYPE` — Kinematics database.

Component	Type	Shape	Default	Meaning
ITEM	TYPE(KINEMATIC_TYPE), ALLOCATABLE	(:)	``	List of items.

Table C.63. Procedures of `MUL2_KINEMATICS`.

Procedure	Kind	Visibility	Purpose
HAS_KINEMATIC_ID	Function → logical	public	True when the kinematic id exists.
APPLY_FIELD_SELECTION	Subroutine	public	The <code>fields</code> record of <code>analysis.dat</code> decides which physics are solved: the expansions of the fields that are switched off are disabled (the kinematics keep their definition), and a field that is switched on must be expanded somewhere.
ANY_FIELD_ACTIVE	Function → logical	public	True when some kinematic expands the field (1..9) at all.
FIND_KINEMATIC_INDEX	Function → integer(i4)	public	Index of a kinematic by id.
PARSE_EXPANSION_TOKEN	Subroutine	public	Parse TE<n>, LE, HLE<n>, M<n> or NONE into an expansion specification.
READ_SUFFIX	Subroutine	private	Read the integer suffix of a token.
INVALID_TOKEN	Subroutine	private	Set the error of an unknown expansion token.

`HAS_KINEMATIC_ID`

True when the kinematic id exists.

Argument	Declaration	Meaning
DATABASE	TYPE(KINEMATICS_DB_TYPE), intent(IN)	Database being filled.
ID	INTEGER(I4), intent(IN)	Identifier.

`APPLY_FIELD_SELECTION`

The `fields` record of `analysis.dat` decides which physics are solved: the expansions of the fields that are switched off are disabled (the kinematics keep their definition), and a field that is switched on must be expanded somewhere.

Argument	Declaration	Meaning
DATABASE	TYPE(KINEMATICS_DB_TYPE), intent(INOUT)	Database being filled.

Argument	Declaration	Meaning
THERMO	LOGICAL, intent(IN)	
PIEZO	LOGICAL, intent(IN)	
STATUS	TYPE(STATUS_TYPE), intent(INOUT)	Status of the call (OK, warning or error with message and source).

ANY_FIELD_ACTIVE

True when some kinematic expands the field (1..9) at all.

Argument	Declaration	Meaning
DATABASE	TYPE(KINEMATICS_DB_TYPE), intent(IN)	Database being filled.
FIELD	INTEGER(I4), intent(IN)	Field index (1=U, 2=V, 3=W, ...).

FIND_KINEMATIC_INDEX

Index of a kinematic by id.

Argument	Declaration	Meaning
DATABASE	TYPE(KINEMATICS_DB_TYPE), intent(IN)	Database being filled.
ID	INTEGER(I4), intent(IN)	Identifier.

PARSE_EXPANSION_TOKEN

Parse TE<n>, LE, HLE<n>, M<n> or NONE into an expansion specification.

Argument	Declaration	Meaning
TOKEN	CHARACTER(LEN=*), intent(IN)	Text token such as TE3, LE or NONE.
SPEC	TYPE(EXPANSION_SPEC_TYPE), intent(OUT)	Expansion specification (family, order, field reference).
STATUS	TYPE(STATUS_TYPE), intent(INOUT)	Status of the call (OK, warning or error with message and source).

C.2.9 MUL2_LAMINATIONS

File SRC/MODEL/mul2_laminations.for.

Material assignments and orientations.

Uses: MUL2_KINDS.

Table C.69. Public constants of MUL2_LAMINATIONS.

Name	Type	Value	Meaning
LAMINATION_CONSTANT	INTEGER(I4)	1_I4	

Table C.70. Derived type LAMINATION_TYPE — One lamination: material and orientation angles.

Component	Type	Shape	Default	Meaning
ID	INTEGER(I4)	``	0_I4	Identifier.
KIND	INTEGER(I4)	``	0_I4	Record kind.
MATERIAL_ID	INTEGER(I4)	``	0_I4	Material id.
ANGLE_Y_DEG	REAL(R8)	``	0.0_R8	Angle about y (degrees).
ANGLE_Z_DEG	REAL(R8)	``	0.0_R8	Angle about z (degrees).

Table C.71. Derived type LAMINATION_DB_TYPE — Lamination database.

Component	Type	Shape	Default	Meaning
ITEM	TYPE(LAMINATION_TYPE), ALLOCATABLE	(:)	``	List of items.

Table C.72. Procedures of `MUL2_LAMINATIONS`.

Procedure	Kind	Visibility	Purpose
<code>FIND_LAMINATION_INDEX</code>	Function → integer(i4)	public	Index of a lamination by id.

`FIND_LAMINATION_INDEX`

Index of a lamination by id.

Argument	Declaration	Meaning
<code>DATABASE</code>	<code>TYPE(LAMINATION_DB_TYPE), intent(IN)</code>	Database being filled.
<code>ID</code>	<code>INTEGER(I4), intent(IN)</code>	Identifier.

C.2.10 `MUL2_MODEL`

File `SRC/MODEL/mul2_model.for`.

Complete input model: one container passed to the high-level steps.

Uses: `MUL2_KINDS`, `MUL2_ANALYSIS_INPUT`, `MUL2_KINEMATICS`, `MUL2_NODES`, `MUL2_ELEMENTS`, `MUL2_EXPANSION_MESHES`, `MUL2_REFERENCE_SYSTEMS`, `MUL2_MATERIALS`, `MUL2_LAMINATIONS`, `MUL2_BOUNDARY_CONDITIONS`, `MUL2_FIELDS`, `MUL2_TIME_INPUT`.

Table C.74. Derived type `MODEL_TYPE` — Complete input model.

Component	Type	Shape	Default	Meaning
<code>INPUT_PATH</code>	<code>CHARACTER(LEN=512)</code>	""	' '	Input directory.
<code>ANALYSIS</code>	<code>TYPE(ANALYSIS_TYPE)</code>	""	""	Analysis data.
<code>POST</code>	<code>TYPE(POST_DB_TYPE)</code>	""	""	Post-processing data.
<code>KINEMATICS</code>	<code>TYPE(KINEMATICS_DB_TYPE)</code>	""	""	Kinematics.
<code>NODES</code>	<code>TYPE(NODE_DB_TYPE)</code>	""	""	Nodes.
<code>ELEMENTS</code>	<code>TYPE(ELEMENT_DB_TYPE)</code>	""	""	Elements.
<code>EXPANSIONS</code>	<code>TYPE(EXPANSION_DB_TYPE)</code>	""	""	Expansion meshes.
<code>VECTORS</code>	<code>TYPE(REFERENCE_VECTOR_DB_TYPE)</code>	""	""	Vectors.
<code>MATERIALS</code>	<code>TYPE(MATERIAL_DB_TYPE)</code>	""	""	Materials.
<code>LAMINATIONS</code>	<code>TYPE(LAMINATION_DB_TYPE)</code>	""	""	Laminations.
<code>BOUNDARIES</code>	<code>TYPE(BOUNDARY_DB_TYPE)</code>	""	""	Boundary conditions.
<code>FIELDS</code>	<code>TYPE(FIELD_DB_TYPE)</code>	""	""	
<code>TIME</code>	<code>TYPE(TIME_INPUT_TYPE)</code>	""	""	
<code>FREQ</code>	<code>TYPE(FREQ_INPUT_TYPE)</code>	""	""	
<code>NL</code>	<code>TYPE(NL_INPUT_TYPE)</code>	""	""	

C.2.11 `MUL2_NODES`

File `SRC/MODEL/mul2_nodes.for`.

Geometric node definitions.

Uses: `MUL2_KINDS`, `MUL2_STATUS`, `MUL2_SORTING`.**Table C.75.** Derived type `NODE_TYPE` — One structural node.

Component	Type	Shape	Default	Meaning
<code>ID</code>	<code>INTEGER(I8)</code>	""	<code>0_I8</code>	Identifier.
<code>COORDINATE</code>	<code>REAL(R8)</code>	(3)	<code>0.0_R8</code>	Coordinates.

Component	Type	Shape	Default	Meaning
KINEMATIC_ID	INTEGER(I4)	``	0_I4	Kinematic id of the node.
HAS_DIRECTOR	LOGICAL	``	.FALSE.	Exact director of a shell node (directors.dat, optional).
DIRECTOR	REAL(R8)	(3)	0.0_R8	

Table C.76. Derived type `NODE_DB_TYPE` — `Id_list/id_order` are the sorted-id lookup built by `build_node_index`.

Component	Type	Shape	Default	Meaning
ITEM	TYPE(NODE_TYPE), ALLOCATABLE	(:)	``	List of items.
ID_LIST	INTEGER(I8), ALLOCATABLE	(:)	``	Sorted node ids.
ID_ORDER	INTEGER(I4), ALLOCATABLE	(:)	``	Permutation of the sorted ids.

Table C.77. Procedures of `MUL2_NODES`.

Procedure	Kind	Visibility	Purpose
HAS_NODE_ID	Function → logical	public	True when a node with this id exists.
FIND_NODE_INDEX	Function → integer(i4)	public	Index of a node by id (binary search).
BUILD_NODE_INDEX	Subroutine	public	Sorted-id lookup ($\mathcal{O}(\log n)$ search); duplicate ids are an error.

HAS_NODE_ID

True when a node with this id exists.

Argument	Declaration	Meaning
NODES	TYPE(NODE_DB_TYPE), intent(IN)	Structural node database.
ID	INTEGER(I8), intent(IN)	Identifier.

FIND_NODE_INDEX

Index of a node by id (binary search).

Argument	Declaration	Meaning
NODES	TYPE(NODE_DB_TYPE), intent(IN)	Structural node database.
ID	INTEGER(I8), intent(IN)	Identifier.

BUILD_NODE_INDEX

Sorted-id lookup ($\mathcal{O}(\log n)$ search); duplicate ids are an error.

Argument	Declaration	Meaning
NODES	TYPE(NODE_DB_TYPE), intent(INOUT)	Structural node database.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.2.12 MUL2_REFERENCE_SYSTEMS

File `SRC/MODEL/mul2_reference_systems.for`.

User reference vectors and orthonormal element frames.

Uses: `MUL2_KINDS`.

Table C.81. Derived type `REFERENCE_VECTOR_TYPE` — One versor.

Component	Type	Shape	Default	Meaning
ID	INTEGER(I4)	``	0_I4	Identifier.
DIRECTION	REAL(R8)	(3)	0.0_R8	Direction vector.

Table C.82. Derived type REFERENCE_VECTOR_DB_TYPE — Versor database.

Component	Type	Shape	Default	Meaning
ITEM	TYPE(REFERENCE_VECTOR_TYPE), ALLOCATABLE	(:)	''	List of items.

Table C.83. Derived type ELEMENT_FRAME_DB_TYPE — Origin and rotation of the local frame of every element.

Component	Type	Shape	Default	Meaning
ORIGIN	REAL(R8), ALLOCATABLE	(:, :)	''	Frame origin (first node).
GLOBAL_TO_LOCAL	REAL(R8), ALLOCATABLE	(:,:, :)	''	Rows = local axes in global components (local = R global).
LOCAL_TO_GLOBAL	REAL(R8), ALLOCATABLE	(:,:, :)	''	Transpose of GLOBAL_TO_LOCAL.
GENERAL_INDEX	INTEGER(I4), ALLOCATABLE	(:)	''	Curved beams and shells (general geometry): index of the element in the compact arrays below (0 = ordinary straight / flat element), the triad of every node (vectors a1, a2, a3 of a section or of a surface: see mul2_general_geometry) and the reference vector.
GENERAL_TRIAD	REAL(R8), ALLOCATABLE	(:,:, :, :)	''	
GENERAL_REFERENCE	REAL(R8), ALLOCATABLE	(:, :)	''	
GENERAL_KINK	INTEGER(I4), ALLOCATABLE	(:, :)	''	Shell nodes shared with elements of another normal (kinks): code 1: the first-order term is the rotation of the node; code -1: the element thickness axis is opposite to the one of the node.

C.2.13 MUL2_TIME_INPUT

File SRC/MODEL/mul2_time_input.for.

TIME-RESPONSE INPUT (TIME_RESP.DAT, HISTORICAL FORMAT) AND THE LOAD AMPLITUDE $a(t)$ THAT MULTIPLIES EVERY LOAD AND PRESCRIBED VALUE. TI TF NSTEP POST_EVERY (OUTPUT EVERY N STEPS) N_SPECIFIC (EXTRA OUTPUT STEPS, ONE PER LINE) N_LOADS STEP t0 A A FOR $t \geq t0$ IMPU t0 A A AT THE FIRST STEP $t \geq t0$ ONLY RAMP t0 t1 A LINEAR RISE FROM 0 TO A SINU PHI OMEGA A A SIN(PHI + OMEGA t) HASU t0 F PHI2 N PHI1 A HANN-WINDOWED SINE BURST HACU t0 F PHI2 N PHI1 A HANN-WINDOWED COSINE BURST WPSU t0 F PHI2 N PHI1 A SINE BURST WITH SIN**2 WINDOW OPTIONAL RECORDS AFTER THE LOADS (ANY ORDER): NEWMARK BETA GAMMA DEFAULT 0.25 0.5 RAYLEIGH ALPHA BETA C = ALPHA M + BETA K (OVERRIDES DAMP) THETA TH THERMAL THETA-METHOD, DEFAULT 1 T0 TEMPERATURE ABSOLUTE REFERENCE TEMPERATURE: SWITCHES ON THE THERMOELASTIC HEATING TERM

Uses: MUL2_KINDS.

Table C.84. Public constants of MUL2_TIME_INPUT.

Name	Type	Value	Meaning
LOAD_STEP	INTEGER(I4)	1_I4	
LOAD_RAMP	INTEGER(I4)	2_I4	
LOAD_IMPULSE	INTEGER(I4)	3_I4	
LOAD_SINE	INTEGER(I4)	4_I4	
LOAD_HANN_SINE	INTEGER(I4)	5_I4	
LOAD_HANN_COSINE	INTEGER(I4)	6_I4	
LOAD_WINDOWED_SINE	INTEGER(I4)	7_I4	

Table C.85. Derived type `TIME_LOAD_TYPE` — component list

Component	Type	Shape	Default	Meaning
KIND	INTEGER(I4)	''	0_I4	Record kind.
ARG	REAL(R8)	(8)	0.0_R8	

Table C.86. Derived type `TIME_INPUT_TYPE` — component list

Component	Type	Shape	Default	Meaning
INITIAL_TIME	REAL(R8)	''	0.0_R8	
FINAL_TIME	REAL(R8)	''	0.0_R8	
STEP_COUNT	INTEGER(I4)	''	0_I4	
POST_EVERY	INTEGER(I4)	''	1_I4	
SPECIFIC_STEP	INTEGER(I4), ALLOCATABLE	(:)	''	
LOAD	TYPE(TIME_LOAD_TYPE), ALLOCATABLE	(:)	''	
NEWMARK_BETA	REAL(R8)	''	0.25_R8	
NEWMARK_GAMMA	REAL(R8)	''	0.5_R8	
RAYLEIGH_MASS	REAL(R8)	''	0.0_R8	
RAYLEIGH_STIFFNESS	REAL(R8)	''	0.0_R8	
HAS_RAYLEIGH	LOGICAL	''	.FALSE.	
THETA	REAL(R8)	''	1.0_R8	
REFERENCE_TEMPERATURE	REAL(R8)	''	0.0_R8	

Table C.87. Derived type `FREQ_INPUT_TYPE` — Freq_resp.dat: fi ff / nstep / post_every, then optional rayleigh alpha beta and t0 records (frequencies in hz).

Component	Type	Shape	Default	Meaning
FIRST_FREQUENCY	REAL(R8)	''	0.0_R8	
LAST_FREQUENCY	REAL(R8)	''	0.0_R8	
STEP_COUNT	INTEGER(I4)	''	0_I4	
POST_EVERY	INTEGER(I4)	''	1_I4	
RAYLEIGH_MASS	REAL(R8)	''	0.0_R8	
RAYLEIGH_STIFFNESS	REAL(R8)	''	0.0_R8	
HAS_RAYLEIGH	LOGICAL	''	.FALSE.	
REFERENCE_TEMPERATURE	REAL(R8)	''	0.0_R8	

Table C.88. Derived type `NL_INPUT_TYPE` — NL_info.dat (nonlinear static): solvtec / nlstep / itmax / toll / post_every (the first five records of the historical file). solvtec 1: load control (nlstep equal steps, lambda = n/nlstep); solvtec 2: arc length (nlstep = maximum number of steps). optional sixth record (arc length only): ds0 dsmin dsmax lambda_max; a zero ds0 / dsmin / dsmax means automatic.

Component	Type	Shape	Default	Meaning
TECHNIQUE	INTEGER(I4)	''	1_I4	
STEP_COUNT	INTEGER(I4)	''	10_I4	
MAX_ITERATIONS	INTEGER(I4)	''	30_I4	
TOLERANCE	REAL(R8)	''	1.0E-6_R8	
POST_EVERY	INTEGER(I4)	''	1_I4	
ARC_LENGTH	REAL(R8)	''	0.0_R8	
ARC_LENGTH_MIN	REAL(R8)	''	0.0_R8	

Component	Type	Shape	Default	Meaning
ARC_LENGTH_MAX	REAL(R8)	``	0.0_R8	
LAMBDA_MAX	REAL(R8)	``	1.0_R8	

Table C.89. Procedures of MUL2_TIME_INPUT .

Procedure	Kind	Visibility	Purpose
STEP_TIME	Function → real(r8)	public	
STEP_IS_OUTPUT	Function → logical	public	
LOAD_AMPLITUDE	Function → real(r8)	public	Amplitude at step i (t = step_time): sum of all the load records. the impulse is a single-step value, as in the historical code.

STEP_TIME

Argument	Declaration	Meaning
TIME	TYPE(TIME_INPUT_TYPE), intent(IN)	
STEP	INTEGER(I4), intent(IN)	

STEP_IS_OUTPUT

Argument	Declaration	Meaning
TIME	TYPE(TIME_INPUT_TYPE), intent(IN)	
STEP	INTEGER(I4), intent(IN)	

LOAD_AMPLITUDE

Amplitude at step i (t = step_time): sum of all the load records. the impulse is a single-step value, as in the historical code.

Argument	Declaration	Meaning
TIME	TYPE(TIME_INPUT_TYPE), intent(IN)	
STEP	INTEGER(I4), intent(IN)	

C.2.14MUL2_TOPOLOGIES

File SRC/MODEL/mul2_topologies.for .

Single registry of supported element topologies.

Uses: MUL2_KINDS , MUL2_STRINGS , MUL2_HLE_SHAPE .

Table C.93. Public constants of MUL2_TOPOLOGIES .

Name	Type	Value	Meaning
TOPOLOGY_UNKNOWN	INTEGER(I4)	0_I4	Topology code UNKNOWN.
TOPOLOGY_B2	INTEGER(I4)	101_I4	Topology code B2.
TOPOLOGY_B3	INTEGER(I4)	102_I4	Topology code B3.
TOPOLOGY_B4	INTEGER(I4)	103_I4	Topology code B4.
TOPOLOGY_Q4	INTEGER(I4)	201_I4	Topology code Q4.
TOPOLOGY_Q9	INTEGER(I4)	202_I4	Topology code Q9.
TOPOLOGY_Q16	INTEGER(I4)	203_I4	Topology code Q16.
TOPOLOGY_T3	INTEGER(I4)	204_I4	Topology code T3.

Name	Type	Value	Meaning
TOPOLOGY_T6	INTEGER(I4)	205_I4	Topology code T6.
TOPOLOGY_H8	INTEGER(I4)	301_I4	Topology code H8.
TOPOLOGY_H20	INTEGER(I4)	302_I4	Topology code H20.
TOPOLOGY_H27	INTEGER(I4)	303_I4	Topology code H27.
TOPOLOGY_T4	INTEGER(I4)	304_I4	Topology code T4.
TOPOLOGY_T10	INTEGER(I4)	305_I4	Topology code T10.
TOPOLOGY_P6	INTEGER(I4)	306_I4	Topology code P6.
TOPOLOGY_S1	INTEGER(I4)	401_I4	Topology code S1.
TOPOLOGY_HB_BASE	INTEGER(I4)	1100_I4	Hle sub-elements: base + polynomial order p (1..99). the base value itself is the family name before the order is known.
TOPOLOGY_HQ_BASE	INTEGER(I4)	2100_I4	Topology code HQ_BASE.

Table C.94. Procedures of `MUL2_TOPOLOGIES`.

Procedure	Kind	Visibility	Purpose
TOPOLOGY_FROM_NAME	Function → integer(i4)	public	Topology code from the name in the input (B2, Q9, H27, ...).
TOPOLOGY_NODE_COUNT	Function → integer(i4)	public	Number of nodes of a topology.
TOPOLOGY_NATURAL_DIMENSION	Function → integer(i4)	public	Natural dimension 0..3 of a topology.
TOPOLOGY_FUNCTION_COUNT	Function → integer(i4)	public	Number of shape functions (nodes for lagrange elements, modes for hle sub-elements).
TOPOLOGY_IS_HLE	Function → logical	public	True for a hle sub-element of a defined order.
TOPOLOGY_IS_HLE_FAMILY	Function → logical	public	True for the family name alone (order not yet read).
TOPOLOGY_HLE_ORDER	Function → integer(i4)	public	

TOPOLOGY_FROM_NAME

Topology code from the name in the input (B2, Q9, H27, ...).

Argument	Declaration	Meaning
NAME	CHARACTER(LEN=*), intent(IN)	Name.

TOPOLOGY_NODE_COUNT

Number of nodes of a topology.

Argument	Declaration	Meaning
TOPOLOGY	INTEGER(I4), intent(IN)	Topology code (B2..S1).

TOPOLOGY_NATURAL_DIMENSION

Natural dimension 0..3 of a topology.

Argument	Declaration	Meaning
TOPOLOGY	INTEGER(I4), intent(IN)	Topology code (B2..S1).

TOPOLOGY_FUNCTION_COUNT

Number of shape functions (nodes for lagrange elements, modes for hle sub-elements).

Argument	Declaration	Meaning
TOPOLOGY	INTEGER(I4), intent(IN)	Topology code (B2..S1).

TOPOLOGY_IS_HLE

True for a hle sub-element of a defined order.

Argument	Declaration	Meaning
TOPOLOGY	INTEGER(I4), intent(IN)	Topology code (B2..S1).

TOPOLOGY_IS_HLE_FAMILY

True for the family name alone (order not yet read).

Argument	Declaration	Meaning
TOPOLOGY	INTEGER(I4), intent(IN)	Topology code (B2..S1).

TOPOLOGY_HLE_ORDER

Argument	Declaration	Meaning
TOPOLOGY	INTEGER(I4), intent(IN)	Topology code (B2..S1).

C.3 Layer IO

Readers of the legacy input files and assembly of the complete model.

C.3.1 MUL2_TEXT_IO

File SRC/IO/mul2_text_io.for.

Shared text-input utilities.

Table C.102. Procedures of MUL2_TEXT_IO.

Procedure	Kind	Visibility	Purpose
NEXT_DATA_LINE	Subroutine	public	Next non-blank, non-comment line of a text file.

NEXT_DATA_LINE

Next non-blank, non-comment line of a text file.

Argument	Declaration	Meaning
UNIT	INTEGER, intent(IN)	Fortran unit number.
LINE	CHARACTER(LEN=*), intent(OUT)	Text line.
IOS	INTEGER, intent(OUT)	I/O status.

C.3.2 MUL2_READ_ANALYSIS

File SRC/IO/read_analysis.for.

Analysis.dat and postprocessing.dat readers (legacy formats).

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_STRINGS, MUL2_TEXT_IO, MUL2_ANALYSIS_INPUT.

Table C.104. Procedures of MUL2_READ_ANALYSIS.

Procedure	Kind	Visibility	Purpose
-----------	------	------------	---------

Procedure	Kind	Visibility	Purpose
READ_ANALYSIS_FILE	Subroutine	public	Record order: solution id, number of modes and the shear treatments (none, redi, seli, mitc) of beam, plate and solid elements. the obsolete solver record of the historical format (a second integer between the solution id and the number of modes) is recognised and ignored with a warning.
READ_JOIN_RECORD	Subroutine	private	join coincident [tolerance]: the dofs of different nodes that are the same field at the same point are joined (tolerance: fraction of the diagonal of the model, default 1e-6).
READ_FIELD_RECORD	Subroutine	private	fields mech thermo piezo: the active physics. mech (displacements) is always solved; stress (mixed rmvt elements) is reserved.
READ_POSTPROCESSING_FILE	Subroutine	public	First record: number of requests. then para / gmsh field outputs, pnt point outputs and the matrix dumps kmat mmatrix frce unkn enrg.

READ_ANALYSIS_FILE

Record order: solution id, number of modes and the shear treatments (none, redi, seli, mitc) of beam, plate and solid elements. the obsolete solver record of the historical format (a second integer between the solution id and the number of modes) is recognised and ignored with a warning.

Argument	Declaration	Meaning
FILE_NAME	CHARACTER(LEN=*), intent(IN)	Path of the file.
ANALYSIS	TYPE(ANALYSIS_TYPE), intent(OUT)	Output: content of ANALYSIS.dat.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

READ_POSTPROCESSING_FILE

First record: number of requests. then para / gmsh field outputs, pnt point outputs and the matrix dumps kmat mmatrix frce unkn enrg.

Argument	Declaration	Meaning
FILE_NAME	CHARACTER(LEN=*), intent(IN)	Path of the file.
POST	TYPE(POST_DB_TYPE), intent(INOUT)	Output: content of POSTPROCESSING.dat.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.3.3 MUL2_READ_BOUNDARY_CONDITIONS

File SRC/IO/read_boundary_conditions.for.

Reader for the first mechanical 101/103 boundary-condition gate.

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_STRINGS, MUL2_TEXT_IO, MUL2_BOUNDARY_CONDITIONS.

Table C.107. Procedures of MUL2_READ_BOUNDARY_CONDITIONS.

Procedure	Kind	Visibility	Purpose
READ_BOUNDARY_CONDITIONS_FILE	Subroutine	public	Read BC.dat (D-PLANE and F-POINT).
READ_FIELD_ID	Subroutine	private	A record of a value may end with a field number (fields.dat): the record is read again with one more item; if it is not there nothing changes.
PARSE_COMPONENTS	Subroutine	private	Parse three displacement tokens; N means free.

READ_BOUNDARY_CONDITIONS_FILE

Read BC.dat (D-PLANE and F-POINT).

Argument	Declaration	Meaning
FILE_NAME	CHARACTER(LEN=*), intent(IN)	Path of the file.

Argument	Declaration	Meaning
DATABASE	TYPE(BOUNDARY_DB_TYPE), intent(INOUT)	Database being filled.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.3.4 MUL2_READ_CONNECTIVITY

File SRC/IO/read_connectivity.for.

Connectivity.dat reader.

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_TEXT_IO, MUL2_SORTING, MUL2_NODES, MUL2_ELEMENTS, MUL2_TOPOLOGIES.

Table C.109. Procedures of MUL2_READ_CONNECTIVITY.

Procedure	Kind	Visibility	Purpose
READ_CONNECTIVITY_FILE	Subroutine	public	Read CONNECTIVITY.dat: topology, node ids, versor and expansion mesh of every element.
ALIAS_NAME	Subroutine	private	S4 s9 s16 are the q4 q9 q16 plate topologies and cb2 cb3 cb4 the b2 b3 b4 beam topologies, with the general (curved) geometry forced.

READ_CONNECTIVITY_FILE

Read CONNECTIVITY.dat: topology, node ids, versor and expansion mesh of every element.

Argument	Declaration	Meaning
FILE_NAME	CHARACTER(LEN=*), intent(IN)	Path of the file.
NODES	TYPE(NODE_DB_TYPE), intent(IN)	Structural node database.
ELEMENTS	TYPE(ELEMENT_DB_TYPE), intent(INOUT)	Structural element database.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.3.5 MUL2_READ_DIRECTORS

File SRC/IO/read_directors.for.

Reader of directors.dat (optional file). exact directors (normals) of the shell nodes, when the geometry is known (cad, analytic surface). without the file, or for the nodes that are not listed, the director is the average of the normals of the elements that share the node. n_directors node_id vx vy vz (n_directors lines, not normalised)

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_TEXT_IO, MUL2_NODES.

Table C.111. Procedures of MUL2_READ_DIRECTORS.

Procedure	Kind	Visibility	Purpose
READ_DIRECTORS_FILE	Subroutine	public	

READ_DIRECTORS_FILE

Argument	Declaration	Meaning
FILE_NAME	CHARACTER(LEN=*), intent(IN)	Path of the file.
NODES	TYPE(NODE_DB_TYPE), intent(INOUT)	Structural node database.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.3.6 MUL2_READ_EXPANSIONS

File SRC/IO/read_expansions.for.

Read exp_mesh_nn.dat and exp_conn_nn.dat files.

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_TEXT_IO, MUL2_TOPOLOGIES, MUL2_HLE_MODES, MUL2_EXPANSION_MESHES.

Table C.113. Procedures of MUL2_READ_EXPANSIONS.

Procedure	Kind	Visibility	Purpose
READ_EXPANSION_FILES	Subroutine	public	Read one EXP_MESH / EXP_CONN pair.
READ_EXPANSION_SET	Subroutine	public	Read all expansion meshes EXP_*_01 ... NN.
READ_EXPANSION_NODES	Subroutine	private	Read EXP_MESH_nn.dat.
READ_EXPANSION_ELEMENTS	Subroutine	private	Read EXP_CONN_nn.dat.
DUPLICATE_NODE_ID	Function → logical	private	True when an id appears twice.
DUPLICATE_ELEMENT_ID	Function → logical	private	True when an id appears twice.

READ_EXPANSION_FILES

Read one EXP_MESH / EXP_CONN pair.

Argument	Declaration	Meaning
MESH_FILE	CHARACTER(LEN=*), intent(IN)	Path of EXP_MESH_nn.dat.
CONNECTIVITY_FILE	CHARACTER(LEN=*), intent(IN)	Path of EXP_CONN_nn.dat.
EXPANSION_ID	INTEGER(I4), intent(IN)	Id given to the mesh (index nn of the file).
MESH	TYPE(EXPANSION_MESH_TYPE), intent(INOUT)	Expansion mesh.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

READ_EXPANSION_SET

Read all expansion meshes EXP_*_01 ... NN.

Argument	Declaration	Meaning
PATH	CHARACTER(LEN=*), intent(IN)	Input directory.
COUNT	INTEGER(I4), intent(IN)	Number of items.
DATABASE	TYPE(EXPANSION_DB_TYPE), intent(INOUT)	Database being filled.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.3.7 MUL2_READ_FIELDS

File SRC/IO/read_fields.for.

Reader of fields.dat (optional file). n_fields field id n_terms term c1 [c2 ...] (n_terms lines) ...

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_STRINGS, MUL2_TEXT_IO, MUL2_FIELDS.

Table C.116. Procedures of MUL2_READ_FIELDS.

Procedure	Kind	Visibility	Purpose
READ_FIELDS_FILE	Subroutine	public	

READ_FIELDS_FILE

Argument	Declaration	Meaning
FILE_NAME	CHARACTER(LEN=*), intent(IN)	Path of the file.
DATABASE	TYPE(FIELD_DB_TYPE), intent(INOUT)	Database being filled.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.3.8 MUL2_READ_KINEMATICS

File SRC/IO/read_kinematics.for.

Kinematics.dat reader.

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_STRINGS, MUL2_TEXT_IO, MUL2_KINEMATICS.

Table C.118. Procedures of MUL2_READ_KINEMATICS.

Procedure	Kind	Visibility	Purpose
READ_KINEMATICS_FILE	Subroutine	public	Read KINEMATICS.dat (field-dependent expansions).
DUPLICATE_ID	Function → logical	private	True when an id appears twice.

READ_KINEMATICS_FILE

Read KINEMATICS.dat (field-dependent expansions).

Argument	Declaration	Meaning
FILE_NAME	CHARACTER(LEN=*), intent(IN)	Path of the file.
DATABASE	TYPE(KINEMATICS_DB_TYPE), intent(INOUT)	Database being filled.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.3.9 MUL2_READ_LAMINATIONS

File SRC/IO/read_laminations.for.

Lamination.dat reader: first gate supports constant lam2 records.

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_TEXT_IO, MUL2_STRINGS, MUL2_MATERIALS, MUL2_LAMINATIONS.

Table C.120. Procedures of MUL2_READ_LAMINATIONS.

Procedure	Kind	Visibility	Purpose
READ_LAMINATIONS_FILE	Subroutine	public	Read LAMINATION.dat (LAM2 records).

READ_LAMINATIONS_FILE

Read LAMINATION.dat (LAM2 records).

Argument	Declaration	Meaning
FILE_NAME	CHARACTER(LEN=*), intent(IN)	Path of the file.
MATERIALS	TYPE(MATERIAL_DB_TYPE), intent(IN)	Material database.
DATABASE	TYPE(LAMINATION_DB_TYPE), intent(INOUT)	Database being filled.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.3.10 MUL2_READ_LEGACY_NODES

File SRC/IO/read_legacy_nodes.for.

Adapter for the historical nodes.dat (id x y z model [order]). the historical file has no kinematics.dat. each distinct (model, order) pair becomes one kinematic with the same expansion for u, v and w and no other active field. le ignores the order column. only the models implemented by v3 (te, le) are accepted.

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_STRINGS, MUL2_TEXT_IO, MUL2_KINEMATICS, MUL2_NODES.

Table C.122. Procedures of MUL2_READ_LEGACY_NODES.

Procedure	Kind	Visibility	Purpose
READ_LEGACY_NODES_FILE	Subroutine	public	Read the historical NODES.dat (TE/LE token) and create the kinematics.
REGISTER_KINEMATIC	Function → integer(i4)	private	Return the id of the (family, order) kinematic, creating it if new.

READ_LEGACY_NODES_FILE

Read the historical NODES.dat (TE/LE token) and create the kinematics.

Argument	Declaration	Meaning
FILE_NAME	CHARACTER(LEN=*), intent(IN)	Path of the file.
KINEMATICS	TYPE(KINEMATICS_DB_TYPE), intent(INOUT)	Field-dependent kinematics database.
NODES	TYPE(NODE_DB_TYPE), intent(INOUT)	Structural node database.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.3.11 MUL2_READ_MATERIALS

File SRC/IO/read_materials.for.

Legacy-compatible material.dat reader for mechanical materials.

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_TEXT_IO, MUL2_STRINGS, MUL2_MATERIALS.

Table C.124. Procedures of MUL2_READ_MATERIALS.

Procedure	Kind	Visibility	Purpose
READ_MATERIALS_FILE	Subroutine	public	Read MATERIAL.dat (ISO-M, ORT-M, MAT-C; other records are accepted and ignored).
STORE_MATERIAL	Subroutine	private	Validate and store one material record.

READ_MATERIALS_FILE

Read MATERIAL.dat (ISO-M, ORT-M, MAT-C; other records are accepted and ignored).

Argument	Declaration	Meaning
FILE_NAME	CHARACTER(LEN=*), intent(IN)	Path of the file.
DATABASE	TYPE(MATERIAL_DB_TYPE), intent(INOUT)	Database being filled.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.3.12 MUL2_READ_MODEL

File SRC/IO/read_model.for.

READ A COMPLETE INPUT DIRECTORY INTO ONE MODEL. THE DIRECTORY FOLLOWS THE HISTORICAL LAYOUT (ANALYSIS, NODES, CONNECTIVITY, EXP_MESH_NN, EXP_CONN_NN, MATERIAL, LAMINATION, VERSORS, BC, POSTPROCESSING). IF KINEMATICS.dat IS PRESENT THE V3 NODES.dat (WITH KINEMATIC_ID) IS EXPECTED, OTHERWISE THE HISTORICAL NODES.dat IS CONVERTED BY MUL2_READ_LEGACY_NODES.

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_KINEMATICS, MUL2_MODEL, MUL2_READ_ANALYSIS, MUL2_READ_KINEMATICS, MUL2_READ_NODES, MUL2_READ_LEGACY_NODES, MUL2_READ_CONNECTIVITY, MUL2_READ_EXPANSIONS, MUL2_READ_REFERENCE_SYSTEMS, MUL2_READ_MATERIALS, MUL2_READ_LAMINATIONS, MUL2_READ_BOUNDARY_CONDITIONS, MUL2_READ_FIELDS, MUL2_READ_DIRECTORS, MUL2_READ_TIME.

Table C.126. Procedures of MUL2_READ_MODEL.

Procedure	Kind	Visibility	Purpose
READ_MODEL	Subroutine	public	Read an input directory into one MODEL_TYPE.
FILE_OF	Function → character(len=600)	private	Full path of a file of the input directory.

READ_MODEL

Read an input directory into one MODEL_TYPE.

Argument	Declaration	Meaning
INPUT_PATH	CHARACTER(LEN=*), intent(IN)	Input directory.

Argument	Declaration	Meaning
MODEL	TYPE(MODEL_TYPE), intent(INOUT)	Complete input model (all databases).
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.3.13 MUL2_READ_NODES

File SRC/IO/read_nodes.for .

Nodes.dat reader for the v3 format.

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_KINEMATICS, MUL2_NODES, MUL2_TEXT_IO.

Table C.128. Procedures of MUL2_READ_NODES .

Procedure	Kind	Visibility	Purpose
READ_NODES_FILE	Subroutine	public	Read the NODES.dat that carries a kinematic id.

READ_NODES_FILE

Read the NODES.dat that carries a kinematic id.

Argument	Declaration	Meaning
FILE_NAME	CHARACTER(LEN=*), intent(IN)	Path of the file.
KINEMATICS	TYPE(KINEMATICS_DB_TYPE), intent(IN)	Field-dependent kinematics database.
NODES	TYPE(NODE_DB_TYPE), intent(INOUT)	Structural node database.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.3.14 MUL2_READ_REFERENCE_SYSTEMS

File SRC/IO/read_reference_systems.for .

Versors.dat reader.

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_TEXT_IO, MUL2_STRINGS, MUL2_REFERENCE_SYSTEMS .

Table C.130. Procedures of MUL2_READ_REFERENCE_SYSTEMS .

Procedure	Kind	Visibility	Purpose
READ_REFERENCE_VECTORS_FILE	Subroutine	public	Read VERSORS.dat.
DUPLICATE_ID	Function → logical	private	True when an id appears twice.

READ_REFERENCE_VECTORS_FILE

Read VERSORS.dat.

Argument	Declaration	Meaning
FILE_NAME	CHARACTER(LEN=*), intent(IN)	Path of the file.
DATABASE	TYPE(REFERENCE_VECTOR_DB_TYPE), intent(INOUT)	Database being filled.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.3.15 MUL2_READ_TIME

File SRC/IO/read_time.for .

Reader of time_resp.dat (see mul2_time_input for the format).

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_STRINGS, MUL2_TEXT_IO, MUL2_TIME_INPUT .

Table C.132. Procedures of MUL2_READ_TIME .

Procedure	Kind	Visibility	Purpose
READ_TIME_FILE	Subroutine	public	
NEXT_DATA	Subroutine	private	Next record, skipping comments and the dashed separators of the historical files.
READ_FREQ_FILE	Subroutine	public	Freq_resp.dat.
READ_NL_FILE	Subroutine	public	Nl_info.dat: five records (solvtec 1 = load control, 2 = arc length) and an optional sixth one ds0 dsmin dsmax lambda_max for the arc length. anything else of the historical file is unused.

READ_TIME_FILE

Argument	Declaration	Meaning
FILE_NAME	CHARACTER(LEN=*), intent(IN)	Path of the file.
TIME	TYPE(TIME_INPUT_TYPE), intent(OUT)	
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

READ_FREQ_FILE

Freq_resp.dat.

Argument	Declaration	Meaning
FILE_NAME	CHARACTER(LEN=*), intent(IN)	Path of the file.
FREQ	TYPE(FREQ_INPUT_TYPE), intent(OUT)	
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

READ_NL_FILE

Nl_info.dat: five records (solvtec 1 = load control, 2 = arc length) and an optional sixth one ds0 dsmin dsmax lambda_max for the arc length. anything else of the historical file is unused.

Argument	Declaration	Meaning
FILE_NAME	CHARACTER(LEN=*), intent(IN)	Path of the file.
NL	TYPE(NL_INPUT_TYPE), intent(OUT)	
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.4 Layer QUADRATURE

Gauss–Legendre and triangle rules.

C.4.1 MUL2_QUADRATURE

File SRC/QUADRATURE/mul2_quadrature.for.

Reference-element gauss quadrature rules.

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_TOPOLOGIES.

Table C.136. Derived type QUADRATURE_RULE_TYPE — Points and weights of a quadrature rule.

Component	Type	Shape	Default	Meaning
NATURAL_DIMENSION	INTEGER(I4)	''	0_I4	Natural dimension.
COORDINATE	REAL(R8), ALLOCATABLE	(:, :)	''	Coordinates.
WEIGHT	REAL(R8), ALLOCATABLE	(:)	''	Weights.

Table C.137. Procedures of MUL2_QUADRATURE.

Procedure	Kind	Visibility	Purpose
-----------	------	------------	---------

Procedure	Kind	Visibility	Purpose
REDUCED_POINTS_PER_DIRECTION	Function → integer(i4)	public	Gauss points per direction of the reduced rule of a structural element (one less than the full rule, as in the baseline: redi and seli). 0 = no reduced rule (triangles and any other topology).
BUILD_REDUCED_QUADRATURE	Subroutine	public	
BUILD_DEFAULT_QUADRATURE	Subroutine	public	Default Gauss rule of a topology.
DEFAULT_POINTS_PER_DIRECTION	Function → integer(i4)	public	Gauss points per direction of the default rule (0 = not tensor).
BUILD_ORDERED_QUADRATURE	Subroutine	public	Tensor rule with order points per direction (lines and quadrilaterals). used for high-order taylor expansions, whose integrands are polynomials of degree 2*n.
BUILD_LINE_RULE	Subroutine	private	Gauss-Legendre rule on [-1,1].
BUILD_TENSOR_RULE	Subroutine	private	Tensor product rule on [-1,1]^d.
BUILD_TRIANGLE_RULE	Subroutine	private	Triangle rule (1 or 3 points).
GAUSS_LEGENDRE_1D	Subroutine	private	Gauss-Legendre points and weights (closed form up to 4, Newton on the Legendre polynomial above).
CLEAR_RULE	Subroutine	private	Release a quadrature rule.

REDUCED_POINTS_PER_DIRECTION

Gauss points per direction of the reduced rule of a structural element (one less than the full rule, as in the baseline: redi and seli). 0 = no reduced rule (triangles and any other topology).

Argument	Declaration	Meaning
TOPOLOGY	INTEGER(I4), intent(IN)	Topology code (B2..S1).

BUILD_REDUCED_QUADRATURE

Argument	Declaration	Meaning
TOPOLOGY	INTEGER(I4), intent(IN)	Topology code (B2..S1).
RULE	TYPE(QUADRATURE_RULE_TYPE), intent(INOUT)	Output: quadrature rule.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

BUILD_DEFAULT_QUADRATURE

Default Gauss rule of a topology.

Argument	Declaration	Meaning
TOPOLOGY	INTEGER(I4), intent(IN)	Topology code (B2..S1).
RULE	TYPE(QUADRATURE_RULE_TYPE), intent(INOUT)	Output: quadrature rule.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

DEFAULT_POINTS_PER_DIRECTION

Gauss points per direction of the default rule (0 = not tensor).

Argument	Declaration	Meaning
TOPOLOGY	INTEGER(I4), intent(IN)	Topology code (B2..S1).

BUILD_ORDERED_QUADRATURE

Tensor rule with order points per direction (lines and quadrilaterals). used for high-order taylor expansions, whose integrands are polynomials of degree 2*n.

Argument	Declaration	Meaning
TOPOLOGY	INTEGER(I4), intent(IN)	Topology code (B2..S1).
ORDER	INTEGER(I4), intent(IN)	Order of the matrix, or polynomial/quadrature order (see routine).
RULE	TYPE(QUADRATURE_RULE_TYPE), intent(INOUT)	Output: quadrature rule.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.5 Layer FEM

Shape functions and natural derivatives of every topology.

C.5.1 MUL2_HLE_MAP

File SRC/FEM/mul2_hle_map.for.

Blending-function map of a hle quadrilateral with curved sides. the sides of a sub-element are straight unless a third point on the side is given: the side is then the circular arc through its two vertices and that point, parameterised by its angle (exact circle, no geometric approximation error). the map is the transfinite (gordon-hall) interpolation of the paper (eqs. 20-25): $q(r,s) = \sum_i n_i(r,s) x_i + \sum_e \beta_e(r,s) (c_e(t) - l_e(t))$ n_i are the bilinear vertex functions, l_e the straight side, c_e the arc, t the natural coordinate along the side and β_e the linear blending function (1 on side e , 0 on the opposite one). the functions of the expansion are not the map: the element is not isoparametric.

Uses: MUL2_KINDS, MUL2_HLE_SHAPE.

Table C.143. Derived type HLE_GEOMETRY_TYPE — component list

Component	Type	Shape	Default	Meaning
ANY_CURVED	LOGICAL	``	.FALSE.	
CURVED	LOGICAL	(4)	.FALSE.	
VERTEX	REAL(R8)	(2,4)	0.0_R8	
CENTER	REAL(R8)	(2,4)	0.0_R8	
RADIUS	REAL(R8)	(4)	0.0_R8	
THETA0	REAL(R8)	(4)	0.0_R8	
DTHETA	REAL(R8)	(4)	0.0_R8	

Table C.144. Procedures of MUL2_HLE_MAP.

Procedure	Kind	Visibility	Purpose
HLE_GEOMETRY_SETUP	Subroutine	public	Vertex(2,4): vertices in the active plane; mid(2,4) and has_mid(4): third point of each side.
HLE_MAP_EVALUATE	Subroutine	public	X(2) and $dx(:,1) = dx/dr$, $dx(:,2) = dx/ds$ at the natural point (r,s).

HLE_GEOMETRY_SETUP

Vertex(2,4): vertices in the active plane; mid(2,4) and has_mid(4): third point of each side.

Argument	Declaration	Meaning
VERTEX	REAL(R8), intent(IN)(2,4)	
MID	REAL(R8), intent(IN)(2,4)	
HAS_MID	LOGICAL, intent(IN)(4)	
GEOMETRY	TYPE(HLE_GEOMETRY_TYPE), intent(OUT)	Combined Gauss geometry (coordinates, weights).

HLE_MAP_EVALUATE

$X(2)$ and $dx(:,1) = dx/dr$, $dx(:,2) = dx/ds$ at the natural point (r,s) .

Argument	Declaration	Meaning
GEOMETRY	TYPE(HLE_GEOMETRY_TYPE), intent(IN)	Combined Gauss geometry (coordinates, weights).
R	REAL(R8), intent(IN)	
S	REAL(R8), intent(IN)	
X	REAL(R8), intent(OUT)(2)	Vector or coordinate.
DX	REAL(R8), intent(OUT)(2,2)	

C.5.2 MUL2_HLE_SHAPE

File SRC/FEM/mul2_hle_shape.for.

Hierarchical legendre expansion (hle) functions of one sub-element. the functions are the ones of pagani, garcia de miguel, carrera (comput mech 2017) and of the p-version finite element method (szabo and babuska): the trunk space of degree p. quadrilateral (natural r,s in $[-1,1]$, vertices 1(-1,-1) 2(1,-1) 3(1,1) 4(-1,1)): vertex $1/4 (1+rt r)(1+st s)$ 4 functions side $k \ 1/2 (1-s) \phi_k(r)$ side 1 (vertex 1 $\rightarrow$ 2) $1/2 (1+r) \phi_k(s)$ side 2 (vertex 2 $\rightarrow$ 3) $1/2 (1+s) \phi_k(r)$ side 3 (vertex 4 $\rightarrow$ 3) $1/2 (1-r) \phi_k(s)$ side 4 (vertex 1 $\rightarrow$ 4) 4 (p-1) functions, $k = 2..p$ internal $\phi_i(r) \phi_j(s)$, $i,j \geq 2$, $i+j \leq p$ (p-2)(p-3)/2 functions (p ≥ 4) line (natural r in $[-1,1]$): vertex $1/2 (1-r)$, $1/2 (1+r)$; internal $\phi_k(r)$, $k = 2..p$ $\phi_k(x) = \sqrt{(2k-1)/2} \int_{-1}^x l_{k-1} \ (k \geq 2)$ local order (the numbering of the paper): the vertices; then, for each $k = 2..p$, the four sides of order k followed ($k \geq 4$) by the internal pairs (i,j) with $i+j = k$, i descending.

Uses: MUL2_KINDS.

Table C.147. Public constants of MUL2_HLE_SHAPE.

Name	Type	Value	Meaning
HLE_VERTEX	INTEGER(I4)	1_I4	
HLE_SIDE	INTEGER(I4)	2_I4	
HLE_INTERNAL	INTEGER(I4)	3_I4	

Table C.148. Procedures of MUL2_HLE_SHAPE.

Procedure	Kind	Visibility	Purpose
HLE_FUNCTION_COUNT	Function $\rightarrow$ integer(i4)	public	
HLE_SIDE_ENDS	Subroutine	public	Start and end vertex of a side along its natural direction.
HLE_MODE_LIST	Subroutine	public	Kind(l), a(l), b(l) of every local function: vertex a = vertex side a = side, b = order k internal a = i, b = j (quadrilateral); a = k (line)
HLE_PHI	Subroutine	public	Φ_k and its derivative for $k = 2..kmax$.
EVALUATE_HLE_SHAPE	Subroutine	public	

HLE_FUNCTION_COUNT

Argument	Declaration	Meaning
QUAD	LOGICAL, intent(IN)	
P	INTEGER(I4), intent(IN)	

HLE_SIDE_ENDS

Start and end vertex of a side along its natural direction.

Argument	Declaration	Meaning
SIDE	INTEGER(I4), intent(IN)	
START	INTEGER(I4), intent(OUT)	
FINISH	INTEGER(I4), intent(OUT)	

HLE_MODE_LIST

Kind(l), a(l), b(l) of every local function: vertex a = vertex side a = side, b = order k internal a = i, b = j (quadrilateral); a = k (line)

Argument	Declaration	Meaning
QUAD	LOGICAL, intent(IN)	
P	INTEGER(I4), intent(IN)	
KIND	INTEGER(I4), intent(OUT)(:)	
A	INTEGER(I4), intent(OUT)(:)	Matrix or operand A.
B	INTEGER(I4), intent(OUT)(:)	Matrix or operand B.

HLE_PHI

Phi_k and its derivative for k = 2..kmax.

Argument	Declaration	Meaning
KMAX	INTEGER(I4), intent(IN)	
X	REAL(R8), intent(IN)	Vector or coordinate.
PHI	REAL(R8), intent(OUT)(:)	
DPHI	REAL(R8), intent(OUT)(:)	

EVALUATE_HLE_SHAPE

Argument	Declaration	Meaning
QUAD	LOGICAL, intent(IN)	
P	INTEGER(I4), intent(IN)	
X	REAL(R8), intent(IN)(:)	Vector or coordinate.
N	REAL(R8), intent(OUT)(:)	Shape function values.
DN	REAL(R8), intent(OUT)(:,:)	Shape derivatives.

C.5.3 MUL2_SHAPE_FUNCTIONS

File SRC/FEM/mul2_shape_functions.for.

Shape functions and natural derivatives.

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_TOPOLOGIES, MUL2_HLE_SHAPE.

Table C.154. Procedures of MUL2_SHAPE_FUNCTIONS.

Procedure	Kind	Visibility	Purpose
EVALUATE_SHAPE	Subroutine	public	Shape functions and natural derivatives of a topology at a natural point.
SHAPE_B	Subroutine	private	Lagrange line shapes (B2, B3, B4).
SHAPE_Q4	Subroutine	private	Bilinear quadrilateral.
SHAPE_Q9	Subroutine	private	Biquadratic quadrilateral.
SHAPE_Q16	Subroutine	private	Bicubic quadrilateral.
SHAPE_T3	Subroutine	private	Linear triangle.

Procedure	Kind	Visibility	Purpose
SHAPE_T6	Subroutine	private	Quadratic triangle.
SHAPE_H8	Subroutine	private	Trilinear hexahedron.
SHAPE_H27	Subroutine	private	Triquadratic hexahedron.
TENSOR_2D	Subroutine	private	Tensor product of 1D Lagrange functions on a quadrilateral.
TENSOR_3D	Subroutine	private	Tensor product of 1D Lagrange functions on a hexahedron.
LAGRANGE_1D	Subroutine	private	1D Lagrange polynomial and derivative of given order and node index.

EVALUATE_SHAPE

Shape functions and natural derivatives of a topology at a natural point.

Argument	Declaration	Meaning
TOPOLOGY	INTEGER(I4), intent(IN)	Topology code (B2..S1).
NATURAL	REAL(R8), intent(IN)(:)	Natural coordinates (xi, eta, nu).
N	REAL(R8), intent(OUT)(:)	Output: shape function values.
DN_DNATURAL	REAL(R8), intent(OUT)(:,:))	Derivatives with respect to natural coordinates.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.6 Layer GEOMETRY

Element reference frames, Jacobians and local-frame gradients.

C.6.1 MUL2_ELEMENT_FRAMES

File SRC/GEOMETRY/mul2_element_frames.for.

Construction of right-handed element reference systems.

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_NODES, MUL2_ELEMENTS, MUL2_REFERENCE_SYSTEMS, MUL2_TOPOLOGIES.

Table C.156. Procedures of MUL2_ELEMENT_FRAMES.

Procedure	Kind	Visibility	Purpose
BUILD_ELEMENT_FRAMES	Subroutine	public	Right-handed local frame of every element from node coordinates and the versor.
BUILD_BEAM_FRAME	Subroutine	private	Beam frame: y along the axis, z from the versor orthogonalised, $x = y \times z$.
BUILD_SURFACE_FRAME	Subroutine	private	Plate frame: z normal to the surface, x from the projected versor, $y = z \times x$.
NORMALIZE	Subroutine	private	Normalise a vector, error if its length is (almost) zero.
CROSS_PRODUCT	Function	private	Vector product.
FIND_VECTOR_INDEX	Function $\rightarrow$ integer(i4)	private	Index of a versor by id.
SET_IDENTITY	Subroutine	private	Identity 3x3 matrix.
CLEAR_FRAMES	Subroutine	private	Release the frame arrays.

BUILD_ELEMENT_FRAMES

Right-handed local frame of every element from node coordinates and the versor.

Argument	Declaration	Meaning
NODES	TYPE(NODE_DB_TYPE), intent(IN)	Structural node database.
ELEMENTS	TYPE(ELEMENT_DB_TYPE), intent(IN)	Structural element database.

Argument	Declaration	Meaning
REFERENCE_VECTORS	TYPE(REFERENCE_VECTOR_DB_TYPE), intent(IN)	Versor database.
FRAMES	TYPE(ELEMENT_FRAME_DB_TYPE), intent(INOUT)	Element reference frames.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.6.2 MUL2_GENERAL_GEOMETRY

File SRC/GEOMETRY/mul2_general_geometry.for.

General (curved) geometry of beams and shells. an ordinary element has one local frame: a straight beam or a flat plate. an element with the general geometry (a curved beam, a shell) carries a triad a1, a2, a3 at every node and the map from the natural coordinates to space is $x(x_i, c) = \sum_i n_i(x_i) [x_i + c_1 a1_i + c_2 a2_i + c_3 a3_i]$ where c is the position of a point of the expansion mesh (the section of a beam, the thickness of a shell, in physical lengths). the triad beam: a1 = a2 x a3, a2 = tangent of the axis, a3 = reference vector (sor) projected on the normal plane; shell: a3 = director (normal), a1 = sor projected on the tangent plane, a2 = a3 x a1 (the frame of a flat plate). the tangents and the directors are averaged over the elements that share a node (only the ones within the feature angle, so a true edge keeps one director per side), which makes the geometry continuous. the thickness comes from the expansion mesh, as for the plates. an element is general when its name says so (s4 s9 s16 cb2 cb3 cb4) or when its nodes are not on a line (b2 b3 b4) or in a plane (q4 q9 q16).

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_NODES, MUL2_ELEMENTS, MUL2_REFERENCE_SYSTEMS, MUL2_TOPOLOGIES, MUL2_SHAPE_FUNCTIONS.

Table C.158. Procedures of MUL2_GENERAL_GEOMETRY.

Procedure	Kind	Visibility	Purpose
IS_GENERAL_ELEMENT	Function → logical	public	
GENERAL_NODE_NATURAL	Subroutine	public	Natural coordinates of node j of a beam (1 value) or plate (2 values) topology, in the order of the shape functions.
GENERAL_MAP	Subroutine	public	Position x and jacobian rows g(a,:) = d x / d t_a of the map, with t = (structural natural coordinates, active expansion coordinates). ds number of structural coordinates (1 beam, 2 shell) coord nodal coordinates (3,n); triad nodal triads (3,3,n) n, dn shape values and derivatives (n, ds) c position in the expansion mesh (3), axis its active axes
GENERAL_FRAME	Subroutine	public	Frame rows (e1; e2; e3) = global_to_local at a point, with the same rules as the ordinary frames: a beam has e2 along the axis (tangent of the axis curve, c = 0) and e3 from the reference vector, a shell has e3 normal to the coordinate surface and e1 from the reference vector.
GENERAL_OFFSET	Subroutine	public	Offset of an expansion point c from the node, in global components (the equivalent of local_to_global * c of an ordinary element).
BUILD_GENERAL_GEOMETRY	Subroutine	public	Finds the general elements and builds the nodal triads.
IS_CURVED	Function → logical	private	True when the nodes are not on the line of a beam / in the plane of a plate.
FIND_VECTOR	Function → integer(i4)	private	
UNIT_VECTOR	Subroutine	private	
CROSS	Function	private	
ITOA	Function	private	

IS_GENERAL_ELEMENT

Argument	Declaration	Meaning
FRAMES	TYPE(ELEMENT_FRAME_DB_TYPE), intent(IN)	Element reference frames.
ELEMENT	INTEGER(I4), intent(IN)	Element index.

GENERAL_NODE_NATURAL

Natural coordinates of node j of a beam (1 value) or plate (2 values) topology, in the order of the shape functions.

Argument	Declaration	Meaning
TOPOLOGY	INTEGER(I4), intent(IN)	Topology code (B2..S1).
J	INTEGER(I4), intent(IN)	
XI	REAL(R8), intent(OUT)(2)	

GENERAL_MAP

Position x and jacobian rows $g(a,:) = d x / d t_a$ of the map, with $t =$ (structural natural coordinates, active expansion coordinates). ds number of structural coordinates (1 beam, 2 shell) coord nodal coordinates (3,n); triad nodal triads (3,3,n) n, dn shape values and derivatives (n, ds) c position in the expansion mesh (3), axis its active axes

Argument	Declaration	Meaning
DS	INTEGER(I4), intent(IN)	
N_NODE	INTEGER(I4), intent(IN)	
COORD	REAL(R8), intent(IN)(:,:)	
TRIAD	REAL(R8), intent(IN)(:,:,:))	
N	REAL(R8), intent(IN)(:)	Shape function values.
DN	REAL(R8), intent(IN)(:,:)	Shape derivatives.
C	REAL(R8), intent(IN)(3)	
AXIS	INTEGER(I4), intent(IN)(:)	Active local axes.
X	REAL(R8), intent(OUT)(3)	Vector or coordinate.
G	REAL(R8), intent(OUT)(3,3)	

GENERAL_FRAME

Frame rows (e1; e2; e3) = global_to_local at a point, with the same rules as the ordinary frames: a beam has e2 along the axis (tangent of the axis curve, $c = 0$) and e3 from the reference vector, a shell has e3 normal to the coordinate surface and e1 from the reference vector.

Argument	Declaration	Meaning
DS	INTEGER(I4), intent(IN)	
N_NODE	INTEGER(I4), intent(IN)	
COORD	REAL(R8), intent(IN)(:,:)	
DN	REAL(R8), intent(IN)(:,:)	Shape derivatives.
G	REAL(R8), intent(IN)(3,3)	
REFERENCE	REAL(R8), intent(IN)(3)	
R	REAL(R8), intent(OUT)(3,3)	
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

GENERAL_OFFSET

Offset of an expansion point *c* from the node, in global components (the equivalent of `local_to_global * c` of an ordinary element).

Argument	Declaration	Meaning
FRAMES	TYPE(ELEMENT_FRAME_DB_TYPE), intent(IN)	Element reference frames.
ELEMENT	INTEGER(I4), intent(IN)	Element index.
LOCAL_NODE	INTEGER(I4), intent(IN)	Local structural node index.
C	REAL(R8), intent(IN)(3)	
OFFSET	REAL(R8), intent(OUT)(3)	

BUILD_GENERAL_GEOMETRY

Finds the general elements and builds the nodal triads.

Argument	Declaration	Meaning
NODES	TYPE(NODE_DB_TYPE), intent(IN)	Structural node database.
ELEMENTS	TYPE(ELEMENT_DB_TYPE), intent(IN)	Structural element database.
VECTORS	TYPE(REFERENCE_VECTOR_DB_TYPE), intent(IN)	DOF vectors (e.g. mode shapes), one per column.
FRAMES	TYPE(ELEMENT_FRAME_DB_TYPE), intent(INOUT)	Element reference frames.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.6.3 MUL2_JACOBIANS

File SRC/GEOMETRY/mul2_jacobians.for.

Square isoparametric jacobian and derivative transformation.

Uses: MUL2_KINDS, MUL2_STATUS.

Table C.165. Procedures of MUL2_JACOBIANS.

Procedure	Kind	Visibility	Purpose
EVALUATE_SQUARE_JACOBIAN	Subroutine	public	Jacobian, determinant, inverse and physical derivatives for a square mapping of dimension 1, 2 or 3.
INVERT_SMALL_MATRIX	Subroutine	private	Invert a 1x1, 2x2 or 3x3 matrix.

EVALUATE_SQUARE_JACOBIAN

Jacobian, determinant, inverse and physical derivatives for a square mapping of dimension 1, 2 or 3.

Argument	Declaration	Meaning
COORDINATE	REAL(R8), intent(IN)(:,:))	Coordinates.
DN_DNATURAL	REAL(R8), intent(IN)(:,:))	Derivatives with respect to natural coordinates.
DIMENSION	INTEGER(I4), intent(IN)	Natural dimension.
JACOBIAN	REAL(R8), intent(OUT)(:,:))	Output: Jacobian matrix.
DETERMINANT	REAL(R8), intent(OUT)	Output: determinant of the Jacobian.
INVERSE_JACOBIAN	REAL(R8), intent(OUT)(:,:))	Output: inverse Jacobian.
DN_DPHYSICAL	REAL(R8), intent(OUT)(:,:))	Output: shape derivatives with respect to the physical coordinates.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.6.4 MUL2_LOCAL_GRADIENTS

File `SRC/GEOMETRY/mul2_local_gradients.for`.

Local-frame gradients of the shape functions of one element. the element lives in its active local axes (beam: axis 2; plate: axes 1,2; solid and expansion elements: their own axes). the jacobian is square in those axes; the gradient is embedded back into the three local coordinates (inactive components are zero).

Uses: `MUL2_KINDS`, `MUL2_STATUS`, `MUL2_TOPOLOGIES`, `MUL2_SHAPE_FUNCTIONS`, `MUL2_JACOBIANS`.

Table C.167. Procedures of `MUL2_LOCAL_GRADIENTS`.

Procedure	Kind	Visibility	Purpose
<code>STRUCTURAL_ACTIVE_AXES</code>	Subroutine	public	Active local axes of a structural element of given dimension.
<code>LOCAL_SHAPE_GRADIENT</code>	Subroutine	public	$\text{Gradient}(:,i)$ = gradient of shape i in the local frame.
<code>EVALUATE_SHAPE_AND_GRADIENT</code>	Subroutine	public	Shapes and local gradients of a structural element at a natural point. $\text{node_local}(:,i)$ is the local coordinate of node i .

`STRUCTURAL_ACTIVE_AXES`

Active local axes of a structural element of given dimension.

Argument	Declaration	Meaning
<code>DIMENSION</code>	<code>INTEGER(I4), intent(IN)</code>	Natural dimension.
<code>AXIS</code>	<code>INTEGER(I4), intent(OUT)(3)</code>	Active local axes.

`LOCAL_SHAPE_GRADIENT`

$\text{Gradient}(:,i)$ = gradient of shape i in the local frame.

Argument	Declaration	Meaning
<code>DIMENSION</code>	<code>INTEGER(I4), intent(IN)</code>	Natural dimension.
<code>AXIS</code>	<code>INTEGER(I4), intent(IN)(3)</code>	Active local axes.
<code>COORDINATE</code>	<code>REAL(R8), intent(IN)(:,:)</code>	Coordinates.
<code>NATURAL_DERIVATIVE</code>	<code>REAL(R8), intent(IN)(:,:)</code>	Derivatives of the shapes with respect to the natural coordinates.
<code>GRADIENT</code>	<code>REAL(R8), intent(OUT)(:,:)</code>	Gradient (3 components).
<code>STATUS</code>	<code>TYPE(STATUS_TYPE), intent(OUT)</code>	Status of the call (OK, warning or error with message and source).

`EVALUATE_SHAPE_AND_GRADIENT`

Shapes and local gradients of a structural element at a natural point. $\text{node_local}(:,i)$ is the local coordinate of node i .

Argument	Declaration	Meaning
<code>TOPOLOGY</code>	<code>INTEGER(I4), intent(IN)</code>	Topology code (B2..S1).
<code>NODE_LOCAL</code>	<code>REAL(R8), intent(IN)(:,:)</code>	Local-frame coordinates of the element nodes.
<code>NATURAL</code>	<code>REAL(R8), intent(IN)(3)</code>	Natural coordinates (ξ , η , ν).
<code>SHAPE</code>	<code>REAL(R8), intent(OUT)(:)</code>	Output: shape function values.
<code>GRADIENT</code>	<code>REAL(R8), intent(OUT)(:,:)</code>	Gradient (3 components).
<code>STATUS</code>	<code>TYPE(STATUS_TYPE), intent(OUT)</code>	Status of the call (OK, warning or error with message and source).

C.7 Layer CUF

Expansion bases (Taylor, Lagrange), point factors and the fundamental nucleus.

C.7.1 MUL2_CUF_BASES

File SRC/CUF/mul2_cuf_bases.for.

Cuf expansion bases. taylor order matches the authoritative code.

Uses: MUL2_KINDS, MUL2_STATUS.

Table C.171. Procedures of MUL2_CUF_BASES.

Procedure	Kind	Visibility	Purpose
TAYLOR_BASIS_TERM_COUNT	Function $\rightarrow$ integer(i4)	public	Number of Taylor monomials of order N in 1D (N+1) or 2D ((N+1)(N+2)/2).
EVALUATE_TAYLOR_BASIS	Subroutine	public	All Taylor monomials and their gradients at a section point.
EVALUATE_TAYLOR_TERM	Subroutine	public	One Taylor monomial $x^a z^b$ and its gradient.
INTEGER_POWER	Function $\rightarrow$ real(r8)	private	x^n by repeated multiplication (no overflow of the generic power).

TAYLOR_BASIS_TERM_COUNT

Number of Taylor monomials of order N in 1D (N+1) or 2D ((N+1)(N+2)/2).

Argument	Declaration	Meaning
ORDER	INTEGER(I4), intent(IN)	Order of the matrix, or polynomial/quadrature order (see routine).
DIMENSION	INTEGER(I4), intent(IN)	Natural dimension.

EVALUATE_TAYLOR_BASIS

All Taylor monomials and their gradients at a section point.

Argument	Declaration	Meaning
ORDER	INTEGER(I4), intent(IN)	Order of the matrix, or polynomial/quadrature order (see routine).
DIMENSION	INTEGER(I4), intent(IN)	Natural dimension.
ACTIVE_AXIS	INTEGER(I4), intent(IN)(2)	Active local axes of the section.
COORDINATE	REAL(R8), intent(IN)(3)	Coordinates.
VALUE	REAL(R8), intent(OUT)(:)	Value(s) (see routine).
GRADIENT	REAL(R8), intent(OUT)(:,:)	Gradient (3 components).
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

EVALUATE_TAYLOR_TERM

One Taylor monomial $x^a z^b$ and its gradient.

Argument	Declaration	Meaning
ORDER	INTEGER(I4), intent(IN)	Order of the matrix, or polynomial/quadrature order (see routine).
DIMENSION	INTEGER(I4), intent(IN)	Natural dimension.
ACTIVE_AXIS	INTEGER(I4), intent(IN)(2)	Active local axes of the section.
COORDINATE	REAL(R8), intent(IN)(3)	Coordinates.
TERM	INTEGER(I4), intent(IN)	Expansion term index.
VALUE	REAL(R8), intent(OUT)	Value(s) (see routine).
GRADIENT	REAL(R8), intent(OUT)(3)	Gradient (3 components).
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.7.2 MUL2_FUNDAMENTAL_NUCLEUS

File SRC/CUF/mul2_fundamental1_nucleus.for .

Generic linear cuf fundamental nuclei at one integration point.

Uses: MUL2_KINDS .

Table C.175. Procedures of MUL2_FUNDAMENTAL_NUCLEUS .

Procedure	Kind	Visibility	Purpose
STIFFNESS_NUCLEUS	Function $\rightarrow$ real(r8)	public	Scalar fundamental nucleus: weight * $B_i^T C B_j$.
STIFFNESS_BLOCK_NUCLEUS	Subroutine	public	3x3 stiffness nucleus of a pair of nodes/terms.
MASS_NUCLEUS	Function $\rightarrow$ real(r8)	public	Scalar mass nucleus: $\rho N_i N_j$ weight for equal components.
MASS_BLOCK_NUCLEUS	Subroutine	public	3x3 mass nucleus (diagonal).
ROTATE_BLOCK_TO_GLOBAL	Subroutine	public	Rotate a 3x3 block from the local to the global frame.

STIFFNESS_NUCLEUS

Scalar fundamental nucleus: weight * $B_i^T C B_j$.

Argument	Declaration	Meaning
TEST_COLUMN	REAL(R8), intent(IN)(6)	Strain column of the test DOF.
STIFFNESS	REAL(R8), intent(IN)(6,6)	Stiffness matrix (or values).
TRIAL_COLUMN	REAL(R8), intent(IN)(6)	Strain column of the trial DOF.
INTEGRATION_WEIGHT	REAL(R8), intent(IN)	Weight including the Jacobians.

STIFFNESS_BLOCK_NUCLEUS

3x3 stiffness nucleus of a pair of nodes/terms.

Argument	Declaration	Meaning
TEST_OPERATOR	REAL(R8), intent(IN)(6,3)	Strain operator of the test DOF.
STIFFNESS	REAL(R8), intent(IN)(6,6)	Stiffness matrix (or values).
TRIAL_OPERATOR	REAL(R8), intent(IN)(6,3)	Strain operator of the trial DOF.
INTEGRATION_WEIGHT	REAL(R8), intent(IN)	Weight including the Jacobians.
BLOCK	REAL(R8), intent(OUT)(3,3)	Output: 3x3 nucleus block.

MASS_NUCLEUS

Scalar mass nucleus: $\rho N_i N_j$ weight for equal components.

Argument	Declaration	Meaning
TEST_COMPONENT	INTEGER(I4), intent(IN)	Displacement component of the test DOF.
TRIAL_COMPONENT	INTEGER(I4), intent(IN)	Displacement component of the trial DOF.
TEST_VALUE	REAL(R8), intent(IN)	Basis value of the test DOF.
TRIAL_VALUE	REAL(R8), intent(IN)	Basis value of the trial DOF.
DENSITY	REAL(R8), intent(IN)	Mass density ρ .
INTEGRATION_WEIGHT	REAL(R8), intent(IN)	Weight including the Jacobians.

MASS_BLOCK_NUCLEUS

3x3 mass nucleus (diagonal).

Argument	Declaration	Meaning
TEST_VALUE	REAL(R8), intent(IN)	Basis value of the test DOF.

Argument	Declaration	Meaning
TRIAL_VALUE	REAL(R8), intent(IN)	Basis value of the trial DOF.
DENSITY	REAL(R8), intent(IN)	Mass density rho.
INTEGRATION_WEIGHT	REAL(R8), intent(IN)	Weight including the Jacobians.
BLOCK	REAL(R8), intent(OUT)(3,3)	Output: 3x3 nucleus block.

ROTATE_BLOCK_TO_GLOBAL

Rotate a 3x3 block from the local to the global frame.

Argument	Declaration	Meaning
BLOCK_LOCAL	REAL(R8), intent(IN)(3,3)	Block in the element frame.
GLOBAL_TO_LOCAL	REAL(R8), intent(IN)(3,3)	Frame rotation (rows = local axes in global components).
BLOCK_GLOBAL	REAL(R8), intent(OUT)(3,3)	Output: block in the global frame.

C.7.3 MUL2_POINT_BASES

File SRC/CUF/mul2_point_bases.for.

Evaluate one field-dependent cuf x fem basis at a point. the basis is the product $\phi = n_{\text{structural}} * f_{\text{expansion}}$. evaluate_expansion_factor f and its gradient at an expansion point evaluate_field_basis composes phi from given n, grad n, f, grad f evaluate_point_factors n, grad n, f, grad f at a cached gauss point evaluate_point_basis phi at a cached gauss point the same composition rule serves cached gauss points, arbitrary post points and the tying points of the shear-locking corrections.

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_ELEMENTS, MUL2_EXPANSION_MESHES, MUL2_KINEMATICS, MUL2_GAUSS_POINTS, MUL2_GAUSS_GEOMETRY, MUL2_CUF_BASES, MUL2_TOPOLOGIES, MUL2_LINEAR_KINEMATICS.

Table C.181. Procedures of MUL2_POINT_BASES.

Procedure	Kind	Visibility	Purpose
EVALUATE_POINT_FACTORS	Subroutine	public	N, grad n (structural node) and f, grad f (expansion term) at a cached gauss point.
EVALUATE_POINT_BASIS	Subroutine	public	Value and gradient of the combined basis N F at a cached Gauss point.
EVALUATE_FIELD_BASIS	Subroutine	public	One scalar basis term of one field at one point. structural_value/gradient: n_i and its local-frame gradient. expansion_shape(:): shapes of the expansion element nodes. expansion_gradient(3,:): their local-frame gradients. coordinate_local: local coordinate of the expansion point.
EVALUATE_EXPANSION_FACTOR	Subroutine	public	The expansion factor f_tau and its local gradient.

EVALUATE_POINT_FACTORS

N, grad n (structural node) and f, grad f (expansion term) at a cached gauss point.

Argument	Declaration	Meaning
POINT	INTEGER(I8), intent(IN)	Global Gauss-point index (or physical point).
STRUCTURAL_NODE	INTEGER(I4), intent(IN)	Local structural node index.
TERM	INTEGER(I4), intent(IN)	Expansion term index.
SPEC	TYPE(EXPANSION_SPEC_TYPE), intent(IN)	Expansion specification (family, order, field reference).
ELEMENTS	TYPE(ELEMENT_DB_TYPE), intent(IN)	Structural element database.
EXPANSIONS	TYPE(EXPANSION_DB_TYPE), intent(IN)	Expansion (section/thickness) mesh database.
RULES	TYPE(REFERENCE_RULE_DB_TYPE), intent(IN)	Reference quadrature rule database.

Argument	Declaration	Meaning
LAYOUT	TYPE(GAUSS_LAYOUT_TYPE), intent(IN)	Gauss-point layout (global point numbering).
STRUCTURAL_CACHE	TYPE(STRUCTURAL_GEOMETRY_CACHE_TYPE), intent(IN)	Geometry cache of the structural points.
EXPANSION_CACHE	TYPE(EXPANSION_GEOMETRY_CACHE_TYPE), intent(IN)	Geometry cache of the expansion points.
GEOMETRY	TYPE(GAUSS_GEOMETRY_TYPE), intent(IN)	Combined Gauss geometry (coordinates, weights).
STRUCTURAL_VALUE	REAL(R8), intent(OUT)	Structural shape function N.
STRUCTURAL_GRADIENT	REAL(R8), intent(OUT)(3)	Gradient of the structural shape function (local frame).
FACTOR_VALUE	REAL(R8), intent(OUT)	Expansion factor F_tau.
FACTOR_GRADIENT	REAL(R8), intent(OUT)(3)	Gradient of the expansion factor (local frame).
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

EVALUATE_POINT_BASIS

Value and gradient of the combined basis N F at a cached Gauss point.

Argument	Declaration	Meaning
POINT	INTEGER(I8), intent(IN)	Global Gauss-point index (or physical point).
STRUCTURAL_NODE	INTEGER(I4), intent(IN)	Local structural node index.
TERM	INTEGER(I4), intent(IN)	Expansion term index.
SPEC	TYPE(EXPANSION_SPEC_TYPE), intent(IN)	Expansion specification (family, order, field reference).
ELEMENTS	TYPE(ELEMENT_DB_TYPE), intent(IN)	Structural element database.
EXPANSIONS	TYPE(EXPANSION_DB_TYPE), intent(IN)	Expansion (section/thickness) mesh database.
RULES	TYPE(REFERENCE_RULE_DB_TYPE), intent(IN)	Reference quadrature rule database.
LAYOUT	TYPE(GAUSS_LAYOUT_TYPE), intent(IN)	Gauss-point layout (global point numbering).
STRUCTURAL_CACHE	TYPE(STRUCTURAL_GEOMETRY_CACHE_TYPE), intent(IN)	Geometry cache of the structural points.
EXPANSION_CACHE	TYPE(EXPANSION_GEOMETRY_CACHE_TYPE), intent(IN)	Geometry cache of the expansion points.
GEOMETRY	TYPE(GAUSS_GEOMETRY_TYPE), intent(IN)	Combined Gauss geometry (coordinates, weights).
VALUE	REAL(R8), intent(OUT)	Value(s) (see routine).
GRADIENT	REAL(R8), intent(OUT)(3)	Gradient (3 components).
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

EVALUATE_FIELD_BASIS

One scalar basis term of one field at one point. structural_value/gradient: n_i and its local-frame gradient. expansion_shape(:): shapes of the expansion element nodes. expansion_gradient(3,:): their local-frame gradients. coordinate_local: local coordinate of the expansion point.

Argument	Declaration	Meaning
SPEC	TYPE(EXPANSION_SPEC_TYPE), intent(IN)	Expansion specification (family, order, field reference).
TERM	INTEGER(I4), intent(IN)	Expansion term index.

Argument	Declaration	Meaning
STRUCTURAL_VALUE	REAL(R8), intent(IN)	Structural shape function N.
STRUCTURAL_GRADIENT	REAL(R8), intent(IN)(3)	Gradient of the structural shape function (local frame).
MESH	TYPE(EXPANSION_MESH_TYPE), intent(IN)	Expansion mesh.
EXPANSION_ELEMENT_INDEX	INTEGER(I4), intent(IN)	Index of the active expansion sub-element.
EXPANSION_DIMENSION	INTEGER(I4), intent(IN)	Natural dimension of the expansion element.
ACTIVE_AXIS	INTEGER(I4), intent(IN)(2)	Active local axes of the section.
COORDINATE_LOCAL	REAL(R8), intent(IN)(3)	Section point in the element local frame.
EXPANSION_SHAPE	REAL(R8), intent(IN)(:)	Shape functions of the expansion sub-element nodes at the point.
EXPANSION_GRADIENT	REAL(R8), intent(IN)(:,:))	Local gradients of the expansion shape functions (3, node).
VALUE	REAL(R8), intent(OUT)	Value(s) (see routine).
GRADIENT	REAL(R8), intent(OUT)(3)	Gradient (3 components).
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

EVALUATE_EXPANSION_FACTOR

The expansion factor f_{τ} and its local gradient.

Argument	Declaration	Meaning
SPEC	TYPE(EXPANSION_SPEC_TYPE), intent(IN)	Expansion specification (family, order, field reference).
TERM	INTEGER(I4), intent(IN)	Expansion term index.
MESH	TYPE(EXPANSION_MESH_TYPE), intent(IN)	Expansion mesh.
EXPANSION_ELEMENT_INDEX	INTEGER(I4), intent(IN)	Index of the active expansion sub-element.
EXPANSION_DIMENSION	INTEGER(I4), intent(IN)	Natural dimension of the expansion element.
ACTIVE_AXIS	INTEGER(I4), intent(IN)(2)	Active local axes of the section.
COORDINATE_LOCAL	REAL(R8), intent(IN)(3)	Section point in the element local frame.
EXPANSION_SHAPE	REAL(R8), intent(IN)(:)	Shape functions of the expansion sub-element nodes at the point.
EXPANSION_GRADIENT	REAL(R8), intent(IN)(:,:))	Local gradients of the expansion shape functions (3, node).
FACTOR_VALUE	REAL(R8), intent(OUT)	Expansion factor F_{τ} .
FACTOR_GRADIENT	REAL(R8), intent(OUT)(3)	Gradient of the expansion factor (local frame).
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.8 Layer KINEMATICS

Linear strain–displacement operator.

C.8.1 MUL2_LINEAR_KINEMATICS

File SRC/KINEMATICS/mul2_linear_kinematics.for.

Linear small-strain operators for field-dependent displacements.

Uses: `MUL2_KINDS`, `MUL2_STATUS`.

Table C.186. Procedures of `MUL2_LINEAR_KINEMATICS`.

Procedure	Kind	Visibility	Purpose
<code>COMPOSE_PRODUCT_BASIS</code>	Subroutine	public	Value and gradient of N F: $N \text{ grad } F + F \text{ grad } N$.
<code>BUILD_DISPLACEMENT_COLUMN</code>	Subroutine	public	Strain column of one displacement component for a given scalar gradient.
<code>BUILD_DISPLACEMENT_OPERATOR</code>	Subroutine	public	6x3 strain operator $B(\text{grad})$ of a scalar basis function.

`COMPOSE_PRODUCT_BASIS`

Value and gradient of N F: $N \text{ grad } F + F \text{ grad } N$.

Argument	Declaration	Meaning
<code>STRUCTURAL_VALUE</code>	<code>REAL(R8), intent(IN)</code>	Structural shape function N.
<code>STRUCTURAL_GRADIENT</code>	<code>REAL(R8), intent(IN)(3)</code>	Gradient of the structural shape function (local frame).
<code>EXPANSION_VALUE</code>	<code>REAL(R8), intent(IN)</code>	Expansion factor F.
<code>EXPANSION_GRADIENT</code>	<code>REAL(R8), intent(IN)(3)</code>	Local gradients of the expansion shape functions (3, node).
<code>VALUE</code>	<code>REAL(R8), intent(OUT)</code>	Value(s) (see routine).
<code>GRADIENT</code>	<code>REAL(R8), intent(OUT)(3)</code>	Gradient (3 components).

`BUILD_DISPLACEMENT_COLUMN`

Strain column of one displacement component for a given scalar gradient.

Argument	Declaration	Meaning
<code>COMPONENT</code>	<code>INTEGER(I4), intent(IN)</code>	Displacement component index (1=u, 2=v, 3=w) or the frame column of the field.
<code>GRADIENT</code>	<code>REAL(R8), intent(IN)(3)</code>	Gradient (3 components).
<code>COLUMN</code>	<code>REAL(R8), intent(OUT)(6)</code>	Output: strain column (6 values).
<code>STATUS</code>	<code>TYPE(STATUS_TYPE), intent(OUT)</code>	Status of the call (OK, warning or error with message and source).

`BUILD_DISPLACEMENT_OPERATOR`

6x3 strain operator $B(\text{grad})$ of a scalar basis function.

Argument	Declaration	Meaning
<code>GRADIENT</code>	<code>REAL(R8), intent(IN)(3)</code>	Gradient (3 components).
<code>OPERATOR</code>	<code>REAL(R8), intent(OUT)(6,3)</code>	Output: 6x3 strain operator.
<code>STATUS</code>	<code>TYPE(STATUS_TYPE), intent(OUT)</code>	Status of the call (OK, warning or error with message and source).

C.9 Layer MATERIALS

Constitutive matrices, rotations and lamination resolution.

C.9.1 `MUL2_MATERIAL_RESOLUTION`

File `SRC/MATERIALS/mul2_material_resolution.for`.

Resolve a lamination into constitutive data at an integration point.

Uses: `MUL2_KINDS`, `MUL2_STATUS`, `MUL2_MATERIALS`, `MUL2_LAMINATIONS`, `MUL2_MATERIAL_ROTATIONS`.

Table C.190. Procedures of `MUL2_MATERIAL_RESOLUTION`.

Procedure	Kind	Visibility	Purpose
RESOLVE_LAMINATION	Subroutine	public	Rotated stiffness (element frame), density and rotation of a lamination.

RESOLVE_LAMINATION

Rotated stiffness (element frame), density and rotation of a lamination.

Argument	Declaration	Meaning
LAMINATION_ID	INTEGER(I4), intent(IN)	Id of the lamination to resolve.
MATERIALS	TYPE(MATERIAL_DB_TYPE), intent(IN)	Material database.
LAMINATIONS	TYPE(LAMINATION_DB_TYPE), intent(IN)	Lamination database.
STIFFNESS_LOCAL	REAL(R8), intent(OUT)(6,6)	Output: stiffness in the element frame.
DENSITY	REAL(R8), intent(OUT)	Mass density rho.
MATERIAL_FROM_LOCAL	REAL(R8), intent(OUT)(3,3)	Output: rotation material <- element frame.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).
PIEZO_LOCAL	REAL(R8), intent(OUT), OPTIONAL(3,6)	
PERMITTIVITY_LOCAL	REAL(R8), intent(OUT), OPTIONAL(3,3)	
HAS_PIEZO	LOGICAL, intent(OUT), OPTIONAL	
EXPANSION_LOCAL	REAL(R8), intent(OUT), OPTIONAL(6)	
CONDUCTIVITY_LOCAL	REAL(R8), intent(OUT), OPTIONAL(3,3)	
HAS_THERMAL	LOGICAL, intent(OUT), OPTIONAL	
PYRO_LOCAL	REAL(R8), intent(OUT), OPTIONAL(3)	
HAS_PYRO	LOGICAL, intent(OUT), OPTIONAL	

C.9.2 MUL2_MATERIAL_ROTATIONS

File SRC/MATERIALS/mul2_material_rotations.for.

Single material-orientation transformation implementation.

Uses: MUL2_KINDS.

Table C.192. Procedures of MUL2_MATERIAL_ROTATIONS.

Procedure	Kind	Visibility	Purpose
BUILD_MATERIAL_ROTATION	Subroutine	public	Rotation matrix material <- element frame from the angles about y and z (degrees).
BUILD_ENGINEERING_STRAIN_TRANSFORM	Subroutine	public	6x6 transformation of engineering strains under a 3x3 rotation.
ROTATE_STIFFNESS	Subroutine	public	Stiffness in the reference frame: $T^A T C T$.
ROTATE_PIEZO	Subroutine	public	$E_{local} = a^t e_{material} t$ (d = e s: d vector, s engineering strain).
ROTATE_PERMITTIVITY	Subroutine	public	
ROTATE_EXPANSION	Subroutine	public	Expansion strains (engineering voigt): $e_{ref} = t(a^t) e_{material}$.

BUILD_MATERIAL_ROTATION

Rotation matrix material <- element frame from the angles about y and z (degrees).

Argument	Declaration	Meaning
ANGLE_Y_DEG	REAL(R8), intent(IN)	Rotation about the local y axis (degrees).
ANGLE_Z_DEG	REAL(R8), intent(IN)	Rotation about the local z axis (degrees).
MATERIAL_FROM_LOCAL	REAL(R8), intent(OUT) (3,3)	Output: matrix that maps element-frame components to material-frame components.

BUILD_ENGINEERING_STRAIN_TRANSFORM

6x6 transformation of engineering strains under a 3x3 rotation.

Argument	Declaration	Meaning
A	REAL(R8), intent(IN)(3,3)	Matrix or operand A.
TRANSFORM	REAL(R8), intent(OUT)(6,6)	Output: 6x6 engineering-strain transformation.

ROTATE_STIFFNESS

Stiffness in the reference frame: $T^T C T$.

Argument	Declaration	Meaning
STIFFNESS_MATERIAL	REAL(R8), intent(IN)(6,6)	Stiffness in the material axes.
MATERIAL_FROM_REFERENCE	REAL(R8), intent(IN)(3,3)	Rotation material <- reference frame.
STIFFNESS_REFERENCE	REAL(R8), intent(OUT)(6,6)	Output: stiffness in the reference frame.

ROTATE_PIEZO

$E_{\text{local}} = a^t e_{\text{material}} t$ ($d = e$: d vector, s engineering strain).

Argument	Declaration	Meaning
PIEZO_MATERIAL	REAL(R8), intent(IN)(3,6)	
MATERIAL_FROM_REFERENCE	REAL(R8), intent(IN)(3,3)	Rotation material <- reference frame.
PIEZO_REFERENCE	REAL(R8), intent(OUT)(3,6)	

ROTATE_PERMITTIVITY

Argument	Declaration	Meaning
PERMITTIVITY_MATERIAL	REAL(R8), intent(IN)(3,3)	
MATERIAL_FROM_REFERENCE	REAL(R8), intent(IN)(3,3)	Rotation material <- reference frame.
PERMITTIVITY_REFERENCE	REAL(R8), intent(OUT)(3,3)	

ROTATE_EXPANSION

Expansion strains (engineering voigt): $e_{\text{ref}} = t(a^t) e_{\text{material}}$.

Argument	Declaration	Meaning
EXPANSION_MATERIAL	REAL(R8), intent(IN)(6)	
MATERIAL_FROM_REFERENCE	REAL(R8), intent(IN)(3,3)	Rotation material <- reference frame.
EXPANSION_REFERENCE	REAL(R8), intent(OUT)(6)	

C.9.3 MUL2_MATERIALS

File SRC/MATERIALS/mul2_materials.for.

Linear mechanical material data and constitutive builders.

Uses: MUL2_KINDS, MUL2_STATUS.

Table C.199. Public constants of `MUL2_MATERIALS`.

Name	Type	Value	Meaning
<code>MATERIAL_ISOTROPIC</code>	<code>INTEGER(I4)</code>	<code>1_I4</code>	Material model code.
<code>MATERIAL_ORTHOTROPIC</code>	<code>INTEGER(I4)</code>	<code>2_I4</code>	Material model code.
<code>MATERIAL_ANISOTROPIC</code>	<code>INTEGER(I4)</code>	<code>3_I4</code>	Material model code.

Table C.200. Derived type `MATERIAL_TYPE` — One material: id, model, density and 6x6 stiffness in the material axes.

Component	Type	Shape	Default	Meaning
<code>ID</code>	<code>INTEGER(I4)</code>	``	<code>0_I4</code>	Identifier.
<code>MODEL</code>	<code>INTEGER(I4)</code>	``	<code>0_I4</code>	Material model code.
<code>DENSITY</code>	<code>REAL(R8)</code>	``	<code>0.0_R8</code>	Mass density.
<code>STIFFNESS</code>	<code>REAL(R8)</code>	(6,6)	<code>0.0_R8</code>	Material axes are (t,l,z); voigt is (tt,ll,zz,tz,lz,tl).
<code>HAS_PIEZO</code>	<code>LOGICAL</code>	``	<code>.FALSE.</code>	Piezoelectric stress coefficients $e(i,j)$: electric direction $i = t,l,z$, strain j in the voigt order of the stiffness, and dielectric permittivity (material axes), both at constant strain.
<code>PIEZO</code>	<code>REAL(R8)</code>	(3,6)	<code>0.0_R8</code>	
<code>PERMITTIVITY</code>	<code>REAL(R8)</code>	(3,3)	<code>0.0_R8</code>	
<code>HAS_EXPANSION</code>	<code>LOGICAL</code>	``	<code>.FALSE.</code>	Thermal expansion (strain per kelvin, voigt order of the stiffness, material axes) and heat conductivity (material axes).
<code>HAS_CONDUCTION</code>	<code>LOGICAL</code>	``	<code>.FALSE.</code>	
<code>EXPANSION</code>	<code>REAL(R8)</code>	(6)	<code>0.0_R8</code>	
<code>CONDUCTIVITY</code>	<code>REAL(R8)</code>	(3,3)	<code>0.0_R8</code>	
<code>HAS_PYRO</code>	<code>LOGICAL</code>	``	<code>.FALSE.</code>	Pyroelectric coefficient (d per kelvin, material axes) measured at zero stress: $d = p \cdot t$ when the body expands freely at $e = 0$.
<code>PYRO</code>	<code>REAL(R8)</code>	(3)	<code>0.0_R8</code>	
<code>SPECIFIC_HEAT</code>	<code>REAL(R8)</code>	``	<code>0.0_R8</code>	Specific heat [J/(kg K)] (thermal capacity = density * c).

Table C.201. Derived type `MATERIAL_DB_TYPE` — Material database.

Component	Type	Shape	Default	Meaning
<code>ITEM</code>	<code>TYPE(MATERIAL_TYPE), ALLOCATABLE</code>	(:)	``	List of items.
<code>DAMP_MASS</code>	<code>REAL(R8)</code>	``	<code>0.0_R8</code>	Global rayleigh damping of the damp record: $c = m_coeff \cdot m + k_coeff \cdot k$ (historical order of the record: k then m).
<code>DAMP_STIFFNESS</code>	<code>REAL(R8)</code>	``	<code>0.0_R8</code>	

Table C.202. Procedures of `MUL2_MATERIALS`.

Procedure	Kind	Visibility	Purpose
<code>BUILD_ISOTROPIC_MATERIAL</code>	Subroutine	public	Isotropic stiffness from E, nu, rho (checks nu in (-1, 0.5)).
<code>BUILD_ORTHOTROPIC_MATERIAL</code>	Subroutine	public	Orthotropic stiffness from the nine engineering constants in the (T,L,Z) axes.
<code>BUILD_ANISOTROPIC_MATERIAL</code>	Subroutine	public	Store a user-supplied 6x6 stiffness.
<code>FIND_MATERIAL_INDEX</code>	Function → integer(i4)	public	Index of a material by id.
<code>INVERT_SYMMETRIC_3X3</code>	Subroutine	private	Invert a symmetric 3x3 matrix.

BUILD_ISOTROPIC_MATERIAL

Isotropic stiffness from E, nu, rho (checks nu in (-1, 0.5)).

Argument	Declaration	Meaning
ID	INTEGER(I4), intent(IN)	Identifier.
YOUNG	REAL(R8), intent(IN)	Young modulus E.
POISSON	REAL(R8), intent(IN)	Poisson ratio nu.
DENSITY	REAL(R8), intent(IN)	Mass density rho.
MATERIAL	TYPE(MATERIAL_TYPE), intent(OUT)	Output: material (id, model, density, stiffness).
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

BUILD_ORTHOTROPIC_MATERIAL

Orthotropic stiffness from the nine engineering constants in the (T,L,Z) axes.

Argument	Declaration	Meaning
ID	INTEGER(I4), intent(IN)	Identifier.
YOUNG_T	REAL(R8), intent(IN)	Young modulus along T.
YOUNG_L	REAL(R8), intent(IN)	Young modulus along L.
YOUNG_Z	REAL(R8), intent(IN)	Young modulus along Z.
NU_LT	REAL(R8), intent(IN)	Poisson ratio nu_LT.
NU_LZ	REAL(R8), intent(IN)	Poisson ratio nu_LZ.
NU_TZ	REAL(R8), intent(IN)	Poisson ratio nu_TZ.
SHEAR_LT	REAL(R8), intent(IN)	Shear modulus G_LT.
SHEAR_LZ	REAL(R8), intent(IN)	Shear modulus G_LZ.
SHEAR_TZ	REAL(R8), intent(IN)	Shear modulus G_TZ.
DENSITY	REAL(R8), intent(IN)	Mass density rho.
MATERIAL	TYPE(MATERIAL_TYPE), intent(OUT)	Output: material (id, model, density, stiffness).
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

BUILD_ANISOTROPIC_MATERIAL

Store a user-supplied 6x6 stiffness.

Argument	Declaration	Meaning
ID	INTEGER(I4), intent(IN)	Identifier.
STIFFNESS	REAL(R8), intent(IN)(6,6)	Stiffness matrix (or values).
DENSITY	REAL(R8), intent(IN)	Mass density rho.
MATERIAL	TYPE(MATERIAL_TYPE), intent(OUT)	Output: material (id, model, density, stiffness).
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

FIND_MATERIAL_INDEX

Index of a material by id.

Argument	Declaration	Meaning
DATABASE	TYPE(MATERIAL_DB_TYPE), intent(IN)	Database being filled.
ID	INTEGER(I4), intent(IN)	Identifier.

C.10 Layer GAUSS

Reference rules, global Gauss-point layout, geometry and material caches.

C.10.1 MUL2_GAUSS_GEOMETRY

File SRC/GAUSS/mul2_gauss_geometry.for.

Element-dependent geometry caches for combined gauss points.

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_NODES, MUL2_ELEMENTS, MUL2_EXPANSION_MESHES, MUL2_REFERENCE_SYSTEMS, MUL2_TOPOLOGIES, MUL2_HLE_MAP, MUL2_GAUSS_POINTS, MUL2_JACOBIANS.

Table C.207. Derived type STRUCTURAL_GEOMETRY_CACHE_TYPE — Jacobian data and local shape gradients of the structural points.

Component	Type	Shape	Default	Meaning
COUNT	INTEGER(I8)	``	0_I8	Number of items.
ELEMENT_FIRST	INTEGER(I8), ALLOCATABLE	(:)	``	First point of each element.
ELEMENT_LAST	INTEGER(I8), ALLOCATABLE	(:)	``	Last point of each element.
CENTER_GLOBAL	REAL(R8), ALLOCATABLE	(:,:)	``	Global coordinate of the structural point.
JACOBIAN	REAL(R8), ALLOCATABLE	(:,:,:)	``	Jacobian matrices.
INVERSE_JACOBIAN	REAL(R8), ALLOCATABLE	(:,:,:)	``	Inverse Jacobians.
DETERMINANT	REAL(R8), ALLOCATABLE	(:)	``	Jacobian determinants.
DERIVATIVE_OFFSET	INTEGER(I8), ALLOCATABLE	(:)	``	Offset of the derivative block of each point.
DERIVATIVE_LOCAL	REAL(R8), ALLOCATABLE	(:,:)	``	Shape gradients in the local frame.

Table C.208. Derived type EXPANSION_GEOMETRY_CACHE_TYPE — Jacobian data and local shape gradients of the expansion points.

Component	Type	Shape	Default	Meaning
COUNT	INTEGER(I8)	``	0_I8	Number of items.
ACTIVE_AXIS	INTEGER(I4), ALLOCATABLE	(:,:)	``	Active local axes of each mesh.
MESH_ELEMENT_BASE	INTEGER(I4), ALLOCATABLE	(:)	``	First flat sub-element of each mesh.
ELEMENT_FIRST	INTEGER(I8), ALLOCATABLE	(:)	``	First point of each element.
ELEMENT_LAST	INTEGER(I8), ALLOCATABLE	(:)	``	Last point of each element.
COORDINATE_LOCAL	REAL(R8), ALLOCATABLE	(:,:)	``	Local coordinate of the expansion point.
JACOBIAN	REAL(R8), ALLOCATABLE	(:,:,:)	``	Jacobian matrices.
INVERSE_JACOBIAN	REAL(R8), ALLOCATABLE	(:,:,:)	``	Inverse Jacobians.
DETERMINANT	REAL(R8), ALLOCATABLE	(:)	``	Jacobian determinants.
DERIVATIVE_OFFSET	INTEGER(I8), ALLOCATABLE	(:)	``	Offset of the derivative block of each point.
DERIVATIVE_LOCAL	REAL(R8), ALLOCATABLE	(:,:)	``	Shape gradients in the local frame.

Table C.209. Derived type GAUSS_GEOMETRY_TYPE — Combined Gauss points: cache indices, global coordinates and integration weights.

Component	Type	Shape	Default	Meaning
COUNT	INTEGER(I8)	``	0_I8	Number of items.
STRUCTURAL_CACHE_INDEX	INTEGER(I8), ALLOCATABLE	(:)	``	Structural cache index of each point.
EXPANSION_CACHE_INDEX	INTEGER(I8), ALLOCATABLE	(:)	``	Expansion cache index of each point.
COORDINATE_GLOBAL	REAL(R8), ALLOCATABLE	(:,:)	``	Global coordinates.
INTEGRATION_WEIGHT	REAL(R8), ALLOCATABLE	(:)	``	Weight including both Jacobians.

Table C.210. Procedures of MUL2_GAUSS_GEOMETRY.

Procedure	Kind	Visibility	Purpose
BUILD_STRUCTURAL_GEOMETRY_CACHE	Subroutine	public	Jacobian, determinant, inverse and local-frame shape gradients of every structural Gauss point.
BUILD_EXPANSION_GEOMETRY_CACHE	Subroutine	public	Same for every expansion sub-element and point (the same cache serves all structural points).
BUILD_COMBINED_GAUSS_GEOMETRY	Subroutine	public	Global coordinate and integration weight of every combined (structural x expansion) point.
STRUCTURAL_ACTIVE_AXES	Subroutine	private	Local axes carried by the structural element: y for beams, x-y for plates, x-y-z for solids.
FIND_EXPANSION_ACTIVE_AXES	Subroutine	private	Local axes spanned by the expansion mesh (the largest node extents).
GET_EXPANSION_NODE_COORDINATE	Subroutine	private	Coordinate of an expansion node by id.
ALLOCATE_STRUCTURAL_CACHE	Subroutine	private	Allocate the structural geometry cache.
ALLOCATE_EXPANSION_CACHE	Subroutine	private	Allocate the expansion geometry cache.
CLEAR_STRUCTURAL_CACHE	Subroutine	private	Release the structural cache.
CLEAR_EXPANSION_CACHE	Subroutine	private	Release the expansion cache.
CLEAR_COMBINED_GEOMETRY	Subroutine	private	Release the combined geometry.

BUILD_STRUCTURAL_GEOMETRY_CACHE

Jacobian, determinant, inverse and local-frame shape gradients of every structural Gauss point.

Argument	Declaration	Meaning
NODES	TYPE(NODE_DB_TYPE), intent(IN)	Structural node database.
ELEMENTS	TYPE(ELEMENT_DB_TYPE), intent(IN)	Structural element database.
FRAMES	TYPE(ELEMENT_FRAME_DB_TYPE), intent(IN)	Element reference frames.
RULES	TYPE(REFERENCE_RULE_DB_TYPE), intent(IN)	Reference quadrature rule database.
CACHE	TYPE(STRUCTURAL_GEOMETRY_CACHE_TYPE), intent(INOUT)	Model cache built by BUILD_MODEL_CACHE.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).
REDUCED	LOGICAL, intent(IN), OPTIONAL	Vector on the free DOFs.

BUILD_EXPANSION_GEOMETRY_CACHE

Same for every expansion sub-element and point (the same cache serves all structural points).

Argument	Declaration	Meaning
EXPANSIONS	TYPE(EXPANSION_DB_TYPE), intent(IN)	Expansion (section/thickness) mesh database.
RULES	TYPE(REFERENCE_RULE_DB_TYPE), intent(IN)	Reference quadrature rule database.
CACHE	TYPE(EXPANSION_GEOMETRY_CACHE_TYPE), intent(INOUT)	Model cache built by BUILD_MODEL_CACHE.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).
EXPANSION_ORDER	INTEGER(I4), intent(IN), OPTIONAL(:)	Gauss points per direction needed by each expansion mesh.

BUILD_COMBINED_GAUSS_GEOMETRY

Global coordinate and integration weight of every combined (structural x expansion) point.

Argument	Declaration	Meaning
ELEMENTS	TYPE(ELEMENT_DB_TYPE), intent(IN)	Structural element database.
EXPANSIONS	TYPE(EXPANSION_DB_TYPE), intent(IN)	Expansion (section/thickness) mesh database.
FRAMES	TYPE(ELEMENT_FRAME_DB_TYPE), intent(IN)	Element reference frames.
LAYOUT	TYPE(GAUSS_LAYOUT_TYPE), intent(IN)	Gauss-point layout (global point numbering).
STRUCTURAL_CACHE	TYPE(STRUCTURAL_GEOMETRY_CACHE_TYPE), intent(IN)	Geometry cache of the structural points.
EXPANSION_CACHE	TYPE(EXPANSION_GEOMETRY_CACHE_TYPE), intent(IN)	Geometry cache of the expansion points.
GEOMETRY	TYPE(GAUSS_GEOMETRY_TYPE), intent(INOUT)	Combined Gauss geometry (coordinates, weights).
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.10.2MUL2_GAUSS_MATERIALS

File SRC/GAUSS/mul2_gauss_materials.for.

Shared constitutive cache and gauss-point material indirection.

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_GAUSS_POINTS, MUL2_MATERIALS, MUL2_LAMINATIONS, MUL2_MATERIAL_RESOLUTION.

Table C.214. Public constants of MUL2_GAUSS_MATERIALS.

Name	Type	Value	Meaning
PART_FULL	INTEGER(I4)	0_I4	
PART_NORMAL	INTEGER(I4)	1_I4	
PART_SHEAR	INTEGER(I4)	2_I4	

Table C.215. Derived type MATERIAL_CACHE_TYPE — Distinct resolved constitutive matrices (one per lamination).

Component	Type	Shape	Default	Meaning
COUNT	INTEGER(I4)	``	0_I4	Number of items.
LAMINATION_ID	INTEGER(I4), ALLOCATABLE	(:)	``	Lamination id.
MATERIAL_ID	INTEGER(I4), ALLOCATABLE	(:)	``	Material id.
DENSITY	REAL(R8), ALLOCATABLE	(:)	``	Mass density.
MATERIAL_FROM_LOCAL	REAL(R8), ALLOCATABLE	(:,:,:)	``	Rotation material <- element frame.
STIFFNESS_LOCAL	REAL(R8), ALLOCATABLE	(:,:,:)	``	Stiffness in the element frame.
ANY_PIEZO	LOGICAL	``	.FALSE.	Piezoelectric coefficients and permittivity in the element frame.
PIEZO_LOCAL	REAL(R8), ALLOCATABLE	(:,:,:)	``	
PERMITTIVITY_LOCAL	REAL(R8), ALLOCATABLE	(:,:,:)	``	
ANY_THERMAL	LOGICAL	``	.FALSE.	Thermal expansion strains and conductivity in the element frame.

Component	Type	Shape	Default	Meaning
EXPANSION_LOCAL	REAL(R8), ALLOCATABLE	(:, :)	``	
CONDUCTIVITY_LOCAL	REAL(R8), ALLOCATABLE	(:, :, :)	``	
ANY_PYRO	LOGICAL	``	.FALSE.	Pyroelectric coefficient at constant strain, element frame: $p_strain = p_stress - e \alpha$ ($d = e s + \epsilon s e + p_strain t$).
PYRO_LOCAL	REAL(R8), ALLOCATABLE	(:, :)	``	
SPECIFIC_HEAT	REAL(R8), ALLOCATABLE	(:)	``	

Table C.216. Derived type GAUSS_MATERIAL_MAP_TYPE — Material cache index of every Gauss point.

Component	Type	Shape	Default	Meaning
COUNT	INTEGER(I8)	``	0_I8	Number of items.
CACHE_INDEX	INTEGER(I4), ALLOCATABLE	(:)	``	Material cache index of each point.

Table C.217. Procedures of MUL2_GAUSS_MATERIALS.

Procedure	Kind	Visibility	Purpose
BUILD_GAUSS_MATERIAL_CACHE	Subroutine	public	Resolve every lamination into a rotated stiffness matrix and map each Gauss point to its entry.
STIFFNESS_PART	Function	public	Constitutive matrix of a lamination, or one of its two parts for the selective integration. the strains that are reduced are the transverse shears of the element (voigt order xx yy zz xz yz xy, element frame): beam (axis y): yz, xy plate (surface x-y): xz, yz solid: xz, yz, xy part_shear keeps the entries whose row or column is a reduced strain, part_normal the others: the two parts add up to the matrix.
GENERALIZED_CONSTITUTIVE	Function	public	Generalised constitutive matrix of the mechanical, electric and thermal fields: rows/columns 1-6 are the strains, 7-9 the potential gradient and 10-12 the temperature gradient (block +k: $q = -k \text{ grad } t$). the thermal expansion only enters the load, not this matrix. $[\sigma] [c e^t] [s] [d] = [e - \epsilon s] [\text{grad}]$ ($d = e s - \epsilon s \text{ grad } \phi$) part_normal / part_shear split the entries as in stiffness_part (the electric rows are never reduced).
THERMAL_STRESS_COEFFICIENT	Function	public	Stress per kelvin of the fully restrained expansion, $\beta = c \alpha$.
SPECIFIC_OF	Function → real(r8)	private	
CLEAR_MATERIAL_CACHE	Subroutine	private	Release the material cache.

BUILD_GAUSS_MATERIAL_CACHE

Resolve every lamination into a rotated stiffness matrix and map each Gauss point to its entry.

Argument	Declaration	Meaning
LAYOUT	TYPE(GAUSS_LAYOUT_TYPE), intent(IN)	Gauss-point layout (global point numbering).
MATERIALS	TYPE(MATERIAL_DB_TYPE), intent(IN)	Material database.
LAMINATIONS	TYPE(LAMINATION_DB_TYPE), intent(IN)	Lamination database.
CACHE	TYPE(MATERIAL_CACHE_TYPE), intent(INOUT)	Model cache built by BUILD_MODEL_CACHE.
POINT_MAP	TYPE(GAUSS_MATERIAL_MAP_TYPE), intent(INOUT)	Output: Gauss point → material cache index.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

STIFFNESS_PART

Constitutive matrix of a lamination, or one of its two parts for the selective integration. the strains that are reduced are the transverse shears of the element (voigt order xx yy zz xz yz xy, element frame): beam (axis y): yz, xy plate (surface x-y): xz, yz solid: xz, yz, xy part_shear keeps the entries whose row or column is a reduced strain, part_normal the others: the two parts add up to the matrix.

Argument	Declaration	Meaning
CACHE	TYPE(MATERIAL_CACHE_TYPE), intent(IN)	Model cache built by BUILD_MODEL_CACHE.
INDEX	INTEGER(I4), intent(IN)	
PART	INTEGER(I4), intent(IN)	
DIM) RESULT(C		

GENERALIZED_CONSTITUTIVE

Generalised constitutive matrix of the mechanical, electric and thermal fields: rows/columns 1-6 are the strains, 7-9 the potential gradient and 10-12 the temperature gradient (block +k: $q = -k \text{ grad } t$). the thermal expansion only enters the load, not this matrix. $[\sigma] [c \ e^t] [s] [d] = [e \ -\epsilon] [grad] (d = e \ s - \epsilon \ grad \ \phi)$ part_normal / part_shear split the entries as in stiffness_part (the electric rows are never reduced).

Argument	Declaration	Meaning
CACHE	TYPE(MATERIAL_CACHE_TYPE), intent(IN)	Model cache built by BUILD_MODEL_CACHE.
INDEX	INTEGER(I4), intent(IN)	
PART	INTEGER(I4), intent(IN)	
DIM) RESULT(M		

THERMAL_STRESS_COEFFICIENT

Stress per kelvin of the fully restrained expansion, $\beta = c \alpha$.

Argument	Declaration	Meaning
CACHE	TYPE(MATERIAL_CACHE_TYPE), intent(IN)	Model cache built by BUILD_MODEL_CACHE.
INDEX) RESULT(BETA		

C.10.3MUL2_GAUSS_POINTS

File SRC/GAUSS/mul2_gauss_points.for.

Shared reference rules and contiguous global gauss-point layout.

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_ELEMENTS, MUL2_EXPANSION_MESHES, MUL2_TOPOLOGIES, MUL2_QUADRATURE, MUL2_SHAPE_FUNCTIONS.

Table C.222. Derived type REFERENCE_RULE_TYPE — One reference quadrature rule with shapes and derivatives at its points.

Component	Type	Shape	Default	Meaning
TOPOLOGY	INTEGER(I4)	``	0_I4	Topology code.
ORDER	INTEGER(I4)	``	0_I4	Order of the matrix / expansion order / quadrature order (see type).
NATURAL_DIMENSION	INTEGER(I4)	``	0_I4	Natural dimension.
NODE_COUNT	INTEGER(I4)	``	0_I4	Number of nodes.
COORDINATE	REAL(R8), ALLOCATABLE	(:, :)	``	Coordinates.
WEIGHT	REAL(R8), ALLOCATABLE	(:)	``	Weights.

Component	Type	Shape	Default	Meaning
SHAPE	REAL(R8), ALLOCATABLE	(:, :)	""	Shape function values at the tying/rule points.
DERIVATIVE	REAL(R8), ALLOCATABLE	(:, :, :)	""	Shape derivatives.

Table C.223. Derived type `REFERENCE_RULE_DB_TYPE` — Collection of reference rules.

Component	Type	Shape	Default	Meaning
ITEM	TYPE(REFERENCE_RULE_TYPE), ALLOCATABLE	(:)	""	List of items.

Table C.224. Derived type `GAUSS_LAYOUT_TYPE` — Global numbering of the Gauss points and its decomposition into element, sub-element, structural and expansion point.

Component	Type	Shape	Default	Meaning
COUNT	INTEGER(I8)	""	0_I8	Number of items.
ELEMENT_FIRST	INTEGER(I8), ALLOCATABLE	(:)	""	First point of each element.
ELEMENT_LAST	INTEGER(I8), ALLOCATABLE	(:)	""	Last point of each element.
ELEMENT_INDEX	INTEGER(I4), ALLOCATABLE	(:)	""	Element index.
EXPANSION_ELEMENT_INDEX	INTEGER(I4), ALLOCATABLE	(:)	""	Sub-element of each point.
STRUCTURAL_RULE_INDEX	INTEGER(I4), ALLOCATABLE	(:)	""	Structural rule of each point.
STRUCTURAL_POINT_INDEX	INTEGER(I4), ALLOCATABLE	(:)	""	Structural point within its rule.
EXPANSION_RULE_INDEX	INTEGER(I4), ALLOCATABLE	(:)	""	Expansion rule of each point.
EXPANSION_POINT_INDEX	INTEGER(I4), ALLOCATABLE	(:)	""	Expansion point within its rule.
LAMINATION_ID	INTEGER(I4), ALLOCATABLE	(:)	""	Lamination id.
REFERENCE_WEIGHT	REAL(R8), ALLOCATABLE	(:)	""	Product of the two reference weights.

Table C.225. Procedures of `MUL2_GAUSS_POINTS`.

Procedure	Kind	Visibility	Purpose
BUILD_REFERENCE_RULE_DATABASE	Subroutine	public	One reference rule (points, weights, shapes, derivatives) per (topology, points-per-direction).
BUILD_REFERENCE_RULE	Subroutine	private	Evaluate shapes and derivatives at the points of one rule.
BUILD_GAUSS_LAYOUT	Subroutine	public	Global Gauss-point numbering: element, sub-element, structural point, expansion point.
ADD_TOPOLOGY	Subroutine	private	Register a (topology, order) pair once.
FIND_REFERENCE_RULE	Function → integer(i4)	public	Index of the rule of a topology and order, 0 if missing.
FIND_STRUCTURAL_RULE	Function → integer(i4)	public	Rule of a structural element: the reduced one (order -1) when requested and available, otherwise the default rule.
EXPANSION_POINTS	Function → integer(i4)	private	Points per direction required by a mesh (0 = default rule).
EXPANSION_MESH_ORDER	Function → integer(i4)	public	Points per direction of an expansion mesh given the per-mesh need.
ALLOCATE_LAYOUT	Subroutine	private	Allocate the Gauss layout arrays.
CLEAR_LAYOUT	Subroutine	private	Release the arrays of a DOF layout.

BUILD_REFERENCE_RULE_DATABASE

One reference rule (points, weights, shapes, derivatives) per (topology, points-per-direction).

Argument	Declaration	Meaning
ELEMENTS	TYPE(ELEMENT_DB_TYPE), intent(IN)	Structural element database.
EXPANSIONS	TYPE(EXPANSION_DB_TYPE), intent(IN)	Expansion (section/thickness) mesh database.
DATABASE	TYPE(REFERENCE_RULE_DB_TYPE), intent(INOUT)	Database being filled.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).
EXPANSION_ORDER	INTEGER(I4), intent(IN), OPTIONAL(:)	Gauss points per direction needed by each expansion mesh.
REDUCED_DIMENSION	LOGICAL, intent(IN), OPTIONAL(3)	Reduced_dimension(d): the structural elements of dimension d also get a reduced rule (order = -1 in the database).

BUILD_GAUSS_LAYOUT

Global Gauss-point numbering: element, sub-element, structural point, expansion point.

Argument	Declaration	Meaning
ELEMENTS	TYPE(ELEMENT_DB_TYPE), intent(IN)	Structural element database.
EXPANSIONS	TYPE(EXPANSION_DB_TYPE), intent(IN)	Expansion (section/thickness) mesh database.
REFERENCE_RULES	TYPE(REFERENCE_RULE_DB_TYPE), intent(IN)	Reference rule database.
LAYOUT	TYPE(GAUSS_LAYOUT_TYPE), intent(INOUT)	Gauss-point layout (global point numbering).
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).
EXPANSION_ORDER	INTEGER(I4), intent(IN), OPTIONAL(:)	Gauss points per direction needed by each expansion mesh.
REDUCED	LOGICAL, intent(IN), OPTIONAL	Reduced: the structural points use the reduced rule where one exists.

FIND_REFERENCE_RULE

Index of the rule of a topology and order, 0 if missing.

Argument	Declaration	Meaning
DATABASE	TYPE(REFERENCE_RULE_DB_TYPE), intent(IN)	Database being filled.
TOPOLOGY	INTEGER(I4), intent(IN)	Topology code (B2..S1).
ORDER	INTEGER(I4), intent(IN), OPTIONAL	Order of the matrix, or polynomial/quadrature order (see routine).

FIND_STRUCTURAL_RULE

Rule of a structural element: the reduced one (order -1) when requested and available, otherwise the default rule.

Argument	Declaration	Meaning
DATABASE	TYPE(REFERENCE_RULE_DB_TYPE), intent(IN)	Database being filled.
TOPOLOGY	INTEGER(I4), intent(IN)	Topology code (B2..S1).
REDUCED	LOGICAL, intent(IN)	Vector on the free DOFs.

EXPANSION_MESH_ORDER

Points per direction of an expansion mesh given the per-mesh need.

Argument	Declaration	Meaning
TOPOLOGY	INTEGER(I4), intent(IN)	Topology code (B2..S1).
MESH	INTEGER(I4), intent(IN)	Expansion mesh.

Argument	Declaration	Meaning
EXPANSION_ORDER	INTEGER(I4), intent(IN), OPTIONAL(:)	Gauss points per direction needed by each expansion mesh.

C.11 Layer ELEMENTS

Element matrices (reference and separable kernels), MITC, dense products.

C.11.1MUL2_DENSE_PRODUCTS

File SRC/ELEMENTS/mul2_dense_products.for.

Dense products used by the element kernels. $\text{matrix}(i,j) += \text{sum_q } a(q,i) * b(q,j)$ for $i \leq j$ only. the caller mirrors the upper triangle. the loops are written without array temporaries (no stack use, safe inside openmp regions) and with a 4-column register block to reuse every streamed column of a.

Uses: MUL2_KINDS.

Table C.231. Procedures of MUL2_DENSE_PRODUCTS.

Procedure	Kind	Visibility	Purpose
ACCUMULATE_UPPER_PRODUCT	Subroutine	public	A, b: (rows,n) with the first rows rows used. index (ascending) selects the columns of a, b and their position in matrix; without it the columns are 1..size(a,2).
COLUMN	Function → integer(i4)	private	Column of the matrix addressed by a (possibly selected) index.
ADD_ENTRY	Subroutine	private	Add a value to one matrix entry.
ACCUMULATE_PRODUCT	Subroutine	public	$\text{Matrix}(i,j) += \text{sum_q } a(q,i) * b(q,j)$ for all i, j.

ACCUMULATE_UPPER_PRODUCT

A, b: (rows,n) with the first rows rows used. index (ascending) selects the columns of a, b and their position in matrix; without it the columns are 1..size(a,2).

Argument	Declaration	Meaning
A	REAL(R8), intent(IN)(:,:)	Left factor, rows = integration rows, columns = DOFs.
B	REAL(R8), intent(IN)(:,:)	Right factor (same shape as A).
ROWS	INTEGER(I4), intent(IN)	Number of rows actually used.
MATRIX	REAL(R8), intent(INOUT)(:,:)	In/out: receives the upper triangle of $A^T B$.
INDEX	INTEGER(I4), intent(IN), OPTIONAL(:)	Optional ascending list of selected columns.

ACCUMULATE_PRODUCT

$\text{Matrix}(i,j) += \text{sum_q } a(q,i) * b(q,j)$ for all i, j.

Argument	Declaration	Meaning
A	REAL(R8), intent(IN)(:,:)	Matrix or operand A.
B	REAL(R8), intent(IN)(:,:)	Matrix or operand B.
ROWS	INTEGER(I4), intent(IN)	Number of rows actually used.
MATRIX	REAL(R8), intent(INOUT)(:,:)	In/out: matrix receiving $A^T B$.

C.11.2MUL2_ELEMENT_MATRICES

File SRC/ELEMENTS/mul2_element_matrices.for.

Dense reference element matrices for linear mechanics.

Uses: `MUL2_KINDS`, `MUL2_STATUS`, `MUL2_NODES`, `MUL2_ELEMENTS`, `MUL2_TOPOLOGIES`, `MUL2_KINEMATICS`, `MUL2_EXPANSION_MESHES`, `MUL2_REFERENCE_SYSTEMS`, `MUL2_GAUSS_POINTS`, `MUL2_GAUSS_GEOMETRY`, `MUL2_GAUSS_MATERIALS`, `MUL2_DOF_LAYOUT`, `MUL2_POINT_BASES`, `MUL2_MITC`, `MUL2_LINEAR_KINEMATICS`, `MUL2_DENSE_PRODUCTS`, `MUL2_SEPARABLE_KERNEL`.

Table C.234. Derived type `ELEMENT_MATRIX_TYPE` — Dense K and M of one element with the description of its local DOFs.

Component	Type	Shape	Default	Meaning
<code>ELEMENT_INDEX</code>	<code>INTEGER(I4)</code>	``	<code>0_I4</code>	Element index.
<code>LOCAL_DOF_COUNT</code>	<code>INTEGER(I4)</code>	``	<code>0_I4</code>	Number of local DOFs.
<code>GLOBAL_DOF</code>	<code>INTEGER(I8)</code> , <code>ALLOCATABLE</code>	<code>(:)</code>	``	Global DOF numbers.
<code>NODE_INDEX</code>	<code>INTEGER(I4)</code> , <code>ALLOCATABLE</code>	<code>(:)</code>	``	Node index of each local node.
<code>KINEMATIC_INDEX</code>	<code>INTEGER(I4)</code> , <code>ALLOCATABLE</code>	<code>(:)</code>	``	Kinematic index of each local node.
<code>STRUCTURAL_NODE</code>	<code>INTEGER(I4)</code> , <code>ALLOCATABLE</code>	<code>(:)</code>	``	Local structural node of each local DOF.
<code>FIELD</code>	<code>INTEGER(I4)</code> , <code>ALLOCATABLE</code>	<code>(:)</code>	``	Field of each local DOF (or request list).
<code>TERM</code>	<code>INTEGER(I4)</code> , <code>ALLOCATABLE</code>	<code>(:)</code>	``	Expansion term of each local DOF.
<code>CSR_POSITION</code>	<code>INTEGER(I8)</code> , <code>ALLOCATABLE</code>	<code>(:, :)</code>	``	Precomputed CSR positions.
<code>STIFFNESS</code>	<code>REAL(R8)</code> , <code>ALLOCATABLE</code>	<code>(:, :)</code>	``	Stiffness values or matrix.
<code>MASS</code>	<code>REAL(R8)</code> , <code>ALLOCATABLE</code>	<code>(:, :)</code>	``	Mass values or matrix.
<code>INTERNAL_FORCE</code>	<code>REAL(R8)</code> , <code>ALLOCATABLE</code>	<code>(:)</code>	``	Internal force of the nonlinear (total lagrangian) element.

Table C.235. Derived type `POINT_WORK_TYPE` — Workspace of the point-by-point kernels. the basis of a degree of freedom is `n_node * f_term`: the structural factors (one per node) and the expansion factors (one per term and per distinct expansion) are evaluated once per point and combined for every dof, instead of re-evaluating both for each of the (often hundreds of) dofs.

Component	Type	Shape	Default	Meaning
<code>N_MEMO</code>	<code>INTEGER(I4)</code>	``	<code>0_I4</code>	
<code>MEMO_OF</code>	<code>INTEGER(I4)</code> , <code>ALLOCATABLE</code>	<code>(:)</code>	``	
<code>MEMO_NODE</code>	<code>INTEGER(I4)</code> , <code>ALLOCATABLE</code>	<code>(:)</code>	``	
<code>MEMO_TERMS</code>	<code>INTEGER(I4)</code> , <code>ALLOCATABLE</code>	<code>(:)</code>	``	
<code>MEMO_KIN</code>	<code>INTEGER(I4)</code> , <code>ALLOCATABLE</code>	<code>(:)</code>	``	
<code>MEMO_FIELD</code>	<code>INTEGER(I4)</code> , <code>ALLOCATABLE</code>	<code>(:)</code>	``	
<code>SVAL</code>	<code>REAL(R8)</code> , <code>ALLOCATABLE</code>	<code>(:)</code>	``	
<code>SGRAD</code>	<code>REAL(R8)</code> , <code>ALLOCATABLE</code>	<code>(:, :)</code>	``	
<code>FVAL</code>	<code>REAL(R8)</code> , <code>ALLOCATABLE</code>	<code>(:, :)</code>	``	
<code>FGRAD</code>	<code>REAL(R8)</code> , <code>ALLOCATABLE</code>	<code>(:, :, :)</code>	``	
<code>VALUE</code>	<code>REAL(R8)</code> , <code>ALLOCATABLE</code>	<code>(:)</code>	``	Values.
<code>BCOL</code>	<code>REAL(R8)</code> , <code>ALLOCATABLE</code>	<code>(:, :)</code>	``	
<code>GRAD</code>	<code>REAL(R8)</code> , <code>ALLOCATABLE</code>	<code>(:, :)</code>	``	

Table C.236. Procedures of `MUL2_ELEMENT_MATRICES`.

Procedure	Kind	Visibility	Purpose
<code>BUILD_LINEAR_ELEMENT_MATRICES</code>	Subroutine	public	K and M of one element: separable kernel when applicable, otherwise batched point-by-point reference kernel.
<code>TRY_SEPARABLE</code>	Subroutine	private	Call the separable kernel (internal helper).

Procedure	Kind	Visibility	Purpose
BUILD_NONLINEAR_ELEMENT_MATRICES	Subroutine	public	Total lagrangian (st. venant-kirchhoff) element at the state u0. $h = \text{grad } u$ (element frame), $e = b \cdot u + (h^t h)/2$ (green strain), $\sigma = m \cdot \gamma$ (pk2 stress, electric displacement, heat flux), $f_{\text{int}} = \sum_p w \cdot b(u)^t \sigma$, $k_t = \sum_p w [b(u)^t m \cdot b(u) + (d_{i,d_j}) \cdot \text{grad } n_i \cdot s \cdot \text{grad } n_j]$ with $b(u)$ the variation of γ (b + the term $h^t \cdot \text{grad } n$). the thermal and pyroelectric loads enter σ , the coupling blocks $k(u,t)$ and $k(\phi,t)$ are added. mitc ties only the linear part.
ADD_MASS	Subroutine	private	$M = \sum_p (w \cdot \rho)_p \cdot n_p \cdot n_{pt}$, with n non-zero only between dofs of the same field: one product per field. <code>basis_value(point,dof)</code> .
MIRROR_UPPER_TRIANGLE	Subroutine	private	Copy the upper triangle of K (and M) into the lower one.
SETUP_POINT_WORK	Subroutine	private	Distinct expansions of the dofs of an element: two dofs share a table of expansion factors when their expansion (family, order, reference) is the same, e.g. the three displacement components.
SAME_EXPANSION	Function → logical	private	
EVALUATE_POINT_COLUMNS	Subroutine	private	Basis value, gradient and generalised-strain column of every dof at one gauss point (<code>work%value</code> , <code>work%grad</code> , <code>work%bcol</code>).
COLUMN_FROM_BASIS	Subroutine	private	Generalised-strain column of a dof from its basis gradient: the displacement operator (with the mitc tying when active), the potential gradient (rows 7-9) or the temperature gradient (10-12).
BUILD_ELEMENT_DOF_LIST	Subroutine	public	Local DOF list of an element: global numbers, structural node, field and term of every local DOF.
VALIDATE_INPUT	Subroutine	private	Consistency checks of the databases before the element kernel runs.
CLEAR_ELEMENT_MATRIX	Subroutine	public	Release an element-matrix container.

BUILD_LINEAR_ELEMENT_MATRICES

K and M of one element: separable kernel when applicable, otherwise batched point-by-point reference kernel.

Argument	Declaration	Meaning
ELEMENT_INDEX	INTEGER(I4), intent(IN)	Index of the element in the element database.
NODES	TYPE(NODE_DB_TYPE), intent(IN)	Structural node database.
ELEMENTS	TYPE(ELEMENT_DB_TYPE), intent(IN)	Structural element database.
KINEMATICS	TYPE(KINEMATICS_DB_TYPE), intent(IN)	Field-dependent kinematics database.
EXPANSIONS	TYPE(EXPANSION_DB_TYPE), intent(IN)	Expansion (section/thickness) mesh database.
DOF_LAYOUT	TYPE(DOF_LAYOUT_TYPE), intent(IN)	Global DOF numbering.
RULES	TYPE(REFERENCE_RULE_DB_TYPE), intent(IN)	Reference quadrature rule database.
GAUSS_LAYOUT	TYPE(GAUSS_LAYOUT_TYPE), intent(IN)	Gauss-point layout (global point numbering).
STRUCTURAL_CACHE	TYPE(STRUCTURAL_GEOMETRY_CACHE_TYPE), intent(IN)	Geometry cache of the structural points.
EXPANSION_CACHE	TYPE(EXPANSION_GEOMETRY_CACHE_TYPE), intent(IN)	Geometry cache of the expansion points.
GEOMETRY	TYPE(GAUSS_GEOMETRY_TYPE), intent(IN)	Combined Gauss geometry (coordinates, weights).
FRAMES	TYPE(ELEMENT_FRAME_DB_TYPE), intent(IN)	Element reference frames.

Argument	Declaration	Meaning
MATERIAL_CACHE	TYPE(MATERIAL_CACHE_TYPE), intent(IN)	Resolved constitutive matrices.
MATERIAL_MAP	TYPE(GAUSS_MATERIAL_MAP_TYPE), intent(IN)	Gauss point -> material cache index.
MATRICES	TYPE(ELEMENT_MATRIX_TYPE), intent(INOUT)	Element matrix container (K, M, DOF lists).
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).
MITC	TYPE(MITC_DATA_TYPE), intent(IN), OPTIONAL	MITC tying data of the element.
WITH_MASS	LOGICAL, intent(IN), OPTIONAL	Also build the mass matrix.
FORCE_GENERAL	LOGICAL, intent(IN), OPTIONAL	Force the reference (point-by-point) kernel.
PART	INTEGER(I4), intent(IN), OPTIONAL	Part: part_full, part_normal or part_shear of the constitutive matrix (selective integration); with_stiffness = .false. skips k.
WITH_STIFFNESS	LOGICAL, intent(IN), OPTIONAL	
COUPLING	LOGICAL, intent(IN), OPTIONAL	Coupling = .true.: stiffness receives only the thermoelastic coupling block $k(u,t) = - \sum_p w b^t (c \alpha) n_t$ (not symmetric: the block $k(t,u)$ is zero in the stationary problem).
GEOMETRIC	LOGICAL, intent(IN), OPTIONAL	Geometric = .true.: stiffness receives the geometric (initial stress) matrix of the state u_0 (global dof vector): $k_g(i,j) = \sum_p w (d_{i,d_j}) \text{grad } n_i \cdot s \text{grad } n_j$, with s the stress tensor of u_0 (element frame) and d the local direction of the displacement component of a dof.
U0	REAL(R8), intent(IN), OPTIONAL(:)	

BUILD_NONLINEAR_ELEMENT_MATRICES

Total lagrangian (st. venant-kirchhoff) element at the state u_0 . $h = \text{grad } u$ (element frame), $e = b u + (h^t h)/2$ (green strain), $\sigma = m \gamma$ (pk2 stress, electric displacement, heat flux), $f_{\text{int}} = \sum_p w b(u)^t \sigma$, $k_t = \sum_p w [b(u)^t m b(u) + (d_{i,d_j}) \text{grad } n_i \cdot s \text{grad } n_j]$ with $b(u)$ the variation of γ (b + the term $h^t \text{grad } n$). the thermal and pyroelectric loads enter σ , the coupling blocks $k(u,t)$ and $k(\phi,t)$ are added. mitc ties only the linear part.

Argument	Declaration	Meaning
ELEMENT_INDEX	INTEGER(I4), intent(IN)	Index of the element in the element database.
NODES	TYPE(NODE_DB_TYPE), intent(IN)	Structural node database.
ELEMENTS	TYPE(ELEMENT_DB_TYPE), intent(IN)	Structural element database.
KINEMATICS	TYPE(KINEMATICS_DB_TYPE), intent(IN)	Field-dependent kinematics database.
EXPANSIONS	TYPE(EXPANSION_DB_TYPE), intent(IN)	Expansion (section/thickness) mesh database.
DOF_LAYOUT	TYPE(DOF_LAYOUT_TYPE), intent(IN)	Global DOF numbering.
RULES	TYPE(REFERENCE_RULE_DB_TYPE), intent(IN)	Reference quadrature rule database.
GAUSS_LAYOUT	TYPE(GAUSS_LAYOUT_TYPE), intent(IN)	Gauss-point layout (global point numbering).
STRUCTURAL_CACHE	TYPE(STRUCTURAL_GEOMETRY_CACHE_TYPE), intent(IN)	Geometry cache of the structural points.
EXPANSION_CACHE	TYPE(EXPANSION_GEOMETRY_CACHE_TYPE), intent(IN)	Geometry cache of the expansion points.
GEOMETRY	TYPE(GAUSS_GEOMETRY_TYPE), intent(IN)	Combined Gauss geometry (coordinates, weights).
FRAMES	TYPE(ELEMENT_FRAME_DB_TYPE), intent(IN)	Element reference frames.
MATERIAL_CACHE	TYPE(MATERIAL_CACHE_TYPE), intent(IN)	Resolved constitutive matrices.

Argument	Declaration	Meaning
MATERIAL_MAP	TYPE(GAUSS_MATERIAL_MAP_TYPE), intent(IN)	Gauss point -> material cache index.
MATRICES	TYPE(ELEMENT_MATRIX_TYPE), intent(INOUT)	Element matrix container (K, M, DOF lists).
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).
MITC	TYPE(MITC_DATA_TYPE), intent(IN)	MITC tying data of the element.
U0	REAL(R8), intent(IN)(:)	

BUILD_ELEMENT_DOF_LIST

Local DOF list of an element: global numbers, structural node, field and term of every local DOF.

Argument	Declaration	Meaning
ELEMENT_INDEX	INTEGER(I4), intent(IN)	Index of the element in the element database.
NODES	TYPE(NODE_DB_TYPE), intent(IN)	Structural node database.
ELEMENTS	TYPE(ELEMENT_DB_TYPE), intent(IN)	Structural element database.
KINEMATICS	TYPE(KINEMATICS_DB_TYPE), intent(IN)	Field-dependent kinematics database.
DOF_LAYOUT	TYPE(DOF_LAYOUT_TYPE), intent(IN)	Global DOF numbering.
MATRICES	TYPE(ELEMENT_MATRIX_TYPE), intent(INOUT)	Element matrix container (K, M, DOF lists).
STATUS	TYPE(STATUS_TYPE), intent(INOUT)	Status of the call (OK, warning or error with message and source).

CLEAR_ELEMENT_MATRIX

Release an element-matrix container.

Argument	Declaration	Meaning
MATRICES	TYPE(ELEMENT_MATRIX_TYPE), intent(INOUT)	Element matrix container (K, M, DOF lists).

C.11.3 MUL2_GENERAL_KERNEL

File SRC/ELEMENTS/mul2_general_kernel.for.

Element matrices of curved beams and shells (general geometry). the basis of a degree of freedom is $\phi = n_i(x_i) f_{\tau}(c)$, with c the position in the expansion mesh (section of the beam, thickness of the shell). the displacements are global cartesian components, so the element is a solid with the map of mul2_general_geometry: $g_a = d x / d t_a$ (rows of the jacobian), $\text{grad } \phi = g^{-1} d \phi / d t$. the strains are formed in the covariant basis, $e_{\sim ab} = 1/2 (g_a \cdot u_{,b} + g_b \cdot u_{,a})$, where the mitc interpolation can replace the components that lock (a shell: the transverse shears and the membrane ones, beam: axial and shears) by the values at tying points of the same position in the thickness; then they are rotated to the frame of the material, $e_l = t(q) e_{\sim}$, $q = r g^{-1}$, $r = \text{global_to_local}$ at the point. without tying this is exactly the cartesian strain of a solid. the electric potential and the temperature use the cartesian gradient. general_context_setup geometry and tying tables of an element general_point_columns strain columns of all the dofs at a point build_general_element_matrices k, m (or the thermoelastic block) the recovery of strain and stress uses the same columns.

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_NODES, MUL2_ELEMENTS, MUL2_TOPOLOGIES, MUL2_KINEMATICS, MUL2_EXPANSION_MESHES, MUL2_REFERENCE_SYSTEMS, MUL2_GAUSS_POINTS, MUL2_GAUSS_GEOMETRY, MUL2_GAUSS_MATERIALS, MUL2_DOF_LAYOUT, MUL2_POINT_BASES, MUL2_DENSE_PRODUCTS, MUL2_ELEMENT_MATRICES, MUL2_GENERAL_GEOMETRY, MUL2_SHAPE_FUNCTIONS.

Table C.241. Derived type TIE_FAMILY_TYPE — One family of tying points: tensor product of a list in xi and one in eta; the rows of the strain that are interpolated from them.

Component	Type	Shape	Default	Meaning
NX	INTEGER(I4)	“	0_I4	

Component	Type	Shape	Default	Meaning
NY	INTEGER(I4)	``	1_I4	
XL	REAL(R8)	(4)	0.0_R8	
YL	REAL(R8)	(4)	0.0_R8	
ROW	LOGICAL	(6)	.FALSE.	
SHAPE	REAL(R8), ALLOCATABLE	(:,:)	``	Shape function values at the tying/rule points.
DSHAPE	REAL(R8), ALLOCATABLE	(:,:,:)	``	

Table C.242. Derived type `GENERAL_CONTEXT_TYPE` — Geometry of one element and its tying tables.

Component	Type	Shape	Default	Meaning
TOPOLOGY	INTEGER(I4)	``	0_I4	Topology code.
DS	INTEGER(I4)	``	0_I4	
NN	INTEGER(I4)	``	0_I4	
AXIS	INTEGER(I4)	(3)	0_I4	
N_FAMILY	INTEGER(I4)	``	0_I4	
COORD	REAL(R8)	(3,16)	0.0_R8	
TRIAD	REAL(R8)	(3,3,16)	0.0_R8	
REFERENCE	REAL(R8)	(3)	0.0_R8	
KINK	INTEGER(I4)	(16)	0_I4	
FAMILY	TYPE(TIE_FAMILY_TYPE)	(3)	``	Expansion family code.

Table C.243. Procedures of `MUL2_GENERAL_KERNEL`.

Procedure	Kind	Visibility	Purpose
GENERAL_CONTEXT_SETUP	Subroutine	public	Geometry (nodes, triads, reference vector), active expansion axes and, with tying, the tying points of the element.
GENERAL_POINT_COLUMNS	Subroutine	public	Strain columns of all the dofs at one point. n, dn structural shapes and natural derivatives (nn, ds) natural structural natural coordinates of the point c position in the expansion mesh (3) dof_node/field, fv, fg local node, field, expansion factor and its gradient (local axes) of every dof bcol(1:6) strain in the local frame (bcol(7:9) potential gradient, bcol(10:12) temperature gradient); value the basis; g, r, det geometry at the point.
BUILD_GENERAL_ELEMENT_MATRICES	Subroutine	public	
ADD_MASS	Subroutine	private	Add the mass contribution of a batch of points, one product per displacement component.
MIRROR	Subroutine	private	
TIE_WEIGHTS	Subroutine	private	Weights of the tying points of a family at a natural point.
COVARIANT_ROWS	Subroutine	private	Covariant strain rows (11, 22, 33, 13, 23, 12; engineering shears) of a displacement in the global direction f with scalar basis derivatives dphi = d phi / d t.
BUILD_T	Subroutine	private	6 x 6 matrix that turns the covariant engineering strains into the ones of the local frame, $q(m,a) = e_m \cdot g^a$.
LAGRANGE	Function $\rightarrow$ real(r8)	private	Lagrange polynomial i of the n points x(1:n) at xi.
INVERT3	Subroutine	private	

GENERAL_CONTEXT_SETUP

Geometry (nodes, triads, reference vector), active expansion axes and, with tying, the tying points of the element.

Argument	Declaration	Meaning
CTX	TYPE(GENERAL_CONTEXT_TYPE), intent(OUT)	
ELEMENT_INDEX	INTEGER(I4), intent(IN)	Index of the element in the element database.
NODES	TYPE(NODE_DB_TYPE), intent(IN)	Structural node database.
ELEMENTS	TYPE(ELEMENT_DB_TYPE), intent(IN)	Structural element database.
EXPANSIONS	TYPE(EXPANSION_DB_TYPE), intent(IN)	Expansion (section/thickness) mesh database.
EXPANSION_CACHE	TYPE(EXPANSION_GEOMETRY_CACHE_TYPE), intent(IN)	Geometry cache of the expansion points.
FRAMES	TYPE(ELEMENT_FRAME_DB_TYPE), intent(IN)	Element reference frames.
TYING	LOGICAL, intent(IN)	
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

GENERAL_POINT_COLUMNS

Strain columns of all the dofs at one point. n, dn structural shapes and natural derivatives (nn, ds) natural structural natural coordinates of the point c position in the expansion mesh (3) dof_node/field, fv, fg local node, field, expansion factor and its gradient (local axes) of every dof bcol(1:6) strain in the local frame (bcol(7:9) potential gradient, bcol(10:12) temperature gradient); value the basis; g, r, det geometry at the point.

Argument	Declaration	Meaning
CTX	TYPE(GENERAL_CONTEXT_TYPE), intent(IN)	
N	REAL(R8), intent(IN)(:)	Shape function values.
DN	REAL(R8), intent(IN)(:,:)	Shape derivatives.
NATURAL	REAL(R8), intent(IN)(3)	Natural coordinates (xi, eta, nu).
C	REAL(R8), intent(IN)(3)	
N_DOF	INTEGER(I4), intent(IN)	
DOF_NODE	INTEGER(I4), intent(IN)(:)	
DOF_FIELD	INTEGER(I4), intent(IN)(:)	
FV	REAL(R8), intent(IN)(:)	
FG	REAL(R8), intent(IN)(:,:)	
BCOL	REAL(R8), intent(OUT)(:,:)	
VALUE	REAL(R8), intent(OUT)(:)	Value(s) (see routine).
G	REAL(R8), intent(OUT)(3,3)	
R	REAL(R8), intent(OUT)(3,3)	
DET	REAL(R8), intent(OUT)	
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

BUILD_GENERAL_ELEMENT_MATRICES

Argument	Declaration	Meaning
ELEMENT_INDEX	INTEGER(I4), intent(IN)	Index of the element in the element database.
NODES	TYPE(NODE_DB_TYPE), intent(IN)	Structural node database.
ELEMENTS	TYPE(ELEMENT_DB_TYPE), intent(IN)	Structural element database.

Argument	Declaration	Meaning
KINEMATICS	TYPE(KINEMATICS_DB_TYPE), intent(IN)	Field-dependent kinematics database.
EXPANSIONS	TYPE(EXPANSION_DB_TYPE), intent(IN)	Expansion (section/thickness) mesh database.
DOF_LAYOUT	TYPE(DOF_LAYOUT_TYPE), intent(IN)	Global DOF numbering.
RULES	TYPE(REFERENCE_RULE_DB_TYPE), intent(IN)	Reference quadrature rule database.
GAUSS_LAYOUT	TYPE(GAUSS_LAYOUT_TYPE), intent(IN)	Gauss-point layout (global point numbering).
STRUCTURAL_CACHE	TYPE(STRUCTURAL_GEOMETRY_CACHE_TYPE), intent(IN)	Geometry cache of the structural points.
EXPANSION_CACHE	TYPE(EXPANSION_GEOMETRY_CACHE_TYPE), intent(IN)	Geometry cache of the expansion points.
GEOMETRY	TYPE(GAUSS_GEOMETRY_TYPE), intent(IN)	Combined Gauss geometry (coordinates, weights).
FRAMES	TYPE(ELEMENT_FRAME_DB_TYPE), intent(IN)	Element reference frames.
MATERIAL_CACHE	TYPE(MATERIAL_CACHE_TYPE), intent(IN)	Resolved constitutive matrices.
MATERIAL_MAP	TYPE(GAUSS_MATERIAL_MAP_TYPE), intent(IN)	Gauss point -> material cache index.
MATRICES	TYPE(ELEMENT_MATRIX_TYPE), intent(INOUT)	Element matrix container (K, M, DOF lists).
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).
TYING	LOGICAL, intent(IN)	
WITH_MASS	LOGICAL, intent(IN), OPTIONAL	Also build the mass matrix.
COUPLING	LOGICAL, intent(IN), OPTIONAL	

C.11.4MUL2_MITC

File SRC/ELEMENTS/mul2_mitc.for.

MITC SHEAR-LOCKING CORRECTION BY ASSUMED STRAIN ROWS. A TIED STRAIN ROW OF THE B OPERATOR IS NOT EVALUATED AT THE QUADRATURE POINT BUT INTERPOLATED FROM ITS VALUES AT THE TYING POINTS OF A TYING SET (TENSOR LAGRANGE INTERPOLATION): $B_R(XI) = \text{SUM}_L M_L(XI) B_R(XI_L)$ THE SAME EXPANSION POINT IS USED AT THE TYING POINTS, SO THE CORRECTION ACTS ON THE FEM DIRECTIONS ONLY. THE TABLE REPRODUCES THE BASELINE (PHYSICAL STRAIN COMPONENTS, $A=1/\text{SQRT}(3)$, $B=\text{SQRT}(3/5)$): ELEMENT ROW(S) TYING POINTS (XI x ETA x ZETA) B2 5,6 {0} B3 5,6 {-A,A} B4 5,6 {-B,0,B} Q4 4 {0} x {1,-1} 5 {1,-1} x {0} Q9 1,4 {-A,A} x {-B,0,B} 2,5 {-B,0,B} x {-A,A} 6 {-A,A} x {-A,A} H8 4,5,6 {0} x {0} x {0} H27 4,5,6 {-A,A} x {-A,A} x {-A,A}

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_TOPOLOGIES, MUL2_LOCAL_GRADIENTS, MUL2_LINEAR_KINEMATICS.

Table C.247. Derived type TYING_SET_TYPE — Points of one tying set: the tensor product of one list per direction.

Component	Type	Shape	Default	Meaning
COUNT_1D	INTEGER(I4)	(3)	1_I4	Tying points per direction.
COORDINATE_1D	REAL(R8)	(3,3)	0.0_R8	Tying coordinates per direction.

Table C.248. Derived type MITC_DATA_TYPE — MITC data of one element: tied rows, tying sets, shapes and gradients at the tying points.

Component	Type	Shape	Default	Meaning
ACTIVE	LOGICAL	""	.FALSE.	Flags.
SET_COUNT	INTEGER(I4)	""	0_I4	Number of tying sets.
ROW_SET	INTEGER(I4)	(6)	0_I4	Tying set of each strain row (0 = not tied).
SET	TYPE(TYING_SET_TYPE)	(MAX_SET)	""	Tying sets.
SHAPE	REAL(R8), ALLOCATABLE	(:,:,:) :	""	Shape function values at the tying/rule points.
GRADIENT	REAL(R8), ALLOCATABLE	(:,:,:,):	""	Local-frame gradients.

Table C.249. Procedures of MUL2_MITC .

Procedure	Kind	Visibility	Purpose
MITC_IS_AVAILABLE	Function → logical	public	True for topologies with a tying table (B2 B3 B4 Q4 Q9 H8 H20 H27).
MITC_PREPARE_ELEMENT	Subroutine	public	Tying table of a topology and the shapes/gradients at its points. node_local(:,i) is the local coordinate of element node i.
MITC_CLEAR	Subroutine	private	Reset MITC data (including the tying sets).
DEFINE_TYING	Subroutine	private	Table of tied rows and tying points of a topology.
SET_LIST	Subroutine	private	Set the tying coordinates of one direction.
POINT_OF_SET	Subroutine	private	Natural coordinates of tying point l (first direction fastest).
INTERPOLATION_WEIGHT	Subroutine	public	M_l(natural): lagrange weight of every tying point of the set.
MITC_TIE_COLUMN	Subroutine	public	Replace the tied rows of the strain column of one dof. natural: structural natural coordinates of the point local_node: structural node of the dof factor, factor_gradient: expansion factor and its gradient at the expansion point (unchanged at the tying points) component: column of the frame rotation for the dof field b_column: in: regular column; out: column with tied rows

MITC_IS_AVAILABLE

True for topologies with a tying table (B2 B3 B4 Q4 Q9 H8 H20 H27).

Argument	Declaration	Meaning
TOPOLOGY	INTEGER(I4), intent(IN)	Topology code (B2..S1).

MITC_PREPARE_ELEMENT

Tying table of a topology and the shapes/gradients at its points. node_local(:,i) is the local coordinate of element node i.

Argument	Declaration	Meaning
TOPOLOGY	INTEGER(I4), intent(IN)	Topology code (B2..S1).
NODE_LOCAL	REAL(R8), intent(IN)(:,:)	Local-frame coordinates of the element nodes.
DATA	TYPE(MITC_DATA_TYPE), intent(INOUT)	MITC data of the element.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

INTERPOLATION_WEIGHT

M_l(natural): lagrange weight of every tying point of the set.

Argument	Declaration	Meaning
SET	TYPE(TYING_SET_TYPE), intent(IN)	Tying set.
NATURAL	REAL(R8), intent(IN)(3)	Natural coordinates (xi, eta, nu).
WEIGHT	REAL(R8), intent(OUT)(:)	Output: interpolation weights of the tying points.

MITC_TIE_COLUMN

Replace the tied rows of the strain column of one dof. natural: structural natural coordinates of the point local_node: structural node of the dof factor, factor_gradient: expansion factor and its gradient at the expansion point (unchanged at the tying points) component: column of the frame rotation for the dof field b_column: in: regular column; out: column with tied rows

Argument	Declaration	Meaning
DATA	TYPE(MITC_DATA_TYPE), intent(IN)	MITC data of the element.

Argument	Declaration	Meaning
NATURAL	REAL(R8), intent(IN)(3)	Natural coordinates (xi, eta, nu).
LOCAL_NODE	INTEGER(I4), intent(IN)	Local structural node index.
FACTOR	REAL(R8), intent(IN)	PARDISO handle with factors and stored matrix.
FACTOR_GRADIENT	REAL(R8), intent(IN)(3)	Gradient of the expansion factor (local frame).
COMPONENT	REAL(R8), intent(IN)(3)	Displacement component index (1=u, 2=v, 3=w) or the frame column of the field.
B_COLUMN	REAL(R8), intent(INOUT)(6)	Strain column (6 values) of the DOF; tied rows are replaced on output.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.11.5 MUL2_SEPARABLE_KERNEL

File SRC/ELEMENTS/mul2_separable_kernel.for.

Separable cuf element matrices. the combined basis of a dof is $n_i(p) f_t(q)$: a structural factor (structural point p) times an expansion factor (expansion point q of a sub-element e). in the element frame, with s the structural axes, $d\phi/dd = \sigma_d(p,i) * \epsilon_d(q,t)$ $\sigma_d = dn/dd$ (d in s), n (d not in s) $\epsilon_d = f$ (d in s), df/dd (d not in s) and the strain column is $b(:,r) = \sum_d w_d f(r) \sigma_d \epsilon_d$, with $w_d f = b_operator(e_d) * frame(:,f)$. tied (mitc) rows replace σ by its tensor-lagrange interpolation from the tying points. with a constitutive matrix c_e constant inside each sub-element e, $k((i,t,f),(j,s,h)) = \sum_e \sum_{d,d'} \lambda_{e,dd'}(i,j,f,h) * \epsilon_{e,dd'}(t,s) \lambda = \sum_{r,r'} c_e(r,r') w_d f(r) w_{d'} h(r') \sigma_{dd'}^{cc'}(i,j) \sigma_{dd'}^{cc'}(i,j) = \sum_p w_p \sigma_d^{cc'}(p,i) \sigma_d^{cc'}(p,i) \epsilon_{e,dd'}(t,s) = \sum_q w_q \epsilon_d(q,t) \epsilon_d(q,s)$ so the cost is additive in the structural and expansion points instead of their product. the result is algebraically the same as the point-by-point kernel (round-off only).

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_ELEMENTS, MUL2_TOPOLOGIES, MUL2_KINEMATICS, MUL2_EXPANSION_MESHES, MUL2_REFERENCE_SYSTEMS, MUL2_GAUSS_POINTS, MUL2_GAUSS_GEOMETRY, MUL2_GAUSS_MATERIALS, MUL2_POINT_BASES, MUL2_MITC, MUL2_LINEAR_KINEMATICS, MUL2_DENSE_PRODUCTS.

Table C.254. Derived type SPEC_BASIS_TYPE — Expansion functions of one distinct kinematic specification at all expansion points of the element: $\epsilon_s(\text{point}, \text{term}, 0:3)$, 0 = value.

Component	Type	Shape	Default	Meaning
SPEC	TYPE(EXPANSION_SPEC_TYPE)	``	``	Expansion specification.
TERMS	INTEGER(I4)	``	0_I4	Number of terms.
EPS	REAL(R8), ALLOCATABLE	(:, :, :)	``	Expansion functions EPS(point, term, 0:3).

Table C.255. Derived type EXPANSION_PRODUCT_TYPE — $\epsilon_m(t,s,k,e)$: $k = 3*(d-1)+d'$ for $d, d' = 1..3$ and $k = 10$ for $f f$.

Component	Type	Shape	Default	Meaning
READY	LOGICAL	``	.FALSE.	Computed flag.
EM	REAL(R8), ALLOCATABLE	(:, :, :, :)	``	Expansion integrals EM(t,s,k,e).

Table C.256. Procedures of MUL2_SEPARABLE_KERNEL.

Procedure	Kind	Visibility	Purpose
BUILD_SEPARABLE_MATRICES	Subroutine	public	Applicable = .false. leaves the matrices untouched (the caller then uses the point-by-point kernel). field/structural_node/term are the local dof lists (node-major, then field, then term).
PRODUCT_OF	Function → integer(i4)	private	Product of the three counts of a tying set.
SAME_SPEC	Function → logical	private	True when two kinematic specifications are identical (family, order, field reference).
PREPARE_PRODUCT	Subroutine	private	$\epsilon_{m,e,k}(t,s) = \sum_q w_q \epsilon_{a,d}(q,t) \epsilon_{b,d'}(q,s)$, once per pair.

BUILD_SEPARABLE_MATRICES

Applicable = .false. leaves the matrices untouched (the caller then uses the point-by-point kernel).
field/structural_node/term are the local dof lists (node-major, then field, then term).

Argument	Declaration	Meaning
ELEMENT_INDEX	INTEGER(I4), intent(IN)	Index of the element in the element database.
ELEMENTS	TYPE(ELEMENT_DB_TYPE), intent(IN)	Structural element database.
KINEMATICS	TYPE(KINEMATICS_DB_TYPE), intent(IN)	Field-dependent kinematics database.
EXPANSIONS	TYPE(EXPANSION_DB_TYPE), intent(IN)	Expansion (section/thickness) mesh database.
RULES	TYPE(REFERENCE_RULE_DB_TYPE), intent(IN)	Reference quadrature rule database.
LAYOUT	TYPE(GAUSS_LAYOUT_TYPE), intent(IN)	Gauss-point layout (global point numbering).
STRUCTURAL_CACHE	TYPE(STRUCTURAL_GEOMETRY_CACHE_TYPE), intent(IN)	Geometry cache of the structural points.
EXPANSION_CACHE	TYPE(EXPANSION_GEOMETRY_CACHE_TYPE), intent(IN)	Geometry cache of the expansion points.
GEOMETRY	TYPE(GAUSS_GEOMETRY_TYPE), intent(IN)	Combined Gauss geometry (coordinates, weights).
FRAMES	TYPE(ELEMENT_FRAME_DB_TYPE), intent(IN)	Element reference frames.
MATERIAL_CACHE	TYPE(MATERIAL_CACHE_TYPE), intent(IN)	Resolved constitutive matrices.
MATERIAL_MAP	TYPE(GAUSS_MATERIAL_MAP_TYPE), intent(IN)	Gauss point -> material cache index.
MITC_DATA	TYPE(MITC_DATA_TYPE), intent(IN)	MITC tying data of the element.
FIELD	INTEGER(I4), intent(IN)(:)	Field index (1=U, 2=V, 3=W, ...).
STRUCTURAL_NODE	INTEGER(I4), intent(IN)(:)	Local structural node index.
TERM	INTEGER(I4), intent(IN)(:)	Expansion term index.
KINEMATIC_INDEX	INTEGER(I4), intent(IN)(:)	Kinematic index of every structural node.
WITH_MASS	LOGICAL, intent(IN)	Also build the mass matrix.
STIFFNESS	REAL(R8), intent(INOUT)(:,:)	Stiffness matrix (or values).
MASS	REAL(R8), intent(INOUT)(:,:)	Mass matrix (or values).
APPLICABLE	LOGICAL, intent(OUT)	Set to .FALSE. when the separable path cannot be used.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).
PART	INTEGER(I4), intent(IN), OPTIONAL	
WITH_STIFFNESS	LOGICAL, intent(IN), OPTIONAL	

C.12 Layer ASSEMBLY

DOF numbering, CSR pattern and assembly, sparse operations.

C.12.1 MUL2_DOF_LAYOUT

File SRC/ASSEMBLY/mul2_dof_layout.for.

Field-major global degree-of-freedom numbering.

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_NODES, MUL2_INCIDENCE, MUL2_ELEMENTS, MUL2_KINEMATICS, MUL2_EXPANSION_MESHES, MUL2_TOPOLOGIES, MUL2_CUF_BASES.

Table C.258. Derived type DOF_LAYOUT_TYPE — Field-major global DOF numbering: terms per (node, field), first DOF, range of every field.

Component	Type	Shape	Default	Meaning
-----------	------	-------	---------	---------

Component	Type	Shape	Default	Meaning
TERM_COUNT	INTEGER(I4), ALLOCATABLE	(:, :)	``	Number of terms of (node, field).
FIRST	INTEGER(I8), ALLOCATABLE	(:, :)	``	First global DOF of (node, field) or first element of a node.
FIELD_FIRST	INTEGER(I8)	(N_FIELDS)	0_I8	First DOF of each field.
FIELD_LAST	INTEGER(I8)	(N_FIELDS)	0_I8	Last DOF of each field.
TOTAL_DOF	INTEGER(I8)	``	0_I8	Total number of DOFs.
ALIAS	INTEGER(I8), ALLOCATABLE	(:)	``	Dofs joined by coincidence (join coincident): provisional number -> final number. not allocated when nothing is joined.

Table C.259. Procedures of `MUL2_DOF_LAYOUT`.

Procedure	Kind	Visibility	Purpose
<code>BUILD_DOF_LAYOUT</code>	Subroutine	public	Compute the number of terms of every (node, field) and the field-major global numbering.
<code>NODE_FIELD_TERM_COUNT</code>	Subroutine	private	Terms of one field at one node from its expansion family and the expansion meshes of the incident elements.
<code>LAGRANGE_TERM_COUNT</code>	Function → integer(i4)	private	Number of nodes of an expansion mesh (terms of a Lagrange expansion).
<code>MESH_IS_HLE</code>	Function → logical	private	
<code>EXPANSION_DIMENSION_FOR_ELEMENT</code>	Function → integer(i4)	private	Natural dimension of the sub-elements of an expansion mesh (-1 unknown, -2 mixed).
<code>GLOBAL_DOF</code>	Function → integer(i8)	public	Global DOF number of (node index, field, term).
<code>CLEAR_LAYOUT</code>	Subroutine	private	Release the arrays of a DOF layout.

`BUILD_DOF_LAYOUT`

Compute the number of terms of every (node, field) and the field-major global numbering.

Argument	Declaration	Meaning
NODES	<code>TYPE(NODE_DB_TYPE), intent(IN)</code>	Structural node database.
ELEMENTS	<code>TYPE(ELEMENT_DB_TYPE), intent(IN)</code>	Structural element database.
KINEMATICS	<code>TYPE(KINEMATICS_DB_TYPE), intent(IN)</code>	Field-dependent kinematics database.
EXPANSIONS	<code>TYPE(EXPANSION_DB_TYPE), intent(IN)</code>	Expansion (section/thickness) mesh database.
LAYOUT	<code>TYPE(DOF_LAYOUT_TYPE), intent(INOUT)</code>	Gauss-point layout (global point numbering).
STATUS	<code>TYPE(STATUS_TYPE), intent(OUT)</code>	Status of the call (OK, warning or error with message and source).

`GLOBAL_DOF`

Global DOF number of (node index, field, term).

Argument	Declaration	Meaning
LAYOUT	<code>TYPE(DOF_LAYOUT_TYPE), intent(IN)</code>	Gauss-point layout (global point numbering).
NODE_INDEX	<code>INTEGER(I4), intent(IN)</code>	Index of the node in the node database.
FIELD	<code>INTEGER(I4), intent(IN)</code>	Field index (1=U, 2=V, 3=W, ...).
TERM	<code>INTEGER(I4), intent(IN)</code>	Term index (1-based) within the (node, field) block.

C.12.2MUL2_SPARSE_ASSEMBLY

File SRC/ASSEMBLY/mul2_sparse_assembly.for.

One-based ilp64 csr pattern and global matrix assembly.

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_ELEMENT_MATRICES.

Table C.262. Derived type SPARSE_SYSTEM_TYPE — CSR (1-based, 64-bit) pattern shared by K and M with their value arrays.

Component	Type	Shape	Default	Meaning
ORDER	INTEGER(I8)	``	0_I8	Order of the matrix / expansion order / quadrature order (see type).
NONZERO_COUNT	INTEGER(I8)	``	0_I8	Number of stored entries.
ROW_POINTER	INTEGER(I8), ALLOCATABLE	(:)	``	CSR row pointer.
COLUMN_INDEX	INTEGER(I8), ALLOCATABLE	(:)	``	CSR column indices.
STIFFNESS	REAL(R8), ALLOCATABLE	(:)	``	Stiffness values or matrix.
MASS	REAL(R8), ALLOCATABLE	(:)	``	Mass values or matrix.

Table C.263. Derived type DOF_LIST_TYPE — Table(k) > 0 marks a lagrange-expansion dof and term(k) its term: two such dofs of the same table are coupled only if coupled(term,term).

Component	Type	Shape	Default	Meaning
GLOBAL_DOF	INTEGER(I8), ALLOCATABLE	(:)	``	Global DOF numbers.
TABLE	INTEGER(I4), ALLOCATABLE	(:)	``	Lagrange coupling table id (0 = fully coupled).
TERM	INTEGER(I4), ALLOCATABLE	(:)	``	Expansion term of each local DOF.

Table C.264. Derived type COUPLING_TABLE_TYPE — COUPLED(t,s): the expansion nodes t and s share a sub-element.

Component	Type	Shape	Default	Meaning
COUPLED	LOGICAL, ALLOCATABLE	(:, :)	``	Coupling flags of expansion terms.

Table C.265. Procedures of MUL2_SPARSE_ASSEMBLY.

Procedure	Kind	Visibility	Purpose
BUILD_AND_ASSEMBLE_SPARSE_SYSTEM	Subroutine	public	Pattern + assembly from dense element matrices (reference path used by the unit tests).
BUILD_SPARSE_PATTERN	Subroutine	public	CSR pattern from the DOF lists of dense element matrices.
BUILD_PATTERN_FROM_DOF_LISTS	Subroutine	public	Csr pattern from the dof lists alone (no element matrices needed). row r collects the dofs of every element touching r, without duplicates, sorted. memory is proportional to the final nnz.
ARE_COUPLED	Function → logical	private	False only for two lagrange-expansion dofs of the same table whose terms do not share a sub-element (their entry is exactly zero).
SORT_SEGMENT	Subroutine	private	Ascending sort of column(first:last) (insertion, heap for long rows).
SIFT_COLUMN	Subroutine	private	Heap-sort helper for SORT_SEGMENT.
SCATTER_ELEMENT	Subroutine	public	Add one element matrix pair into the shared pattern (binary search in every row; no allocation, no pattern change).
BUILD_SCATTER_MAPS	Subroutine	private	Precompute CSR_POSITION of every element entry.
ASSEMBLE_ELEMENT_MATRICES	Subroutine	public	Add dense element matrices through precomputed scatter maps.

Procedure	Kind	Visibility	Purpose
FIND_CSR_POSITION	Function → integer(i8)	private	Position of (row, column) in the CSR arrays, 0 when absent.
SORT_COORDINATES	Subroutine	private	Sort (row, column) pairs lexicographically.
SIFT_DOWN	Subroutine	private	Heap-sort helper.
PAIR_LESS	Function → logical	private	Lexicographic comparison of two (row, column) pairs.
SWAP_PAIR	Subroutine	private	Swap two (row, column) pairs.
CLEAR_SPARSE_SYSTEM	Subroutine	public	Release a CSR system.

BUILD_AND_ASSEMBLE_SPARSE_SYSTEM

Pattern + assembly from dense element matrices (reference path used by the unit tests).

Argument	Declaration	Meaning
TOTAL_DOF	INTEGER(I8), intent(IN)	Total number of DOFs.
ELEMENT_MATRICES	TYPE(ELEMENT_MATRIX_TYPE), intent(INOUT) (:)	Dense element matrices with their DOF lists.
SYSTEM	TYPE(SPARSE_SYSTEM_TYPE), intent(INOUT)	Sparse system (CSR pattern, K, M).
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

BUILD_SPARSE_PATTERN

CSR pattern from the DOF lists of dense element matrices.

Argument	Declaration	Meaning
TOTAL_DOF	INTEGER(I8), intent(IN)	Total number of DOFs.
ELEMENT_MATRICES	TYPE(ELEMENT_MATRIX_TYPE), intent(INOUT) (:)	Dense element matrices with their DOF lists.
SYSTEM	TYPE(SPARSE_SYSTEM_TYPE), intent(INOUT)	Sparse system (CSR pattern, K, M).
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

BUILD_PATTERN_FROM_DOF_LISTS

Csr pattern from the dof lists alone (no element matrices needed). row r collects the dofs of every element touching r, without duplicates, sorted. memory is proportional to the final nnz.

Argument	Declaration	Meaning
TOTAL_DOF	INTEGER(I8), intent(IN)	Total number of DOFs.
LISTS	TYPE(DOF_LIST_TYPE), intent(IN)(:)	DOF lists of all elements.
SYSTEM	TYPE(SPARSE_SYSTEM_TYPE), intent(INOUT)	Sparse system (CSR pattern, K, M).
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).
WITH_MASS	LOGICAL, intent(IN), OPTIONAL	Also build the mass matrix.
TABLES	TYPE(COUPLING_TABLE_TYPE), intent(IN), OPTIONAL(:)	Lagrange coupling tables (one per expansion mesh).

SCATTER_ELEMENT

Add one element matrix pair into the shared pattern (binary search in every row; no allocation, no pattern change).

Argument	Declaration	Meaning
SYSTEM	TYPE(SPARSE_SYSTEM_TYPE), intent(INOUT)	Sparse system (CSR pattern, K, M).
GLOBAL_DOF	INTEGER(I8), intent(IN)(:)	Global DOF numbers of the local DOFs.
STIFFNESS	REAL(R8), intent(IN)(:,:)	Dense element stiffness.
MASS	REAL(R8), intent(IN)(:,:)	Dense element mass (may be empty).
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

ASSEMBLE_ELEMENT_MATRICES

Add dense element matrices through precomputed scatter maps.

Argument	Declaration	Meaning
ELEMENT_MATRICES	TYPE(ELEMENT_MATRIX_TYPE), intent(IN)(:)	Dense element matrices with their DOF lists.
SYSTEM	TYPE(SPARSE_SYSTEM_TYPE), intent(INOUT)	Sparse system (CSR pattern, K, M).
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

CLEAR_SPARSE_SYSTEM

Release a CSR system.

Argument	Declaration	Meaning
SYSTEM	TYPE(SPARSE_SYSTEM_TYPE), intent(INOUT)	Sparse system (CSR pattern, K, M).

C.12.3MUL2_SPARSE_OPERATIONS

File SRC/ASSEMBLY/mul2_sparse_operations.for .

Basic operations on the shared csr pattern. reduction to the free dofs keeps a map from every reduced entry to its position in the full pattern, so that any value array (k, m or a combination) is reduced without re-building the pattern.

Uses: MUL2_KINDS.

Table C.272. Derived type REDUCED_PATTERN_TYPE — CSR pattern of the free-DOF sub-matrix and the maps to the full numbering.

Component	Type	Shape	Default	Meaning
ORDER	INTEGER(I8)	''	0_I8	Order of the matrix / expansion order / quadrature order (see type).
NONZERO_COUNT	INTEGER(I8)	''	0_I8	Number of stored entries.
FREE_TO_GLOBAL	INTEGER(I8), ALLOCATABLE	(:)	''	Free DOF -> global DOF.
ROW_POINTER	INTEGER(I8), ALLOCATABLE	(:)	''	CSR row pointer.
COLUMN_INDEX	INTEGER(I8), ALLOCATABLE	(:)	''	CSR column indices.
SOURCE_POSITION	INTEGER(I8), ALLOCATABLE	(:)	''	Position in the full CSR of every reduced entry.

Table C.273. Procedures of MUL2_SPARSE_OPERATIONS .

Procedure	Kind	Visibility	Purpose
SPARSE_MULTIPLY	Subroutine	public	$Y = a \times x$ for a full (non-triangular) csr matrix.
BUILD_REDUCED_PATTERN	Subroutine	public	Pattern of the submatrix on the dofs with constrained(dof) = false.
REDUCE_VALUES	Subroutine	public	Copy the values of a full matrix into the reduced pattern.

Procedure	Kind	Visibility	Purpose
EXPAND_VECTOR	Subroutine	public	Scatter a reduced vector back to the full dof numbering.

SPARSE_MULTIPLY

$Y = a \cdot x$ for a full (non-triangular) csr matrix.

Argument	Declaration	Meaning
ORDER	INTEGER(I8), intent(IN)	Order of the matrix, or polynomial/quadrature order (see routine).
ROW_POINTER	INTEGER(I8), intent(IN)(:)	CSR row pointer (1-based, size N+1).
COLUMN_INDEX	INTEGER(I8), intent(IN)(:)	CSR column indices (1-based).
VALUE	REAL(R8), intent(IN)(:)	Value(s) (see routine).
X	REAL(R8), intent(IN)(:)	Vector or coordinate.
Y	REAL(R8), intent(OUT)(:)	Result vector.

BUILD_REDUCED_PATTERN

Pattern of the submatrix on the dofs with constrained(dof) = false.

Argument	Declaration	Meaning
ORDER	INTEGER(I8), intent(IN)	Order of the matrix, or polynomial/quadrature order (see routine).
ROW_POINTER	INTEGER(I8), intent(IN)(:)	CSR row pointer (1-based, size N+1).
COLUMN_INDEX	INTEGER(I8), intent(IN)(:)	CSR column indices (1-based).
CONSTRAINED	LOGICAL, intent(IN)(:)	Flags of constrained DOFs.
PATTERN	TYPE(REDUCED_PATTERN_TYPE), intent(INOUT)	Reduced CSR pattern with maps.

REDUCE_VALUES

Copy the values of a full matrix into the reduced pattern.

Argument	Declaration	Meaning
PATTERN	TYPE(REDUCED_PATTERN_TYPE), intent(IN)	Reduced CSR pattern with maps.
FULL_VALUE	REAL(R8), intent(IN)(:)	Values of the full CSR matrix.
REDUCED_VALUE	REAL(R8), intent(OUT)(:)	Output: values on the reduced pattern.

EXPAND_VECTOR

Scatter a reduced vector back to the full dof numbering.

Argument	Declaration	Meaning
PATTERN	TYPE(REDUCED_PATTERN_TYPE), intent(IN)	Reduced CSR pattern with maps.
REDUCED	REAL(R8), intent(IN)(:)	Vector on the free DOFs.
FULL	REAL(R8), intent(INOUT)(:)	Output: vector in the full DOF numbering.

C.13 Layer BOUNDARY

Constraints and loads.

C.13.1MUL2_BOUNDARY_APPLICATION

File SRC/BOUNDARY/mul2_boundary_application.for.

Geometric mechanical bc resolution and exact static elimination.

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_BOUNDARY_CONDITIONS, MUL2_NODES, MUL2_INCIDENCE, MUL2_ELEMENTS, MUL2_KINEMATICS, MUL2_EXPANSION_MESHES, MUL2_REFERENCE_SYSTEMS, MUL2_GENERAL_GEOMETRY, MUL2_DOF_LAYOUT, MUL2_CUF_BASIS, MUL2_TOPOLOGIES, MUL2_SPARSE_ASSEMBLY, MUL2_FIELDS.

Table C.278. Derived type CONSTRAINT_SET_TYPE — Constrained-DOF flags and prescribed values.

Component	Type	Shape	Default	Meaning
COUNT	INTEGER(I8)	``	0_I8	Number of items.
ACTIVE	LOGICAL, ALLOCATABLE	(:)	``	Flags.
VALUE	REAL(R8), ALLOCATABLE	(:)	``	Values.
HAS_TIES	LOGICAL	``	.FALSE.	Ties (floating electrodes): master(i) = i, or the dof that holds the value of the dof i (a slave, also marked active).
MASTER	INTEGER(I8), ALLOCATABLE	(:)	``	

Table C.279. Procedures of MUL2_BOUNDARY_APPLICATION.

Procedure	Kind	Visibility	Purpose
BUILD_MECHANICAL_BOUNDARY_DATA	Subroutine	public	Constraints and loads of bc.dat; fields (fields.dat) are optional.
BUILD_BOUNDARY_WITH_FIELDS	Subroutine	private	
ADD_PLANE_TIE	Subroutine	private	Floating electrode: the electric-potential dofs of the lagrange (or hierarchical vertex) section nodes on the plane are tied to the first one; the others become inactive slaves (value 0).
APPLY_DOF_TIES	Subroutine	public	$K = t^t k t$ and $f = t^t f$ for the ties: the rows and columns of the slaves are added to those of their master; a slave keeps a unit diagonal (it is constrained to zero and copied from the master after the solution). k and, when allocated, m share the pattern.
SHIFT_ENTRY	Subroutine	private	Move entry i to position target ($\leq i$), shifting the others up.
EXPAND_TIES	Subroutine	public	Copy the value of every master to its slaves.
ADD_PLANE_CONSTRAINT	Subroutine	private	Constrain the DOFs of every node/term whose physical position lies on the plane $A x + B y + C z + D = 0$.
HLE_TERM_ON_PLANE	Function → logical	private	A hle term is on the plane when its vertex, both ends of its side or all the vertices of its sub-element are.
ADD_POINT_FORCE	Subroutine	private	Add a point load at a physical position: nodal term for Lagrange, projection on all monomials for Taylor.
ADD_POINT_VALUE	Subroutine	private	Prescribed field value (e.g. electric potential) at a section node of a lagrange or hierarchical expansion.
APPLY_STATIC_CONSTRAINTS	Subroutine	public	Exact symmetric elimination: $F = F - K u_c$, zero rows/columns of constrained DOFs, unit diagonal.
SET_DOF_CONSTRAINT	Subroutine	private	Mark one DOF as constrained; conflicting values are an error.
FIRST_ELEMENT	Function → integer(i4)	private	First element incident to a node (0 if none).
EXPANDED_POINT	Subroutine	private	Physical position of a section point: node + frame * local section coordinate.
PLACED_POINT	Subroutine	public	Global position of a point of the expansion mesh at a node, seen by the element: the frame of an ordinary element, the triad of the node of a curved beam / shell.
POINT_ON_PLANE	Function → logical	private	Plane test with a scaled tolerance.

Procedure	Kind	Visibility	Purpose
FIND_EXPANSION_AXES	Subroutine	private	Active local axes and dimension of an expansion mesh from its node extent.
CLEAR_BOUNDARY_DATA	Subroutine	private	Release the constraint set and force vector.

BUILD_MECHANICAL_BOUNDARY_DATA

Constraints and loads of bc.dat; fields (fields.dat) are optional.

Argument	Declaration	Meaning
BOUNDARIES	TYPE(BOUNDARY_DB_TYPE), intent(IN)	Boundary-condition database.
NODES	TYPE(NODE_DB_TYPE), intent(IN)	Structural node database.
ELEMENTS	TYPE(ELEMENT_DB_TYPE), intent(IN)	Structural element database.
KINEMATICS	TYPE(KINEMATICS_DB_TYPE), intent(IN)	Field-dependent kinematics database.
EXPANSIONS	TYPE(EXPANSION_DB_TYPE), intent(IN)	Expansion (section/thickness) mesh database.
FRAMES	TYPE(ELEMENT_FRAME_DB_TYPE), intent(IN)	Element reference frames.
DOF_LAYOUT	TYPE(DOF_LAYOUT_TYPE), intent(IN)	Global DOF numbering.
TOLERANCE	REAL(R8), intent(IN)	Geometric tolerance.
CONSTRAINTS	TYPE(CONSTRAINT_SET_TYPE), intent(INOUT)	Constrained-DOF set (flags and prescribed values).
FORCE	REAL(R8), ALLOCATABLE, intent(INOUT)(:)	Global force vector.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).
FIELDS	TYPE(FIELD_DB_TYPE), intent(IN), OPTIONAL	

APPLY_DOF_TIES

$K = t^t k t$ and $f = t^t f$ for the ties: the rows and columns of the slaves are added to those of their master; a slave keeps a unit diagonal (it is constrained to zero and copied from the master after the solution). k and, when allocated, m share the pattern.

Argument	Declaration	Meaning
SYSTEM	TYPE(SPARSE_SYSTEM_TYPE), intent(INOUT)	Sparse system (CSR pattern, K, M).
FORCE	REAL(R8), intent(INOUT), OPTIONAL(:)	Global force vector.
CONSTRAINTS	TYPE(CONSTRAINT_SET_TYPE), intent(IN)	Constrained-DOF set (flags and prescribed values).
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

EXPAND_TIES

Copy the value of every master to its slaves.

Argument	Declaration	Meaning
CONSTRAINTS	TYPE(CONSTRAINT_SET_TYPE), intent(IN)	Constrained-DOF set (flags and prescribed values).
VECTOR	REAL(R8), intent(INOUT)(:)	Vector(s) / eigenvectors.

APPLY_STATIC_CONSTRAINTS

Exact symmetric elimination: $F = F - K u_c$, zero rows/columns of constrained DOFs, unit diagonal.

Argument	Declaration	Meaning
SYSTEM	TYPE(SPARSE_SYSTEM_TYPE), intent(INOUT)	Sparse system (CSR pattern, K, M).
FORCE	REAL(R8), intent(INOUT)(:)	Global force vector.
CONSTRAINTS	TYPE(CONSTRAINT_SET_TYPE), intent(IN)	Constrained-DOF set (flags and prescribed values).
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

PLACED_POINT

Global position of a point of the expansion mesh at a node, seen by the element: the frame of an ordinary element, the triad of the node of a curved beam / shell.

Argument	Declaration	Meaning
NODES	TYPE(NODE_DB_TYPE), intent(IN)	Structural node database.
ELEMENTS	TYPE(ELEMENT_DB_TYPE), intent(IN)	Structural element database.
FRAMES	TYPE(ELEMENT_FRAME_DB_TYPE), intent(IN)	Element reference frames.
ELEMENT_INDEX	INTEGER(I4), intent(IN)	Index of the element in the element database.
NODE_INDEX	INTEGER(I4), intent(IN)	
LOCAL_POINT	REAL(R8), intent(IN)(3)	
PHYSICAL_POINT	REAL(R8), intent(OUT)(3)	
MID_SURFACE	LOGICAL, intent(IN), OPTIONAL	Mid_surface: a plane selects the nodes of a shell by their mid-surface point (the director is not exactly the normal of a symmetry plane), and constrains the whole thickness.

C.13.2MUL2_COINCIDENT_JOIN

File SRC/BOUNDARY/mul2_coincident_join.for.

Joining of degrees of freedom by coincidence of their positions. the ordinary joint is the shared node (same node number in the elements). with `join coincident` in `analysis.dat` the degrees of freedom of different nodes are also identified when they are the same field at the same point of space, as in the historical code: a beam whose section node lies on a plate joins the plate even if the structural nodes of the two elements are different. le the dof of a node of the expansion mesh is the field at the point $x(\text{node}) + \text{offset}(\text{expansion node})$. te the term 1 (the field at the node) is joined at the same point; the higher terms only when the two elements have the same orientation (they are derivatives along the element axes). hle not joined (side and internal modes are not values). the dof are renumbered: the alias table of the layout maps every provisional dof to its final number (global_dof applies it).

Uses: `MUL2_KINDS`, `MUL2_STATUS`, `MUL2_NODES`, `MUL2_INCIDENCE`, `MUL2_ELEMENTS`, `MUL2_KINEMATICS`, `MUL2_EXPANSION_MESHES`, `MUL2_REFERENCE_SYSTEMS`, `MUL2_GENERAL_GEOMETRY`, `MUL2_DOF_LAYOUT`, `MUL2_BOUNDARY_APPLICATION`.

Table C.285. Procedures of `MUL2_COINCIDENT_JOIN`.

Procedure	Kind	Visibility	Purpose
JOIN_COINCIDENT_DOFS	Subroutine	public	Relative_tolerance is a fraction of the diagonal of the model.
ADD_ENTRY	Subroutine	private	Add a value to one matrix entry.
GLOBAL_DOF_RAW	Function → integer(i8)	private	Dof number before any alias (the layout has none yet).
CELL_OF	Subroutine	private	
BUCKET_INDEX	Function → integer(i4)	private	
FIND_ROOT	Function → integer(i8)	private	

Procedure	Kind	Visibility	Purpose
JOINABLE	Function → logical	private	

JOIN_COINCIDENT_DOFS

Relative_tolerance is a fraction of the diagonal of the model.

Argument	Declaration	Meaning
NODES	TYPE(NODE_DB_TYPE), intent(IN)	Structural node database.
ELEMENTS	TYPE(ELEMENT_DB_TYPE), intent(IN)	Structural element database.
KINEMATICS	TYPE(KINEMATICS_DB_TYPE), intent(IN)	Field-dependent kinematics database.
EXPANSIONS	TYPE(EXPANSION_DB_TYPE), intent(IN)	Expansion (section/thickness) mesh database.
FRAMES	TYPE(ELEMENT_FRAME_DB_TYPE), intent(IN)	Element reference frames.
RELATIVE_TOLERANCE	REAL(R8), intent(IN)	
LAYOUT	TYPE(DOF_LAYOUT_TYPE), intent(INOUT)	Gauss-point layout (global point numbering).
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.13.3MUL2_SURFACE_LOADS

File SRC/BOUNDARY/mul2_surface_loads.for.

Surface heat loads on the exposed skin of the model. a surface piece is the product of a part of the structural element and a part of the expansion sub-element, with two parameters in all: beam (1d x 2d): axis x edge of the section (lateral skin) end node x whole section (end faces) plate (2d x 1d): surface x end of the thickness (top, bottom) edge x whole thickness (sides) solid (3d x 0d): face of the element a piece is exposed when no other piece has the same centre (shared faces are interior). the outward normal points away from the centre of the element. loads ($q > 0$ heats the body), all on the temperature field (4): 1 flux $q = q_0$ on the pieces of a plane 2 sun $q = \text{absorptivity } g \max(0, n_s)$ (s : towards the sun) 3 convection $q = h (t_{\text{inf}} - t)$ on the pieces of a plane the convection adds $h \int n n^t$ to the conduction matrix.

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_LOG, MUL2_TIMER, MUL2_MODEL, MUL2_MODEL_CACHE, MUL2_BOUNDARY_CONDITIONS, MUL2_RECOVERY, MUL2_TOPOLOGIES, MUL2_SHAPE_FUNCTIONS, MUL2_QUADRATURE, MUL2_NODES, MUL2_KINEMATICS, MUL2_DOF_LAYOUT, MUL2_POINT_BASES, MUL2_SPARSE_ASSEMBLY, MUL2_SORTING, MUL2_FIELDS.

Table C.287. Derived type PART_TYPE — Part of a natural domain: natural = a + m (u, v). kind: 0 point, 1 line, 2 square, 3 triangle (the parameters are the first columns).

Component	Type	Shape	Default	Meaning
KIND	INTEGER(I4)	''	0_I4	Record kind.
WHOLE	LOGICAL	''	.FALSE.	
A	REAL(R8)	(3)	0.0_R8	
M	REAL(R8)	(3,2)	0.0_R8	

Table C.288. Procedures of MUL2_SURFACE_LOADS.

Procedure	Kind	Visibility	Purpose
APPLY_SURFACE_LOADS	Subroutine	public	
MESH_OF	Function → integer(i4)	private	
STRUCTURAL_PARTS	Subroutine	private	Parts of the natural domain of a topology that can build a surface.
PART_DIMENSION	Function → integer(i4)	private	

Procedure	Kind	Visibility	Purpose
PIECE_IS_SURFACE	Function → logical	private	A piece is a surface when the two parts have two parameters in all and are not both the whole domain.
PART_CENTRE	Subroutine	private	Natural coordinates of the centre of a part.
PIECE_CENTRE	Subroutine	private	Physical position of the centre of a piece.
ELEMENT_REFERENCE	Subroutine	private	Centre of the element (sub-element): the normals point away from it.
FIND_EXPOSED	Subroutine	private	Exposed = no other piece has the same centre.
PART_RULE	Subroutine	private	
INTEGRATE_PIECE	Subroutine	private	Integral of all the surface loads over one exposed piece.
ON_PLANE	Function → logical	private	The zero plane selects everything.

APPLY_SURFACE_LOADS

Argument	Declaration	Meaning
MODEL	TYPE(MODEL_TYPE), intent(IN)	Complete input model (all databases).
CACHE	TYPE(MODEL_CACHE_TYPE), intent(IN)	Model cache built by BUILD_MODEL_CACHE.
SYSTEM	TYPE(SPARSE_SYSTEM_TYPE), intent(INOUT)	Sparse system (CSR pattern, K, M).
FORCE	REAL(R8), intent(INOUT)(:)	Global force vector.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.14 Layer SOLVERS

PARDISO wrappers and the modal eigensolver (ARPACK / dense).

C.14.1 MUL2_MODAL_SOLVER

File SRC/SOLVERS/mul2_modal_solver.for.

Generalized symmetric eigenproblem $k x = \lambda m x$. arpack (dsaupd/dseupd, mode 3, bmat = g) with spectral transformation $op = inv(k - \sigma m) m$; the linear systems are solved with one pardiso factorization. small problems (or nev near n) use dense lapack dsygv. the k-shift residual is re-computed from the matrices. arpack is built with 64-bit integers and logicals (ilp64 mkl). eigenvalues are returned in ascending order; eigenvectors are m-orthonormal: $x(i)^* t m x(j) = \delta(i,j)$.

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_SPARSE_OPERATIONS, MUL2_PARDISO_FACTOR.

Table C.290. Derived type MODAL_INFO_TYPE — Statistics of the eigensolver run.

Component	Type	Shape	Default	Meaning
BACKEND	CHARACTER(LEN=8)	``	' '	Solver used.
ORDER	INTEGER(I8)	``	0_I8	Order of the matrix / expansion order / quadrature order (see type).
REQUESTED	INTEGER(I8)	``	0_I8	Requested modes.
CONVERGED	INTEGER(I8)	``	0_I8	Converged modes.
SUBSPACE	INTEGER(I8)	``	0_I8	Krylov subspace.
ITERATIONS	INTEGER(I8)	``	0_I8	Iterations.
OPERATIONS	INTEGER(I8)	``	0_I8	Operator applications.
SIGMA	REAL(R8)	``	0.0_R8	Shift.

Table C.291. Procedures of MUL2_MODAL_SOLVER.

Procedure	Kind	Visibility	Purpose
-----------	------	------------	---------

Procedure	Kind	Visibility	Purpose
DSAUPD	Subroutine	external (MKL/ARPACK/LAPACK)	ARPACK reverse-communication driver (external).
DSEUPD	Subroutine	external (MKL/ARPACK/LAPACK)	ARPACK eigenvalue/eigenvector extraction (external).
DSYGV	Subroutine	external (MKL/ARPACK/LAPACK)	LAPACK dense generalised symmetric eigenproblem (external).
DSYSV	Subroutine	external (MKL/ARPACK/LAPACK)	
SOLVE_MODAL_PROBLEM	Subroutine	public	Solve for the nev eigenvalues closest to sigma. row_pointer/column_index: full symmetric csr shared by k and m. eigenvalue(1:nev) ascending; vector(:,i) m-orthonormal; residual(i) = $\ k x - \lambda m x\ / \ k x\ $.
SOLVE_ARPACK	Subroutine	private	Arpack, shift-invert, pardiso
HAS_MASSLESS_ROW	Function → logical	private	Dense lapack fallback a row of m without any non-zero value (a dof that carries no mass, e.g. an electric potential).
SOLVE_DENSE	Subroutine	private	Dense fallback with dsygv for $N \leq 400$ or almost all modes requested.
DENSE_GENERALIZED	Subroutine	private	$A x = \lambda b x$ for symmetric a and b. b singular: $m x = \mu k x$ with k positive definite. returns the lambda in ascending order and the b-orthonormal vectors (a and b are destroyed).
SORT_MODES	Subroutine	private	Ascending order of the eigenvalues (selection sort, vectors follow).
COMPUTE_RESIDUALS	Subroutine	private	Relative residual $\ K x - \lambda M x\ $ of every mode.
SOLVE_BUCKLING_DENSE	Subroutine	public	Linear buckling $(k + \lambda b) x = 0$ on a full symmetric csr pattern, by lapack. rows without geometric stiffness (electric potential) are condensed out. with k positive definite the pencil $g x = \mu k x$ has $\mu = -1/\lambda$: the negative mu give the positive load factors, returned in ascending order (the modes are k-orthonormal).

SOLVE_MODAL_PROBLEM

Solve for the nev eigenvalues closest to sigma. row_pointer/column_index: full symmetric csr shared by k and m. eigenvalue(1:nev) ascending; vector(:,i) m-orthonormal; residual(i) = $\|k x - \lambda m x\| / \|k x\|$.

Argument	Declaration	Meaning
ORDER	INTEGER(I8), intent(IN)	Order of the matrix, or polynomial/quadrature order (see routine).
ROW_POINTER	INTEGER(I8), intent(IN)(:)	CSR row pointer of the reduced (free-DOF) matrices.
COLUMN_INDEX	INTEGER(I8), intent(IN)(:)	CSR column indices (1-based).
STIFFNESS	REAL(R8), intent(IN)(:)	Stiffness matrix (or values).
MASS	REAL(R8), intent(IN)(:)	Mass matrix (or values).
NEV	INTEGER(I4), intent(IN)	Number of requested modes.
SIGMA	REAL(R8), intent(IN)	Spectral shift.
EIGENVALUE	REAL(R8), intent(OUT)(:)	Eigenvalues (ω^2).
VECTOR	REAL(R8), intent(OUT)(:,:)	Vector(s) / eigenvectors.
RESIDUAL	REAL(R8), intent(OUT)(:)	Relative residuals.
INFO	TYPE(MODAL_INFO_TYPE), intent(OUT)	Solver information (backend, iterations, subspace).
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

SOLVE_BUCKLING_DENSE

Linear buckling $(k + \lambda g) x = 0$ on a full symmetric csr pattern, by lapack. rows without geometric stiffness (electric potential) are condensed out. with k positive definite the pencil $g x = \mu k x$ has $\mu = -1/\lambda$: the negative μ give the positive load factors, returned in ascending order (the modes are k -orthonormal).

Argument	Declaration	Meaning
ORDER	INTEGER(I8), intent(IN)	Order of the matrix, or polynomial/quadrature order (see routine).
ROW_POINTER	INTEGER(I8), intent(IN)(:)	CSR row pointer (1-based, size N+1).
COLUMN_INDEX	INTEGER(I8), intent(IN)(:)	CSR column indices (1-based).
STIFFNESS	REAL(R8), intent(IN)(:)	Stiffness matrix (or values).
GEOMETRIC	REAL(R8), intent(IN)(:)	
NEV	INTEGER(I4), intent(IN)	Number of requested modes.
FACTOR	REAL(R8), intent(OUT)(:)	PARDISO handle with factors and stored matrix.
VECTOR	REAL(R8), intent(OUT)(:,:)	Vector(s) / eigenvectors.
COUNT	INTEGER(I4), intent(OUT)	Number of items.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.14.2MUL2_PARDISO_FACTOR

File SRC/SOLVERS/mul2_pardiso_factor.for.

Mkl pardiso ilp64 factorization of real symmetric csr matrices. the caller passes the full symmetric csr (1-based, ilp64). the upper triangle required by mtype 2/-2 is extracted here and kept inside the factor handle. the factors are released with pardiso_release.

Uses: MUL2_KINDS, MUL2_STATUS.

Table C.294. Public constants of MUL2_PARDISO_FACTOR.

Name	Type	Value	Meaning
MTYPE_SYMMETRIC_POSITIVE	INTEGER(I8)	2_I8	PARDISO matrix type.
MTYPE_SYMMETRIC_INDEFINITE	INTEGER(I8)	-2_I8	PARDISO matrix type.
MTYPE_NONSYMMETRIC	INTEGER(I8)	11_I8	Real non-symmetric (structurally symmetric) matrix: the full csr is kept in the handle.

Table C.295. Derived type PARDISO_FACTOR_TYPE — PARDISO handle: internal pointers, parameters and the stored upper triangle.

Component	Type	Shape	Default	Meaning
ACTIVE	LOGICAL	``	.FALSE.	Flags.
ORDER	INTEGER(I8)	``	0_I8	Order of the matrix / expansion order / quadrature order (see type).
MTYPE	INTEGER(I8)	``	0_I8	Matrix type.
PT	INTEGER(I8)	(64)	0_I8	PARDISO internal pointers.
IPARM	INTEGER(I8)	(64)	0_I8	PARDISO parameters.
ROW_POINTER	INTEGER(I8), ALLOCATABLE	(:)	``	CSR row pointer.
COLUMN_INDEX	INTEGER(I8), ALLOCATABLE	(:)	``	CSR column indices.
VALUE	REAL(R8), ALLOCATABLE	(:)	``	Values.

Table C.296. Procedures of MUL2_PARDISO_FACTOR.

Procedure	Kind	Visibility	Purpose
-----------	------	------------	---------

Procedure	Kind	Visibility	Purpose
PARDISO	Subroutine	external (MKL/ARPACK/LAPACK)	MKL PARDISO entry point (external, ILP64).
PARDISO_FACTORIZE	Subroutine	public	Reorder and factorize (phase 12) a full symmetric csr system.
PARDISO_SET_SYSTEM	Subroutine	public	Copy the upper triangle into the handle. afterwards the caller may free its own copy of the system before the factorization.
PARDISO_FACTOR_LOADED	Subroutine	public	Phase 12 on the matrix stored in the handle. a cholesky breakdown (round-off in a very ill-conditioned spd matrix, e.g. high-order taylor expansions) is retried with the symmetric indefinite factorization and pivot perturbation.
PARDISO_FREE_INTERNAL	Subroutine	private	Free the internal memory of pardiso but keep the matrix.
PARDISO_SOLVE	Subroutine	public	Solve $a x = b$ with the factors of the handle (phase 33).
PARDISO_RELEASE	Subroutine	public	Free the internal memory of pardiso (phase -1). safe to repeat.
EXTRACT_UPPER_TRIANGLE	Subroutine	private	Upper triangle of a symmetric CSR matrix.
PARDISO_HINT	Function → character(len=64)	private	Meaning of the most common pardiso error codes.

PARDISO_FACTORIZE

Reorder and factorize (phase 12) a full symmetric csr system.

Argument	Declaration	Meaning
ORDER	INTEGER(I8), intent(IN)	Order of the matrix, or polynomial/quadrature order (see routine).
ROW_POINTER	INTEGER(I8), intent(IN)(:)	CSR row pointer (1-based, size N+1).
COLUMN_INDEX	INTEGER(I8), intent(IN)(:)	CSR column indices (1-based).
VALUE	REAL(R8), intent(IN)(:)	Value(s) (see routine).
MTYPE	INTEGER(I8), intent(IN)	PARDISO matrix type (2 SPD, -2 symmetric indefinite).
FACTOR	TYPE(PARDISO_FACTOR_TYPE), intent(INOUT)	PARDISO handle with factors and stored matrix.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

PARDISO_SET_SYSTEM

Copy the upper triangle into the handle. afterwards the caller may free its own copy of the system before the factorization.

Argument	Declaration	Meaning
ORDER	INTEGER(I8), intent(IN)	Order of the matrix, or polynomial/quadrature order (see routine).
ROW_POINTER	INTEGER(I8), intent(IN)(:)	CSR row pointer (1-based, size N+1).
COLUMN_INDEX	INTEGER(I8), intent(IN)(:)	CSR column indices (1-based).
VALUE	REAL(R8), intent(IN)(:)	Value(s) (see routine).
FACTOR	TYPE(PARDISO_FACTOR_TYPE), intent(INOUT)	PARDISO handle with factors and stored matrix.

Argument	Declaration	Meaning
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).
FULL	LOGICAL, intent(IN), OPTIONAL	Full = .true.: keep the whole (non-symmetric) matrix.

PARDISO_FACTOR_LOADED

Phase 12 on the matrix stored in the handle. a cholesky breakdown (round-off in a very ill-conditioned spd matrix, e.g. high-order taylor expansions) is retried with the symmetric indefinite factorization and pivot perturbation.

Argument	Declaration	Meaning
MTYPE	INTEGER(I8), intent(IN)	PARDISO matrix type (2 SPD, -2 symmetric indefinite).
FACTOR	TYPE(PARDISO_FACTOR_TYPE), intent(INOUT)	PARDISO handle with factors and stored matrix.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

PARDISO_SOLVE

Solve $a x = b$ with the factors of the handle (phase 33).

Argument	Declaration	Meaning
FACTOR	TYPE(PARDISO_FACTOR_TYPE), intent(INOUT)	PARDISO handle with factors and stored matrix.
RIGHT_HAND_SIDE	REAL(R8), intent(IN)(:)	Right-hand side vector.
SOLUTION	REAL(R8), intent(OUT)(:)	Output: solution vector.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

PARDISO_RELEASE

Free the internal memory of pardiso (phase -1). safe to repeat.

Argument	Declaration	Meaning
FACTOR	TYPE(PARDISO_FACTOR_TYPE), intent(INOUT)	PARDISO handle with factors and stored matrix.

C.14.3 MUL2_PARDISO_SOLVER

File SRC/SOLVERS/mul2_pardiso_solver.for.

One-shot linear solve for real symmetric positive-definite csr systems (static analysis) on top of mul2_pardiso_factor.

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_SPARSE_ASSEMBLY, MUL2_PARDISO_FACTOR.

Table C.302. Procedures of MUL2_PARDISO_SOLVER.

Procedure	Kind	Visibility	Purpose
SOLVE_SYMMETRIC_POSITIVE_DEFINITE	Subroutine	public	Static solver: set, optionally release the caller matrix, factorize, solve.

SOLVE_SYMMETRIC_POSITIVE_DEFINITE

Static solver: set, optionally release the caller matrix, factorize, solve.

Argument	Declaration	Meaning
SYSTEM	TYPE(SPARSE_SYSTEM_TYPE), intent(INOUT)	Sparse system (CSR pattern, K, M).
RIGHT_HAND_SIDE	REAL(R8), intent(IN)(:)	Right-hand side vector.
SOLUTION	REAL(R8), ALLOCATABLE, intent(INOUT) (:)	Global displacement vector.

Argument	Declaration	Meaning
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).
RELEASE_SYSTEM	LOGICAL, intent(IN), OPTIONAL	True frees the caller matrix right after PARDISO has copied it.

C.15 Layer ANALYSES

Model cache, parallel assembly, analyses 101 and 103, driver.

C.15.1MUL2_ANALYSIS_101

File SRC/ANALYSES/mul2_analysis_101.for.

Solution 101: linear static mechanical analysis. assemble k -> loads and boundary conditions -> exact symmetric elimination -> pardiso -> displacement coefficients

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_LOG, MUL2_TIMER, MUL2_MODEL, MUL2_MODEL_CACHE, MUL2_MODEL_ASSEMBLY, MUL2_KINEMATICS, MUL2_SURFACE_LOADS, MUL2_PARDISO_FACTOR, MUL2_BOUNDARY_APPLICATION, MUL2_PARDISO_SOLVER, MUL2_ANALYSIS_RESULTS.

Table C.304. Procedures of MUL2_ANALYSIS_101.

Procedure	Kind	Visibility	Purpose
RUN_STATIC_ANALYSIS	Subroutine	public	Analysis 101: assemble K, build loads and constraints, eliminate constraints exactly, factorize with PARDISO and solve $K u = F$.
SOLVE_THERMOELASTIC	Subroutine	private	Monolithic thermoelasticity: the system $[k_{uu} \ k_{ut}] [u] [f] [0 \ k_{tt}] [t] = [q]$ is non-symmetric and solved in one pardiso factorization (mtype 11).

RUN_STATIC_ANALYSIS

Analysis 101: assemble K, build loads and constraints, eliminate constraints exactly, factorize with PARDISO and solve $K u = F$.

Argument	Declaration	Meaning
MODEL	TYPE(MODEL_TYPE), intent(IN)	Complete input model (all databases).
CACHE	TYPE(MODEL_CACHE_TYPE), intent(IN)	Model cache built by BUILD_MODEL_CACHE.
RESULTS	TYPE(ANALYSIS_RESULTS_TYPE), intent(INOUT)	Analysis results container.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.15.2MUL2_ANALYSIS_103

File SRC/ANALYSES/mul2_analysis_103.for.

Solution 103: linear free vibrations. assemble k, m -> eliminate constrained dofs -> arpack/pardiso -> expand modes -> frequencies $f = \sqrt{\lambda} / (2 \pi)$ constrained dofs are removed from the problem (the baseline adds a large penalty to k instead). prescribed non-zero displacements and forces have no meaning here and are ignored with a warning.

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_LOG, MUL2_TIMER, MUL2_MODEL, MUL2_KINEMATICS, MUL2_MODEL_CACHE, MUL2_MODEL_ASSEMBLY, MUL2_BOUNDARY_APPLICATION, MUL2_SPARSE_OPERATIONS, MUL2_MODAL_SOLVER, MUL2_ANALYSIS_RESULTS.

Table C.306. Procedures of MUL2_ANALYSIS_103.

Procedure	Kind	Visibility	Purpose
RUN_MODAL_ANALYSIS	Subroutine	public	Analysis 103: assemble K and M, reduce to the free DOF, solve $K x = \lambda M x$ with shift-invert ARPACK (or dense LAPACK) and sort the modes.

RUN_MODAL_ANALYSIS

Analysis 103: assemble K and M, reduce to the free DOF, solve $K x = \lambda M x$ with shift-invert ARPACK (or dense LAPACK) and sort the modes.

Argument	Declaration	Meaning
MODEL	TYPE(MODEL_TYPE), intent(IN)	Complete input model (all databases).
CACHE	TYPE(MODEL_CACHE_TYPE), intent(IN)	Model cache built by BUILD_MODEL_CACHE.
RESULTS	TYPE(ANALYSIS_RESULTS_TYPE), intent(INOUT)	Analysis results container.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.15.3MUL2_ANALYSIS_104

File SRC/ANALYSES/mul2_analysis_104.for.

SOLUTION 104: LINEAR TIME RESPONSE (NEWMARK). MONOLITHIC SYSTEM FOR ALL THE PHYSICS IN ONE EFFECTIVE MATRIX, U (FIELDS 1-3) SECOND ORDER: $M A + C V + K X = F$ (NEWMARK) T (FIELD 4) FIRST ORDER: $CAP TDOT + K_T T + \dots = Q$ (THETA) P (FIELD 5) ALGEBRAIC: NO INERTIA WITH $C = \alpha M + \beta K$ ON THE MECHANICAL DOFS (RAYLEIGH). WHEN THE ABSOLUTE REFERENCE TEMPERATURE T_0 IS GIVEN THE THERMOELASTIC HEATING TERM $T_0 (-K_{UT})^T V$ IS ADDED TO THE TEMPERATURE ROWS (TWO-WAY COUPLING, THERMOELASTIC DAMPING). EVERY LOAD AND EVERY PRESCRIBED VALUE IS MULTIPLIED BY THE AMPLITUDE $a(t)$ OF TIME_RESP.DAT. THE INITIAL STATE IS $A(0)$ TIMES THE PRESCRIBED VALUES, ZERO VELOCITY, TEMPERATURE ZERO. THE EFFECTIVE MATRIX IS FACTORED ONCE (CONSTANT STEP).

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_LOG, MUL2_MODEL, MUL2_MODEL_CACHE, MUL2_MODEL_ASSEMBLY, MUL2_BOUNDARY_APPLICATION, MUL2_SURFACE_LOADS, MUL2_SPARSE_ASSEMBLY, MUL2_SPARSE_OPERATIONS, MUL2_PARDISO_FACTOR, MUL2_KINEMATICS, MUL2_DYNAMIC_COMMON, MUL2_TIME_INPUT, MUL2_POST_OUTPUT, MUL2_ANALYSIS_RESULTS.

Table C.308. Procedures of MUL2_ANALYSIS_104.

Procedure	Kind	Visibility	Purpose
RUN_TIME_ANALYSIS	Subroutine	public	
INITIAL_ACCELERATION	Subroutine	private	$A(0)$ from $m a = f - k x$ (free mechanical dofs only).

RUN_TIME_ANALYSIS

Argument	Declaration	Meaning
MODEL	TYPE(MODEL_TYPE), intent(IN)	Complete input model (all databases).
CACHE	TYPE(MODEL_CACHE_TYPE), intent(IN)	Model cache built by BUILD_MODEL_CACHE.
RESULTS	TYPE(ANALYSIS_RESULTS_TYPE), intent(INOUT)	Analysis results container.
WARNINGS	TYPE(WARNING_LIST_TYPE), intent(INOUT)	List of warnings for REPORT/WARNING_file.dat.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.15.4MUL2_ANALYSIS_105

File SRC/ANALYSES/mul2_analysis_105.for.

SOLUTION 105: LINEAR BUCKLING. STATIC SOLUTION UNDER THE REFERENCE LOADS (101, THERMAL LOADS INCLUDED) $\rightarrow$ GEOMETRIC MATRIX K_G OF ITS STRESS FIELD $\rightarrow (K + \lambda K_G) X = 0$ ON THE FREE MECHANICAL DOFS. THE TEMPERATURE IS A GIVEN FIELD OF THE PRE-STRESS (ITS DOFS ARE NOT PART OF THE EIGENPROBLEM); THE ELECTRIC POTENTIAL IS CONDENSED OUT (CLOSED-CIRCUIT STIFFNESS WITH THE

PRESCRIBED ELECTRODES). LAMBDA IS THE FACTOR THAT MULTIPLIES THE REFERENCE LOAD, THE CRITICAL LOAD IS LAMBDA TIMES THE APPLIED ONE. RESULTS: DYNAMIC/ BUCKLING_FACTORS.dat AND THE BUCKLING MODES (RESULTS_DYN_*.vtk/msh).

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_LOG, MUL2_MODEL, MUL2_MODEL_CACHE, MUL2_MODEL_ASSEMBLY, MUL2_SPARSE_OPERATIONS, MUL2_MODAL_SOLVER, MUL2_DOF_LAYOUT, MUL2_ANALYSIS_RESULTS, MUL2_ANALYSIS_101.

Table C.310. Procedures of MUL2_ANALYSIS_105.

Procedure	Kind	Visibility	Purpose
RUN_BUCKLING_ANALYSIS	Subroutine	public	

RUN_BUCKLING_ANALYSIS

Argument	Declaration	Meaning
MODEL	TYPE(MODEL_TYPE), intent(IN)	Complete input model (all databases).
CACHE	TYPE(MODEL_CACHE_TYPE), intent(IN)	Model cache built by BUILD_MODEL_CACHE.
RESULTS	TYPE(ANALYSIS_RESULTS_TYPE), intent(INOUT)	Analysis results container.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.15.5MUL2_ANALYSIS_106

File SRC/ANALYSES/mul2_analysis_106.for.

SOLUTION 106: HARMONIC (FREQUENCY) RESPONSE. $(K - W^{*2} M + I W C) X = F$ F: THE LOADS OF BC.dat, PRESCRIBED VALUES AT PHASE 0 ON EVERY FREQUENCY OF FREQ_RESP.dat. THE COMPLEX SYSTEM IS SOLVED AS THE REAL BLOCK SYSTEM $[AR - AI] [XR] [FR] [AI AR] [XI] = [FI]$ (PARDISO, ONE FACTORIZATION PER FREQUENCY). ALL THE PHYSICS OF THE TIME ANALYSIS ARE INCLUDED: THE MECHANICAL DOFS HAVE INERTIA AND THE RAYLEIGH DAMPING $C = \alpha M + \beta K$, THE ELECTRIC POTENTIAL IS ALGEBRAIC, THE TEMPERATURE ROWS GET $I W$ CAPACITY AND, WITH T_0 , THE THERMOELASTIC HEATING $I W T_0 (-K_{UT})^T$. RESULT: DYNAMIC/FREQ_HISTORY.dat (REAL AND IMAGINARY PARTS).

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_LOG, MUL2_MODEL, MUL2_MODEL_CACHE, MUL2_MODEL_ASSEMBLY, MUL2_BOUNDARY_APPLICATION, MUL2_SURFACE_LOADS, MUL2_SPARSE_ASSEMBLY, MUL2_PARDISO_FACTOR, MUL2_KINEMATICS, MUL2_DYNAMIC_COMMON, MUL2_POST_OUTPUT, MUL2_ANALYSIS_RESULTS.

Table C.312. Procedures of MUL2_ANALYSIS_106.

Procedure	Kind	Visibility	Purpose
RUN_FREQUENCY_ANALYSIS	Subroutine	public	

RUN_FREQUENCY_ANALYSIS

Argument	Declaration	Meaning
MODEL	TYPE(MODEL_TYPE), intent(IN)	Complete input model (all databases).
CACHE	TYPE(MODEL_CACHE_TYPE), intent(IN)	Model cache built by BUILD_MODEL_CACHE.
RESULTS	TYPE(ANALYSIS_RESULTS_TYPE), intent(INOUT)	Analysis results container.
WARNINGS	TYPE(WARNING_LIST_TYPE), intent(INOUT)	List of warnings for REPORT/WARNING_file.dat.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.15.6MUL2_ANALYSIS_108

File SRC/ANALYSES/mul2_analysis_108.for.

SOLUTION 108: GEOMETRICALLY NONLINEAR STATIC (TOTAL LAGRANGIAN). NEWTON-RAPHSON WITH LOAD / DISPLACEMENT INCREMENTS. EVERY LOAD AND PRESCRIBED VALUE IS SCALED BY THE FACTOR $\text{LAMBDA} = N / \text{NLSTEP}$ OF STEP N; THE PHYSICS ARE THOSE OF 101 (MECHANICAL, ELECTRIC POTENTIAL, TEMPERATURE, SURFACE HEAT LOADS) IN THE ONE MONOLITHIC SYSTEM $R = \text{LAMBDA } F - F_{\text{INT}}(X) - K_S X = 0$, $K_T DX = R$ WITH F_{INT} FROM THE GREEN STRAIN AND THE PK2 STRESS OF THE MATERIAL (ST. VENANT-KIRCHHOFF). THE TANGENT IS RE-FACTORED AT EVERY ITERATION. K_S IS THE (LINEAR) CONVECTION OF THE SURFACE LOADS. SOLVTEC 2 OF NL_INFO.DAT SELECTS THE ARC-LENGTH METHOD (CRISFIELD, CYLINDRICAL CONSTRAINT ON THE DISPLACEMENTS, ADAPTIVE STEP): LAMBDA BECOMES AN UNKNOWN AND THE PATH CAN PASS LIMIT POINTS (SNAP-THROUGH, SNAP-BACK). IT NEEDS ZERO PRESCRIBED VALUES (THE LOAD IS THE PARAMETER). OUTPUT PER POST STEP: DYNAMIC/TIME_HISTORY.dat (THE TIME COLUMN IS LAMBDA) AND, AT THE END, THE STATIC FILES. STRESSES AND STRAINS OF THE OUTPUT ARE THE LINEARISED MEASURES OF THE FINAL DISPLACEMENTS.

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_LOG, MUL2_TIMER, MUL2_MODEL, MUL2_MODEL_CACHE, MUL2_MODEL_ASSEMBLY, MUL2_BOUNDARY_APPLICATION, MUL2_SURFACE_LOADS, MUL2_SPARSE_ASSEMBLY, MUL2_SPARSE_OPERATIONS, MUL2_PARDISO_FACTOR, MUL2_KINEMATICS, MUL2_DYNAMIC_COMMON, MUL2_POST_OUTPUT, MUL2_ANALYSIS_RESULTS.

Table C.314. Procedures of MUL2_ANALYSIS_108.

Procedure	Kind	Visibility	Purpose
RUN_NONLINEAR_ANALYSIS	Subroutine	public	
SOLVE_TWO	Subroutine	private	Factorises the tangent once and solves for two right-hand sides (the residual and the reference load). the prescribed increments are zero, so the constrained entries of both solutions are zero.
ARC_LENGTH_STEPS	Subroutine	private	Arc-length path following (crisfield, cylindrical constraint $ du ^2 = ds^2$ on the displacement degrees of freedom). the reference load is the force of the model ($\text{lambda} = 1$); the path ends when lambda reaches lambda_max or after nlstep steps (lambda may become negative on the way: snap-back, inverted snap-through). a step that does not converge is repeated with half the arc length; after a converged step ds is rescaled by $\sqrt{5/\text{iterations}}$.

RUN_NONLINEAR_ANALYSIS

Argument	Declaration	Meaning
MODEL	TYPE(MODEL_TYPE), intent(IN)	Complete input model (all databases).
CACHE	TYPE(MODEL_CACHE_TYPE), intent(IN)	Model cache built by BUILD_MODEL_CACHE.
RESULTS	TYPE(ANALYSIS_RESULTS_TYPE), intent(INOUT)	Analysis results container.
WARNINGS	TYPE(WARNING_LIST_TYPE), intent(INOUT)	List of warnings for REPORT/WARNING_file.dat.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.15.7MUL2_ANALYSIS_RESULTS

File SRC/ANALYSES/mul2_analysis_results.for.

Results shared by the analysis drivers and the post-processing.

Uses: MUL2_KINDS, MUL2_SPARSE_ASSEMBLY, MUL2_BOUNDARY_APPLICATION.

Table C.316. Derived type ANALYSIS_RESULTS_TYPE — System: k (after the static boundary conditions for 101, full and unconstrained for 103) and, for 103, m on the same pattern. solution: displacement coefficients in field-major dof order. mode(:,k): k-th mode, m-orthonormal, in the same order (dofs on constraints are zero). frequency is in hz, eigenvalue in $(\text{rad/s})^{**2}$.

Component	Type	Shape	Default	Meaning
SOLUTION_ID	INTEGER(I4)	''	0_I4	Analysis number (101 or 103).
SYSTEM	TYPE(SPARSE_SYSTEM_TYPE)	''	''	Sparse K and M.
CONSTRAINTS	TYPE(CONSTRAINT_SET_TYPE)	''	''	Constrained DOFs.

Component	Type	Shape	Default	Meaning
FORCE	REAL(R8), ALLOCATABLE	(:)	''	Global force vector.
SOLUTION	REAL(R8), ALLOCATABLE	(:)	''	Displacement coefficients in field-major DOF order.
MODE_COUNT	INTEGER(I4)	''	0_I4	Number of modes (requested / converged).
EIGENVALUE	REAL(R8), ALLOCATABLE	(:)	''	Eigenvalues omega^2.
FREQUENCY	REAL(R8), ALLOCATABLE	(:)	''	Natural frequencies in Hz.
MODE	REAL(R8), ALLOCATABLE	(:, :)	''	M-orthonormal mode shapes (constrained DOFs = 0).
RESIDUAL	REAL(R8), ALLOCATABLE	(:)	''	Relative residual of every mode.
SOLVER_BACKEND	CHARACTER(LEN=8)	''	' '	ARPACK or DENSE.
SOLVER_ITERATIONS	INTEGER(I8)	''	0_I8	Arnoldi update iterations.
SOLVER_OPERATIONS	INTEGER(I8)	''	0_I8	Operator applications.
SOLVER_SUBSPACE	INTEGER(I8)	''	0_I8	Dimension of the Krylov subspace.
ASSEMBLY_SECONDS	REAL(R8)	''	0.0_R8	Wall time of the assembly.
SOLVER_SECONDS	REAL(R8)	''	0.0_R8	Wall time of the solver.

C.15.8MUL2_DRIVER

File SRC/ANALYSES/mul2_driver.for.

Top-level flow of one run: read_model -> preprocess -> run_analysis -> write_results

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_LOG, MUL2_TIMER, MUL2_MODEL, MUL2_MODEL_CACHE, MUL2_READ_MODEL, MUL2_ANALYSIS_INPUT, MUL2_ANALYSIS_RESULTS, MUL2_ANALYSIS_101, MUL2_ANALYSIS_103, MUL2_ANALYSIS_104, MUL2_ANALYSIS_105, MUL2_ANALYSIS_106, MUL2_ANALYSIS_108, MUL2_POST_OUTPUT.

Table C.317. Procedures of MUL2_DRIVER.

Procedure	Kind	Visibility	Purpose
RUN_MUL2	Subroutine	public	Program driver: create the output directories, read the model, build the cache, run 101 or 103, write all outputs and the warning report.
REMEMBER_WARNING	Subroutine	private	WARNINGS OF A STEP GO INTO REPORT/WARNING_file.dat.

RUN_MUL2

Program driver: create the output directories, read the model, build the cache, run 101 or 103, write all outputs and the warning report.

Argument	Declaration	Meaning
INPUT_PATH	CHARACTER(LEN=*), intent(IN)	Input directory.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.15.9MUL2_DYNAMIC_COMMON

File SRC/ANALYSES/mul2_dynamic_common.for.

Pieces shared by the time and frequency analyses (104, 106). a degree of freedom belongs to one of three classes that decide how it enters the dynamic equations: 1 displacement (fields 1-3): inertia and viscous damping 2 temperature (field 4): heat capacity (first order in time) 3 potential (field 5): no inertia (algebraic) the damping matrix lives on the csr pattern of k.

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_SPARSE_ASSEMBLY, MUL2_DOF_LAYOUT.

Table C.319. Public constants of MUL2_DYNAMIC_COMMON.

Name	Type	Value	Meaning
CLASS_DISPLACEMENT	INTEGER(I4)	1_I4	

Name	Type	Value	Meaning
CLASS_TEMPERATURE	INTEGER(I4)	2_I4	
CLASS_POTENTIAL	INTEGER(I4)	3_I4	

Table C.320. Procedures of MUL2_DYNAMIC_COMMON.

Procedure	Kind	Visibility	Purpose
BUILD_DOF_CLASSES	Subroutine	public	Class(dof) for every degree of freedom of the layout.
BUILD_DAMPING_VALUES	Subroutine	public	Damping values on the pattern of the system: displacement rows and columns: rayleigh $c = \alpha m + \beta k$; temperature row, displacement column (only with $t_0 > 0$): the thermoelastic heating $-t_0 k_{ut}^t$ (k_{ut} is the transposed entry).
CSR_POSITION	Function → integer(i8)	public	Position of (row, column) in the csr pattern (binary search in the sorted columns of the row), 0 when the entry is not in the pattern.

BUILD_DOF_CLASSES

Class(dof) for every degree of freedom of the layout.

Argument	Declaration	Meaning
NODE_COUNT	INTEGER(I4), intent(IN)	
LAYOUT	TYPE(DOF_LAYOUT_TYPE), intent(IN)	Gauss-point layout (global point numbering).
ORDER	INTEGER(I8), intent(IN)	Order of the matrix, or polynomial/quadrature order (see routine).
CLASS	INTEGER(I4), ALLOCATABLE, intent(OUT) (:)	
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

BUILD_DAMPING_VALUES

Damping values on the pattern of the system: displacement rows and columns: rayleigh $c = \alpha m + \beta k$; temperature row, displacement column (only with $t_0 > 0$): the thermoelastic heating $-t_0 k_{ut}^t$ (k_{ut} is the transposed entry).

Argument	Declaration	Meaning
SYSTEM	TYPE(SPARSE_SYSTEM_TYPE), intent(IN)	Sparse system (CSR pattern, K, M).
CLASS	INTEGER(I4), intent(IN) (:)	
ALPHA_M	REAL(R8), intent(IN)	
BETA_K	REAL(R8), intent(IN)	
T0	REAL(R8), intent(IN)	
CDAMP	REAL(R8), ALLOCATABLE, intent(OUT) (:)	

CSR_POSITION

Position of (row, column) in the csr pattern (binary search in the sorted columns of the row), 0 when the entry is not in the pattern.

Argument	Declaration	Meaning
SYSTEM	TYPE(SPARSE_SYSTEM_TYPE), intent(IN)	Sparse system (CSR pattern, K, M).
ROW	INTEGER(I8), intent(IN)	
COLUMN	INTEGER(I8), intent(IN)	Output: strain column (6 values).

C.15.10 MUL2_MODEL_ASSEMBLY

File SRC/ANALYSES/mul2_model_assembly.for.

Global k and m of a preprocessed model on one shared csr pattern. 1. the dof list of every element gives the csr pattern (once); 2. elements are evaluated in chunks, in parallel (openmp, elements are independent and write only their own matrices); 3. every chunk is scattered into the csr arrays serially and in element order, so the result does not depend on the number of threads (deterministic round-off). the memory of the element matrices is bounded by the chunk size.

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_MITC, MUL2_MODEL, MUL2_MODEL_CACHE, MUL2_ANALYSIS_INPUT, MUL2_TOPOLOGIES, MUL2_GAUSS_MATERIALS, MUL2_ELEMENT_MATRICES, MUL2_GENERAL_KERNEL, MUL2_GENERAL_GEOMETRY, MUL2_SPARSE_ASSEMBLY, MUL2_KINEMATICS, MUL2_NODES, MUL2_EXPANSION_MESHES.

Table C.324. Procedures of MUL2_MODEL_ASSEMBLY.

Procedure	Kind	Visibility	Purpose
ASSEMBLE_MODEL_SYSTEM	Subroutine	public	Assemble global K (and M) in CSR: DOF lists, pattern, parallel element evaluation in chunks, serial deterministic scatter.
ASSEMBLE_NONLINEAR_VALUES	Subroutine	public	Tangent matrix and internal force of the nonlinear (total lagrangian) model at the state u0. tangent(:) is aligned with system%column_index. elements are evaluated in parallel in chunks and scattered in element order (the result does not depend on the number of threads).
NONLINEAR_ELEMENT	Subroutine	private	
ASSEMBLE_GEOMETRIC_VALUES	Subroutine	public	Geometric (initial-stress) matrix of the state u0 on the pattern of the system: values(:) is aligned with system%column_index.
GEOMETRIC_ELEMENT	Subroutine	private	Geometric matrix of one element (thread-safe: only reads shared data).
MARK_LAGRANGE_DOFs	Subroutine	private	Lagrange-expansion dofs of an element: table = expansion mesh and term = mesh node. (taylor dofs keep table = 0: fully coupled.)
BUILD_COUPLING_TABLES	Subroutine	private	Coupled(t,s) = terms t and s (nodes of the mesh) share a sub-element.
EVALUATE_ELEMENT	Subroutine	private	K and m of one element; with coupled the thermoelastic block k(u,t) (full integration) is added when the element has a temperature field.
EVALUATE_ELEMENT_BASE	Subroutine	private	K and m of one element (thread-safe: only reads the shared data).
GENERAL_ELEMENT	Subroutine	private	K and m (or the thermoelastic block) of a curved beam / shell. none: compatible strains; mitc: tied strains; redi / seli are not defined for the general geometry.

ASSEMBLE_MODEL_SYSTEM

Assemble global K (and M) in CSR: DOF lists, pattern, parallel element evaluation in chunks, serial deterministic scatter.

Argument	Declaration	Meaning
MODEL	TYPE(MODEL_TYPE), intent(IN)	Complete input model (all databases).
CACHE	TYPE(MODEL_CACHE_TYPE), intent(IN)	Model cache built by BUILD_MODEL_CACHE.
SYSTEM	TYPE(SPARSE_SYSTEM_TYPE), intent(INOUT)	Output: K and optionally M in CSR.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).
WITH_MASS	LOGICAL, intent(IN), OPTIONAL	Also build the mass matrix.
THERMOELASTIC	LOGICAL, intent(IN), OPTIONAL	Thermoelastic: add the non-symmetric coupling block k(u,t).

ASSEMBLE_NONLINEAR_VALUES

Tangent matrix and internal force of the nonlinear (total lagrangian) model at the state u0. tangent(:) is aligned with system%column_index. elements are evaluated in parallel in chunks and scattered in element order (the result does not depend on the number of threads).

Argument	Declaration	Meaning
MODEL	TYPE(MODEL_TYPE), intent(IN)	Complete input model (all databases).
CACHE	TYPE(MODEL_CACHE_TYPE), intent(IN)	Model cache built by BUILD_MODEL_CACHE.
U0	REAL(R8), intent(IN)(:)	
SYSTEM	TYPE(SPARSE_SYSTEM_TYPE), intent(IN)	Sparse system (CSR pattern, K, M).
TANGENT	REAL(R8), ALLOCATABLE, intent(INOUT)(:)	
INTERNAL	REAL(R8), intent(OUT)(:)	
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

ASSEMBLE_GEOMETRIC_VALUES

Geometric (initial-stress) matrix of the state u0 on the pattern of the system: values(:) is aligned with system%column_index.

Argument	Declaration	Meaning
MODEL	TYPE(MODEL_TYPE), intent(IN)	Complete input model (all databases).
CACHE	TYPE(MODEL_CACHE_TYPE), intent(IN)	Model cache built by BUILD_MODEL_CACHE.
U0	REAL(R8), intent(IN)(:)	
SYSTEM	TYPE(SPARSE_SYSTEM_TYPE), intent(IN)	Sparse system (CSR pattern, K, M).
VALUES	REAL(R8), ALLOCATABLE, intent(OUT)(:)	
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.15.11 MUL2_MODEL_CACHE

File SRC/ANALYSES/mul2_model_cache.for.

Preprocessing of a validated model: frames, dofs, gauss databases and material cache. nothing here depends on the analysis type.

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_MODEL, MUL2_ANALYSIS_INPUT, MUL2_NODES, MUL2_TOPOLOGIES, MUL2_MITC, MUL2_KINEMATICS, MUL2_EXPANSION_MESHES, MUL2_REFERENCE_SYSTEMS, MUL2_ELEMENT_FRAMES, MUL2_GENERAL_GEOMETRY, MUL2_DOF_LAYOUT, MUL2_COINCIDENT_JOIN, MUL2_GAUSS_POINTS, MUL2_GAUSS_GEOMETRY, MUL2_GAUSS_MATERIALS, MUL2_QUADRATURE, MUL2_LAMINATIONS.

Table C.328. Derived type REDUCED_SET_TYPE — Second integration set of the reduced and selective integration: the structural points use the reduced rule (the expansion points are the same as in the full set).

Component	Type	Shape	Default	Meaning
GAUSS_LAYOUT	TYPE(GAUSS_LAYOUT_TYPE)	''	''	Gauss layout.
STRUCTURAL_GEOMETRY	TYPE(STRUCTURAL_GEOMETRY_CACHE_TYPE)	''	''	Structural geometry cache.
GEOMETRY	TYPE(GAUSS_GEOMETRY_TYPE)	''	''	Combined geometry.
MATERIAL_MAP	TYPE(GAUSS_MATERIAL_MAP_TYPE)	''	''	Gauss point to material map.

Table C.329. Derived type MODEL_CACHE_TYPE — Everything derived from the model that does not depend on the analysis: frames, DOF layout, rules, Gauss layout and geometry, material cache.

Component	Type	Shape	Default	Meaning
FRAMES	TYPE(ELEMENT_FRAME_DB_TYPE)	''	''	Element frames.
DOF_LAYOUT	TYPE(DOF_LAYOUT_TYPE)	''	''	DOF numbering.

Component	Type	Shape	Default	Meaning
RULES	TYPE(REFERENCE_RULE_DB_TYPE)	``	``	Reference rules.
GAUSS_LAYOUT	TYPE(GAUSS_LAYOUT_TYPE)	``	``	Gauss layout.
STRUCTURAL_GEOMETRY	TYPE(STRUCTURAL_GEOMETRY_CACHE_TYPE)	``	``	Structural geometry cache.
EXPANSION_GEOMETRY	TYPE(EXPANSION_GEOMETRY_CACHE_TYPE)	``	``	Expansion geometry cache.
GEOMETRY	TYPE(GAUSS_GEOMETRY_TYPE)	``	``	Combined geometry.
MATERIAL_CACHE	TYPE(MATERIAL_CACHE_TYPE)	``	``	Material cache.
EXPANSION_ORDER	INTEGER(I4), ALLOCATABLE	(:)	``	Gauss points per direction of each expansion mesh.
MATERIAL_MAP	TYPE(GAUSS_MATERIAL_MAP_TYPE)	``	``	Gauss point to material map.
REDUCED_DIMENSION	LOGICAL	(3)	. FALSE .	Reduced_dimension(d): elements of dimension d use the reduced set (redi or seli with an available reduced rule).
HAS_REDUCED	LOGICAL	``	. FALSE .	
REDUCED	TYPE(REDUCED_SET_TYPE)	``	``	

Table C.330. Procedures of MUL2_MODEL_CACHE .

Procedure	Kind	Visibility	Purpose
CHECK_MECHANICAL_MODEL	Subroutine	public	V3 101/103 are purely mechanical: only u, v and w may be active.
BUILD_MODEL_CACHE	Subroutine	public	Preprocessing pipeline: DOF layout, element frames, Gauss rules and layout, structural/expansion/combined geometry and material cache.
CHECK_PHYSICS_DATA	Subroutine	private	Every lamination of an element whose nodes solve the temperature (the electric potential) needs its conductivity (permittivity): without them the matrix is singular and the solver only reports not-a-number.
FIND_REDUCED_DIMENSIONS	Subroutine	private	Dimensions (1 beam, 2 plate, 3 solid) whose elements use the reduced structural rule. a dimension requesting redi/seli with no element that has a reduced rule (triangles) stays fully integrated: warning.
ELEMENT_SHEAR_MODE	Function → integer(i4)	public	Shear treatment requested for the dimension of an element.
BUILD_EXPANSION_POINT_ORDERS	Subroutine	private	Gauss points per direction needed by each expansion mesh. a taylor expansion of order n gives integrands of degree 2n: n+1 points are required (the baseline uses the same count). Ie nodes need nothing beyond the default rule of the expansion element.
PREPARE_ELEMENT_MITC	Subroutine	public	Mitc tying data of one element when the analysis requests mitc for its dimension. a topology without a mitc table is integrated fully and a warning is raised (the baseline silently produces no matrix).

CHECK_MECHANICAL_MODEL

V3 101/103 are purely mechanical: only u, v and w may be active.

Argument	Declaration	Meaning
MODEL	TYPE(MODEL_TYPE), intent(IN)	Complete input model (all databases).
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

BUILD_MODEL_CACHE

Preprocessing pipeline: DOF layout, element frames, Gauss rules and layout, structural/expansion/combined geometry and material cache.

Argument	Declaration	Meaning
MODEL	TYPE(MODEL_TYPE), intent(IN)	Complete input model (all databases).
CACHE	TYPE(MODEL_CACHE_TYPE), intent(INOUT)	Model cache built by BUILD_MODEL_CACHE.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

ELEMENT_SHEAR_MODE

Shear treatment requested for the dimension of an element.

Argument	Declaration	Meaning
MODEL	TYPE(MODEL_TYPE), intent(IN)	Complete input model (all databases).
ELEMENT	INTEGER(I4), intent(IN)	Element index.

PREPARE_ELEMENT_MITC

Mitc tying data of one element when the analysis requests mitc for its dimension. a topology without a mitc table is integrated fully and a warning is raised (the baseline silently produces no matrix).

Argument	Declaration	Meaning
MODEL	TYPE(MODEL_TYPE), intent(IN)	Complete input model (all databases).
CACHE	TYPE(MODEL_CACHE_TYPE), intent(IN)	Model cache built by BUILD_MODEL_CACHE.
ELEMENT	INTEGER(I4), intent(IN)	Element index.
MITC	TYPE(MITC_DATA_TYPE), intent(INOUT)	MITC tying data of the element.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.16 Layer POST

Point location and recovery, output grids, output files.

C.16.1MUL2_POST_GRID

File SRC/POST/mul2_post_grid.for.

Visualization grid: hexahedral cells sampling every element. each element / expansion sub-element is divided into $r \times s \times l$ cells. a cell has $3 \times 3 \times 3$ (quadratic) or $2 \times 2 \times 2$ (linear) nodes at natural coordinates (xi, eta, nu) in $[-1,1]$, ordered with xi outermost and nu innermost. the natural triple is split between the structural element and the expansion element by element dimension: beam (1d): structural = (eta) expansion = (xi, nu) plate (2d): structural = (xi, eta) expansion = (nu) solid (3d): structural = (xi, eta, nu) triangular parents are reached with the collapsed map $a = (1+x)/2$, $b = (1+y)/2$ (1-a).

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_MODEL, MUL2_EXPANSION_MESHES, MUL2_TOPOLOGIES.

Table C.335. Derived type POST_GRID_TYPE — Output grid: cells with their element, sub-element and the natural coordinates of every grid node.

Component	Type	Shape	Default	Meaning
NODES_PER_CELL	INTEGER(I4)	``	27_I4	Nodes per cell.
CELL_COUNT	INTEGER(I4)	``	0_I4	Number of cells.
NODE_COUNT	INTEGER(I4)	``	0_I4	Number of nodes.
CELL_ELEMENT	INTEGER(I4), ALLOCATABLE	(:)	``	Element of each cell.
CELL_SUB_ELEMENT	INTEGER(I4), ALLOCATABLE	(:)	``	Sub-element of each cell.
CELL_DIMENSION	INTEGER(I4), ALLOCATABLE	(:)	``	Dimension of each cell.
NATURAL_STRUCTURAL	REAL(R8), ALLOCATABLE	(:, :)	``	Structural natural coordinates of grid nodes.
NATURAL_EXPANSION	REAL(R8), ALLOCATABLE	(:, :)	``	Expansion natural coordinates of grid nodes.

Table C.336. Procedures of MUL2_POST_GRID.

Procedure	Kind	Visibility	Purpose
BUILD_POST_GRID	Subroutine	public	Split(1:9) = beam (xi, eta, nu), plate (xi, eta, nu), solid (xi, eta, nu) subdivisions (the postprocessing.dat layout).
ELEMENT_SPLITS	Subroutine	private	Subdivisions (x, y, z) of the cells of an element of given dimension.
CELL_LIMITS	Subroutine	private	Natural coordinates of the nodes of cell number i of n along one direction: points = 3 (end, middle, end) or 2 (ends).
SPLIT_NATURAL	Subroutine	private	Split a 3-component natural point into structural and expansion natural coordinates.
COLLAPSE_TRIANGLE	Subroutine	private	Map a square point to a triangle point (collapsed map).
VTK_CELL_ORDER	Subroutine	public	Node order of one cell in the output formats (1-based positions in the 3x3x3 or 2x2x2 node block).
GMSH_CELL_ORDER	Subroutine	public	Node order of a cell in GMSH output.
CLEAR_POST_GRID	Subroutine	public	Release a post grid.

BUILD_POST_GRID

Split(1:9) = beam (xi, eta, nu), plate (xi, eta, nu), solid (xi, eta, nu) subdivisions (the postprocessing.dat layout).

Argument	Declaration	Meaning
MODEL	TYPE(MODEL_TYPE), intent(IN)	Complete input model (all databases).
NODES_PER_CELL	INTEGER(I4), intent(IN)	Nodes per output cell: 8, 20 or 27.
SPLIT	INTEGER(I4), intent(IN)(9)	Nine subdivision counts of the record.
GRID	TYPE(POST_GRID_TYPE), intent(INOUT)	Output grid.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

VTK_CELL_ORDER

Node order of one cell in the output formats (1-based positions in the 3x3x3 or 2x2x2 node block).

Argument	Declaration	Meaning
NODES_PER_CELL	INTEGER(I4), intent(IN)	Nodes per output cell: 8, 20 or 27.
ORDER	INTEGER(I4), intent(OUT)(20)	Order of the matrix, or polynomial/quadrature order (see routine).
COUNT	INTEGER(I4), intent(OUT)	Number of items.

GMSH_CELL_ORDER

Node order of a cell in GMSH output.

Argument	Declaration	Meaning
NODES_PER_CELL	INTEGER(I4), intent(IN)	Nodes per output cell: 8, 20 or 27.
ORDER	INTEGER(I4), intent(OUT)(27)	Order of the matrix, or polynomial/quadrature order (see routine).
COUNT	INTEGER(I4), intent(OUT)	Number of items.

CLEAR_POST_GRID

Release a post grid.

Argument	Declaration	Meaning
GRID	TYPE(POST_GRID_TYPE), intent(INOUT)	Output grid.

C.16.2MUL2_POST_OUTPUT

File SRC/POST/mul2_post_output.for.

SCIENTIFIC OUTPUT FILES OF THE STATIC AND MODAL ANALYSES. STATIC/POST_POINT.dat POINT RESULTS (PNT RECORDS) STATIC/RESULTS_PARA_NN.vtk PARAVIEW FIELDS (PARA RECORDS) STATIC/RESULTS_GMSH_NN.msh GMSH FIELDS (GMSH RECORDS) DYNAMIC/FREQUENCIES.dat NATURAL FREQUENCIES [HZ] DYNAMIC/RESULTS_DYN_PARA.vtk MODE SHAPES (PARAVIEW) DYNAMIC/RESULTS_DYN_GMSH.msh MODE SHAPES (GMSH) WORK/K_MAT.dat M_MAT.dat FORCES.dat UNKNOWN.dat MATRIX DUMPS FILE NAMES AND COLUMN LAYOUTS FOLLOW THE BASELINE. THE DOF NUMBERING OF THE DUMPS IS THE V3 FIELD-MAJOR ONE.

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_LOG, MUL2_FILES, MUL2_MODEL, MUL2_MODEL_CACHE, MUL2_ANALYSIS_INPUT, MUL2_ANALYSIS_RESULTS, MUL2_SPARSE_ASSEMBLY, MUL2_POST_GRID, MUL2_RECOVERY.

Table C.341. Derived type WARNING_LIST_TYPE — Warnings collected for REPORT/WARNING_file.dat.

Component	Type	Shape	Default	Meaning
COUNT	INTEGER(I4)	``	0_I4	Number of items.
TEXT	CHARACTER(LEN=200)	(64)	' '	Warning texts.

Table C.342. Procedures of MUL2_POST_OUTPUT.

Procedure	Kind	Visibility	Purpose
PREPARE_OUTPUT_DIRECTORIES	Subroutine	public	Create STATIC, DYNAMIC, WORK and REPORT.
WRITE_STATIC_OUTPUT	Subroutine	public	Static analysis output
EVALUATE_GRID_STATES	Subroutine	private	Values of displacement, strain and stress at every grid node. cells are independent and evaluated in parallel.
EVALUATE_CELL_STATES	Subroutine	private	Same for one cell.
LOCATE_ALL_POINTS	Subroutine	private	Locate every point request once and keep the result (see located).
WRITE_TIME_STEP_OUTPUT	Subroutine	public	OUTPUT OF ONE STEP OF A TIME ANALYSIS: DYNAMIC/TIME_HISTORY.dat GETS ONE ROW PER POINT (THE POST_POINT COLUMNS AFTER THE TIME) AND EVERY FIELD REQUEST WRITES DYNAMIC/RESULTS_PARA_NN_SSSSS.vtk.
WRITE_FREQUENCY_OUTPUT	Subroutine	public	OUTPUT OF ONE FREQUENCY OF A HARMONIC RESPONSE: DYNAMIC/ FREQ_HISTORY.dat HAS ONE ROW PER POINT AND FREQUENCY: F[HZ] ID RE/IM OF UX UY UZ TEMPERATURE VOLTAGE
WRITE_POST_POINTS	Subroutine	private	Point results
WRITE_POINT_HEADER	Subroutine	private	Header line of POST_POINT.dat.
WRITE_POINT_ROW	Subroutine	private	One result row (46 columns).
WRITE_MISSING_POINT_ROW	Subroutine	private	Row of NaN for a point outside the model.
FIND_MESH	Function → integer(i4)	private	Expansion mesh index of an element.
WRITE_VTK_GEOMETRY	Subroutine	private	Paraview / gmsch writers (shared by static and modal output)
WRITE_VTK_VECTOR	Subroutine	private	Write a VTK vector field.
WRITE_VTK_SCALAR	Subroutine	private	Write a VTK scalar field.
SELECT_FRAME	Subroutine	private	Nodal data in the frame requested by the post-processing record.
VON_MISES	Function → real(r8)	private	Von Mises equivalent stress.
WRITE_STATIC_VTK	Subroutine	private	RESULTS_PARA_nn.vtk.
WRITE_GMSH_GEOMETRY	Subroutine	private	Write nodes and elements of a GMSH file.
WRITE_GMSH_DATA	Subroutine	private	Write one GMSH view.
WRITE_STATIC_GMSH	Subroutine	private	RESULTS_GMSH_nn.msh.

Procedure	Kind	Visibility	Purpose
WRITE_MODAL_OUTPUT	Subroutine	public	Modal analysis output
WRITE_SOLVER_INFO	Subroutine	private	WORK/ARPACK_SOLVER_INFO.dat: EIGENVALUES, RESIDUALS, ITERATIONS.
EVALUATE_GRID_MODES	Subroutine	private	Mode displacements at the grid nodes: displacement(3,node,mode).
EVALUATE_CELL_MODES	Subroutine	private	Same for one cell.
WRITE_MODAL_VTK	Subroutine	private	RESULTS_DYN_PARA.vtk.
WRITE_MODAL_GMSH	Subroutine	private	RESULTS_DYN_GMSH.msh.
WRITE_MATRIX_DUMPS	Subroutine	private	Matrix and vector dumps (work directory)
WRITE_CSR	Subroutine	private	Dump a CSR matrix as row, column, value lines.
WRITE_VECTOR	Subroutine	private	Dump a vector.
ADD_WARNING	Subroutine	private	Report files
WRITE_WARNING_FILE	Subroutine	public	REPORT/WARNING_file.dat IN THE BASELINE LAYOUT.
INTEGER_TEXT	Function → character(len=16)	private	Integer to text without blanks.

PREPARE_OUTPUT_DIRECTORIES

Create STATIC, DYNAMIC, WORK and REPORT.

Argument	Declaration	Meaning
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

WRITE_STATIC_OUTPUT

Static analysis output

Argument	Declaration	Meaning
MODEL	TYPE(MODEL_TYPE), intent(IN)	Complete input model (all databases).
CACHE	TYPE(MODEL_CACHE_TYPE), intent(IN)	Model cache built by BUILD_MODEL_CACHE.
RESULTS	TYPE(ANALYSIS_RESULTS_TYPE), intent(IN)	Analysis results container.
WARNINGS	TYPE(WARNING_LIST_TYPE), intent(INOUT)	List of warnings for REPORT/WARNING_file.dat.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

WRITE_TIME_STEP_OUTPUT

OUTPUT OF ONE STEP OF A TIME ANALYSIS: DYNAMIC/TIME_HISTORY.dat GETS ONE ROW PER POINT (THE POST_POINT COLUMNS AFTER THE TIME) AND EVERY FIELD REQUEST WRITES DYNAMIC/RESULTS_PARA_NN_SSSSS.vtk.

Argument	Declaration	Meaning
MODEL	TYPE(MODEL_TYPE), intent(IN)	Complete input model (all databases).
CACHE	TYPE(MODEL_CACHE_TYPE), intent(IN)	Model cache built by BUILD_MODEL_CACHE.
SOLUTION	REAL(R8), intent(IN)(:)	Global displacement vector.
STEP	INTEGER(I4), intent(IN)	
TIME_VALUE	REAL(R8), intent(IN)	
WARNINGS	TYPE(WARNING_LIST_TYPE), intent(INOUT)	List of warnings for REPORT/WARNING_file.dat.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

WRITE_FREQUENCY_OUTPUT

OUTPUT OF ONE FREQUENCY OF A HARMONIC RESPONSE: DYNAMIC/ FREQ_HISTORY.dat HAS ONE ROW PER POINT AND FREQUENCY: F[HZ] ID RE/IM OF UX UY UZ TEMPERATURE VOLTAGE

Argument	Declaration	Meaning
MODEL	TYPE(MODEL_TYPE), intent(IN)	Complete input model (all databases).
CACHE	TYPE(MODEL_CACHE_TYPE), intent(IN)	Model cache built by BUILD_MODEL_CACHE.
REAL_PART	REAL(R8), intent(IN)(:)	
IMAGINARY_PART	REAL(R8), intent(IN)(:)	
STEP	INTEGER(I4), intent(IN)	
FREQUENCY	REAL(R8), intent(IN)	Natural frequencies in Hz.
WARNINGS	TYPE(WARNING_LIST_TYPE), intent(INOUT)	List of warnings for REPORT/WARNING_file.dat.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

WRITE_MODAL_OUTPUT

Modal analysis output

Argument	Declaration	Meaning
MODEL	TYPE(MODEL_TYPE), intent(IN)	Complete input model (all databases).
CACHE	TYPE(MODEL_CACHE_TYPE), intent(IN)	Model cache built by BUILD_MODEL_CACHE.
RESULTS	TYPE(ANALYSIS_RESULTS_TYPE), intent(IN)	Analysis results container.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

WRITE_WARNING_FILE

REPORT/WARNING_file.dat IN THE BASELINE LAYOUT.

Argument	Declaration	Meaning
WARNINGS	TYPE(WARNING_LIST_TYPE), intent(IN)	List of warnings for REPORT/WARNING_file.dat.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.16.3MUL2_RECOVERY

File SRC/POST/mul2_recovery.for.

Recovery of displacement, strain and stress at arbitrary points. a point is given by an element, one sub-element of its expansion mesh and the natural coordinates of both (structural, expansion). the field is composed with the same basis rule used by the element matrices (mul2_point_bases), so recovery and stiffness share one implementation of the kinematics. conventions strain = [exx, eyy, ezz, gxz, gyx, gxy] (engineering shear) stress = [sxx, syy, szz, sxz, syz, sxy] local = element frame, global = model frame

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_MODEL, MUL2_MODEL_CACHE, MUL2_ANALYSIS_INPUT, MUL2_GENERAL_GEOMETRY, MUL2_GENERAL_KERNEL, MUL2_MITC, MUL2_LOCAL_GRADIENTS, MUL2_LINEAR_KINEMATICS, MUL2_NODES, MUL2_KINEMATICS, MUL2_EXPANSION_MESHES, MUL2_LAMINATIONS, MUL2_TOPOLOGIES, MUL2_SHAPE_FUNCTIONS, MUL2_HLE_MAP, MUL2_JACOBIANS, MUL2_DOF_LAYOUT, MUL2_POINT_BASES, MUL2_MATERIAL_ROTATIONS.

Table C.349. Derived type PLACE_TYPE — Geometry and shape data of one (element, sub-element) pair.

Component	Type	Shape	Default	Meaning
ELEMENT	INTEGER(I4)	''	0_I4	Element index.
MESH	INTEGER(I4)	''	0_I4	Expansion mesh index.
SUB_ELEMENT	INTEGER(I4)	''	0_I4	Sub-element index.

Component	Type	Shape	Default	Meaning
STRUCTURAL_DIMENSION	INTEGER(I4)	``	0_I4	Structural dimension.
EXPANSION_DIMENSION	INTEGER(I4)	``	0_I4	Expansion dimension.
STRUCTURAL_TOPOLOGY	INTEGER(I4)	``	0_I4	Structural topology.
EXPANSION_TOPOLOGY	INTEGER(I4)	``	0_I4	Expansion topology.
STRUCTURAL_AXIS	INTEGER(I4)	(3)	[1_I4,2_I4,3_I4]	Structural axes.
EXPANSION_AXIS	INTEGER(I4)	(3)	[1_I4,2_I4,3_I4]	Expansion axes.
NODE_GLOBAL	REAL(R8), ALLOCATABLE	(:,:)``	``	Global node coordinates.
NODE_LOCAL	REAL(R8), ALLOCATABLE	(:,:)``	``	Local node coordinates.
EXPANSION_NODE	REAL(R8), ALLOCATABLE	(:,:)``	``	Expansion node coordinates.
SHAPE_STRUCTURAL	REAL(R8), ALLOCATABLE	(:)	``	Structural shapes.
NATURAL_DERIVATIVE_S	REAL(R8), ALLOCATABLE	(:,:)``	``	Structural natural derivatives.
GRADIENT_STRUCTURAL	REAL(R8), ALLOCATABLE	(:,:)``	``	Structural local gradients.
SHAPE_EXPANSION	REAL(R8), ALLOCATABLE	(:)	``	Expansion shapes.
NATURAL_DERIVATIVE_E	REAL(R8), ALLOCATABLE	(:,:)``	``	Expansion natural derivatives.
GRADIENT_EXPANSION	REAL(R8), ALLOCATABLE	(:,:)``	``	Expansion local gradients.
EXPANSION_POINT_LOCAL	REAL(R8)	(3)	0.0_R8	Expansion point in the local frame.
CURVED	LOGICAL	``	.FALSE.	Hq sub-element with curved sides: blending-function map.
HLE_MAP	TYPE(HLE_GEOMETRY_TYPE)	``	``	
MAP_DX	REAL(R8)	(2,2)	0.0_R8	
POINT_GLOBAL	REAL(R8)	(3)	0.0_R8	Physical point.
NATURAL_S	REAL(R8)	(3)	0.0_R8	Structural natural coordinates.
MITC	TYPE(MITC_DATA_TYPE)	``	``	MITC tying data.
GENERAL	LOGICAL	``	.FALSE.	Curved beam / shell (mul2_general_kernel): geometry and tying.
GCTX	TYPE(GENERAL_CONTEXT_TYPE)	``	``	

Table C.350. Derived type POINT_STATE_TYPE — Displacement, strain and stress at one point.

Component	Type	Shape	Default	Meaning
ELEMENT	INTEGER(I4)	``	0_I4	Element index.
SUB_ELEMENT	INTEGER(I4)	``	0_I4	Sub-element index.
ELEMENT_DIMENSION	INTEGER(I4)	``	0_I4	Element dimension.
LAMINATION_ID	INTEGER(I4)	``	0_I4	Lamination id.
MATERIAL_ID	INTEGER(I4)	``	0_I4	Material id.
COORDINATE	REAL(R8)	(3)	0.0_R8	Coordinates.
DISPLACEMENT	REAL(R8)	(3)	0.0_R8	Displacement (global).
STRAIN_LOCAL	REAL(R8)	(6)	0.0_R8	Strain in the element frame (xx,yy,zz,xz,yz,xy).

Component	Type	Shape	Default	Meaning
STRAIN_GLOBAL	REAL(R8)	(6)	0.0_R8	Strain in the global frame.
STRESS_LOCAL	REAL(R8)	(6)	0.0_R8	Stress in the element frame.
STRESS_GLOBAL	REAL(R8)	(6)	0.0_R8	Stress in the global frame.
POTENTIAL	REAL(R8)	``	0.0_R8	Electric fields (piezoelectric models): potential, field $e = -\text{grad } \phi$ and displacement $d = e_{pz} s - \epsilon s - \text{grad } \phi$.
ELECTRIC_FIELD_LOCAL	REAL(R8)	(3)	0.0_R8	
ELECTRIC_FIELD_GLOBAL	REAL(R8)	(3)	0.0_R8	
ELECTRIC_DISPLACEMENT_LOCAL	REAL(R8)	(3)	0.0_R8	
ELECTRIC_DISPLACEMENT_GLOBAL	REAL(R8)	(3)	0.0_R8	
TEMPERATURE	REAL(R8)	``	0.0_R8	Thermal fields: temperature and heat flux $q = -k \text{ grad } t$.
HEAT_FLUX_LOCAL	REAL(R8)	(3)	0.0_R8	
HEAT_FLUX_GLOBAL	REAL(R8)	(3)	0.0_R8	
FRAME	REAL(R8)	(3,3)	RESHAPE([1.0_R8,0.0_R8,0.0_R8,0.0_R8,1.0_R8,0.0_R8,0.0_R8,0.0_R8,0.0_R8,1.0_R8],[3,3])	Frame of the point (global_to_local): the one of the element, or the one of the point for a curved beam / shell.

Table C.351. Procedures of MUL2_RECOVERY .

Procedure	Kind	Visibility	Purpose
PLACE_SETUP	Subroutine	public	Load the coordinates of the nodes of one element and of one expansion sub-element.
SETUP_CURVED_PLACE	Subroutine	private	Geometry of a hq sub-element with curved sides.
FREE_PLACE	Subroutine	private	Release a PLACE_TYPE.
PLACE_EVALUATE	Subroutine	public	Shapes, local gradients and physical position at a natural point. with_gradient = .false. skips the jacobian inversions (newton).
CURVED_GRADIENT	Subroutine	private	Physical gradients of the expansion functions with the blending map.
STATE_FROM_PLACE	Subroutine	public	Displacement, strain and stress at the point last evaluated by place_evaluate(..., with_gradient = .true.).
GENERAL_STATE_PART	Subroutine	private	Displacement, potential, temperature and the strain / gradients of the local frame at a point of a curved beam / shell.
DISPLACEMENT_FROM_PLACE	Subroutine	public	Global displacement of several dof vectors (e.g. mode shapes) at the point last evaluated by place_evaluate. no gradient is needed. vectors(dof,k) -> displacement(1:3,k)
TRANSFORM_INVERSE	Function	private	Inverse of the strain transform of an orthogonal rotation.

Procedure	Kind	Visibility	Purpose
LOCATE_POINT	Subroutine	public	Find the (element, sub-element, natural coordinates) containing the target point. newton iteration on $x_s(x_i_s) + r^{**}t x_e(x_i_e) = \text{target}$ whose unknowns are all the natural coordinates of the point.
POINT_NEAR_ELEMENT	Function → logical	private	Cheap rejection: the point lies within the bounding box of the structural nodes expanded by the largest expansion extent.
NEWTON_LOCATE	Subroutine	private	Newton iteration on the isoparametric map to find the natural coordinates.
NATURAL_INSIDE	Function → logical	private	True when natural coordinates lie inside the reference element.
INVERT_3X3	Subroutine	private	Invert a 3x3 matrix with its determinant.

PLACE_SETUP

Load the coordinates of the nodes of one element and of one expansion sub-element.

Argument	Declaration	Meaning
MODEL	TYPE(MODEL_TYPE), intent(IN)	Complete input model (all databases).
CACHE	TYPE(MODEL_CACHE_TYPE), intent(IN)	Model cache built by BUILD_MODEL_CACHE.
ELEMENT	INTEGER(I4), intent(IN)	Element index.
SUB_ELEMENT	INTEGER(I4), intent(IN)	Index of the expansion sub-element.
PLACE	TYPE(PLACE_TYPE), intent(INOUT)	Evaluation context: node coordinates, shapes and frames of one element/sub-element.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

PLACE_EVALUATE

Shapes, local gradients and physical position at a natural point. with_gradient = .false. skips the jacobian inversions (newton).

Argument	Declaration	Meaning
PLACE	TYPE(PLACE_TYPE), intent(INOUT)	Evaluation context: node coordinates, shapes and frames of one element/sub-element.
MODEL	TYPE(MODEL_TYPE), intent(IN)	Complete input model (all databases).
CACHE	TYPE(MODEL_CACHE_TYPE), intent(IN)	Model cache built by BUILD_MODEL_CACHE.
NATURAL_S	REAL(R8), intent(IN)(3)	Structural natural coordinates.
NATURAL_E	REAL(R8), intent(IN)(3)	Expansion natural coordinates.
WITH_GRADIENT	LOGICAL, intent(IN)	Also compute local gradients.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

STATE_FROM_PLACE

Displacement, strain and stress at the point last evaluated by place_evaluate(..., with_gradient = .true.).

Argument	Declaration	Meaning
MODEL	TYPE(MODEL_TYPE), intent(IN)	Complete input model (all databases).
CACHE	TYPE(MODEL_CACHE_TYPE), intent(IN)	Model cache built by BUILD_MODEL_CACHE.
PLACE	TYPE(PLACE_TYPE), intent(IN)	Evaluation context: node coordinates, shapes and frames of one element/sub-element.
SOLUTION	REAL(R8), intent(IN)(:)	Global displacement vector.

Argument	Declaration	Meaning
STATE	TYPE(PPOINT_STATE_TYPE), intent(OUT)	Displacement, strain, stress at output points.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

DISPLACEMENT_FROM_PLACE

Global displacement of several dof vectors (e.g. mode shapes) at the point last evaluated by place_evaluate. no gradient is needed. vectors(dof,k) -> displacement(1:3,k)

Argument	Declaration	Meaning
MODEL	TYPE(MODEL_TYPE), intent(IN)	Complete input model (all databases).
CACHE	TYPE(MODEL_CACHE_TYPE), intent(IN)	Model cache built by BUILD_MODEL_CACHE.
PLACE	TYPE(PLACE_TYPE), intent(IN)	Evaluation context: node coordinates, shapes and frames of one element/sub-element.
VECTORS	REAL(R8), intent(IN)(:,:))	DOF vectors (e.g. mode shapes), one per column.
DISPLACEMENT	REAL(R8), intent(OUT)(:,:))	Output: global displacement components (one column per vector).
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

LOCATE_POINT

Find the (element, sub-element, natural coordinates) containing the target point. newton iteration on $x_s(x_i_s) + r^{**}t$ $x_e(x_i_e) = \text{target}$ whose unknowns are all the natural coordinates of the point.

Argument	Declaration	Meaning
MODEL	TYPE(MODEL_TYPE), intent(IN)	Complete input model (all databases).
CACHE	TYPE(MODEL_CACHE_TYPE), intent(IN)	Model cache built by BUILD_MODEL_CACHE.
TARGET	REAL(R8), intent(IN)(3)	Physical point to locate.
TOLERANCE	REAL(R8), intent(IN)	Geometric tolerance.
ELEMENT	INTEGER(I4), intent(OUT)	Element index.
SUB_ELEMENT	INTEGER(I4), intent(OUT)	Index of the expansion sub-element.
NATURAL_S	REAL(R8), intent(OUT)(3)	Structural natural coordinates.
NATURAL_E	REAL(R8), intent(OUT)(3)	Expansion natural coordinates.
FOUND	LOGICAL, intent(OUT)	True when the point lies inside an element.
STATUS	TYPE(STATUS_TYPE), intent(OUT)	Status of the call (OK, warning or error with message and source).

C.17 Layer APP

Program entry points.

C.17.1None

File SRC/APP/mul2_main.for.

MUL2_V3 EXECUTABLE ENTRY POINT. MUL2_V3 [INPUT_DIRECTORY] (DEFAULT: FIRST LINE OF PATH_input.dat, THEN INPUT) MUL2_V3 --version RESULTS ARE WRITTEN TO THE REPORT, STATIC, DYNAMIC AND WORK DIRECTORIES OF THE WORKING DIRECTORY. THE EXIT CODE IS 0 ON SUCCESS.

Uses: MUL2_KINDS, MUL2_STATUS, MUL2_LOG, MUL2_RUNTIME, MUL2_TIMER, MUL2_DRIVER.

C.17.2None

File SRC/APP/mul2_main_core.for.

Entry point of the build without mkl: only the core library is available, there is no linear or eigenvalue solver.

Uses: `MUL2_LOG`, `MUL2_RUNTIME`.